Relax.js

Relax.js

A Deep Build Guide

Ravi Kishan

A practical, test-driven walk through Relax.js internals—VDOM rendering, diffing/patching, scheduling, and compiler-assisted rendering (HRBR).

Relax.js: A Deep Build Guide

© 2026 Ravi Kishan and contributors.
Licensed under the MIT License (see `LICENSE` in the repository).

Disclaimer. This book reverse-engineers and explains the Relax.js codebase. APIs and internals may change. Performance discussions are environment-dependent; use the benchmark methodology chapter for fair comparisons.

Developer Portfolio https://ravikishan.me
GitHub. https://github.com/Ravikisha
Source. https://github.com/Ravikisha/Relax.js
NPM. https://www.npmjs.com/package/relaxcore

Contents

1 What we're building (and why) 1
1.1 What you'll build . . . 1
1.2 What you'll learn . . . 1
1.3 Prereqs . . . 1
1.4 Start from first principles (not from frameworks) . . . 1
1.5 The elevator pitch (Relax in one minute) . . . 2
1.6 A tiny contract: what does Relax.js promise? . . . 2
1.6.1 A quick terminology primer . . . 3
1.7 Public API entry points (code references) . . . 4
1.8 Why a second runtime? (HRBR in one paragraph) . . . 4
1.8.1 What problem does HRBR try to solve? . . . 4
1.9 How we'll prove correctness throughout this book . . . 5
1.9.1 What "tests as spec" means in this repository . . . 5
1.10 Build-along: the smallest end-to-end loop (with real code) . . . 6
1.10.1 createApp(): mount and unmount . . . 6
1.10.2 A small snippet you should internalize . . . 7
1.10.3 The test that proves it works . . . 7
1.11 Why this book is written the way it is (my bias as the author) . . . 7
1.12 Common pitfalls (what usually goes wrong) . . . 8
1.13 Mini-exercise . . . 8
1.14 Online resources . . . 8
1.15 Walkthrough: patching is "same element, new values" . . . 8
1.16 Mini-exercise (with intent) . . . 9
1.17 Online resources (optional, but good context) . . . 9

2 Repo tour & project constraints 10
2.1 What you'll build . . . 10
2.2 What you'll learn . . . 10
2.3 Prereqs . . . 10
2.4 Concept: read the system before you read the code . . . 10
2.5 What you're looking at . . . 11
2.5.1 Directory map (first pass) . . . 11
2.6 The constraint that shapes everything: DOM correctness in jsdom . . . 12
2.6.1 Why jsdom is a meaningful constraint . . . 12
2.7 How to read code in this repo (recommended path) . . . 12
2.7.1 A second reading path: follow a user action . . . 13
2.8 Build-along: follow one interaction end-to-end . . . 13

2.8.1 The spec (test) as the narrative . . . 13
2.8.2 Step 1: the root lifecycle boundary (createApp) . . . 13
2.8.3 Step 2: DOM event binding (how a handler gets installed) . . . 14
2.8.4 Step 3: component-to-component communication (Dispatcher) . . . 14
2.8.5 Step 4: where does the DOM update happen? . . . 14
2.9 A concrete example: createApp() is tiny on purpose . . . 15
2.9.1 Code reference . . . 15
2.9.2 Test reference (what proves the contract) . . . 15
2.10 Tests as specs (my rule for this repo) . . . 15
2.11 Common pitfalls . . . 16
2.12 Mini-exercise . . . 16
2.13 Project constraint: small API surface . . . 16
2.13.1 Where to see the supported surface area . . . 16
2.14 Tests: how this repo measures correctness . . . 16
2.14.1 The single most important helper: singleHtmlLine . . . 17
2.14.2 Where to learn semantics fastest (test tour) . . . 17
2.15 Mini-exercise . . . 17
2.16 Diagram: a "follow the tests" reading workflow . . . 18
2.17 Online resources . . . 18

3 TSX/JSX: an authoring layer on top of `h()` 19
3.1 The goal . . . 19
3.2 How TSX works in this repository (TypeScript configuration) . . . 19
3.3 Two authoring styles you'll see . . . 20
3.3.1 Direct VDOM authoring (manual `h()`) . . . 20
3.3.2 React-like TSX authoring (compiled via a JSX runtime) . . . 20
3.4 The Relax JSX runtime (the real implementation) . . . 21
3.4.1 Fragments: why `Fragment` is exported . . . 21
3.4.2 Dev runtime: jsxDEV . . . 21
3.5 Tests as spec (the subset is intentional) . . . 21
3.5.1 Test 1: TSX can mount via the Relax JSX runtime . . . 21
3.5.2 Test 2: React-like conveniences are supported . . . 22
3.6 What's not the same as React . . . 22
3.6.1 Practical limitations of the TSX subset . . . 22
3.7 Mini-exercise . . . 23
3.8 Online resources (optional, citation-ready) . . . 23

4 The VNode model (what the renderer *really* consumes) 24
4.1 What you will build in your head . . . 24
4.2 The one invariant I defend everywhere . . . 24
4.3 Why we need VNodes in the first place . . . 25
4.4 Why VNodes exist . . . 25
4.5 The VNode types: `DOM_TYPES` . . . 26
4.6 Contract: `h(tag, props, children)` . . . 26
4.7 Child normalization: `withoutNulls()` + `mapTextNodes()` . . . 27
4.8 Fragments: `hFragment(children)` and `extractChildren()` . . . 28
4.9 Slots: `hSlot(children)` as a component-level placeholder . . . 29
4.10 HRBR interop: `hBlock(mount)` and VDOM identity . . . 30

4.11 VNode identity and diffing: a preview . . . 31
4.12 Mini-exercises . . . 32
4.13 Online resources . . . 32

5 Mounting DOM (`src/mount-dom.ts`) 33
5.1 Contract: `mountDOM` . . . 33
5.2 The central pattern: dispatch by `vdom.type` . . . 34
5.3 Text nodes: `createTextNode` . . . 36
5.4 Element nodes: `createElementNode` and the props/events split . . . 36
5.4.1 `addProps`: extracting attrs vs. events . . . 37
5.5 Event handler binding: the meaning of `hostComponent` . . . 38
5.6 Fragments: `createFragmentNodes` and insertion indices . . . 39
5.6.1 The index problem . . . 40
5.7 Components: `createComponentNode` and asynchronous `onMounted` . . . 40
5.7.1 Lifecycle scheduling and the job queue . . . 41
5.8 HRBR interop: `createHrbrNode` and why `el` points to the host . . . 42
5.9 Insertion mechanics: `insert(el, parentEl, index)` . . . 43
5.10 Mini-exercises . . . 43
5.11 Online resources . . . 43

6 Attributes, properties, classes, and styles (`src/attributes.ts`) 45
6.1 The mental model: the browser has *two* prop channels . . . 45
6.2 Contract: `setAttributes(el, attrs)` . . . 46
6.3 The core decision: `setAttribute(el, name, value)` . . . 47
6.4 Tests as spec: what does "property first" buy us? . . . 48
6.5 Removal is not symmetric: `removeAttribute(el, name)` . . . 48
6.6 Class and style are special-cased (on purpose) . . . 48
6.7 Props vs events: keeping the shape sane . . . 49
6.8 Design notes: why Relax stays small here . . . 49
6.9 Mini-exercises . . . 50
6.10 Online resources . . . 50

7 Events and the Command Dispatcher 51
7.1 DOM events: the two problems every renderer must solve . . . 51
7.2 DOM event listener wiring (`src/events.ts`) . . . 52
7.2.1 Contract . . . 52
7.2.2 Implementation: the wrapper listener (real code) . . . 52
7.2.3 Multiple listeners: `addEventListeners(events, el, hostComponent?)` . . . 53
7.2.4 Cleanup symmetry: `removeEventListeners(listeners, el)` . . . 54
7.3 Tests for DOM event semantics (`src/__tests__/events.test.ts`) . . . 54
7.3.1 Invariant 1: wrappers don't break sync or async handlers . . . 54
7.3.2 Invariant 2: with a host component, `this` is correct . . . 54
7.4 The command dispatcher (`src/dispatcher.ts`) . . . 54
7.4.1 A minimal contract . . . 55
7.4.2 Subscription semantics: dedupe by function identity . . . 55
7.4.3 Dispatch semantics: warn on unhandled commands, always run after-hooks . . . 55
7.5 Tests for dispatcher semantics (`src/__tests__/dispatcher.test.ts`) . . . 56
7.6 Common pitfalls and edges . . . 56

7.7 Mini-exercises . . . 56
7.8 Where this plugs into the renderer . . . 56
7.9 Online resources . . . 57

8 Patching and Diffing: Updating the DOM Efficiently 58
8.1 Diagram: the patch decision tree . . . 58
8.2 `patchDOM`: the top-level operation . . . 59
8.2.1 Contract . . . 59
8.2.2 Replace vs patch: areNodesEqual . . . 60
8.2.3 When nodes are equal: preserve el . . . 61
8.3 Node-type specific patching . . . 61
8.3.1 Text nodes: patchText . . . 61
8.3.2 Element nodes: patchElement . . . 61
8.3.3 Attributes: patchAttrs uses objectsDiff . . . 62
8.3.4 Classes: patchClasses uses arraysDiff . . . 62
8.3.5 Styles: patchStyles uses objectsDiff . . . 62
8.3.6 Events: patchEvents . . . 62
8.3.7 Component nodes: patchComponent . . . 63
8.3.8 HRBR nodes: patchHrbr . . . 64
8.3.9 A tiny but important helper: `patchDOMSameElement` . . . 64
8.4 Patching children: from fast paths to general diff . . . 64
8.4.1 Step 0: child extraction . . . 64
8.4.2 Fast path A (opt-in): delegate keyed list reconciliation to HRBR reconciler . 65
8.4.3 Fast path B (opt-in): _textOnly rows . . . 65
8.4.4 Fast path C: keyed reorder via LIS . . . 65
8.4.5 Fast path D: stable keyed same-order . . . 66
8.4.6 General path: arraysDiffSequence . . . 67
8.4.7 What the patch-children tests specify (without running them) . . . 70
8.4.8 What the diff-sequence tests prove . . . 70
8.5 Tests to lean on . . . 71
8.6 Takeaways . . . 71
8.7 Online resources . . . 71

9 Destroying DOM and Unmounting Components 72
9.1 Diagram: render lifecycle as a loop (mount →patch →destroy) . . . 72
9.2 A tiny contract for teardown . . . 73
9.2.1 Two different things you must destroy (from scratch) . . . 73
9.3 destroyDOM(): the teardown dispatcher . . . 73
9.3.1 The shared post-step: deleting `vdom.el` . . . 74
9.3.2 A quick mental trace: what gets touched? . . . 74
9.4 Type-based teardown . . . 75
9.4.1 Text nodes . . . 75
9.4.2 Element nodes: remove element + recurse + listener cleanup . . . 75
9.4.3 Fragment nodes . . . 76
9.4.4 Component nodes: unmount + schedule `onUnmounted` . . . 76
9.5 Inside Component.unmount() (`src/component.ts`) . . . 77
9.5.1 Unmount sequence . . . 78
9.5.2 Unmount in one annotated snippet (as JavaScript) . . . 78

9.6 HRBR nodes: dispose, destroy, and remove host . . . 78
9.6.1 From scratch: HRBR is about *ownership boundaries* . . . 79
9.6.2 The destroy sequence (as JavaScript) . . . 80
9.6.3 How HRBR teardown relates to patching (what the tests prove) . . . 80
9.7 A practical mental model . . . 81
9.7.1 A leak-prevention checklist you can reuse . . . 81
9.8 Takeaways . . . 81
9.9 Walking the tests: what they prove . . . 82
9.9.1 `destroy a text element` . . . 82
9.9.2 `destroy an html element and its children` . . . 82
9.9.3 `remove an html element event listeners` . . . 82
9.9.4 Recursive destruction and fragments . . . 82
9.9.5 `destroy a component with subcomponents` . . . 83
9.9.6 Lifecycle scheduling tests (`component-lifecycle.test.ts`) . . . 83
9.9.7 HRBR teardown behavior (`hrbr-integration.test.ts`) . . . 83
9.10 Online resources . . . 83
9.11 Failure modes: the bugs youĺl actually hit . . . 83
9.11.1 Zombie event handlers (detached element still fires) . . . 84
9.11.2 Use-after-destroy (stale VNodes still referenced) . . . 84
9.11.3 Unmount hook re-entrancy . . . 84
9.11.4 HRBR interop leaks . . . 84
9.12 Edge cases worth understanding . . . 84
9.12.1 Fragments and multi-root components . . . 84
9.12.2 Double destroy . . . 85
9.12.3 Destroy order (parent removed before child teardown) . . . 85
9.13 Practical patterns for leak-free components . . . 85
9.13.1 Common resources you must clean up . . . 85
9.13.2 A small `resourceBag` pattern (plain JavaScript) . . . 85
9.14 Exercises . . . 86
9.15 Chapter summary . . . 86

10 Scheduling, nextTick, and Deterministic Work **87**
10.1 Diagram: two schedulers, two contracts . . . 87
10.2 The VDOM microtask job queue (`src/scheduler.ts`) . . . 88
10.2.1 From scratch: one thread, a call stack, and two queues . . . 88
10.2.2 Why it exists . . . 88
10.2.3 Data model . . . 88
10.2.4 enqueueJob and queueMicrotask . . . 89
10.2.5 processJobs: ordering and async behavior . . . 89
10.2.6 nextTick(): what it guarantees (and what it doesn't) . . . 91
10.3 The runtime lane scheduler (`runtime/scheduler.ts`) . . . 93
10.3.1 Creating a scheduler . . . 93
10.3.2 Budgets and time slicing . . . 93
10.3.3 Lane priority and deterministic ordering . . . 93
10.3.4 Aging and promotion (starvation prevention) . . . 94
10.3.5 withBudget(): integrating user work . . . 95
10.4 How these schedulers relate to the book . . . 95
10.5 Takeaways . . . 95

10.6 Walking the tests: what they prove . . . 95
10.6.1 VDOM scheduler tests (`src/__tests__/scheduler.test.ts`) . . . 95
10.6.2 Runtime scheduler tests (`runtime/__tests__/scheduler.test.ts`) . . . 96
10.7 Online resources . . . 96
10.8 Pitfalls and debugging (senior notes) . . . 96
10.8.1 Mistaking `nextTick()` for "wait for all async" . . . 96
10.8.2 Microtask storms (starving input/paint) . . . 97
10.8.3 Re-entrancy and why hooks are scheduled . . . 97
10.9 Exercises . . . 97
10.10 Chapter summary . . . 97

11 Slots and Content Projection **98**
11.1 Diagram: component render →slot rewrite →normal VDOM . . . 99
11.2 Slot VNodes: `hSlot` and the SLOT marker . . . 99
11.2.1 Why track `didCreateSlot`? . . . 99
11.2.2 Authoring from scratch: what the user writes vs. what Relax sees . . . 100
11.3 fillSlots(): projecting external children into a component . . . 101
11.3.1 Contract . . . 101
11.3.2 A more precise contract: invariants you can rely on . . . 101
11.3.3 Choosing between default and external content . . . 102
11.3.4 Slots with no content are removed . . . 103
11.3.5 A subtle but crucial point: `children` is not "DOM children" . . . 103
11.3.6 A code-walk of `fillSlots` (in plain JavaScript) . . . 103
11.3.7 Shape detail: why `hSlot([defaultContent])` is special . . . 104
11.4 Why fillSlots skips component children during traversal . . . 105
11.4.1 Why this boundary matters in real apps . . . 105
11.4.2 From-scratch example: why crossing the boundary would be a bug . . . 106
11.5 Slots in real components (`component-slots.test.ts`) . . . 106
11.5.1 Basic projection . . . 106
11.5.2 Default content . . . 106
11.5.3 Conditional slots . . . 107
11.5.4 Updating slot content between renders . . . 107
11.5.5 Nested slots inside lists . . . 107
11.6 Walking the integration tests in slow motion (event-loop + render pipeline) . . . 108
11.7 Takeaways . . . 108
11.8 Walking the tests: what they prove . . . 109
11.8.1 Unit semantics (`src/__tests__/slots.test.ts`) . . . 109
11.8.2 Integration semantics (`src/__tests__/component-slots.test.ts`) . . . 109
11.9 Online resources . . . 109
11.10 Common pitfalls (senior notes) . . . 110
11.11 Performance notes (why this is fast in practice) . . . 110
11.12 Exercises . . . 110
11.13 Chapter summary . . . 110

12 HRBR Signals: a Small Deterministic Reactive Graph **112**
12.1 Why signals exist (and why HRBR needs them) . . . 112
12.2 Public API: the contract we have to satisfy . . . 113
12.3 Mental model . . . 113
12.4 The tiny contract (keep this in your head) . . . 114
12.5 Diagram: signals graph (sources →computations →derived values) . . . 114
12.6 Core data structures: intrusive lists for stable order . . . 114
12.6.1 Why two linked lists (observer list + source list)? . . . 115
12.6.2 Diagram: forward observers vs. reverse sources . . . 115
12.7 Dependency capture: `currentComputation` . . . 116
12.7.1 Signal reads . . . 116
12.7.2 `untrack(fn)` . . . 117
12.8 Writes: schedule observers, preserve order . . . 117
12.8.1 A precise code-walk: `scheduleComputation` and the `pending` set . . . 118
12.9 Effects: cleanup, re-run, disposal . . . 120
12.9.1 Cleanup semantics . . . 120
12.9.2 Detaching and resubscribing: `unlinkAllSources(comp)` . . . 120
12.9.3 Conditional dependencies in slow motion (the reason `unlinkAllSources` exists) 121
12.10 The tests as specs: what behavior is non-negotiable . . . 122
12.10.1 Core semantics: `runtime/__tests__/signals.test.ts` . . . 122
12.10.2 Determinism + leak-ish checks: `signals.determinism.test.ts` . . . 122
12.10.3 Fuzz/property checks: `signals.fuzz.test.ts` . . . 123
12.10.4 Dispose . . . 123
12.11 Re-entrancy: scheduling while running . . . 123
12.11.1 Two reasons computations are delayed . . . 123
12.12 Batching: a flush boundary . . . 123
12.13 Memos: derived signals with equality . . . 124
12.14 Lanes: effects can be scheduled via the runtime scheduler . . . 124
12.14.1 How non-sync lanes are tested . . . 124
12.15 Determinism tests and fuzz tests . . . 125
12.16 Walking the tests: what they prove . . . 125
12.16.1 Core semantics (`runtime/__tests__/signals.test.ts`) . . . 125
12.16.2 Determinism suite (`signals.determinism.test.ts`) . . . 126
12.16.3 Fuzz/property tests (`signals.fuzz.test.ts`) . . . 126
12.17 Online resources . . . 126
12.18 Takeaways . . . 126
12.19 Exercises . . . 126
12.20 Chapter summary . . . 127

13 HRBR Deterministic Scheduler: Lanes, Budgets, and Aging **128**
13.1 The contract: schedules tasks, flushes deterministically . . . 128
13.1.1 Why a scheduler exists (not just "because React has one") . . . 128
13.1.2 A concrete mental model: 5 queues + 1 deterministic pick loop . . . 129
13.2 Lanes: latency categories and fixed priority order . . . 130
13.2.1 What lanes are (and what they are not) . . . 130
13.2.2 What is a lane in practice? . . . 131
13.2.3 The sync lane . . . 131
13.3 Flush strategies: how work gets a time slice . . . 131

13.3.1 createRequestFlush(strategy) . . . 132
13.3.2 createBrowserScheduler() convenience . . . 132
13.4 Deterministic ordering inside a lane . . . 132
13.4.1 Where deadlines come from . . . 132
13.4.2 Three tie-breakers, one guarantee (and why we need all three) . . . 133
13.5 Budgets: splitting work over multiple flushes . . . 133
13.5.1 What flush(budgetMs) does . . . 133
13.5.2 setFrameBudget(ms) . . . 134
13.5.3 Frame budget versus task budget (important distinction) . . . 134
13.5.4 Walkthrough: tight budget slicing (mapping a test to code) . . . 134
13.5.5 Timeline example: lane priority, aging, and budget all at once . . . 135
13.5.6 Edge cases to keep in mind . . . 136
13.6 Deadlines and aging: starvation prevention via promotion . . . 136
13.6.1 Fairness without nondeterminism . . . 137
13.6.2 Why promotion checks only queue heads . . . 137
13.7 withBudget(): bridging user work and scheduler work . . . 137
13.7.1 A practical use case: big event handler + pending effects . . . 137
13.7.2 Global helpers in `runtime/index.ts` . . . 138
13.8 Devtools and instrumentation hooks . . . 138
13.8.1 Why "events + counters" is a good runtime pattern . . . 138
13.9 Walking the tests: what they prove . . . 138
13.9.1 Core semantics (`runtime/__tests__/scheduler.test.ts`) . . . 138
13.9.2 How to read these tests like a spec . . . 139
13.10 Online resources . . . 139
13.11 Takeaways . . . 139

14 HRBR Blocks: Template + Slots (the Compiled Fast Path) 140
14.1 Why blocks exist (what VDOM cannot do efficiently) . . . 141
14.2 A minimal block from scratch (end-to-end) . . . 141
14.3 A tiny contract (what a block guarantees) . . . 142
14.4 Block vocabulary . . . 142
14.4.1 TemplateHTML . . . 142
14.4.2 Slot . . . 143
14.5 Why paths (and not querySelector)? . . . 143
14.5.1 Path stability: whitespace is part of your ABI . . . 144
14.6 The public API (surface area) . . . 144
14.7 defineBlock(): the type boundary . . . 145
14.8 mountBlock(): instantiate template + resolve slot nodes . . . 145
14.8.1 Path resolution and caching . . . 146
14.8.2 Dev diagnostics . . . 146
14.9 update(values): slot patching rules . . . 146
14.9.1 Slot kind dispatch at a glance . . . 147
14.9.2 text slots . . . 147
14.9.3 attr slots (including boolean attributes and SVG xlink) . . . 148
14.9.4 prop slots (input correctness) . . . 148
14.9.5 class slots . . . 149
14.9.6 style slots . . . 149
14.9.7 event slots: bind once, swap handler . . . 149

14.10 Lifecycle: destroy vs dispose, and idempotency . . . 150
14.10.1 Why the destroyed guard matters in a scheduled world . . . 150
14.11 mountCompiledBlock(): wiring signals to slot writes . . . 150
14.11.1 The contract the compiler relies on . . . 150
14.11.2 Why coalescing matters (and how it stays deterministic) . . . 151
14.11.3 Timeline: what happens when two signals change in the same tick . . . 152
14.11.4 Annotated walkthrough: `mountCompiledBlock()` line-by-line . . . 152
14.11.5 Lane choice is part of the compiled contract . . . 155
14.11.6 Disposal stops reactive updates . . . 155
14.12 Composition: nested blocks and fallback regions . . . 155
14.13 Walking the tests (the executable spec) . . . 156
14.13.1 `runtime/__tests__/block.test.ts` . . . 156
14.13.2 `runtime/__tests__/block-slot-matrix.test.ts` . . . 157
14.13.3 `runtime/__tests__/block-input-controls.test.ts` . . . 157
14.13.4 `runtime/__tests__/block-disposal.test.ts` . . . 157
14.14 Online resources . . . 157
14.15 Takeaways . . . 157

15 Hydration: Attaching HRBR to Server HTML 158
15.1 Why hydration exists (from scratch) . . . 158
15.2 The big picture: where hydration sits in HRBR . . . 159
15.3 A tiny contract (what hydrateBlock promises) . . . 160
15.4 Build it: a minimal end-to-end hydration loop . . . 160
15.4.1 Why these steps matter . . . 161
15.5 Hydration contract (v1) . . . 161
15.6 The fast path: attach without rewriting . . . 162
15.7 Mismatch detection: when to bail out . . . 163
15.7.1 Root tag mismatch . . . 163
15.7.2 Slot path mismatch: element tagName mismatch . . . 163
15.8 Why this design? (tradeoffs, alternatives, and invariants) . . . 164
15.9 Hydrated update semantics: same slot kinds as mountBlock . . . 165
15.9.1 Redundant write skipping . . . 165
15.10 Event slots: v1 difference vs mountBlock . . . 166
15.11 SSR helper: renderBlockToString() . . . 166
15.12 Takeaways . . . 167
15.13 Walking the tests (the executable spec) . . . 167
15.13.1 `runtime/__tests__/hydration.test.ts` . . . 167
15.13.2 `runtime/__tests__/ssr.test.ts` . . . 168
15.14 Online resources . . . 168
15.15 Common pitfalls . . . 168
15.16 Mini-exercise . . . 169

16 SSR Authoring (v1): Rendering Blocks to HTML Strings 170
16.1 What you'll build . . . 170
16.2 What you'll learn . . . 170
16.3 Prerequisites . . . 170
16.4 Why SSR exists in HRBR . . . 171
16.5 A tiny contract (what SSR v1 guarantees) . . . 171

16.6 The contract: `renderBlockToString(def, options?) -> string` . . . 172
16.6.1 A minimal end-to-end usage (SSR → hydrate → update) . . . 172
16.7 Implementation: DOM-based SSR for correctness . . . 173
16.7.1 Why DOM-based SSR is acceptable in v1 . . . 173
16.7.2 The implementation is short on purpose . . . 174
16.8 SSR must round-trip with hydration . . . 174
16.8.1 Authoring rule: structure changes are breaking changes . . . 175
16.9 The end-to-end spec: render → hydrate → update . . . 175
16.9.1 Why the test checks identity, not just text . . . 176
16.10 Limitations and the intended direction . . . 176
16.11 Takeaways . . . 176
16.12 Walking the tests (the executable spec) . . . 176
16.12.1 `runtime/__tests__/ssr.test.ts` . . . 177
16.13 Common pitfalls . . . 177
16.14 Mini-exercise . . . 177
16.15 Online resources . . . 178

17 Fallback Reconciler: Dynamic Structure When Blocks Aren't Enough 179
17.1 What you'll build . . . 179
17.2 What you'll learn . . . 180
17.3 Prerequisites . . . 180
17.4 Why fallback exists (the honest reason) . . . 180
17.5 A tiny contract (what fallback + reconciler promise) . . . 181
17.6 Contracts . . . 181
17.6.1 The reconciler contract: direct children only . . . 181
17.6.2 From scratch: what is a `ReconcileItem`? . . . 182
17.6.3 Keyed vs unkeyed mode . . . 182
17.7 Why ranges exist (one logical item => multiple DOM nodes) . . . 183
17.7.1 Range authoring example (fragment-like output) . . . 183
17.8 Unkeyed reconciliation: simple and predictable . . . 184
17.9 Keyed reconciliation: preserve identity and minimize moves . . . 184
17.9.1 Two-ended scan + LIS . . . 185
17.9.2 Why keyed mode still needs dev diagnostics . . . 185
17.10 Fallback mount: wiring reconciliation to reactivity . . . 185
17.10.1 Fallback example (mirrors the integration test) . . . 186
17.10.2 Dev diagnostics for suspicious usage . . . 186
17.10.3 Shape-change safety: node ↔ range . . . 187
17.11 Property/fuzz testing as an executable oracle . . . 187
17.12 Fallback mount: reconciling signal-driven dynamic children . . . 188
17.13 Takeaways . . . 188
17.14 Walking the tests (the executable spec) . . . 189
17.14.1 `runtime/__tests__/reconciler.test.ts` . . . 189
17.14.2 `runtime/__tests__/reconciler.fuzz.test.ts` . . . 189
17.14.3 `runtime/__tests__/fallback.test.ts` . . . 189
17.15 Online resources . . . 189

18 VDOM ↔ HRBR interop (mixed trees) 191
18.1 The tiny contract: an HRBR vnode is just a mount factory . . . 192
18.1.1 VNode shape . . . 192
18.1.2 A tiny contract you can hold in your head . . . 193
18.1.3 HRBR instances: a minimal lifecycle interface . . . 193
18.1.4 Creating HRBR vnodes: `hBlock()` . . . 193
18.2 Mounting: why HRBR gets its own host container . . . 194
18.2.1 `createHrbrNode()`: mount into a dedicated `<span>` . . . 194
18.2.2 Edge cases this design avoids . . . 195
18.3 Patching: stability by mount factory identity . . . 195
18.3.1 Fast path: stable mount function means reuse . . . 196
18.3.2 Slow path: mount changed means dispose+destroy+remount . . . 197
18.3.3 A note on where remounts happen . . . 198
18.4 Destroying: one call site, two optional hooks . . . 198
18.4.1 Why both `dispose()` and `destroy()`? . . . 199
18.5 Tests as specs: what the integration tests guarantee . . . 199
18.5.1 Mounting an HRBR block returned from a VDOM component . . . 199
18.5.2 Patching: stable mount function preserves instance . . . 200
18.5.3 Patching: changing mount function forces remount . . . 200
18.6 Common pitfalls (and how to avoid them) . . . 200
18.6.1 Pitfall: creating a new mount function every render . . . 200
18.6.2 Pitfall: forgetting that VDOM owns the wrapper, not the inner nodes . . . 200
18.6.3 Pitfall: leaking event handlers without a teardown hook . . . 200
18.7 Mini-exercise: design an HRBR block API that updates without remounting . . . 201
18.8 Tests-as-spec: the integration suite . . . 201
18.8.1 Test harness: what this suite is actually exercising . . . 201
18.8.2 1) Mount an HRBR block returned from a VDOM component . . . 201
18.8.3 2) Event semantics: bind to the host component . . . 202
18.8.4 3) Patch: stable mount identity preserves instance . . . 202
18.8.5 4) Patch: changing mount identity remounts . . . 202
18.9 Practical guidance (and foot-guns) . . . 203
18.9.1 Common failure modes in mixed trees . . . 203
18.10 Online resources . . . 203

19 Compiler architecture overview (prototype + Babel transform) 204
19.1 The output contract (what the runtime needs) . . . 205
19.1.1 Tiny contract (2 bullets) . . . 205
19.1.2 1) A `BlockDef`: stable template + slot metadata . . . 206
19.1.3 2) A `CompiledSlot[]` list: how to read live values . . . 206
19.1.4 A 3-part boundary you should memorize . . . 206
19.2 Routing decision: block vs fallback . . . 207
19.2.1 Dynamic structure . . . 207
19.2.2 A useful rule of thumb . . . 207
19.2.3 How "block vs fallback" relates to Chapter 18 . . . 208
19.3 TypeScript prototype compiler (Phase 5 reference) . . . 208
19.3.1 `compileTSXToBlock()`: parse -> extract -> render template . . . 208
19.3.2 Determinism: why the TS compiler is a good executable spec . . . 208
19.3.3 Paths: the compiler and runtime must agree . . . 209

19.3.4 The one gotcha that causes most bugs: `childNodes` includes text nodes . . . 209
19.3.5 Walkthrough: compute a slot path by hand . . . 209
19.3.6 What the TS prototype actually returns (and why the reads are empty) . . . 210
19.4 Babel transform: how the output program is shaped . . . 210
19.4.1 The shape (as asserted by tests) . . . 210
19.4.2 Lane pragmas: compiler-to-scheduler hints without changing semantics . . . 211
19.5 Debugging: when a slot path is wrong . . . 211
19.5.1 Text expression handling: sentinel + slot path . . . 212
19.5.2 Void elements and template correctness . . . 213
19.5.3 Attributes: literals vs expression slots . . . 213
19.5.4 Fallback decision wrapper: `compileTSXOrFallback()` . . . 213
19.6 Dynamic structure detection (what triggers fallback) . . . 214
19.6.1 Decision table: TS prototype vs Babel transform . . . 214
19.6.2 Babel's `isTextLikeExpr`: the tiny heuristic that buys you a lot . . . 214
19.7 Babel compilation from scratch: how one JSX element becomes a block . . . 215
19.7.1 Step 1: pick a stable root (or give up) . . . 215
19.7.2 Step 2: render a string that the browser will parse deterministically . . . 215
19.7.3 Step 3: allocate slots: "static HTML with holes" . . . 216
19.7.4 Step 4: emit reads as closures (the key Babel advantage) . . . 216
19.7.5 Step 5: fallback routing is just "emit a different mount factory" . . . 216
19.8 Tests as specs: how to validate compiler changes safely . . . 216
19.8.1 Spec-to-test mapping (start here when debugging) . . . 217
19.8.2 Decision table: TS prototype vs Babel transform . . . 217
19.8.3 Babelś `isTextLikeExpr`: the tiny heuristic that buys you a lot . . . 217
19.9 How fallback routing looks in the Babel output (scaffold) . . . 218
19.10 Babel transform scaffold (real-world integration) . . . 219
19.10.1 Roles of the Babel plugin . . . 219
19.10.2 "Block mode" is best-effort: return `null` to fall back . . . 219
19.10.3 Babel transform: what "best-effort block mode" means in code . . . 219
19.10.4 Slot key strategies: sequential vs stable (dev mode) . . . 219
19.10.5 Lane pragmas (compiler-to-scheduler hint) . . . 220
19.11 Tests-as-spec . . . 220
19.11.1 Unit tests (TS): compile + mount sanity . . . 220
19.11.2 TS end-to-end: compile -> mount -> update without recreating root . . . 221
19.11.3 Babel transform unit tests: shape of emitted program . . . 221
19.11.4 Babel transform errors: the intentional block-mode subset . . . 221
19.11.5 Babel transform E2E (jsdom): evaluate emitted literals + mount . . . 222
19.12 Worked example: derive `templateHTML` and slot paths from the snapshot . . . 222
19.12.1 The input JSX (what the author writes) . . . 222
19.12.2 Step 1: the static template string (structure + placeholders) . . . 222
19.12.3 Step 2: the slot declarations (what to write + where to write it) . . . 223
19.12.4 Step 3: compute `path: [3, 1]` by hand (the whitespace trap) . . . 223
19.12.5 Step 4: compiled slot readers (how values cross the boundary) . . . 224
19.12.6 Debugging checklist (tests-first) . . . 224
19.12.7 Babel fallback E2E (jsdom): structural expression routes to `mountFallback` . 224
19.13 Where we go next . . . 224
19.14 Online resources . . . 225

20 Babel plugin in detail (JSX transform) 226
20.1 Public entrypoint and options . . . 226
20.1.1 Tiny contract: what this plugin must guarantee . . . 226
20.2 High-level rewrite shape . . . 227
20.3 Program-level state and synthesis (imports + wrappers) . . . 227
20.3.1 Why inject at `Program.exit`? . . . 228
20.4 The core visitor: `JSXElement(path)` . . . 228
20.4.1 Success path: compile into a block . . . 228
20.4.2 Failure path: route to fallback . . . 229
20.4.3 Fallback synthesis: what gets captured (and what doesn't) . . . 230
20.5 Template extraction: `buildTemplateFromJsxElement()` . . . 230
20.5.1 Supported subset (today) . . . 230
20.6 Babel plugins from scratch: the minimum you need to know . . . 231
20.7 A tiny contract for this file (inputs/outputs/error modes) . . . 231
20.8 Why this scaffold emits "raw AST objects" instead of Babel helpers . . . 232
20.9 Program-level synthesis: `ensureRuntimeImport` and `ensureWrapper` . . . 232
20.9.1 How imports are synthesized . . . 232
20.9.2 Why we have a wrapper helper at all . . . 232
20.10 Template extraction as a recursive renderer . . . 233
20.10.1 Key data structures . . . 233
20.10.2 Paths: tracked while rendering . . . 233
20.10.3 The "text-like" gate: why `isTextLikeExpr` exists . . . 233
20.11 Fallback synthesis: the semantic firewall (minimal but real) . . . 233
20.11.1 From scratch: what the fallback compiler is building . . . 234
20.12 Failure modes are part of the design (and are tested) . . . 234
20.12.1 How paths are computed in the Babel transform . . . 235
20.12.2 Slot keys: sequential vs location-based . . . 235
20.12.3 Lane pragmas: `/*@lane transition*/` . . . 235
20.12.4 Text-like safety . . . 235
20.13 Failure modes are part of the API . . . 236
20.14 End-to-end tests: jsdom execution proves output shape . . . 236
20.14.1 Block mode e2e: transformed output mounts and updates . . . 236
20.14.2 Fallback routing e2e: structural text expression goes to `mountFallback` . . . 237
20.15 What to improve next (guided by tests) . . . 237
20.16 Full trace: one `JSXElement` visitor call, end to end . . . 237
20.16.1 Author input (JSX in user code) . . . 237
20.16.2 Step A: `Program.enter` resets per-file state . . . 238
20.16.3 Step B: `JSXElement(path)` attempts block compilation . . . 238
20.16.4 Step C: `buildTemplateFromJsxElement` renders HTML and records slots . . . 238
20.16.5 Step D: the visitor hoists `defineBlock(...)` at module scope . . . 239
20.16.6 Step E: the JSX expression becomes a mount factory . . . 239
20.16.7 Step F: `Program.exit` injects imports and the wrapper helper . . . 239
20.17 Exercises (tests-first, like a real compiler engineer) . . . 239
20.17.1 Exercise 1: explain a slot path without running code . . . 240
20.17.2 Exercise 2: add a new slot kind (carefully) . . . 240
20.17.3 Exercise 3: widen `isTextLikeExpr` with a test . . . 240
20.18 Online resources . . . 240

21 Devtools and instrumentation (optional) 241
21.1 The API surface: five functions and two helpers . . . 241
21.1.1 Tiny contract (what you can rely on) . . . 242
21.2 Event model . . . 242
21.3 Counters: what is tracked and when . . . 243
21.3.1 Defensive reads: `getDevtoolsCounters()` . . . 243
21.3.2 Why DOM ops and allocs are gated . . . 243
21.4 Emission: increment + forward . . . 244
21.4.1 A subtle performance point: "no hook" short-circuit . . . 244
21.5 Where events come from in the runtime . . . 245
21.5.1 Blocks: slot writes and DOM operations . . . 245
21.5.2 Scheduler: flush timing . . . 245
21.5.3 Signals: scheduled computations . . . 246
21.6 Tests-as-spec . . . 246
21.6.1 `runtime/__tests__/devtools.test.ts` . . . 246
21.6.2 `runtime/__tests__/instrumentation.test.ts` . . . 246
21.7 How you might use this in practice . . . 247
21.7.1 From scratch: what problem are we solving? . . . 247
21.7.2 A tiny devtools "panel" in 50 lines (hook + DOM) . . . 247
21.7.3 How to interpret the counters (and common misreads) . . . 248
21.7.4 A tiny contract you can build tooling around . . . 249
21.8 Exercises (tests-first) . . . 249
21.8.1 A simple pattern: hook + manual aggregation . . . 250
21.9 Online resources . . . 250

22 Benchmarks methodology (repeatability + fairness) 251
22.1 A tiny contract for what we are measuring . . . 251
22.2 Benchmarks run in a real browser . . . 252
22.3 Benchmarks run in a real browser . . . 252
22.3.1 Why "real browser" matters . . . 252
22.3.2 A small but important detail: Node/jsdom fallback exists . . . 253
22.4 The harness contract . . . 253
22.4.1 Benchmark case shape . . . 253
22.4.2 What counts as "one tick"? (and why that matters) . . . 253
22.4.3 Tiny contract for a fair `BenchmarkCase` . . . 254
22.4.4 Warmup vs measure windows . . . 254
22.4.5 Reported metrics: avg + p95 + drops . . . 254
22.4.6 How to read results (avg vs p95 vs drops) . . . 254
22.5 Profiles and seeded runs . . . 255
22.5.1 Profiles via URL parameters . . . 255
22.5.2 Benchmark variables (every knob you can turn) . . . 255
22.5.3 Examples: URL parameters in practice . . . 256
22.5.4 Seeded RNG . . . 257
22.6 The benchmark runner UI . . . 257
22.6.1 Why `host.innerHTML = ''` between runs matters . . . 257
22.7 Cases: what they measure . . . 257
22.7.1 Case study: `text-1m` . . . 258
22.7.2 What should you expect from `text-1m`? . . . 258

22.7.3 Case study: `list-10k-1pct` . . . 258
22.7.4 What `list-10k-1pct` tries to model . . . 258
22.7.5 Case study: `widgets-200` . . . 259
22.7.6 Extra cases in `cases/more.ts` (and what each means) . . . 259
22.8 Build modes matter (dev vs prod) . . . 259
22.8.1 What the Rollup configs actually do . . . 260
22.9 Perf smoke (CI-friendly) . . . 260
22.9.1 What perf-smoke is actually specifying . . . 260
22.9.2 Why the thresholds are wide . . . 260
22.10 Instrumentation and benchmarks: keep them separate . . . 261
22.11 Online resources . . . 261
22.12 Checklist: how to run a fair comparison . . . 261
22.13 Interpreting a real output (example walkthrough) . . . 261
22.14 Why this benchmark approach is better than "single number" benchmarks . . . 262
22.15 Online resources . . . 263

23 Release-quality discipline: tests, typecheck, lint, builds **264**
23.1 The quality-gates contract . . . 264
23.1.1 A tiny contract (inputs, outputs, failure modes) . . . 265
23.2 Typecheck: make illegal states unrepresentable . . . 265
23.2.1 Common typecheck failures (and how to debug) . . . 266
23.2.2 Why there are multiple `tsconfig` files . . . 266
23.3 Lint: keep the surface area consistent . . . 266
23.3.1 What lint catches that typecheck doesn't . . . 267
23.4 Tests: executable specs with jsdom . . . 267
23.4.1 The test suite structure . . . 267
23.4.2 Tests-as-spec example: TSX wiring . . . 267
23.4.3 Debugging failing tests (a workflow) . . . 268
23.4.4 What to do when a test failure seems "impossible" . . . 268
23.5 Builds: ESM, CJS, and types . . . 268
23.5.1 Why builds are a quality gate . . . 269
23.5.2 Build discipline: match `exports` to outputs . . . 269
23.6 Release checklist (practical) . . . 270
23.7 Evidence from this repo . . . 270
23.8 Online resources . . . 270

Appendix: notation **271**

References **272**

Chapter 1

What we're building (and why)

1.1 What you'll build

In this chapter, you will build a mental model for the whole project:

- A crisp definition of what a UI runtime must do (model →mount →patch →destroy).
- The two execution paths in Relax.js (VDOM baseline and HRBR compiled fast path).
- A practical reading workflow: tests are the spec.

1.2 What you'll learn

- Why UI libraries exist: the DOM is an output device, not a state container.
- The minimum set of concepts you need to build a correct renderer.
- How to interpret benchmark results without lying to yourself (preview).

1.3 Prereqs

No JavaScript/TypeScript tricks are required. I only assume you can read code, and you understand that DOM nodes are mutable objects.

1.4 Start from first principles (not from frameworks)

I want to start from a logical place, not from a JS API. Here is the problem statement that led me to build Relax:

> We want a deterministic way to turn *state* into *DOM*, and then update that DOM efficiently as state changes.

This gives us four responsibilities. If you understand these, you understand 80% of any UI runtime:

1. **Model**: represent desired UI in memory.

2. **Mount**: turn the model into DOM for the first time.
3. **Patch**: update DOM when the model changes.
4. **Destroy**: cleanup (remove nodes, listeners, and references).

Relax.js is my implementation of those responsibilities, with strong emphasis on:

- **Correctness** (tests as a spec)
- **Performance** (avoid unnecessary DOM work)
- **Predictability** (clear scheduling and lifecycle boundaries)

1.5 The elevator pitch (Relax in one minute)

Relax.js is a small frontend library built around a Virtual DOM (VDOM) and a component model. What makes this repository especially interesting is that it contains *two* rendering strategies:

1. A traditional VDOM runtime under `src/`.
2. An experimental compiler-assisted runtime (HRBR) designed to update the DOM through precomputed **slots**.

The goal of this book is not only to show *how to use* Relax.js, but to explain *how to build it*. We'll read the real source code, map it to a mental model, and connect every important concept to tests.

This isn't a conceptual "frameworks 101" book. It's a guided reverse-engineering and rebuild:

- You will learn the smallest set of abstractions needed to describe UI.
- You will see how those abstractions are represented in memory.
- You will trace how a rendering pass turns them into real DOM.
- You will follow how updates are scheduled and applied.
- You will use the test suite as the specification that locks behavior in place.

By the end, you should be able to (1) understand every important subsystem in this repository, and (2) rebuild something similar with confidence.

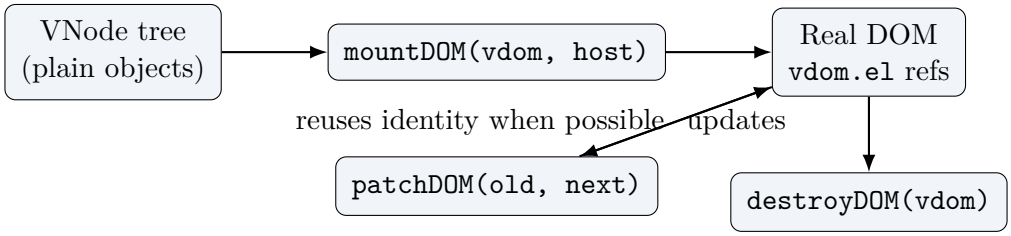
1.6 A tiny contract: what does Relax.js promise?

At a high level, Relax.js promises:

- You describe UI as a tree (VNode tree or compiled block).
- The runtime mounts it into a DOM host element.
- When state changes, only the necessary DOM updates happen.
- Components have lifecycle hooks and can emit/handle events.

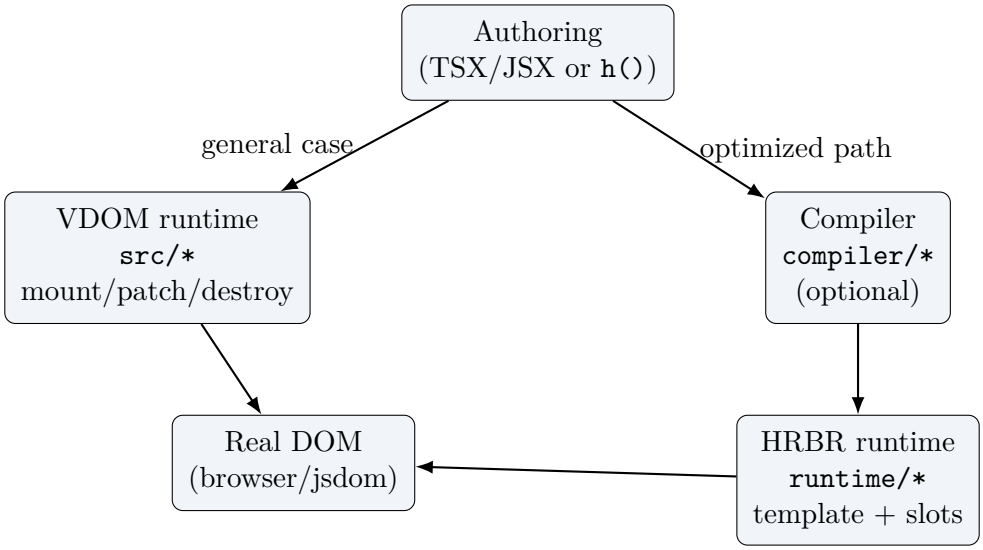
Diagram: the end-to-end VDOM update loop (mount →patch →destroy)

Even if you later adopt the HRBR compiled fast path, the VDOM loop is the baseline contract. Most tests in `src/__tests__/*` are describing one of these arrows.



This is the book's core promise: if two VNodes represent the "same" thing, Relax tries to keep the same DOM node and just update its values.

Diagram: the two execution paths (VDOM vs HRBR)



This diagram is the mental model for the rest of the book: Relax always ends at *real DOM updates*, but it can reach them either by interpreting a VNode tree (VDOM) or by executing precomputed slot writes (HRBR).

Those promises are verified by tests under `src/__tests__/` and (for the HRBR runtime) under `runtime/__tests__/`.

1.6.1 A quick terminology primer

You'll see these words throughout the code and tests:

- **VNode**: a plain object that describes what DOM you want.
- **Mount**: create DOM from a VNode for the first time.
- **Patch**: update existing DOM by comparing old/new VNodes.
- **Destroy**: remove DOM nodes and detach listeners.

- **Host element**: the DOM element you mount the app into (often `document.body` in tests).
- **HRBR block**: a compiled representation: template HTML + dynamic *slots*.
- **Slot**: a precomputed "write location" (text/attr/prop/class/style/event) in a template.

1.7 Public API entry points (code references)

The *public* exports live in `src/index.ts`:

```
export { createApp } from ./app'
export { defineComponent } from ./component'
export { DOM_TYPES, h, hFragment, hSlot, hString } from ./h'
export { nextTick } from ./scheduler'
```

This tells you the learning order:

1. `createApp()` (app lifecycle)
2. `defineComponent()` (component model)
3. `h()/hFragment()` (VNode model)
4. `mountDOM()/patchDOM()/destroyDOM()` (DOM mechanics)
5. `nextTick()` (async scheduling in the VDOM runtime)

1.8 Why a second runtime? (HRBR in one paragraph)

The VDOM approach is flexible, but it has overhead: you allocate VNodes, diff lists, and interpret the tree every update. HRBR is the opposite trade-off:

- The compiler extracts a stable template (static HTML) once.
- Dynamic values become **slots** with stored DOM paths.
- Updates become *direct writes* to known nodes.

You'll see HRBR show up in VDOM code via `DOM_TYPES.HRBR` and `hBlock()` in `src/h.ts`.

1.8.1 What problem does HRBR try to solve?

If a UI subtree is structurally stable (same elements in the same shape, only values change), a classic VDOM update does extra work:

- allocate and traverse VNodes on every update
- interpret the tree to find where to write
- diff children arrays (sometimes expensive)

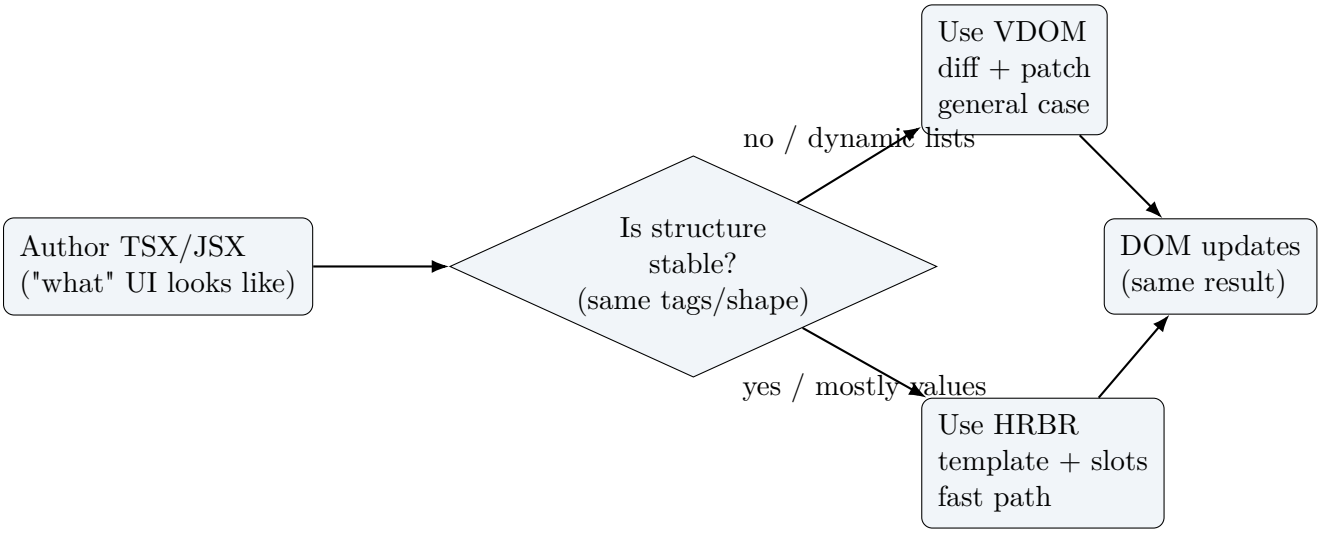
HRBR moves some of that cost earlier ("compile time"):

- build a static DOM template once
- record paths to dynamic locations (slots)
- update by writing directly to those DOM nodes

That doesn't make VDOM obsolete. It creates a spectrum: VDOM is the flexible general case; HRBR is an optimized fast path when the structure is known.

Diagram: when HRBR is a good fit (static structure, dynamic data)

This isn't a binary choice; think of HRBR as a selective optimization for subtrees that are structurally stable.



1.9 How we'll prove correctness throughout this book

This repository uses **Vitest** and **jsdom**. Our rule is simple: every time we explain a behavior, we should be able to point to a unit test.

Examples:

- Mounting basics: `src/__tests__/mount-dom.test.ts`
- Patching/diffing: `src/__tests__/patch-dom.test.ts`
- Components + lifecycle: `src/__tests__/component*.test.ts`
- HRBR block runtime: `runtime/__tests__/block.test.ts`

1.9.1 What "tests as spec" means in this repository

You should treat tests here as *contracts*. When you change runtime behavior, you don't just "fix failing tests"—you decide whether the contract changed.

Most tests follow the same simple pattern:

1. Set up a small DOM (usually `document.body`).
2. Build a VNode tree (or HRBR block).
3. Mount, patch, or destroy.
4. Assert on resulting `innerHTML` or element identity.

That gives you two superpowers as a reader:

- You can infer semantics without reading every implementation detail.
- You can reproduce bugs by writing a tiny test first.

1.10 Build-along: the smallest end-to-end loop (with real code)

Let's walk through the smallest app lifecycle you can understand in one sitting. This is the loop I use when I sanity-check any renderer:

1. Build a model of UI (VNode tree).
2. Mount it into a host element.
3. Change state by producing a new model.
4. Patch the DOM to match.
5. Destroy and ensure cleanup is complete.

1.10.1 createApp(): mount and unmount

The application API is defined in `src/app.ts`. The critical part is that `createApp()` remembers three pieces of state:

- `parentEl`: where we mounted
- `isMounted`: guard against double mount/unmount
- `vdom`: the root VNode tree

When you mount:

1. it throws if already mounted
2. it creates the root VNode by calling `h(RootComponent, props)`
3. it calls `mountDOM(vdom, parentEl)`

When you unmount:

1. it throws if not mounted
2. it calls `destroyDOM(vdom)`
3. it resets internal references

This is a small but important architectural choice: *the App owns the root VNode*. That makes "unmount" a reliable cleanup boundary.

1.10.2 A small snippet you should internalize

When you strip away everything else, most UI libraries are some version of this:

```
let oldTree = null

function render(nextTree, host) {
  if (oldTree == null) {
    mountDOM(nextTree, host)
  } else {
    patchDOM(oldTree, nextTree, host)
  }
  oldTree = nextTree
}

function dispose() {
  if (oldTree != null) destroyDOM(oldTree)
  oldTree = null
}
```

extbfWhy this matters: if you can reason about this loop, you can reason about the entire Relax VDOM runtime. All the complexity is in the details: fragments, keyed reorders, events, components, and scheduling.

1.10.3 The test that proves it works

Open `src/__tests__/app.test.ts`. It reads like a story:

- It creates an app via `createApp(App, { todos: [...] })`.
- It mounts to `document.body`.
- It waits for `nextTick()` to flush async work.
- It asserts a stable HTML snapshot via `singleHtmlLine...`.

Notice the discipline here: the tests make DOM updates predictable by explicitly awaiting `nextTick()`. That foreshadows Chapter 10, where we explain why scheduling exists and how it interacts with DOM patching.

1.11 Why this book is written the way it is (my bias as the author)

I built Relax by insisting on a few simple rules:

- **Small surface area**: if it's not essential, it stays internal.
- **Contracts before cleverness**: tests come first, performance work comes second.
- **Measure what users feel**: tail latency and dropped frames matter more than averages.

This book follows the same rules. You'll see me repeat the same pattern constantly: define the contract, show the smallest implementation, then harden it against edge cases.

1.12 Common pitfalls (what usually goes wrong)

When you build a UI runtime, these are the mistakes that create the nastiest bugs:

- Confusing **attributes** with **properties** (it works for `div`, fails for `input`).
- Forgetting to remove event listeners on destroy (leaks + ghost behavior).
- Replacing DOM nodes on update instead of reusing identity when structure matches.
- Using only averages to judge performance (p95 and drops are the real story).
- Letting scheduling semantics become implicit (tests get flaky, bugs become nondeterministic).

1.13 Mini-exercise

Pick one of the test files below and write a one-paragraph "contract" description for it:

- `src/__tests__/mount-dom.test.ts`
- `src/__tests__/patch-dom.test.ts`
- `src/__tests__/destroy-dom.test.ts`

Your goal isn't to understand every line. Your goal is to extract the invariants: what must always be true after mount/patch/destroy.

1.14 Online resources

These sources are helpful for orienting yourself before diving into implementation details:

- Virtual DOM as a concept (implementation details differ, but the mental model is useful): React docs: Render and Commit.
- DOM node basics (text nodes vs element nodes): MDN: Node.nodeType.
- Microtasks vs macrotasks (relevant for `nextTick()` / `queueMicrotask`): MDN: queueMicrotask.

1.15 Walkthrough: patching is "same element, new values"

Relax's VDOM runtime is built around a simple principle:

> If the old and new nodes are the "same kind" of node, we reuse the existing DOM element and update attributes/children. If they are different, we replace.

The test suite encodes this principle over and over. For example, `src/__tests__/patch-dom.test.ts` starts with extremely small cases:

- **no change**: patching identical trees should reuse `el`.

- **change the root node**: patching `div -> span` replaces the root element.
- **patch text**: `"foo" -> "bar"` updates only the Text node.

Those tests tell you what to look for when reading `src/patch-dom.ts`: element identity reuse is a correctness requirement, not an optimization.

1.16 Mini-exercise (with intent)

Before moving on, do this intentionally (don't just skim):

1. Read `src/app.ts` and answer: what runtime calls does `createApp()` delegate to?
2. Read `src/__tests__/app.test.ts` and find every `await nextTick()`. For each one, explain what DOM change it is waiting for.
3. Read the first 5 tests in `src/__tests__/patch-dom.test.ts`. For each, write a one-sentence rule (e.g., "patching text updates textContent without replacing parent elements").

1.17 Online resources (optional, but good context)

This book is grounded in the Relax.js repository, but it helps to know the broader ecosystem it draws from. These are references you can read alongside chapters; they are not prerequisites.

- Virtual DOM background (high-level): React — Render and Commit.
- The diffing strategy commonly used in VDOM children reconciliation: Neil Fraser, "An O(ND) Difference Algorithm and Its Variations".
- Fine-grained reactivity conceptual background (signals): SolidJS docs: Signals.
- Scheduling and keeping UIs responsive: MDN: requestAnimationFrame, MDN: MessageChannel.
- Testing DOM code in Node: Vitest Guide, jsdom.

Chapter 2

Repo tour & project constraints

2.1 What you'll build

By the end of this chapter, you will have:

- A system-level map of the Relax.js repository (what subsystems exist and why).
- A repeatable workflow to go from a question →tests →implementation.
- One complete mental trace of a user interaction from DOM event →component method →DOM patch.

2.2 What you'll learn

- How I split this repo into three cooperating products (VDOM runtime, HRBR runtime, compiler).
- Why jsdom constraints make the runtime *more* deterministic and teachable.
- The smallest set of files you should read to understand an end-to-end update.

2.3 Prereqs

Read Chapter 1. You should be comfortable with the idea that DOM nodes have identity and can be mutated.

2.4 Concept: read the system before you read the code

When you're learning a new codebase, the worst mistake is to start at `src/index.ts` and keep scrolling. You'll drown in details.

Instead, I want you to start with two maps:

- a **module map** (what subsystems exist)
- a **causal map** (what happens when a user clicks / types)

This chapter gives you both.

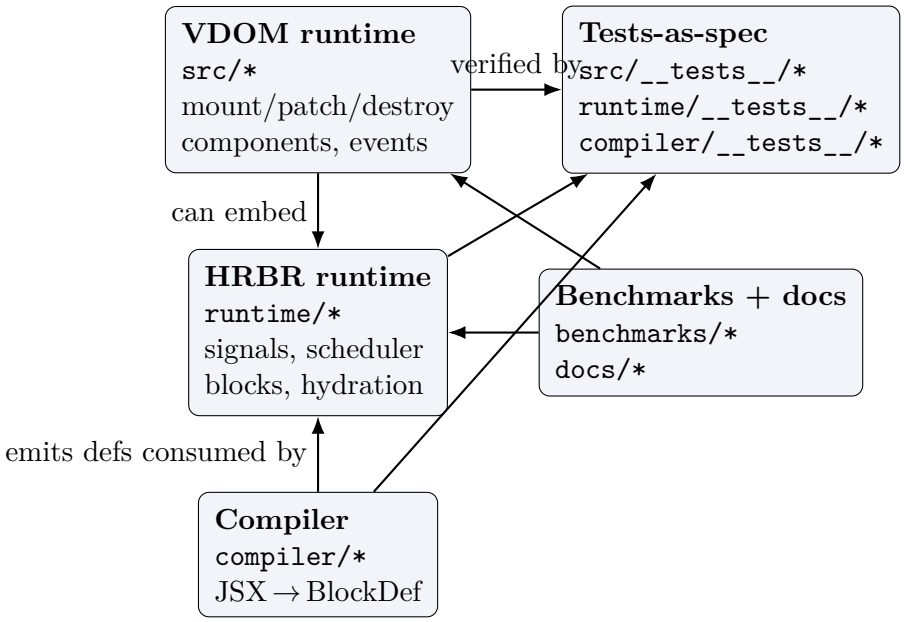
2.5 What you're looking at

This repository is intentionally small, but it isn't a single "runtime". It is a collection of cooperating subsystems, each with its own contract and its own tests.

At a high level, you should think of Relax.js as three products living in one repo:

1. **Relax VDOM runtime** (the teaching-friendly baseline) under `src/`.
2. **HRBR runtime** (experimental, compiled fast-path) under `runtime/`.
3. **Compiler** (experimental transform from TSX/JSX subset to HRBR blocks) under `compiler/`.

Diagram: repository architecture at a glance



The key idea: tests are the stable external contract, while the implementation is split into a flexible VDOM baseline and an optional compiled runtime.

Everything else exists to support one of these (docs, benchmarks, build tooling).

2.5.1 Directory map (first pass)

This repository is split into a few major areas:

- `src/`: the Relax.js Virtual DOM runtime.
- `runtime/`: an experimental runtime (HRBR) based on signals, scheduling, and compiled blocks.
- `compiler/`: a compiler/transform that converts a JSX subset into HRBR block definitions.
- `benchmarks/`: a browser harness to compare performance.
- `docs/`: design notes and whitepapers.

Two more folders matter when you're thinking about correctness:

- `src/__tests__/`: the VDOM runtime spec.
- `runtime/__tests__/` and `compiler/__tests__/`: the HRBR + compiler spec.

2.6 The constraint that shapes everything: DOM correctness in jsdom

Almost every implementation choice is constrained by one hard rule:

> The runtime must be correct in a headless DOM (jsdom), and tests are the contract.

That affects:

- Attribute vs property behavior (inputs/selects are tricky in jsdom).
- Event binding and cleanup.
- Diff algorithms (keyed moves must preserve node identity).

2.6.1 Why jsdom is a meaningful constraint

jsdom is *not* a full browser, and that is exactly why it is useful here. If your runtime depends on unspecified browser behavior or timing quirks, your test suite will become flaky.

Instead, tests in this repo focus on:

- DOM structure (`innerHTML`)
- identity (whether an element is reused or replaced)
- cleanup (event listeners and references are removed)

This shapes file boundaries: many helpers exist to make DOM updates deterministic and assertable.

2.7 How to read code in this repo (recommended path)

When you see a source file, immediately find its tests. That's the spec.

Recommended reading order:

1. `src/index.ts` (public exports)
2. `src/h.ts` (VNode model)
3. `src/mount-dom.ts` (mount)
4. `src/patch-dom.ts` (patch)
5. `src/destroy-dom.ts` (cleanup)
6. `src/component.ts` (components)

2.7.1 A second reading path: follow a user action

The fastest way to build intuitions is to follow a single user interaction end-to-end. For example: "typing in an input and clicking a button adds a todo".

You can trace that across:

- event normalization: `src/events.ts`
- component event binding: `src/dispatcher.ts` and `src/component.ts`
- scheduling: `src/scheduler.ts` (`nextTick()`)
- patching: `src/patch-dom.ts`

Doing this with the tests open beside you is the theme of this book.

2.8 Build-along: follow one interaction end-to-end

Now let's do the thing I personally do when I need to understand or debug a UI runtime: I pick one user interaction and trace it until I can explain every boundary.

We'll use the Todo app test as our "story". The specification is `src/__tests__/app.test.ts`.

2.8.1 The spec (test) as the narrative

In that test file, the interesting lines look like this:

```
input.value = 'Buy␣milk'
input.dispatchEvent(new Event('input'))

button.click()

await nextTick()
expect(document.body.innerHTML).toBe(...)
```

Why this matters: this is an executable description of the user story. The only reason we can trust the runtime is because this story is repeatable.

In the rest of this chapter, I'm going to connect this story to the implementation.

2.8.2 Step 1: the root lifecycle boundary (createApp)

The first system boundary is the application lifecycle. In Relax, `createApp()` is intentionally tiny. Its entire job is to create the root VNode and unmount it.

Implementation: `src/app.ts`.

```
export function createApp(RootComponent, props = {}) {
  let vdom = null

  return {
    mount(parentEl) {
      vdom = h(RootComponent, props)
      mountDOM(vdom, parentEl)
    },
    unmount() {
```

```
      destroyDOM(vdom)
      vdom = null
    },
  }
}
```

extbfWhy this design: I want exactly one owner for the root VNode. If you don't have a clear ownership boundary, "unmount" becomes best-effort cleanup and bugs get subtle.

2.8.3 Step 2: DOM event binding (how a handler gets installed)

When a VNode contains an `on` map (events), Relax wires listeners during mount. The low-level helper lives in `src/events.ts`:

```
export function addEventListener(eventName, handler, el, hostComponent = null) {
  const listener = (event) => {
    if (hostComponent) handler.call(hostComponent, event)
    else handler(event)
  }

  el.addEventListener(eventName, listener)
  return listener
}
```

extbfWhy this matters: the listener wrapper is a tiny but crucial semantic guarantee. It means component methods can rely on `this` being the component instance. (We'll return to this in the events chapter.)

2.8.4 Step 3: component-to-component communication (Dispatcher)

In Relax, components don't reach into each other directly. They communicate through a tiny command dispatcher: `src/dispatcher.ts`.

```
dispatch(commandName, payload) {
  if (this.#subs.has(commandName)) {
    this.#subs.get(commandName).forEach((handler) => handler(payload))
  }

  this.#afterHandlers.forEach((handler) => handler())
}
```

extbfWhy this matters: this gives me a clean abstraction boundary: event handlers emit events, components react, and then we run after-hooks (often "schedule a patch").

2.8.5 Step 4: where does the DOM update happen?

This is the final thing you must always locate:

> Where is the code that turns a "new desired UI" into DOM mutations?

In the VDOM runtime, that answer is `src/patch-dom.ts` (covered in depth later). In this chapter, the important idea is simply: *there is exactly one patching engine*, and everything funnels into it.

2.9 A concrete example: createApp() is tiny on purpose

texttttcreateApp() exists so the user has a simple lifecycle API. The full file is short: `src/app.ts`.

Key behavior:

- It creates a root VNode with `h(RootComponent, props)`.
- It mounts it once.
- It can unmount and fully cleanup.

2.9.1 Code reference

See `src/app.ts`. The central calls are `mountDOM(...)` and `destroyDOM(...)`.

2.9.2 Test reference (what proves the contract)

The high-level application lifecycle is specified by `src/__tests__/app.test.ts`. This test file shows a full mount-to-interaction flow:

- It mounts the app.
- It waits for async work with `await nextTick()`.
- It simulates user input by setting `input.value` and dispatching an `input` event.
- It simulates clicks via `button.click()`.
- It asserts DOM structure with `singleHtmlLine`...`.

That single test is a great "reading harness" for learning how Relax wires components + events + patching.

2.10 Tests as specs (my rule for this repo)

If you only remember one workflow from this chapter, make it this:

1. Start from a test that matches the behavior you care about.
2. Read the assertions and treat them as the contract.
3. Jump to implementation and locate the code that makes those assertions true.
4. If you're changing behavior, change the test first.

Concrete anchors for this chapter:

- App lifecycle story: `src/__tests__/app.test.ts`
- Event semantics: `src/__tests__/events.test.ts`
- Dispatcher semantics: `src/__tests__/dispatcher.test.ts`

2.11 Common pitfalls

- Reading implementation first and assuming the behavior is intentional.
- Ignoring jsdom differences (inputs/selection are special).
- Treating "unmount" as optional cleanup.
- Not understanding what is synchronous vs deferred to `nextTick()`.

2.12 Mini-exercise

Open `src/__tests__/app.test.ts` and answer:

- Why does the test call `await nextTick()` after mount?
- Which user actions are simulated (exact DOM APIs)?
- Which pieces of the runtime must exist for that simulation to work (mounting, events, patching)?

2.13 Project constraint: small API surface

Relax exports only a few primitives (h, fragments, components, scheduler). Everything else is internal. That keeps the library small and also makes the tests more important.

2.13.1 Where to see the supported surface area

The public exports are in `src/index.ts` and mirrored by the `exports` map in `package.json`.

In other words:

- if it's not in `src/index.ts`, it's not a stable public API
- tests are free to depend on internals, but users shouldn't

This book leans on that design: we can explain the internals without promising them as user-facing contracts.

2.14 Tests: how this repo measures correctness

This repository uses Vitest with a jsdom environment. The configuration lives in `vitest.config.js`:

- test patterns: `**/__tests__/**/*.test.ts(x)`
- reporters: verbose (good for learning)
- environment: jsdom

2.14.1 The single most important helper: singleHtmlLine

Many tests assert against HTML strings. If you try to compare raw multi-line template literals to `innerHTML`, whitespace will fight you.

This repo solves that in `src/__tests__/utils.ts` with `singleHtmlLine()`:

- it collapses whitespace
- it removes spaces between tags (for example, `>\s+<` becomes `><`)
- it trims

This is a micro-lesson in test design: you want assertions that are strict about structure, but forgiving about irrelevant formatting.

2.14.2 Where to learn semantics fastest (test tour)

If you're new to the codebase, these tests teach the contracts in the right order:

- `src/__tests__/mount-dom.test.ts`: mount behavior, fragment behavior, saving `vdom.el` references.
- `src/__tests__/patch-dom.test.ts`: identity reuse vs replacement, attribute updates, keyed children, and edge cases.
- `src/__tests__/destroy-dom.test.ts`: cleanup and idempotency, especially "remove event listeners".
- `src/__tests__/component.test.ts`: component lifecycle, props/state updates, and event binding to the host component.
- `src/__tests__/jsx-react-syntax.test.tsx`: TSX compatibility contract (className/onClick-/fragments/components).

Each of these will be revisited later in the book when we dive into the implementation files they protect.

2.15 Mini-exercise

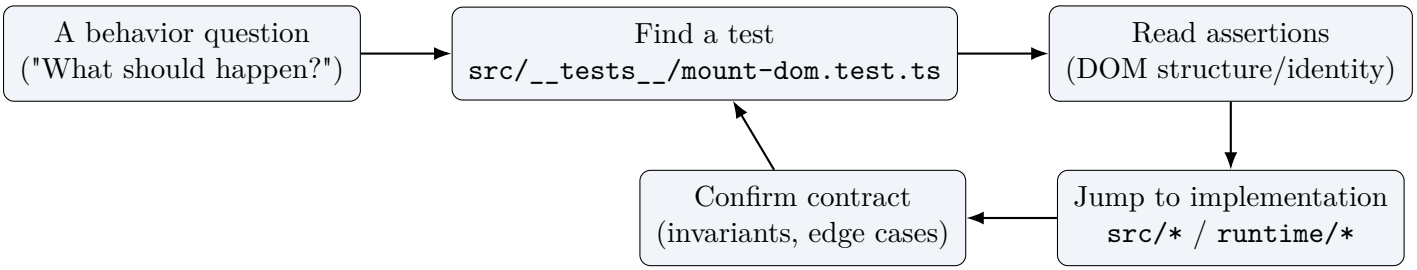
Open these tests and write down what they prove (one sentence each). Then locate the implementation file they exercise.

- `src/__tests__/app.test.ts`
- `src/__tests__/component.test.ts`
- `src/__tests__/mount-dom.test.ts`
- `src/__tests__/destroy-dom.test.ts`

If you can answer these, you're ready for Part II.

2.16 Diagram: a "follow the tests" reading workflow

When you don't know where to start, start from the tests and work backward to the implementation.



This loop is also how you safely add features: you extend the spec (tests) and then evolve the implementation.

2.17 Online resources

This chapter is about reading *this repo*, but a few external references help explain why these constraints and structures are common.

- DOM living standard (event/bubbling/capturing model): WHATWG DOM Standard.
- Event handling basics and differences between properties vs attributes: MDN: addEventListener, MDN: HTMLElement.
- Testing DOM in Node: Vitest Guide, jsdom.
- Module packaging and exports maps: Node.js docs: `exports`.

For formal citations, you can map key links into the backmatter `References` list in `book/main.tex`.

Chapter 3

TSX/JSX: an authoring layer on top of `h()`

3.1 The goal

Relax.js supports writing UI in TSX/JSX with a React-like syntax. But I want to demystify it immediately: JSX is only *syntax*. It becomes function calls (or, in TypeScript's modern mode, calls to a small "JSX runtime" module).

In Relax, the important function is still `h()` from `src/h.ts`. The TSX layer exists to make authoring more ergonomic and familiar.

This chapter answers four practical questions:

1. What does TypeScript emit for TSX in this repo?
2. Where is the TSX runtime implemented (and what does it do)?
3. Which React-like conveniences are supported (and which are not)?
4. How do tests lock the TSX contract in place?

3.2 How TSX works in this repository (TypeScript configuration)

The TSX behavior in this repo is defined in `tsconfig.json`:

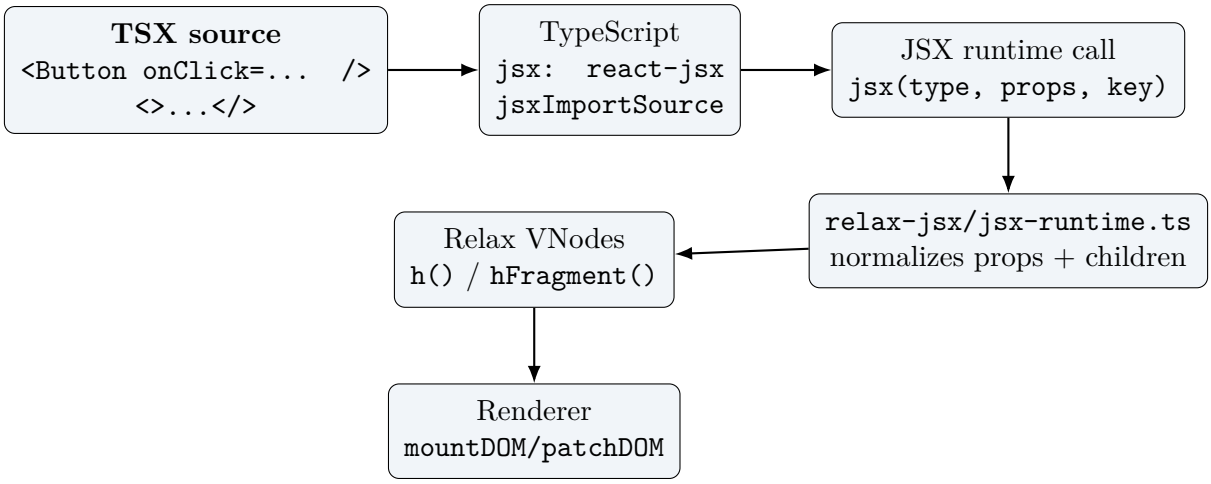
- `"jsx": "react-jsx"`
- `"jsxImportSource": "relax-jsx"`

This combination means:

- You *do not* configure a global JSX factory function.
- TypeScript emits imports from `relax-jsx/jsx-runtime` (and sometimes `relax-jsx/jsx-dev-runtime`).

So the contract becomes very clean: "TSX syntax must compile into objects that Relax can mount/patch/destroy."

Diagram: TSX syntax to VNodes (what TypeScript actually emits)



When TSX "works" in this repo, it means the right-hand side of this diagram produces the same VNode shapes as writing `h()` manually.

3.3 Two authoring styles you'll see

3.3.1 Direct VDOM authoring (manual `h()`)

You can write VNodes manually:

```
import { h, hFragment } from 'relax.js'

const view = hFragment([
  h('h1', {}, ['Hello']),
  h('button', { on: { click: () => console.log('clicked') } }, ['Click'])
])
```

3.3.2 React-like TSX authoring (compiled via a JSX runtime)

TypeScript can compile TSX into calls to a configured JSX runtime. This repo uses a React-like authoring style where:

- `className` maps to Relax's `class` prop.
- `onClick` maps to Relax's event prop form (internally becomes `on: { click: fn }`).
- Fragments (`<>...</>`) are supported.

There's an important nuance here:

> Relax supports both "native" Relax props (like `class` and `on: { click: ...}`) *and* React-like compatibility props (like `className` and `onClick`).

This design keeps the underlying runtime simple (it only needs to understand one internal prop shape) while letting TSX authoring feel familiar.

3.4 The Relax JSX runtime (the real implementation)

The TSX runtime lives in `relax-jsx/jsx-runtime.ts`. It's intentionally tiny.

At a high level it does four jobs:

1. Normalizes `children` to an array.
2. Applies React-like prop compatibility:
   - `className -> class`
   - `onClick/onInput/... → on: { click/input/...}`
3. Threads the optional `key` through into Relax props.
4. Delegates to Relax's real VDOM constructors: exttth(type, props, children) or `hFragment(children)`.

3.4.1 Fragments: why `Fragment` is exported

In the modern TS transform, fragments compile to a special "Fragment" value. Relax's JSX runtime exports:

- `export const Fragment = hFragment as any`

Then `jsxImpl()` checks whether the element type is `Fragment` and returns `hFragment(children)`. That makes fragment VNodes indistinguishable from ones created explicitly via `hFragment()`.

3.4.2 Dev runtime: jsxDEV

Tooling sometimes emits `jsxDEV` calls in development builds. This repo includes `relax-jsx/jsx-dev-runtime.ts`, which simply re-exports `Fragment`, `jsx`, and `jsxs`, and provides `jsxDEV(...)` as an alias to `jsx(...)`.

This is not a performance hack; it's a compatibility bridge so Vite/Vitest can run TSX code in different modes.

3.5 Tests as spec (the subset is intentional)

The definitive spec for supported TSX/JSX features lives in:

- `src/__tests__/jsx-tsx.test.ts`
- `src/__tests__/jsx-react-syntax.test.tsx`

3.5.1 Test 1: TSX can mount via the Relax JSX runtime

`src/__tests__/jsx-tsx.test.ts` is the smallest end-to-end proof:

- It imports and runs `examples/jsx/hello.tsx`.
- That example defines a component whose `render()` returns TSX.
- It mounts using `createApp()`, waits `nextTick()`, and returns `innerHTML`.

If this test fails, it usually means one of these broke:

- TS didn't resolve `relax-jsx/jsx-runtime` the way we expect (config/alias problem).
- The TSX runtime produced VNodes in a shape `mountDOM()` doesn't handle.

3.5.2 Test 2: React-like conveniences are supported

extttsrc/___tests___/jsx-react-syntax.test.tsx is a feature matrix. It's intentionally written as "contracts" in a comment at the top of the file.

It specifies (and proves) things like:

- intrinsic elements accept `id`, `className`, and text children
- fragments (`<>...</>`) work
- `onClick` attaches a DOM click handler
- list rendering via `items.map(...)` works (including `key`)
- patching TSX trees updates DOM without reconstructing the whole host
- TSX works for Relax components created via `defineComponent()`

This test file is a great example of how to design compatibility: pick the exact subset you want, and encode it as executable assertions.

3.6 What's not the same as React

Even though the syntax is familiar, semantics are Relax's:

- Components are created via `defineComponent()`.
- The event binding model binds handlers to the host component instance.
- Slots in Relax are a component feature (external/default content), not React children.

3.6.1 Practical limitations of the TSX subset

This repo intentionally does *not* try to be a full React clone. Two examples:

- The TSX runtime only rewrites "event props" when the value is a function. If you pass non-function values to `onClick`, you should expect it to behave as a normal prop.
- "React children" as a conceptual model is not the point. In Relax, children are just VNodes (or arrays of VNodes) passed to `h()`.

The tests are the source of truth for what is supported. If you'd like to extend the supported subset, the safest workflow is:

1. Add a failing test first in `src/__tests__/jsx-react-syntax.test.tsx`.
2. Update `relax-jsx/jsx-runtime.ts` to normalize the new case.
3. Ensure mount/patch/destroy tests still pass.

3.7 Mini-exercise

Read `src/__tests__/jsx-react-syntax.test.tsx` and answer:

1. How are fragments compiled?
2. How are events passed?
3. How do components appear in JSX and how do props flow?

Then open `relax-jsx/jsx-runtime.ts` and locate the exact code that implements each answer.

3.8 Online resources (optional, citation-ready)

- TypeScript handbook: JSX (explains `react-jsx` and `jsxImportSource`): TypeScript: JSX.
- React JSX runtime reference (useful for understanding why `jsx`/`jsxs`/`jsxDEV` exist): React docs: jsx-runtime.
- Vite + Vitest module resolution (useful when JSX runtime imports fail): Vite: JSX, Vitest Guide.
- DOM events (what an `onClick` becomes): MDN: click event.

Chapter 4

The VNode model (what the renderer *really* consumes)

In Chapter 3 we treated TSX as "just syntax" that boils down to calling `h()` and `hFragment()`. This chapter is where I make that concrete.

If you strip Relax down to its bones, the renderer is a pair of functions:

- `mountDOM(vdom, host)`: allocate DOM for a VNode tree.
- `patchDOM(oldVdom, newVdom, host)`: reconcile two VNode trees (reuse DOM).

Both functions only work if their input has a stable, predictable data shape. That data shape is the **VNode model** in `src/h.ts`.

I like to describe `src/h.ts` as "the type-level constitution of the renderer". If you understand the invariants enforced here, every later chapter gets easier.

4.1 What you will build in your head

By the end of this chapter you should be able to do these three things confidently:

1. Read a random VNode object in a debugger and know what it means.
2. Predict what `h('div', {}, ['x'])` returns (including text node normalization).
3. Explain why `areNodesEqual()` is written the way it is and how it drives patching.

4.2 The one invariant I defend everywhere

Relax keeps the VNode model intentionally small, and then makes the runtime depend on one invariant:

> **Invariant:** after creation, every value inside `VNode.children` is already a VNode. No raw strings, no raw numbers, no `null`, no `undefined`.

This is not a "nice-to-have". It's the thing that keeps `mountDOM()` and `patchDOM()` from degenerating into dozens of slow type checks.

4.3 Why we need VNodes in the first place

The DOM is mutable state. The renderer's job is to move a lot of mutable state toward a desired UI with minimal work.

So we need a representation of "desired UI" that is:

- **Pure data:** can be created without touching the DOM.
- **Stable:** has explicit node kinds and predictable fields.
- **Diffable:** can be compared against a previous version.

In Relax, that's the VNode.

4.4 Why VNodes exist

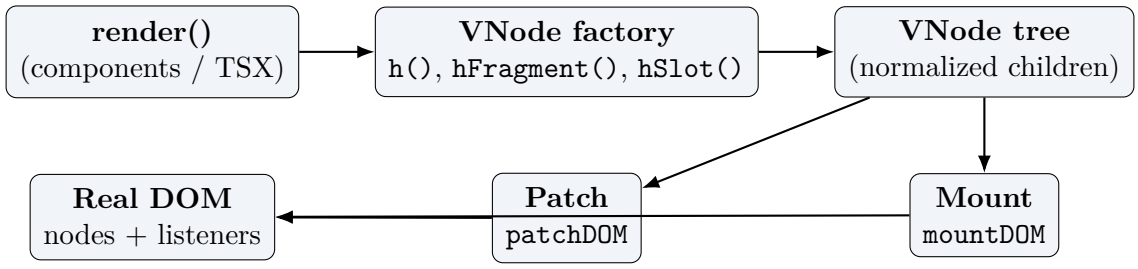
A virtual DOM implementation needs a stable, serializable representation of UI. In Relax, that representation is the **VNode**. The renderer can then be split into three big phases:

1. **Create**: build a VNode tree (usually by calling `h()` directly, or indirectly via TSX as described in Chapter 3).
2. **Mount**: allocate real DOM nodes for that tree (`src/mount-dom.ts`).
3. **Patch**: reconcile an "old tree" with a "new tree" (`src/patch-dom.ts`), reusing as much DOM as possible.

The "create" step is deceptively important: the more regular the VNode tree is, the less branching the mount/patch code needs.

Diagram: from render to real DOM

The following diagram is the "wiring diagram" for the rest of Part II. Every later chapter is basically "zooming in" on one box:



If you keep this picture in mind, a lot of micro-details (like `vdom.el` back-references or `areNodesEqual`) stop feeling ad-hoc: they're glue that keeps the arrows valid.

4.5 The VNode types: DOM_TYPES

Relax defines node kinds as string constants in `DOM_TYPES` (`src/h.ts`):

- `TEXT`: a text node (`TextVNode`).
- `ELEMENT`: an HTML/SVG element (`ElementVNode`).
- `FRAGMENT`: a list of siblings without an extra wrapper element (`FragmentVNode`).
- `COMPONENT`: a component constructor + props/children (`ComponentVNode`).
- `SLOT`: a placeholder for "external children" in component trees (`SlotVNode`).
- `HRBR`: a special vnode wrapping an HRBR "mount factory" (`HrbrVNode`).

Two details are worth calling out because they influence the rest of the runtime:

1. **Not every VNode has a DOM element.** `TEXT` uses a real `Text` node (`el?: Text`). `ELEMENT` uses a real `HTMLElement` (`el?: HTMLElement`). But `FRAGMENT` is special: it represents multiple siblings, so it can't point at a single "root node".
2. **Keys are part of identity.** You'll see `props.key` used later by the reconciliation algorithm to decide whether a node is "the same logical thing".

4.6 Contract: h(tag, props, children)

Relax's primary constructor is `h()` (`src/h.ts`). It's intentionally close to "hyperscript" functions found in other VDOM systems.

When I'm writing a renderer, I want the constructors to be boring. "Boring" here means: strongly asserted inputs, aggressively normalized children, minimal implicit behavior. Relax follows that.

Code: the actual h() contract in src/h.ts

The implementation is short enough that you should read it end-to-end at least once:

```
export function h(tag: string | unknown, props: Record<string, unknown> = {}, children:
    unknown[] = []): ElementVNode | ComponentVNode {
  const type = typeof tag === 'string' ? DOM_TYPES.ELEMENT : DOM_TYPES.COMPONENT

  assert(
    typeof props === 'object' && !Array.isArray(props),
    '[vdom]␣h()␣expects␣an␣object␣as␣props␣(2nd␣argument)'
  )
  assert(
    Array.isArray(children),
    `[vdom] h() expects an array of children (3rd argument), but got '${typeof␣children}'`

  )

  return {
    tag: tag as any,
```

```
    props: props as ElementVNodeProps,
    type,
    children: mapTextNodes(withoutNulls(children as any[])) as VNode[],
  } as any
}
```

Two things to notice:

- **Fast fail:** `assert(...)` catches incorrect usage immediately.
- **Normalization happens right here:** `withoutNulls(...)` and `mapTextNodes(...)` enforce the children invariant.

Inputs

- `tag`: either a string (intrinsic element like `div`) or a component constructor/function.
- `props`: *must be a plain object.* Relax checks this using `assert(...)` and fails fast with a helpful message.
- `children`: *must be an array.* This is also asserted.

Output

- If `typeof tag === 'string'`, `h()` produces an `ELEMENT` vnode.
- Otherwise it produces a `COMPONENT` vnode.

Even though Relax supports TSX, TSX still ends up calling this layer (via the `relax-jsx` runtime), so understanding `h()` is foundational.

4.7 Child normalization: `withoutNulls()` + `mapTextNodes()`

Relax does two child normalization passes inside `h()` and `hFragment()`:

1. **Drop nullish children**: `withoutNulls(children)` removes `null` and `undefined` (see `src/utils/arrays.ts`). This means "conditional children" can be written as `show ? node : null`.
2. **Map primitives to `TEXT` nodes**: `mapTextNodes()` converts `string | number | boolean | bigint | symbol` into `TextVNode` via `hString()`.

Why normalize eagerly?

Normalization at creation time is a performance and correctness tactic:

- **Performance:** mount/patch are on the hot path; you want them to pay as little "figure out what this is" cost as possible.
- **Correctness:** by forcing children into a single representation (VNodes), you avoid accidental differences between "first render" and "update" paths.

- **Debuggability:** tests can assert on VNode shapes without having to include JS coercion rules.

You can see this philosophy reflected in the tests: instead of testing dozens of mount/patch branches for primitives, the test suite locks down the normalization helpers and then relies on the invariant.

This is one of those design choices that pays rent everywhere:

- Mount code can check `vNode.type` and never needs to special-case "child is a string".
- Patch code can treat text nodes uniformly; it doesn't have to worry about coercion rules.

4.8 Fragments: `hFragment(children)` and `extractChildren()`

Fragments represent "multiple siblings" and are heavily used by TSX fragment syntax (`<>...</>`).

Constructing fragments

extttthFragment(vNodes) asserts the input is an array, filters nullish children, and maps primitives to text nodes (exactly like `h()`).

Code: fragment constructor

This is the whole function (again: boring on purpose):

```
export function hFragment(vNodes: unknown[]): FragmentVNode {
  assert(Array.isArray(vNodes), '[vdom]␣hFragment()␣expects␣an␣array␣of␣vNodes')

  return {
    type: DOM_TYPES.FRAGMENT,
    children: mapTextNodes(withoutNulls(vNodes as any[])) as VNode[],
  }
}
```

Flattening fragments

Later, other parts of the runtime sometimes need to treat "a tree that may contain fragments" as a flat list (for example, when calculating insertion positions). extttextractChildren(vdom) provides that flattening:

- If it finds a fragment child, it recursively collects that fragment's children.
- Otherwise it includes the child as-is.

The unit test `extract children from a tree with fragments` in `src/__tests__/h.test.ts` is your executable spec for this behavior.

Code: the spec is literally an object equality

This is a good example of what I mean by "tests as documentation":

```
test('extract␣children␣from␣a␣tree␣with␣fragments', () => {
  const vNode = h('div', {}, [
    'A',
    hFragment([
      hFragment([hString('B')]),
      hString('C'),
      hFragment([hString('D')]),
    ]),
    'E',
  ])
  const children = extractChildren(vNode)

  expect(children).toEqual([
    { type: DOM_TYPES.TEXT, value: 'A' },
    { type: DOM_TYPES.TEXT, value: 'B' },
    { type: DOM_TYPES.TEXT, value: 'C' },
    { type: DOM_TYPES.TEXT, value: 'D' },
    { type: DOM_TYPES.TEXT, value: 'E' },
  ])
})
```

Notice what this test gives us: a guarantee that fragments behave like "transparent containers" for the purposes of child lists.

4.9 Slots: `hSlot(children)` as a component-level placeholder

Slots are how Relax expresses "children passed into a component" without forcing the component to take opaque `children` and do manual array plumbing.

At the VNode level, a slot is just:

- `type: DOM_TYPES.SLOT`
- optional `children`: default content to use when no external children are provided.

Then, to connect slot VNodes to component semantics, `src/h.ts` keeps a small piece of module-scoped state: `hSlotCalled`.

Why the module flag exists

When rendering a component, Relax needs to know whether the component output contained at least one slot. Rather than scanning the render output tree after the fact, Relax uses this fast signal:

1. `hSlot()` sets `hSlotCalled = true`.
2. Component rendering checks `didCreateSlot()`.
3. If a slot was created, the runtime calls the slot-filling algorithm (`src/slots.ts`) and then `resetDidCreateSlot()` to avoid leaking the flag into unrelated renders.

This is one of Relax's "tiny global state" trade-offs: it keeps render fast and simple, but it also means slot creation is effectively a side-effect.

What the tests prove

The best way to internalize slot semantics is to read `src/__tests__/component-slots.test.ts`. A few key cases:

- **External content wins:** A component that renders `hSlot()` will receive the parent's children in that position.
- **Default content:** `hSlot([ ...default ])` is used when the parent provides no children.
- **Empty slot:** if there's no external content and no default content, the slot becomes empty (the test asserts `<div></div>`).
- **Conditional slots:** slot nodes can appear/disappear across re-renders, and patching handles it.

4.10 HRBR interop: `hBlock(mount)` and VDOM identity

HRBR (the compiled "block" runtime) needs to integrate into the same mounting/patching pipeline as regular VNodes. Relax does that by wrapping an HRBR mount factory in an `HRBR` vnode:

- `hBlock(mount)` returns `{ type: DOM_TYPES.HRBR, mount }`.
- Mounting creates a host element region and calls `mount(host)`.
- The returned instance exposes at least `destroy()`, and may expose `update()` / `dispose()`.

There's an important identity rule here, and the repo encodes it explicitly in `areNodesEqual()` (`src/nodes-equal.ts`):

> extbfHRBR vnode identity is defined by mount factory identity: `old.mount === new.mount`.

That single line is the difference between:

- **Patch preserves instance:** stable mount function keeps the same HRBR instance.
- **Patch remounts:** new mount function destroys and mounts a fresh instance.

What the HRBR tests prove

Read `src/__tests__/hrbr-integration.test.ts` as the contract:

- **Mount and destroy:** an HRBR block returned from a VDOM component mounts into the DOM and gets destroyed on `destroyDOM`.
- **Event semantics:** HRBR event slots bind handler `this` to the host Relax component instance (the test asserts `calls[0].thisArg === vdom.component`).
- **Patch behavior:** stable vs. changing mount factory identity changes whether the HRBR instance is preserved.

This is a great example of the book's theme: treat tests as executable documentation.

4.11 VNode identity and diffing: a preview

This chapter is about creation, but VNode shape is designed with patching in mind. The function `areNodesEqual()` (`src/nodes-equal.ts`) shows you the identity rules Relax uses later:

Code: `areNodesEqual()` is tiny, but it's a design statement

Read this function carefully. Everything in patching depends on this decision about identity:

```
export function areNodesEqual(oldNode: any, newNode: any) {
  if (oldNode.type !== newNode.type) {
    return false
  }

  if (oldNode.type === DOM_TYPES.HRBR) {
    return oldNode.mount === newNode.mount
  }

  if (oldNode.type === DOM_TYPES.TEXT) {
    return true
  }

  if (oldNode.type === DOM_TYPES.ELEMENT) {
    return oldNode.tag === newNode.tag && oldNode.props?.key === newNode.props?.key
  }

  if (oldNode.type === DOM_TYPES.COMPONENT) {
    return oldNode.tag === newNode.tag && oldNode.props?.key === newNode.props?.key
  }

  return true
}
```

From a systems perspective, this is the policy wall between "update in place" and "destroy + remount". If `areNodesEqual(old, next)` returns `false`, patching treats it as a different logical node and re-creates DOM / component instances.

Tests as spec: identity rules are enumerated

The tests in `src/__tests__/node-equals.test.ts` are effectively a checklist:

- Text nodes: always equal (value patches, identity doesn't change).
- Fragments: always equal.
- Elements: equal if tag matches, and key matches when present.
- Components: equal if constructor matches, and key matches when present.

This is why I keep the function small: it's easier to reason about and easier to lock down with tests.

- text nodes: always equal (Relax patches text by updating value rather than replacing node type)

- fragments: always equal
- elements: equal if tag matches and `props.key` matches
- components: equal if constructor matches and `props.key` matches
- HRBR: equal if mount factory function is the same

The tests in `src/__tests__/node-equals.test.ts` enumerate these cases in small, sharp assertions.

4.12 Mini-exercises

1. **Read like a maintainer:** In `src/__tests__/h.test.ts`, locate the test named `"h() maps 'number', 'boolean', 'bigint', and 'symbol' values to text vNodes"`. Explain (to yourself) why `Symbol(5)` becomes the string `"Symbol(5)"`.
2. **Write a new invariant test:** Add a small test that asserts `h('div', , [null, undefined, 'x']).children.length === 1`. (This should already be true, but the test helps lock the behavior down.)
3. **Identity thought experiment:** Why do you think Relax treats text nodes as "always equal"? What does that allow `patchDOM()` to do as an optimization?

4.13 Online resources

This chapter describes patterns that show up across many UI runtimes. These references help if you want to compare Relax's choices to other ecosystems:

- Hyperscript and virtual node factories (conceptual background): https://github.com/hyperhype/hyperscript
- DOM node types (Text vs Element) on MDN: https://developer.mozilla.org/en-US/docs/Web/API/Node/node
- React's reconciliation keys (why `key` exists): https://react.dev/learn/rendering-lists
- Web Components slots (a parallel mental model for "default vs external" content): https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_templates_and_slots

Chapter 5

Mounting DOM (`src/mount-dom.ts`)

This chapter is about the first half of the renderer: taking the data structures from `src/h.ts` and producing real DOM nodes.

I'm going to teach mounting the way I teach any system that mutates state:

1. Define the **contract** (inputs/outputs/failure modes).
2. Define the **invariants** we keep true after each step.
3. Walk the code path *by node type*, because real renderers are dispatchers.

In Relax there's a clean separation:

- **Creation** builds VNode trees (Chapter 4).
- **Mounting** turns VNodes into DOM and stores back-references (this chapter).
- **Patching** reconciles trees (Chapter 8).
- **Destroying** cleans up listeners / component instances / HRBR blocks (Chapter 9).

The entry point is `mountDOM(vdom, parentEl, index?, hostComponent?)` in `src/mount-dom.ts`.

5.1 Contract: `mountDOM`

Inputs

- `vdom`: a VNode (one of the `DOM_TYPES`).
- `parentEl`: the DOM element where the node should be inserted. This must be non-null.
- `index` (optional): insert position inside `parentEl.childNodes`.
- `hostComponent` (optional): the component instance that should become `this` for event handlers.

Outputs / side-effects

- Inserts DOM into `parentEl`.
- Mutates the vnode: for most node types, `vdom.el` becomes a back-reference to the created DOM.
- For elements, saves listener cleanup hooks on `vdom.listeners`.
- For components, creates and stores `vdom.component`.

Failure modes

- If `parentEl == null`, it throws `[mountDOM] Parent element is null`.
- If `vdom.type` doesn't match any known tag in `DOM_TYPES`, it throws `Can't mount DOM of type: ...`.

The tests in `src/__tests__/mount-dom.test.ts` include a couple of "guardrail" cases that assert these errors exist.

Two subtle but crucial invariants

When mounting is correct, these invariants hold and the rest of the runtime can be simpler:

1. **Back-references exist:** every mounted vnode that conceptually "owns" a DOM node sets `vdom.el`.
2. **Cleanup data exists:** element vnodes store their concrete listener wrapper functions in `vdom.listeners`.

Those two fields are what make later patching and destroying possible without extra DOM queries.

5.2 The central pattern: dispatch by `vdom.type`

extttmountDOM() is a type-dispatcher. In code it's a `switch (vdom.type)` that delegates to one helper per vnode kind.

Code: the actual dispatch loop

This is the heart of `src/mount-dom.ts`:

```
export function mountDOM(vdom, parentEl, index, hostComponent = null) {
  if (parentEl == null) {
    throw new Error('[mountDOM] Parent element is null')
  }

  switch (vdom.type) {
    case DOM_TYPES.TEXT:
      createTextNode(vdom, parentEl, index)
      break
    case DOM_TYPES.ELEMENT:
```

```
      createElementNode(vdom, parentEl, index, hostComponent)
      break
    case DOM_TYPES.FRAGMENT:
      createFragmentNodes(vdom, parentEl, index, hostComponent)
      break
    case DOM_TYPES.COMPONENT:
      createComponentNode(vdom, parentEl, index, hostComponent)
      enqueueJob(() => vdom.component.onMounted())
      break
    case DOM_TYPES.HRBR:
      createHrbrNode(vdom, parentEl, index)
      break
    default:
      throw new Error(`Can't␣mount␣DOM␣of␣type:␣${vdom.type}`)
␣␣}
}
```

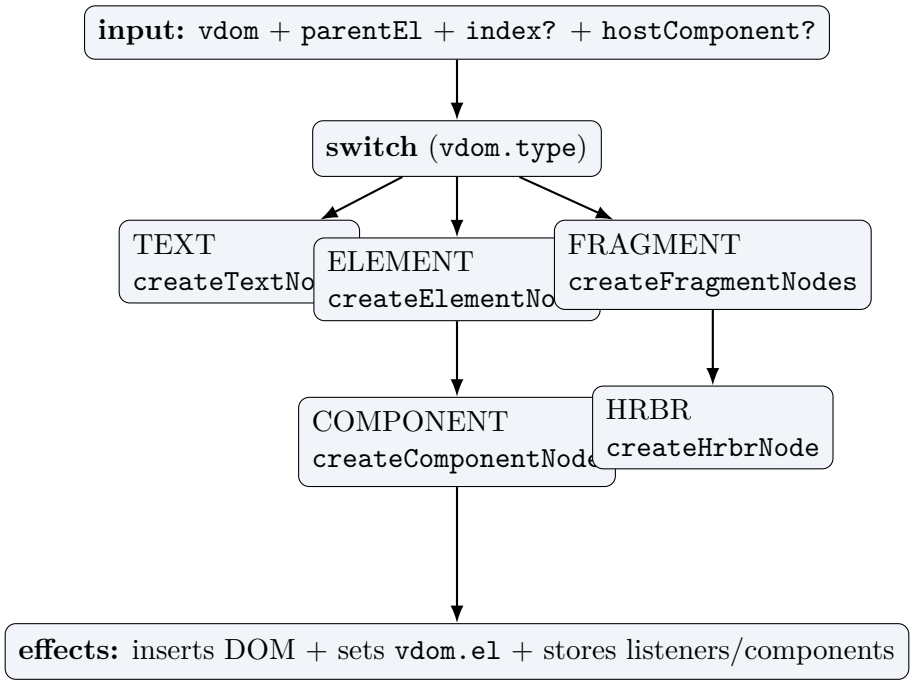
There are two design choices worth noticing right away:

- Components schedule `onMounted()` via the job queue instead of running it inline.
- HRBR mounting is isolated behind a host container element so it can be treated as a unit.

- `createTextNode` for `DOM_TYPES.TEXT`
- `createElementNode` for `DOM_TYPES.ELEMENT`
- `createFragmentNodes` for `DOM_TYPES.FRAGMENT`
- `createComponentNode` for `DOM_TYPES.COMPONENT`
- `createHrbrNode` for `DOM_TYPES.HRBR`

Diagram: mount dispatcher

This is the mental model for the file `src/mount-dom.ts`:



This is not just an organizational trick: it is what makes patching symmetric later. Chapter 8 will mirror this shape with a patch dispatcher.

5.3 Text nodes: `createTextNode`

For `TextVNode`, mounting is:

1. Create a real text node: `document.createTextNode(vdom.value)`.
2. Store a stable back-pointer: `vdom.el = textNode`.
3. Insert it into the parent using `insert(textNode, parentEl, index)`.

What the tests prove

texttsrc/___tests___/mount-dom.test.ts contains two small tests that act like a contract:

- `mount a text element in a host element`: asserts the DOM contains `hello`.
- `save the created text element in the vdom`: asserts `vdom.el` is a `Text` instance and points at the right content.

Read those tests as: "If we ever refactor mounting, these properties must remain true or patching/destroying will become unreliable."

5.4 Element nodes: `createElementNode` and the props/events split

For `ElementVNode`, Relax mounts in a deliberate order:

1. Create `const element = document.createElement(tag)`.

2. Apply props and events via `addProps(element, vdom, hostComponent)`.
3. Store `vdom.el = element`.
4. Mount children into `element` (recursively via `mountDOM(child, element, null, hostComponent)`).
5. Insert the final element into the parent.

The most important concept here is that Relax treats events specially.

5.4.1 `addProps`: extracting attrs vs. events

texttttaddProps() calls `extractPropsAndEvents(vdom)` (from `src/utils/props.ts`). The return value is split into:

- **attrs**: everything that becomes DOM attributes/properties (`id`, `class`, `style`, `data-*`, etc).
- **events**: the event map stored under `props.on`.

Then mounting wires two subsystems:

- `addEventListeners(events, el, hostComponent)` returns a map of cleanup functions; this is saved as `vdom.listeners`.
- `setAttributes(el, attrs)` applies the non-event attributes.

That `vdom.listeners` back-reference is critical later in Chapter 9: destroying uses it to remove listeners even if the vnode is no longer in the DOM.

Code: attributes vs. events are wired by two different subsystems

This little helper function is worth memorizing:

```
function addProps(el, vdom, hostComponent) {
  const { props: attrs, events } = extractPropsAndEvents(vdom)

  vdom.listeners = addEventListeners(events, el, hostComponent)
  setAttributes(el, attrs)
}
```

The important point isn't "how to set an attribute"; it's *where the boundary is*:

- Events are installed through a wrapper so patching can update handlers without leaking listeners.
- Attributes are applied directly and are purely data-driven.

We'll dive deeper into listener stability in Chapter 7.

What the tests prove

The tests in `src/__tests__/mount-dom.test.ts` cover a small but meaningful matrix:

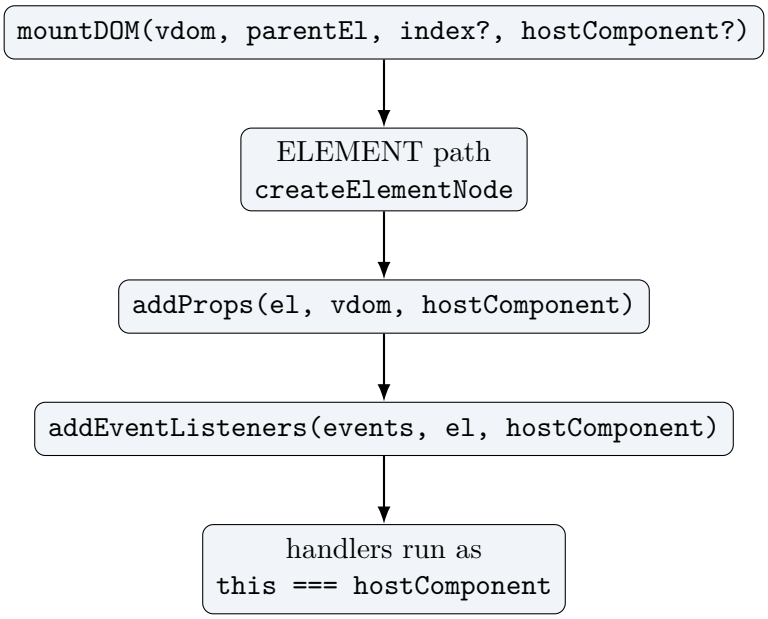
- **attributes:** id, class, list-of-classes
- **style:** a style object becomes an inline style string
- **events:** clicking the element calls the handler and `vdom.listeners` is populated

These are not "just tests", they are a spec for how Relax converts the VNode `props` object into real DOM behavior.

5.5 Event handler binding: the meaning of `hostComponent`

texttmountDOM() accepts an optional `hostComponent` argument, which gets threaded into `createElementNode(...)` and ultimately into `addEventListeners(...)`.

Diagram: how `hostComponent` flows into events



The intent is: when a component mounts elements, those element event handlers should run with `this === componentInstance` so handler methods can mutate component state.

What the tests prove

The test `where there is a host component, the event handlers are bound to it` builds a fragment containing two buttons and passes a plain object `comp = { count: 5 }` as the host. Both click handlers increment `this.count`. The assertions show two things:

- the same host is used for nested mounts (a button nested inside a div still binds `this` correctly)
- binding is not limited to direct children

This is a key piece of "component semantics" even though it's implemented entirely inside the mounting layer.

From scratch: why I like this explicit `hostComponent` thread

Many frameworks hide handler binding behind closures. That works, but it makes patching harder because you constantly allocate new function identities.

Relax takes a more systems-y stance:

> Event listeners are installed once, and the wrapper decides what `this` should be.

That gives you an explicit place to encode semantics (bind to component instance when present) and it makes garbage pressure more predictable.

5.6 Fragments: `createFragmentNodes` and insertion indices

Fragments are the trickiest type to mount because a fragment is "many nodes".

Relax's move is simple:

> A fragment vnode stores `vdom.el = parentEl`.

That is, a fragment doesn't own a DOM node. Instead, it keeps a reference to the container it was mounted in.

Code: the fragment mount loop and the shifting index

This loop is the entire reason fragments are a little tricky:

```
function createFragmentNodes(vdom, parentEl, index, hostComponent) {
  const { children } = vdom
  vdom.el = parentEl

  let idx = index ?? null
  for (const child of children) {
    mountDOM(child, parentEl, idx, hostComponent)

    if (idx == null) {
      continue
    }

    switch (child.type) {
      case DOM_TYPES.FRAGMENT:
        idx += child.children.length
        break
      case DOM_TYPES.COMPONENT:
        idx += child.component.elements.length
        break
      default:
        idx++
    }
  }
}
```

If you want to understand why patching later is careful about indices, start here.

5.6.1 The index problem

When mounting into an explicit `index`, you want fragment children to appear starting at that position. But as soon as you insert the first child, the parent's `childNodes` list changes, and the next child's target index must shift.

Relax solves this by maintaining a local counter `idx`:

- If `idx == null`, it just appends each child and doesn't care.
- Otherwise, after mounting each child, it increments `idx` by the number of DOM nodes that child contributed.

The "how many DOM nodes did this child contribute?" logic is where fragments and components matter:

- child fragment: contributes `child.children.length`
- child component: contributes `child.component.elements.length`
- other nodes: contribute `1`

This design implies a subtle contract with the component implementation: component instances must maintain an `elements` array listing the DOM nodes they own. You'll revisit that in the component lifecycle chapters.

What the tests prove

texttt src/__tests__/mount-dom.test.ts contains several important fragment tests:

- `mount a fragment in a host element` and `mount a fragment inside a fragment...`: show that fragments flatten naturally at mount time.
- `all nested fragments el references point to the parent element`: asserts the design choice `fragment.el === parentEl` even for nested fragments.
- A parametrized `test.each` suite (later in the file) checks insertion-index behavior for complex nested fragment/component scenarios.

5.7 Components: `createComponentNode` and asynchronous `onMounted`

Component mounting bridges VDOM to component instances. The algorithm in `createComponentNode()` is:

1. Split vnode props into `props` and `events` via `extractPropsAndEvents(...)`.
2. Allocate the instance: `new Component(props, events, hostComponent)`.
3. Store external children: `component.setExternalContent(vdom.children)`.
4. Delegate to the component to mount itself: `component.mount(parentEl, index)`.
5. Store `vdom.component = component`.

6. Store `vdom.el = component.firstElement`.

The last step is important: a component vnode cannot point to "the component" as a DOM node, so Relax points it at the first DOM element the component produced. Patching uses this to locate and replace component output.

5.7.1 Lifecycle scheduling and the job queue

Immediately after creating a component node, `mountDOM()` enqueues a lifecycle callback:

textttenqueueJob(() => vdom.component.onMounted())

So `onMounted()` is intentionally asynchronous relative to "the DOM nodes now exist". This approach avoids re-entrancy hazards (mount causing mount causing mount) and aligns with how many UI systems schedule lifecycle effects.

From scratch: why schedule `onMounted` instead of calling it directly?

Calling lifecycle hooks synchronously during mount makes it easy to write code that accidentally mutates the tree while you are still building it. That creates re-entrancy hazards like:

- a component mounting triggers state updates
- those updates trigger patching
- patching now operates on a partially mounted tree

The job queue turns that into a two-phase commit:

1. Mount builds the DOM and installs listeners.
2. Jobs run afterward, when invariants are established.

This theme will show up again in scheduling (Chapter 10).

What the tests prove

texttsrc/___tests___/mount-dom.test.ts exercises component creation and parent-child links:

- `mount a component with props`: shows that props are passed in and the instance is stored.
- `mount a component with children`: shows that children can be used to build deeper trees.
- `mount a component with a parent component` and `child components keep a reference to their parent component`: show that `hostComponent` threads the parent/child relationship.
- `when a component with multiple elements is mounted, the vdom keeps a reference to the first element`: locks down the `vdom.el = firstElement` rule.

5.8 HRBR interop: `createHrbrNode` and why `el` points to the host

HRBR nodes are mounted by allocating a dedicated host container element:

- Create a `<span>` host.
- Mark it with a dataset flag: `host.dataset.hrbr = '1'`.
- Insert the host into the parent.
- Call the mount factory: `instance = vdom.mount(host)`.
- Store `vdom.instance = instance`.
- Store `vdom.el = host` (and also `vdom.host = host`).

The comment in `src/mount-dom.ts` is worth repeating: **VDOM owns the host container, not the host's first child.** Patch logic later locates nodes by `oldVdom.el`; if `el` pointed inside HRBR output, replacement/removal would get the wrong DOM index.

Code: HRBR host container rationale as an explicit comment

The implementation makes this policy explicit:

```
function createHrbrNode(vdom, parentEl, index) {
  const host = document.createElement('span')
  host.dataset.hrbr = '1'
  vdom.host = host
  insert(host, parentEl, index)

  const instance = vdom.mount(host)
  vdom.instance = instance

  // IMPORTANT: `vdom.el` must point at the actual DOM node the VDOM owns.
  // For HRBR we own the host container, not the host's first child.
  vdom.el = host
}
```

What the tests prove

While the mounting tests live in `mount-dom.test.ts`, HRBR mounting is exercised via integration tests in `src/__tests__/hrbr-integration.test.ts`. Those tests treat HRBR blocks as "just another vnode type" and assert:

- the DOM output appears after mounting
- the block is destroyed when the VDOM is destroyed
- patching preserves vs remounts based on mount factory identity

5.9 Insertion mechanics: insert(el, parentEl, index)

Everything ultimately goes through `insert()`, which implements four rules:

- **append:** if `index == null`, append.
- **reject negative indices:** if `index < 0`, throw.
- **append out-of-range:** if `index >= parent.childNodes.length`, append.
- **insertBefore otherwise:** insert before `childNodes[index]`.

The tests include a negative-index case (throws) and several index-sensitive fragment suites.

Code: insert() encodes your DOM insertion policy

This helper is the "last mile" for every vnode type:

```
function insert(el, parentEl, index) {
  if (index == null) {
    parentEl.append(el)
    return
  }

  if (index < 0) {
    throw new Error(`Index must be a positive integer, got ${index}`)
  }

  const children = parentEl.childNodes
  if (index >= children.length) {
    parentEl.append(el)
  } else {
    parentEl.insertBefore(el, children[index] ?? null)
  }
}
```

If you ever see a renderer bug where nodes show up in the wrong order, `insert()` and fragment indexing are the first two places I look.

5.10 Mini-exercises

1. Add a focused test to `mount-dom.test.ts` proving that mounting with an out-of-range index appends for element vnodes. (You'll be asserting the same rule `insert()` implements.)
2. In `mount-dom.test.ts`, find the parameterized index tests and explain why nested components contribute `component.elements.length` to the index offset.

5.11 Online resources

- DOM basics: `document.createElement` (MDN): https://developer.mozilla.org/en-US/docs/Web/API/Docum
- DOM basics: `Node.insertBefore` (MDN): https://developer.mozilla.org/en-US/docs/Web/API/Node/insertI

- DOM basics: `Node.appendChild` / `Element.append` (MDN): https://developer.mozilla.org/en-US/docs/Web/API/Element/append
- Event targets and listener binding (MDN): https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/a
- "Why schedule effects" background (React's mental model is a good reference even if implementation differs): https://react.dev/learn/synchronizing-with-effects

Chapter 6

Attributes, properties, classes, and styles (`src/attributes.ts`)

In Chapter 5 we learned how to mint DOM nodes. This chapter is about the part that looks boring in demos but dominates correctness in real apps: *wiring props into the DOM in a way that matches how the browser actually works.*

I call this the **DOM edge**. It's where frameworks either:

- accumulate a huge and fragile compatibility layer, or
- pick a small set of rules and defend them with tests.

Relax chooses the second path. The module `src/attributes.ts` is intentionally tiny, and the "spec" is mostly the test matrix in `src/__tests__/attributes.test.ts`.

6.1 The mental model: the browser has *two* prop channels

When people say "set an attribute", they often mean three different things:

1. **Attributes**: string key/value pairs in the element's attribute map (example: `el.getAttribute('id')`).
2. **Properties**: live fields on the element object (example: `input.value`).
3. **Special APIs**: `className`/`classList` and `style` are their own worlds.

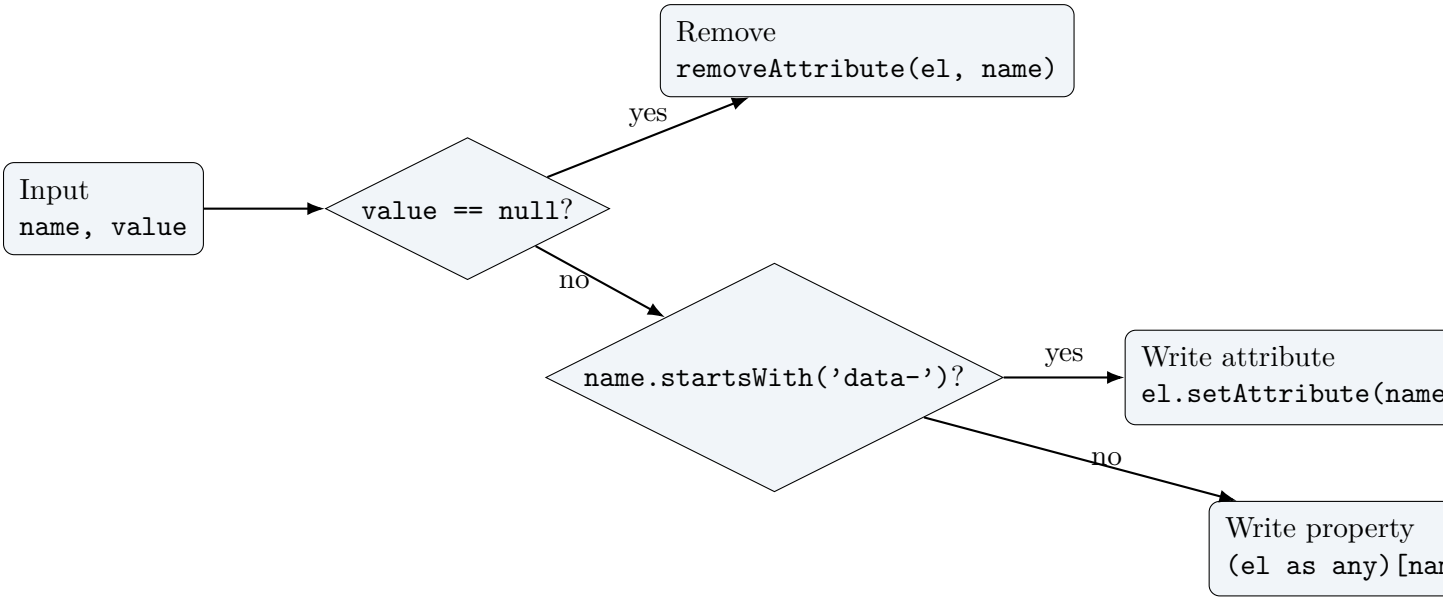
The key renderer lesson is that **attributes and properties are not interchangeable**. The classic foot-gun is forms: exttt<input value="x"> and `input.value = "x"` look similar, but behave differently across updates.

Relax's rule of thumb is extremely pragmatic:

> extbfRelax rule: only `data-*` is forced through the attribute channel. Everything else is treated as a property.

This rule is small enough to memorize, and it keeps the "boring" cases correct by default.

Diagram: deciding attribute vs property



6.2 Contract: `setAttributes(el, attrs)`

In Relax, element props are written by one function:

extttsetAttributes(el: HTMLElement, attrs: ElementVNodeProps)

You saw calls to this during element mounting in Chapter 5. You'll see it again during patching in Chapter 8.

Inputs

- `el`: a real DOM element you already created.
- `attrs`: the element props object from the VNode (which may contain `class`, `style`, `key`, and many DOM-ish names).

Outputs

- The DOM element is mutated in-place.
- No return value; this is a pure "effect" boundary.

Non-obvious invariants

- **`key` never reaches the DOM.** It's identity metadata only.
- **`class` and `style` are special-cased.** They do not go through the generic channel.
- **Most props become properties, not attributes.** This is a correctness-first bias.

The implementation (real code)

This is `setAttributes()` from `src/attributes.ts`. Notice the three-stage pipeline: split special fields, delete `key`, then apply the rest.

```
export function setAttributes(el: HTMLElement, attrs: ElementVNodeProps) {
  const { class: className, style, ...otherAttrs } = attrs

  // Delete the "key" property if it exists
  delete (otherAttrs as any).key

  if (className) {
    setClass(el, className)
  }

  if (style) {
    Object.entries(style).forEach(([prop, value]) => {
      setStyle(el, prop, value)
    })
  }

  for (const [name, value] of Object.entries(otherAttrs)) {
    setAttribute(el, name, value as any)
  }
}
```

That deletion might look redundant, because other modules also delete `key`. This redundancy is a feature. I want every layer that can touch "props" to be safe even if you use it in isolation.

Diagram: `setAttributes()` pipeline

6.3 The core decision: `setAttribute(el, name, value)`

The entire attribute/property policy collapses into one small function:

```
export function setAttribute(el: Element, name: string, value: string | number | null) {
  if (value == null) {
    removeAttribute(el, name)
  } else if (name.startsWith('data-')) {
    el.setAttribute(name, String(value))
  } else {
    ;(el as any)[name] = value
  }
}
```

There's an important design choice hiding here:

- Relax does *not* try to maintain a massive allowlist of "attribute names".

- Instead, it assumes **"property first"** and relies on browser behavior.

This is why Relax feels predictable: most names you type behave like real DOM fields.

6.4 Tests as spec: what does "property first" buy us?

If you want to understand this module quickly, don't start by reading framework wars on the internet. Start by reading the invariants encoded in `src/__tests__/attributes.test.ts`.

Here are a few highlights that act like a minimal spec:

- **Class replacement semantics:** setting `class` again replaces the old value.
- **Style is assignment-based:** setting `style` twice accumulates fields unless you explicitly remove them.
- **Form controls behave like fields:** `input.value`, `input.checked`, `select.value` follow DOM property rules.
- **Null means remove:** writing `null` maps to removal behavior (with element-specific defaults).

For example, the test suite proves that `tabIndex` removal yields `-1` (DOM default), and that a `<select>` value that doesn't match an option becomes `''`.

6.5 Removal is not symmetric: `removeAttribute(el, name)`

Removing looks trivial until you meet element-specific behavior and host quirks. Relax uses a two-level cleanup:

```
export function removeAttribute(el: Element, name: string) {
  try {
    ;(el as any)[name] = null
  } catch {
    if (typeof console?.warn === 'function') {
      console.warn(`Failed to set "${name}" to null on ${(el as any).tagName}`)
    }
  }

  el.removeAttribute(name)
}
```

This is one of those places where being "correct" means respecting the platform. Some DOM properties throw when assigned `null` (or behave differently between real browsers and jsdom), so I prefer a best-effort property clear plus a guaranteed attribute removal.

6.6 Class and style are special-cased (on purpose)

There are pragmatic reasons to split out `class` and `style`:

- **`class`:** I want a stable, replacement-based behavior and I want arrays to work.
- **`style`:** I want to assign fields directly (it matches CSSOM), and I don't want to build a diff engine yet.

The tests at the top of `src/__tests__/attributes.test.ts` act as the contract for both.

A subtle style invariant

extttsetAttributes() doesn't know the previous style object. So a later call like `{ style: { backgroundColor: 'blue' } }` will *not* remove an earlier `color` field. That's not a bug; that's a design constraint. In Chapter 8, you'll see how patching handles this by diffing old vs new props.

6.7 Props vs events: keeping the shape sane

Relax stores event handlers under `props.on`. Before mounting an element or instantiating a component, we split the object into two:

extttextractPropsAndEvents(vdom) -> { props, events }

This is the real implementation from `src/utils/props.ts`:

```
export function extractPropsAndEvents(vdom: { props: Record<string, unknown> }) {
  const { on: events = {}, ...props } = vdom.props as any
  delete (props as any).key
  return { props, events }
}
```

Again, note the invariant: `key` is consumed by the renderer and must never leak into the DOM or component props.

Tests as spec

The tests in `src/__tests__/props.test.ts` prove five simple facts:

- empty props → empty `props`
- missing `on` → empty `events`
- `key` is ignored
- non-event fields stay in `props`
- `on: {...}` becomes `events`

When you're building systems like this, these are the invariants you defend. Everything else is just API surface.

6.8 Design notes: why Relax stays small here

The attribute layer is a great place to accidentally create a maintenance nightmare. Relax deliberately keeps the rule set tiny:

- property-first by default
- explicit `data-*` rule
- explicit `class/style` behavior
- tests as a compatibility net

If you were extending Relax, the first realistic feature requests would be:

- **SVG support**: namespaces and attribute/property differences.
- **Style removal semantics**: tracking old style keys (or letting patching own it entirely).
- **Boolean attribute reflection**: some teams prefer attribute-presence semantics.

6.9 Mini-exercises

1. Add a new case to the matrix in `src/__tests__/attributes.test.ts` for `<textarea>` value behavior. (Does it behave like `input.value`?)
2. Add a test that asserts `setAttributes(el, { key: 'x', id: 'y' })` does not create a `key` attribute in the DOM.
3. Thought experiment: if you changed Relax to treat `aria-*` the same way it treats `data-*`, what would break? Which tests would you add first?

6.10 Online resources

- DOM attributes and element APIs (MDN): https://developer.mozilla.org/en-US/docs/Web/API/Element
- `HTMLElement.dataset` and `data-*` (MDN): https://developer.mozilla.org/en-US/docs/Web/API/HTMLElem
- Form control properties like `value`/`checked` (MDN): https://developer.mozilla.org/en-US/docs/Web/API/HT
- CSSOM style API (MDN): https://developer.mozilla.org/en-US/docs/Web/API/CSSStyleDeclaration

Chapter 7

Events and the Command Dispatcher

In Chapter 6 we learned how props become DOM configuration. Now we hit the other half of interactivity: **input flowing back into the system**.

Relax uses the word "event" in two different senses, and it helps to separate them in your head:

- **DOM events**: `click`, `input`, `change`, ... created by the browser and delivered to listeners on real elements.
- **Commands**: named messages with payloads delivered through an in-memory bus.

These are implemented as two tiny modules:

- `src/events.ts` — converts Relax's `props.on` map into real DOM listeners with predictable binding.
- `src/dispatcher.ts` — a deterministic pub/sub for named commands.

And, as usual in Relax, the real contract is pinned down by tests:

- `src/__tests__/events.test.ts`
- `src/__tests__/dispatcher.test.ts`

7.1 DOM events: the two problems every renderer must solve

If a UI library wants to support authoring like:

```
<button onClick={this.onClick}>Hit</button>
```

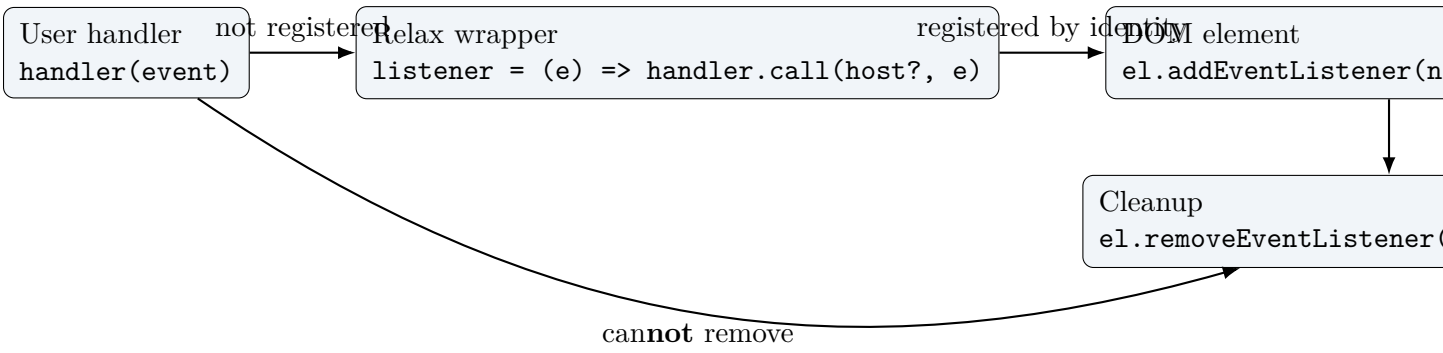
it has to solve two very unglamorous problems:

1. **Binding:** when the browser calls the handler, what should `this` be?
2. **Removal by identity:** when the tree unmounts (or events update), can we remove exactly what we added?

The second one is the bigger deal in practice. The DOM removes listeners by *function identity*, not by comparing source code or "equivalent" functions.

Relax's answer is: **always register a wrapper listener**. The wrapper is what gets stored and later removed.

Diagram: why wrapper identity matters for removal



The DOM can only remove listeners by the exact function object identity that was registered. That's why `events.ts` returns the wrapper.

7.2 DOM event listener wiring (`src/events.ts`)

The central helper is:

extttaddEventListener(eventName, handler, el, hostComponent?)

7.2.1 Contract

- **Inputs:**
  - `eventName`: a DOM event name like `"click"`.
  - `handler`: a function that should run when the event fires.
  - `el`: the real DOM element.
  - `hostComponent`: optional object used as the `this` binding.
- **Output:**
  - returns the concrete `listener` function object that was actually registered with `el.addEventListener`.

Returning the registered listener is not a convenience; it's required for correctness.

7.2.2 Implementation: the wrapper listener (real code)

This is the actual implementation from `src/events.ts`. Notice the wrapper chooses between `handler.call(hostComponent, event)` and `handler(event)`:

```
export function addEventListener(
  eventName: string,
  handler: (payload: unknown) => void,
  el: Element,
  hostComponent: any = null
) {
  const listener = (event: Event) => {
```

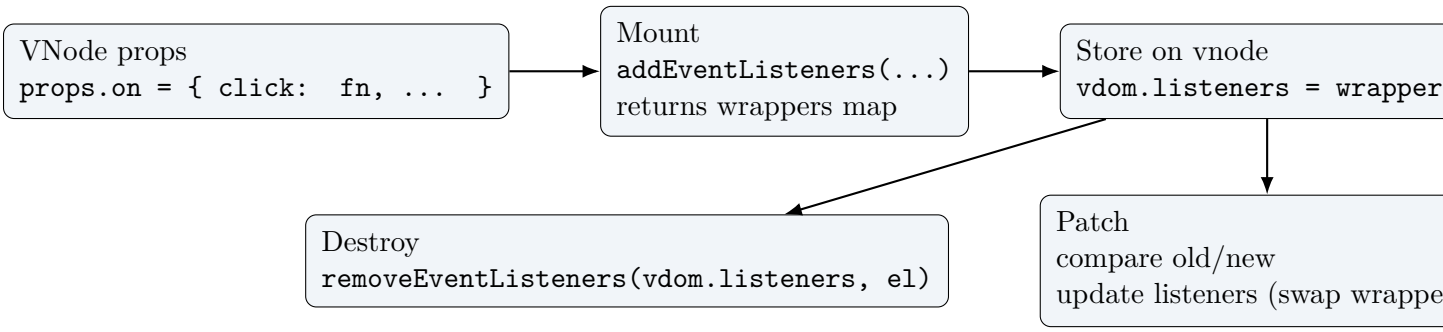
```
    if (hostComponent) {
      handler.call(hostComponent, event)
    } else {
      handler(event)
    }
  }

  el.addEventListener(eventName, listener)
  return listener
}
```

This is one of those renderer tricks that's simple but non-negotiable. If you ever "refactor" this into returning `handler` directly, unmount will leak listeners.

This immediately gives Relax three desirable properties:

- **Stable binding semantics:** the wrapper decides whether to call with `handler.call(hostComponent, event)`.
- **No mutation of user handlers:** the original function object is untouched.
- **Reliable cleanup:** the wrapper function object can be stored and later removed.

Diagram: event wiring lifecycle across mount, patch, destroy



Even if patch-time logic changes, notice the invariant: destroy always removes *whatever wrappers are currently stored on the vnode.*

7.2.3 Multiple listeners: `addEventListeners(events, el, hostComponent?)`

Mounting and patching usually deal with an event map (a record of event name to handler). Relax provides a convenience:

extttaddEventListeners(events, el, hostComponent)

Implementation is straightforward:

- iterate `Object.entries(events)`
- call `addEventListener` for each
- store returned wrappers in `listeners: Record<string, EventListener>`

The important artifact is the returned `listeners` map. In `mount-dom.ts`, Relax stores it on the vnode (`vdom.listeners`) so that unmounting can remove exactly what was added.

7.2.4 Cleanup symmetry: `removeEventListeners(listeners, el)`

Removal is the mirror image:

```
Object.entries(listeners).forEach(([eventName, listener]) => {
  el.removeEventListener(eventName, listener)
})
```

Note what's *not* present: the original user handlers. Only wrappers can be removed.

7.3 Tests for DOM event semantics (`src/__tests__/events.test.ts`)

The tests are small, but they lock down the two things that tend to regress when you wrap handlers.

7.3.1 Invariant 1: wrappers don't break sync or async handlers

extttevents.test.ts contains:

- `synchronous event without host component, called with argument`
- `asynchronous event without host component`

The async test uses `flushPromises()` (`src/utils/promises.ts`) after `btn.click()` to prove that:

- the wrapper does not swallow async behavior
- the handler is actually invoked and can `await` microtasks

This matters because event wrappers are a common source of subtle bugs where promises aren't awaited or exceptions get lost.

7.3.2 Invariant 2: with a host component, `this` is correct

Two tests cover binding (sync and async):

- `synchronous event with host component, the handler is bound to component`
- `asynchronous event with host component, the handler is bound to component`

Both assert:

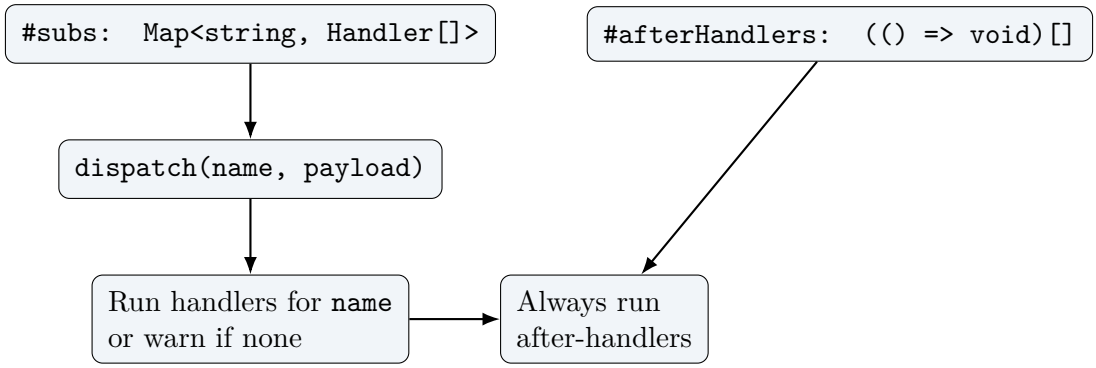
- `actualBinding === hostComponent`
- the handler argument is an `Event`

If you remember only one idea from this chapter, make it this: **binding and cleanup are owned by the wrapper function.**

7.4 The command dispatcher (`src/dispatcher.ts`)

Where `events.ts` is about *browser integration*, `Dispatcher` is an internal message bus. I use it for things like devtools hooks, app-level commands, and decoupled communication that doesn't want to bounce through the DOM.

Diagram: dispatcher data structures and dispatch flow



7.4.1 A minimal contract

extttDispatcher has three public methods:

- `subscribe(commandName, handler)` → returns `unsubscribe()`
- `afterEveryCommand(handler)` → returns `unsubscribe()`
- `dispatch(commandName, payload)`

Internally it stores:

- `#subs`: `Map<string, Array<(payload: unknown) => void»`
- `#afterHandlers`: `Array<() => void>`

So you can subscribe to a particular command name (payload-aware) and/or attach a hook that runs after every dispatch (payload-agnostic).

7.4.2 Subscription semantics: dedupe by function identity

An important policy choice appears in `subscribe()`:

> registering the same handler function twice is a no-op.

In code, that's `handlers.includes(handler)`.

This makes the dispatcher deterministic and prevents accidental handler explosions if a component subscribes in a loop.

7.4.3 Dispatch semantics: warn on unhandled commands, always run after-hooks

extttdispatch(commandName, payload) does:

1. if there are handlers for `commandName`, call them with `payload`
2. otherwise, `console.warn` (if available): `No handlers for command: <name>`
3. always run `#afterHandlers`

That last rule is important: after-hooks run even if there were no command handlers. It allows patterns like global logging, tracing, or devtools hooks.

7.5 Tests for dispatcher semantics (`src/__tests__/dispatcher.test.ts`)

The dispatcher tests are a clean, readable spec for the API:

- **subscribe/unsubscribe works:** the handler is called once, then not called after unsubscribe.
- **dedupe works:** subscribing the same handler multiple times still results in one call per dispatch.
- **multiple handlers work:** two different handlers both fire, and both can unsubscribe.
- **afterEveryCommand works:** runs once per dispatch and unsubscribes cleanly.

Notice how these tests avoid implementation details (maps, arrays, indices) and instead assert the observable contract.

7.6 Common pitfalls and edges

- **Listener removal requires wrapper identity.** Never attempt to remove a handler by passing the original user function if you registered a wrapper.
- **Dedupe is per-function identity.** If you pass a new `() => ...` lambda each time, the dispatcher will treat it as a different handler.
- **Dispatcher is synchronous.** If a handler throws, that throw will propagate from `dispatch()`.

7.7 Mini-exercises

1. In `src/events.ts`, add a new helper `updateEventListeners(old, next, el, hostComponent)` that returns the new wrappers map without leaking old listeners. (Don't implement it here; design the contract and write the tests first.)
2. Extend `dispatcher.test.ts` with a test that proves after-handlers run even when there are *no* command handlers for a dispatch.
3. Thought exercise: should `Dispatcher.subscribe()` remove the command entry from `#subs` when the last handler unsubscribes? What trade-offs does that create?

7.8 Where this plugs into the renderer

You've already seen the integration point in Chapter 5:

- element vnodes store `props.on` (after TSX mapping)
- mounting calls `addEventListeners(...)` and saves wrapper functions on `vdom.listeners`
- destroying later calls `removeEventListeners(vdom.listeners, el)`

The remaining questions, how listeners are updated during patch without leaking, and how host binding is preserved across updates, live in `src/patch-dom.ts` and `src/destroy-dom.ts`.

7.9 Online resources

- DOM events and bubbling (MDN): https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Event_bubbling
- `EventTarget.addEventListener` / `removeEventListener` (MDN): https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener
- Function `call` and `this` binding (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/call
- Message bus / pub-sub as a pattern (Wikipedia overview): https://en.wikipedia.org/wiki/Publish-subscribe_pattern

Chapter 8

Patching and Diffing: Updating the DOM Efficiently

Mounting is the "first render" story. But real apps are dominated by *updates*: state changes, input events, list reorders, and component re-renders.

Relax's patcher has one job: given an *old* VNode tree that is already mounted and a *new* VNode tree produced by the latest render, mutate the real DOM so it matches the new tree while reusing as much existing DOM as is safe.

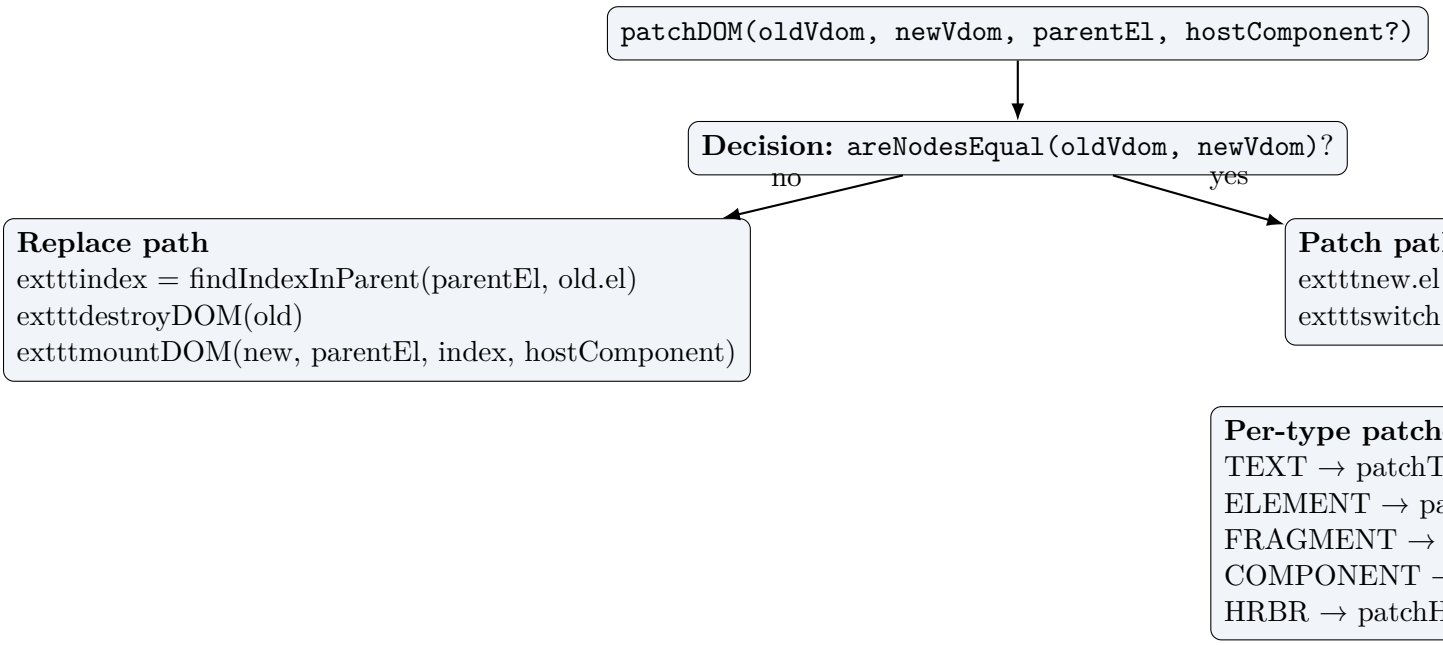
In Relax the patcher is both:

- **small enough to read end-to-end** (in `src/patch-dom.ts`), and
- **heavily specified by tests** (especially `src/__tests__/patch-dom.test.ts`).

This chapter is long because patching is where most correctness and performance edge cases live.

8.1 Diagram: the patch decision tree

Before reading any helper functions, keep this diagram in mind. It summarizes the control flow inside `patchDOM`:



This picture also explains why Chapter 4's invariant (children are always VNodes) matters: it makes the `switch` and its helper patchers tractable.

8.2 `patchDOM`: the top-level operation

The public entry point is:

extttpatchDOM(oldVdom, newVdom, parentEl, hostComponent?)

It lives in `src/patch-dom.ts`.

8.2.1 Contract

- **Inputs**
  - `oldVdom`: the currently mounted VNode tree.
  - `newVdom`: the VNode tree produced by the latest render.
  - `parentEl`: the DOM parent that contains `oldVdom.el`.
  - `hostComponent`: optional component instance used to bind events and to account for fragment offsets.
- **Output**
  - Returns the patched `newVdom` (with `newVdom.el` pointing at the reused DOM).
- **Key invariant**
  - After patching, the returned tree (the *new* VDOM) must contain correct `.el` references.

8.2.2 Replace vs patch: areNodesEqual

The first decision is whether two nodes are "the same" (so we can patch in place) or different (so we must replace).

Relax defines "same" in `src/nodes-equal.ts` via `areNodesEqual(oldNode, newNode)`.

This is not a philosophical definition of equality; it is *exactly* the predicate the patcher uses to decide between:

- patching in place (reuse), or
- destroying and remounting (replace).

The current identity rules are:

- If `type` differs: not equal.
- **TEXT**: always equal (text nodes are patched by content).
- **ELEMENT**: equal when `tag` matches and `key` matches.
- **COMPONENT**: equal when component function identity (`tag`) matches and `key` matches.
- **HRBR**: equal when `mount` factory identity matches.

Why identity is intentionally strict Virtual DOM patching is always a trade-off between reuse and correctness. Relax chooses a conservative identity rule:

- If identity says "not equal", Relax pays the cost of teardown + remount.
- The upside is that "equal" becomes a promise: the patcher can safely assume the DOM node kind matches what it needs to mutate.

In other words, `areNodesEqual` isn't about "deep equality". It's a *safety gate* that defends the patcher.

A renderer mental model: identity answers "can I mutate in place?" When I say two VNodes are "equal" in Relax, I don't mean their props match. I mean something narrower and more operational:

> If I keep the existing DOM node, do I still have the right *kind* of DOM node to mutate into the new node?

That's why:

- TEXT nodes are always patchable (you can always update `nodeValue`).
- ELEMENT nodes require the same `tag` and the same `key`.
- COMPONENT nodes require the same component function identity (`tag`) and the same `key`.

That strictness is what keeps the rest of the patcher simple. When identity says "yes", every per-type patch function can assume it's operating on the right DOM shape.

If nodes are not equal, `patchDOM` does a replacement:

1. Find the current DOM index of `oldVdom.el` inside `parentEl` (`findIndexInParent`).
2. Destroy the old subtree (`destroyDOM(oldVdom)`).
3. Mount the new subtree at the same index (`mountDOM(newVdom, parentEl, index, hostComponent)`).

This behavior is covered early in `src/__tests__/patch-dom.test.ts` by:

- `change the root node`
- `change the root node, mount in same index`

8.2.3 When nodes are equal: preserve el

If nodes are equal, patching starts by reusing the DOM reference:

```
newVdom.el = oldVdom.el
```

This is asserted by tests like `no change` and `sets the el in the new vdom`.

8.3 Node-type specific patching

After equality is established, patching dispatches on `newVdom.type` (see `DOM_TYPES` in `src/h.ts`).

8.3.1 Text nodes: patchText

`patchText(oldVdom, newVdom)` compares `oldVdom.value` and `newVdom.value`. If different, it updates `Text.nodeValue`.

Test: `patch text`.

8.3.2 Element nodes: patchElement

`patchElement(oldVdom, newVdom, hostComponent)` updates four categories of props:

- Attributes/properties (everything except `class`, `style`, `on`)
- `class`
- `style`
- `on` (DOM event handlers)

Implementation detail: it destructures props like:

```
const { class: oldClass, style: oldStyle, on: oldEvents, ...oldAttrs } = oldVdom.props
const { class: newClass, style: newStyle, on: newEvents, ...newAttrs } = newVdom.props
```

and then applies targeted patchers:

- `patchAttrs(el, oldAttrs, newAttrs)`
- `patchClasses(el, oldClass, newClass)`
- `patchStyles(el, oldStyle, newStyle)`
- `patchEvents(el, oldListeners, oldEvents, newEvents, hostComponent)`

Micro-optimization: object identity fast paths Notice the checks in `patchElement`:

```
if (oldAttrs !== newAttrs) { ... }
if (oldClass !== newClass) { ... }
if (oldStyle !== newStyle) { ... }
```

In large lists, applications often reuse the same props objects (or stable sub-objects) for unchanged rows. These identity checks avoid unnecessary diff work.

8.3.3 Attributes: patchAttrs uses objectsDiff

`patchAttrs` calls `objectsDiff(oldAttrs, newAttrs)` (from `src/utils/objects.ts`) to compute `added`, `removed`, `updated`.

It then:

- removes: `removeAttribute(el, name)`
- adds/updates: `setAttribute(el, name, value)`

Important: `setAttribute` and `removeAttribute` are the same helpers discussed in Chapter 6 (`src/attributes.ts`), including the property-vs-attribute rules.

8.3.4 Classes: patchClasses uses arraysDiff

`patchClasses` normalizes both old and new values into an array of class tokens using `toClassList`.

Then it runs `arraysDiff(oldClasses, newClasses)` to compute `added` and `removed`, and performs:

- `el.classList.remove(...removed)`
- `el.classList.add(...added)`

The behavior is tested extensively in `patch class` cases in `src/__tests__/patch-dom.test.ts`.

8.3.5 Styles: patchStyles uses objectsDiff

`patchStyles` again uses `objectsDiff` and applies:

- remove: `removeStyle(el, key)`
- add/update: `setStyle(el, key, value)`

Test group: `patch style`.

8.3.6 Events: patchEvents

`patchEvents` compares old and new `on` objects using `objectsDiff` and performs two phases:

1. **Detach** removed and updated handlers by removing their old wrapper listeners.
2. **Attach** added and updated handlers using `addEventListener` from `src/events.ts`.

Why detach+attach for *updated*? Because the wrapper listener function identity changes when the handler changes.

Why this detail matters so much When people build their first patcher, they often try to "update" an event handler by doing nothing: “the new handler is different, but the event name is the same, so surely the browser will call the new one.”

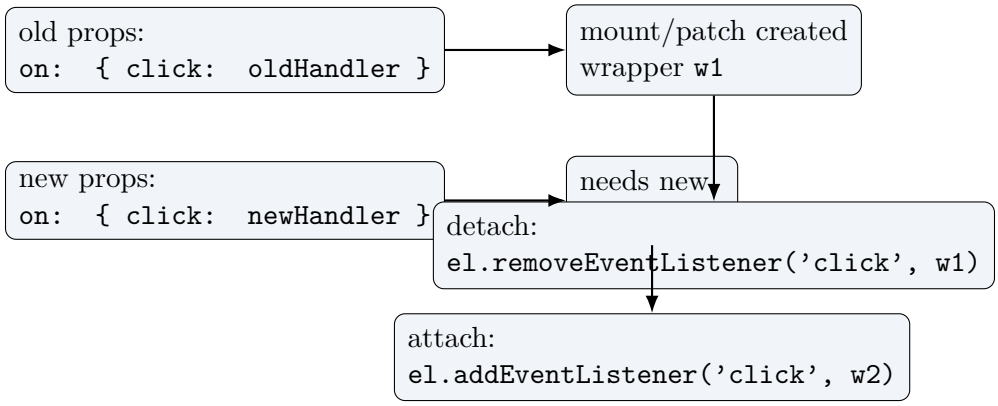
That doesn’t work. In the DOM, `addEventListener` doesn’t register "an event called click". It registers a *pair*: `(eventName, functionObject)`. Removal uses the same pair.

Relax solves this with the wrapper identity trick from Chapter 7, and the patcher follows the same rule:

- if a handler is removed or updated, remove the old wrapper function
- if a handler is added or updated, add a fresh wrapper function
- store the wrapper map on the vnode (`vdom.listeners`) so destroy can remove correctly

If you look at the real implementation in `src/patch-dom.ts`, the important part is the ordering: we remove first, then add. That prevents accidental double-binding during an update.

Diagram: event patching as identity management This is the subtle part: the browser removes listeners by the *exact function object* you originally passed.



Tests like `patch event handlers` effectively specify this: "updating" a handler is implemented as remove+add because the wrapper identity changes.

Tests to read:

- `patch event handlers` in `src/__tests__/patch-dom.test.ts`
- The binding rules in `src/__tests__/events.test.ts`

8.3.7 Component nodes: patchComponent

`patchComponent(oldVdom, newVdom)` preserves the component instance and pushes changes into it.

Key steps:

- Extract external children and props (see `extractPropsAndEvents` in `src/utils/props.ts`).
- Call `component.setExternalContent(children)`.
- Call `component.updateProps(props)`.

- Copy instance fields:
  - `newVdom.component = component`
  - `newVdom.el = component.firstElement`

This is how Relax achieves the "component instance is preserved" semantics tested in `patch component with props`.

8.3.8 HRBR nodes: patchHrbr

8.3.9 A tiny but important helper: `patchDOMSameElement`

You will notice a second patch entry point inside `src/patch-dom.ts`:

extttpatchDOMSameElement(oldVdom, newVdom, hostComponent?)

It is identical to `patchDOM` except it *skips the replacement path* and therefore does not need `parentEl`.

This helper is used in child diffing paths where:

- we already know the node should be kept (for example, keyed nodes that exist in both old and new), and
- we want to avoid scanning `parentEl.childNodes` to find the current index for every child.

When you're reading patching code, seeing `patchDOMSameElement` is a clue: "we're in an optimization path, and we've already committed to reusing the node identity."

Relax contains an experimental HRBR interop mode. In `patch-dom.ts`, `patchHrbr(old, next)`:

- Reuses the existing mounted instance when `old.mount === next.mount`.
- Otherwise, disposes/destroys the old instance and remounts into the same host element.

This behavior matches the spec tests in `src/__tests__/hrbr-integration.test.ts`.

8.4 Patching children: from fast paths to general diff

After patching the node itself, Relax patches children with:

```
patchChildren(oldVdom, newVdom, hostComponent)
```

This is where most performance work lives, and `src/patch-dom.ts` contains several layered strategies.

8.4.1 Step 0: child extraction

Children are normalized by `extractChildren(oldVdom)` / `extractChildren(newVdom)` from `src/h.ts`. This lets fragments and components present children uniformly.

8.4.2 Fast path A (opt-in): delegate keyed list reconciliation to HRBR reconciler

There is an optional integration with the runtime reconciler:

```
if (newVdom.props.\_reconcile === 'hrbr') { ... reconcileChildren(...) }
```

This path only applies when *all direct children are keyed* (keyed list). It builds `ReconcileNode` specs and lets `reconcileChildren` (from `runtime/reconciler`) patch/move nodes with a single keyed pass.

You can think of it as: "VDOM delegates list diffing to a specialized keyed reconciler".

8.4.3 Fast path B (opt-in): __textOnly rows

For large lists of rows shaped like `<li>label</li>`, Relax supports an opt-in hint:

- If `oldVdom.props._textOnly === true` and `newVdom.props._textOnly === true`, and
- both old and new children are exactly one TEXT node,

then patchChildren patches only the text node via `patchDOMSameElement`.

Test name: `patchChildren row fast path (_textOnly)`.

8.4.4 Fast path C: keyed reorder via LIS

The LIS (Longest Increasing Subsequence) trick is a classic VDOM strategy:

- If you can identify the subsequence of children that are already in correct relative order,
- then you only need to move the remaining nodes.

Relax's implementation is in `patchKeyedChildrenReorder(...)` and uses an internal helper `lisIndices(arr)` which computes LIS indices over a sequence of old positions (ignoring `-1` markers for new nodes).

One very practical detail from the code: `elementsForChild(child)` returns *multiple* DOM elements for components (using `child.component.elements`) but only one for normal elements. That's how the reorder path correctly moves a component that renders multiple siblings.

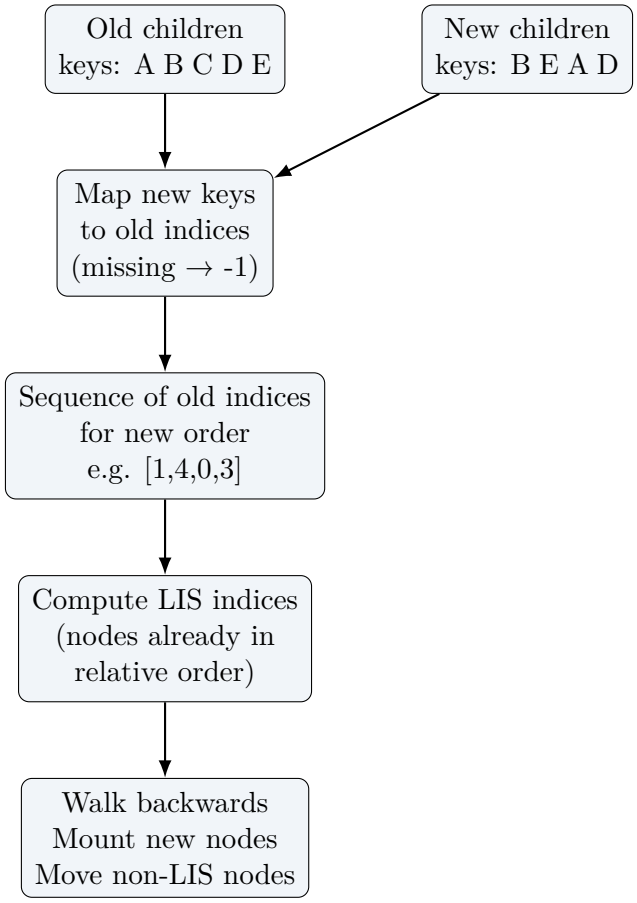
`patchKeyedChildrenReorder(oldChildren, newChildren, parentEl, hostComponent)` implements keyed reordering with minimal DOM moves:

1. Ensure both sides are fully keyed.
2. Remove old nodes not present in the new key set.
3. Patch existing nodes and mark new nodes with `-1` in a sequence.
4. Compute the Longest Increasing Subsequence (LIS) over old indices.
5. Walk backwards, mounting new nodes and moving only nodes not in LIS.

This is a classic strategy: nodes in the LIS are already in relative order, so moving the rest yields the fewest inserts/reorders.

Diagram: keyed children reconciliation with LIS

This is the bird's-eye view of what `patchKeyedChildrenReorder` is doing:



The reason for the backward walk is practical DOM surgery: when you insert/move nodes from the end, you don't invalidate the insertion positions you haven't processed yet.

Test names to read:

- `patchChildren keyed reorder fast path (LIS)`
- The many keyed reorder cases in `src/__tests__/patch-dom.test.ts`

8.4.5 Fast path D: stable keyed same-order

If keys are present and the order is identical, Relax avoids diff work entirely:

```
if (isStableKeyedSameOrder(oldChildren, newChildren)) {
  for (i) patchDOMSameElement(oldChildren[i], newChildren[i])
  return
}
```

Test name: `patchChildren fast path (stable keyed order)`.

8.4.6 General path: arraysDiffSequence

If none of the fast paths apply, Relax computes an edit script:

```
const diffSeq = arraysDiffSequence(oldChildren, newChildren, areNodesEqual)
```

extttarraysDiffSequence is a general-purpose utility in `src/utils/arrays.ts`. It's worth reading because it is one of the algorithmic cores of Relax.

It has extensive unit tests in:

- `src/__tests__/diff-sequence.test.ts`
- `src/__tests__/diff-sequence.fuzzy.test.ts`

It produces operations of types `ADD`, `REMOVE`, `MOVE`, `NOOP` (see `ARRAY_DIFF_OP`).

How to read the diff algorithm (from scratch)

extttarraysDiffSequence(oldArr, newArr, equals) returns an *edit script*. An edit script is just a list of operations that, when applied left-to-right, transforms `oldArr` into `newArr`.

Relax's implementation in `src/utils/arrays.ts` is practical, not academic:

- It walks `newArr` from left to right.
- At each index it decides whether the current old item must be removed, can stay (NOOP), needs to be added (ADD), or needs to be moved (MOVE).
- It tracks *original indices* so tests can assert stable semantics even as the working array mutates.

This is exactly the kind of algorithm that's easy to get "mostly working" and still be wrong in edge cases. That's why Relax has both:

- exact expected sequences in `diff-sequence.test.ts`, and
- randomized fuzz tests in `diff-sequence.fuzzy.test.ts`.

I recommend reading one fuzzy test before you touch the diff logic. It teaches you what you're *actually* allowed to change.

Then `patchChildren` executes them:

- **ADD**: `mountDOM(item, parentEl, index + offset, hostComponent)`
- **REMOVE**: `destroyDOM(item)`
- **MOVE**: move DOM nodes (component = multiple elements) and patch the moved subtree
- **NOOP**: patch the subtree in place

How Relax executes the edit script (the part people usually hand-wave)

This is the section that makes the whole chapter "click".

At this point we have:

- a concrete `parentEl` (the real DOM element we will mutate),
- `oldChildren` (already mounted; each node has `.el` pointers),
- `newChildren` (fresh objects from render; `.el` mostly unset), and
- `diffSeq`, a list of ops that converts old → new.

The important subtlety is this: **the DOM has one truthy linear order, but our algorithm temporarily has two competing linear orders**. As we apply operations, `oldChildren` is being mutated (conceptually) and the DOM is being mutated (physically). That is why `patchChildren` needs an `offset`.

What is `offset`? Think of `index` in the edit script as an index in the *new* children list. But when we insert or remove DOM nodes, the "current DOM index we should insert at" shifts relative to that new index.

Relax models that shift with:

- `offset += 1` when it mounts an item (the DOM got longer than the old list),
- `offset -= 1` when it destroys an item (the DOM got shorter than the old list).

So the effective DOM insertion index becomes `index + offset`.

Operation semantics in detail

Below is a readable version of what the real code is doing (names simplified; still faithful to intent):

```
let offset = 0

for (const op of diffSeq) {
  const { type, index, item, from } = op

  if (type === 'ADD') {
    // Insert a brand-new subtree.
    mountDOM(item, parentEl, index + offset, hostComponent)
    offset += 1
    continue
  }

  if (type === 'REMOVE') {
    // Tear down a mounted subtree (also unbinds events).
    destroyDOM(item)
    offset -= 1
    continue
  }

  if (type === 'MOVE') {
```

```
    // 1) Move DOM nodes to the new position
    // 2) Then patch the subtree in place (so props/events update)
    // (MOVE is not just reorder; it implies we still need to patch.)
    const oldChild = oldChildren[from]
    const newChild = item

    moveDomForVNode(oldChild, parentEl, index + offset)
    patchDOMSameElement(oldChild, newChild, hostComponent)
    continue
  }

  // NOOP: same identity at this position, but props may differ.
  patchDOMSameElement(item.old, item.new, hostComponent)
}
```

This is also why Relax is strict about `areNodesEqual`: if a node is "equal", we assume we can move/patch it without changing its fundamental DOM shape.

MOVE for components (multi-element reality) Moving an *element vnode* means moving one DOM element. Moving a *component vnode* often means moving **multiple** DOM siblings, because a component can render "fragment-like" output.

That is why the implementation builds the move list like:

```
const elementsToMove = isComponent(oldChild)
  ? oldChild.component.elements
  : [oldChild.el]
```

Then it inserts each element in order at the target position. If you only moved `component.firstElement` and forgot the rest, you'd silently corrupt the DOM order.

A concrete mental simulation

Here is a small sequence you can replay by hand:

- old keys: `[A, B, C]`
- new keys: `[B, D, A]`

A sensible edit script is:

- MOVE `B` to index 0
- ADD `D` at index 1
- MOVE `A` to index 2
- REMOVE `C`

As soon as `D` is added, the DOM becomes longer than the original, so the correct target indices for later operations must account for that. That's exactly what `offset` encodes.

8.4.7 What the patch-children tests specify (without running them)

The tests in `src/__tests__/patch-dom.test.ts` are effectively a spec for these semantics. In particular, they lock down:

- **No-op patching still updates**: even when the vnode identity matches, prop diffs still apply.
- **Moves preserve identity but update content**: moved nodes keep their DOM element(s), but still get patched.
- **Component move moves all owned elements**: multi-element components reorder correctly.
- **Index stability**: when replacing the root, new nodes mount into the same parent index.

If you ever change `patchChildren`, read those tests first. They tell you what behavior is intentional and what behavior would be a breaking change.

Component moves A subtle point: moving a component means moving *all* DOM elements owned by that component. That's why the code does:

```
const elementsToMove = isComponent(oldChild)
  ? oldChild.component.elements
  : [oldChild.el]
```

This matches the "component as a fragment" mental model: the component is a logical node that may span multiple DOM siblings.

8.4.8 What the diff-sequence tests prove

The patcher relies on `arraysDiffSequence` to produce a valid edit script. The contract is captured in two test files:

- `src/__tests__/diff-sequence.test.ts`: a human-readable set of exact expected operation sequences for many small cases
- `src/__tests__/diff-sequence.fuzzy.test.ts`: a property-style test that generates random arrays and asserts that applying the operations reproduces the target array

The workhorse assertion is essentially:

> extttapplyArraysDiffSequence(oldArray, arraysDiffSequence(oldArray, newArray)) === newArray

This is a great example of how Relax uses tests: not just to assert a single result, but to lock down the correctness of an algorithm under many variations.

8.5 Tests to lean on

If you want to understand Relax patch semantics quickly, scan these groups in `src/__tests__/patch-dom.test.ts`:

- Root replacement and `.el` preservation.
- Fragment patching (nested add/remove/move).
- Attribute/class/style patching.
- Event handler updates and removals.
- Children patching: keyed/unkeyed, moves, fast paths.
- Component patching and component list semantics.

8.6 Takeaways

- Relax's patcher is layered: correct general diff + several pragmatic fast paths for real-world performance.
- Keys are not just an optimization; they unlock better move semantics and fewer DOM operations.
- Correctness depends on two invariants:
  - `areNodesEqual` must be consistent across mount/patch.
  - `newVdom.el` (and component element arrays) must be updated to reflect reality.

8.7 Online resources

- React keys and list rendering (conceptual background; implementation differs): https://react.dev/learn/renderi lists
- Longest Increasing Subsequence (LIS) algorithm overview: https://en.wikipedia.org/wiki/Longest_increasing_
- DOM operations cost model (MDN on `Node.insertBefore` / `Node.removeChild`): https://developer.mozilla.c US/docs/Web/API/Node/insertBefore
- DOM tree mutation semantics (MDN on `Node.replaceChild`): https://developer.mozilla.org/en-US/docs/Web/API/Node/replaceChild
- Virtual DOM diffing background reading (generic): https://increment.com/frontend/a-deep-dive-into-virtual-dom/

Chapter 9

Destroying DOM and Unmounting Components

So far we've treated the renderer as a one-way pipeline:

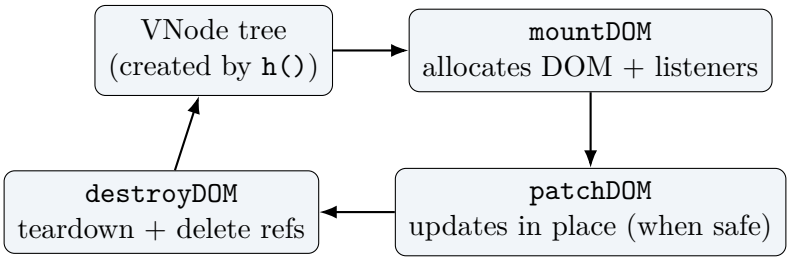
texttttVNode →mountDOM() →DOM →patchDOM()

But a real UI runtime lives and dies by teardown. If teardown is incomplete or inconsistent, it manifests as:

- **leaked event listeners** (handlers still firing after nodes are removed)
- **leaked component instances** (dispatch subscriptions never removed)
- **zombie DOM references** (stale `vdom.el` pointers used accidentally)
- **interop leaks** (third-party instances never disposed)

9.1 Diagram: render lifecycle as a loop (mount →patch →destroy)

It's useful to stop thinking of rendering as a straight line and instead as a loop that eventually ends in teardown:



This chapter focuses on the *only irreversible edge* in that loop: once you've destroyed, the old VNode tree should no longer be considered usable.

Relax centralizes teardown in `destroyDOM(vdom)` in `src/destroy-dom.ts`. The executable specs are primarily in:

- `src/__tests__/destroy-dom.test.ts`
- `src/__tests__/component-lifecycle.test.ts`
- `src/__tests__/hrbr-integration.test.ts` (interop)

9.2 A tiny contract for teardown

We'll use a tight contract so you can re-derive the implementation from the tests:

- **Input**: a *mounted* VNode subtree root.
- **Effects**: remove DOM nodes, dispatch cleanup, interop cleanup, and clear runtime references.
- **Idempotence**: not guaranteed. Calling destroy twice is a programmer error.
- **Success criteria**: after teardown, the DOM no longer contains the rendered subtree *and* the VNode tree no longer retains `.el` pointers.

That last bullet is important: removing DOM nodes is easy—cleaning the runtime state is the hard part.

9.2.1 Two different things you must destroy (from scratch)

When people say *"unmount"* or *"destroy"*, they're often mixing two separate jobs:

1. **Destroy the DOM nodes** (the browser objects).
2. **Destroy the runtime graph** (your VNodes, component instances, subscriptions, and cached references).

In Relax, the first job is the visible one: the subtree disappears from the document. The second job is the reliability one: you prevent memory leaks and you prevent *use-after-destroy* bugs (where stale VNodes are accidentally reused).

This is why the contract mentions both: the DOM must no longer contain the subtree, *and* the old VNode tree must no longer retain `.el` pointers.

Why this matters in practice Even when an element has been removed from the document, JavaScript can still hold a reference to it. If a VNode still points at a removed element (`vdom.el`), you can accidentally:

- call DOM APIs on a node you believe is "gone"
- keep a whole subtree alive in memory because some object graph still references it
- keep event listeners alive (the browser keeps a reference from the element to the listener function)

For browser reference on why listener identity matters (same function object to remove), see MDN: `removeEventListener()`.

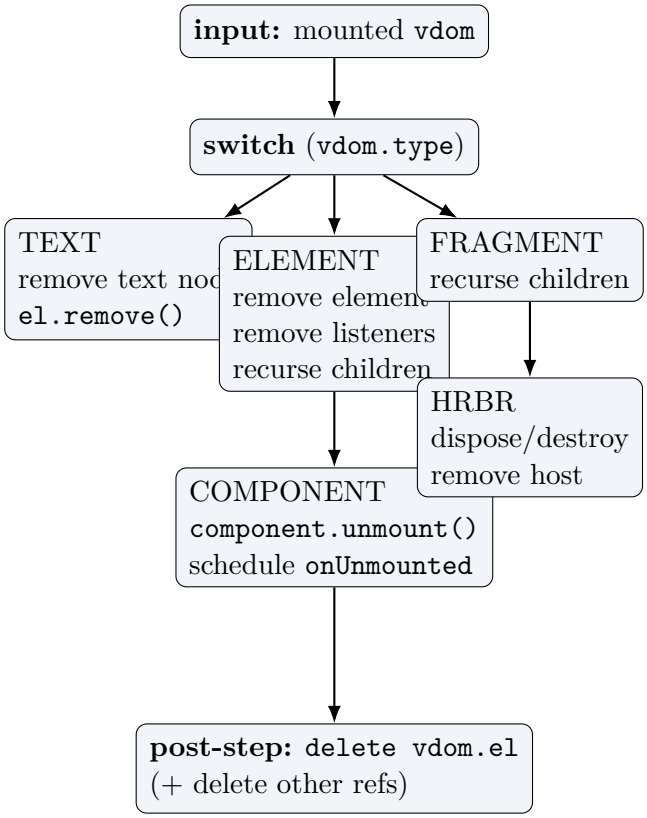
9.3 destroyDOM(): the teardown dispatcher

The public entry point is: `destroyDOM(vdom)`.

The implementation is built as a type-dispatcher over `vdom.type` (from `DOM_TYPES` in `src/h.ts`).

Diagram: destroy dispatcher

The file `src/destroy-dom.ts` mirrors `src/mount-dom.ts`: it is also a type-dispatcher.



9.3.1 The shared post-step: deleting `vdom.el`

At the end of `destroyDOM`, Relax does:

```
delete vdom.el
```

This isn't cosmetic.

- It enforces a strong invariant in tests ("every destroyed node lost its `.el`").
- It makes stale VNodes fail fast if reused accidentally.
- It reduces accidental retention of DOM nodes by application code holding onto VNodes.

9.3.2 A quick mental trace: what gets touched?

If you want a simple way to sanity-check teardown, trace what `destroyDOM(vdom)` touches:

- the browser objects (DOM nodes and event listeners)
- the VNode object graph (delete `.el` and other stored pointers)
- the component graph (call `unmount()` and clear subscriptions)

- the scheduler queue (lifecycle hooks are queued, not executed inline)

If any one of those is skipped, teardown can look "fine" in the DOM while still leaking runtime state.

9.4 Type-based teardown

`destroyDOM` dispatches on `vdom.type` (`DOM_TYPES` is defined in `src/h.ts`).

9.4.1 Text nodes

For `DOM_TYPES.TEXT`, `removeTextNode` asserts `el instanceof Text` and calls `el.remove()`.

Test: `destroy a text element`.

9.4.2 Element nodes: remove element + recurse + listener cleanup

For `DOM_TYPES.ELEMENT`, `removeElementNode` performs three distinct actions:

1. Remove the element from the DOM: `el.remove()`.
2. Recursively destroy children: `children.forEach(destroyDOM)`.
3. If there are DOM event listeners stored on the VNode, detach them:

```
removeEventListeners(listeners, el)
delete vdom.listeners
```

The listener removal uses `removeEventListeners` from `src/events.ts`, which removes by the wrapper listener identity that was created during mount/patch.

Tests:

- `destroy an html element and its children`
- `destroy an html element and its children recursively`
- `remove an html element event listeners`

Ordering note: remove first, then cleanup textttremoveElementNode() removes the element from the DOM *before* walking children. This ordering is deliberate and safe:

- In the DOM, removing a parent implicitly detaches descendants from the document.
- But Relax still needs to recursively teardown the VNode subtree (delete `.el`, detach listeners, unmount nested components).
- Running the recursion after removal avoids intermediate states where logic could observe partially-destroyed nodes in the live document.

A short code reading (as JavaScript)

Here is the important shape of `removeElementNode()` from `src/destroy-dom.ts`. Notice how it does three things: DOM removal, recursive teardown, and listener cleanup.

```
function removeElementNode(vdom) {
  const { el, children, listeners } = vdom

  el.remove()
  children.forEach(destroyDOM)

  if (listeners) {
    removeEventListeners(listeners, el)
    delete vdom.listeners
  }
}
```

Two details to internalize:

- **The recursion is about runtime cleanup, not document removal.** The browser already detached child nodes when the parent was removed.
- **Listener cleanup depends on stored wrapper identity.** You must remove *the exact function object* you added.

9.4.3 Fragment nodes

Fragments are a logical grouping; they don't own a single DOM node. For `DOM_TYPES.FRAGMENT`, `removeFragmentNodes` just destroys each child.

Tests:

- `destroy a fragment`
- `destroy a fragment recursively`

9.4.4 Component nodes: unmount + schedule `onUnmounted`

For `DOM_TYPES.COMPONENT`, `destroyDOM` delegates to the component instance:

```
vdom.component.unmount()
enqueueJob(() => vdom.component.onUnmounted())
```

Two important details are embedded here, and both are tested:

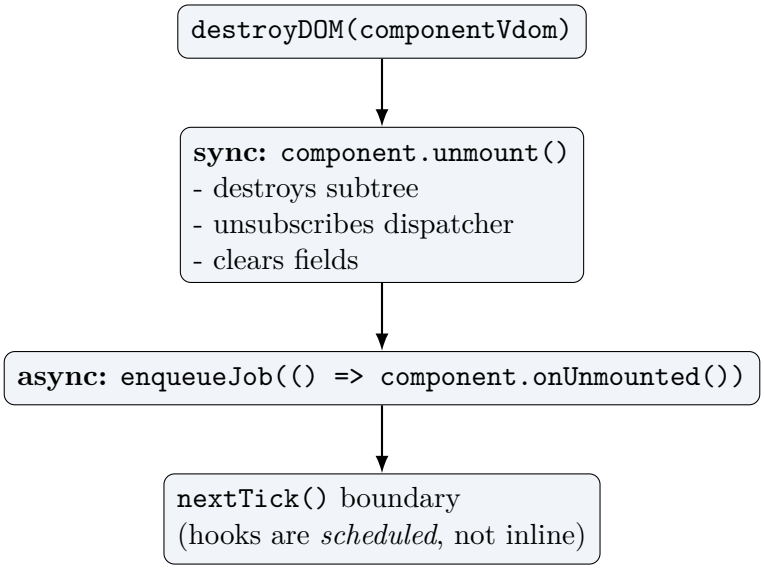
- **Unmount is synchronous structural cleanup.** It destroys the component's rendered subtree and unsubscribes internal dispatcher subscriptions.
- **`onUnmounted` is scheduled.** It runs later via `enqueueJob` (`src/scheduler.ts`), matching the mount side where `onMounted` is also scheduled.

This scheduling is why the component lifecycle tests await `nextTick()` after calling `destroyDOM()`:

- `a component reacts to being unmounted`
- `a component reacts to being unmounted asynchronously`

Diagram: component unmount is two-phase

Component teardown is deliberately split into "structural cleanup now" and "user hook later":



This pattern matches mount-side semantics too (`onMounted` is also scheduled). The intent is: the renderer finishes DOM surgery first, then user hooks run without re-entrancy hazards.

From scratch: why is `onUnmounted` scheduled?

Scheduling isn't just an implementation detail; it's an API promise. If user hooks ran inline during `destroyDOM()`:

- hooks could observe a partially-destroyed tree (some nodes already removed, others not yet)
- hooks could trigger state updates that attempt to patch while teardown is mid-flight
- hooks could throw errors that leave teardown half-complete

By deferring hooks to the scheduler queue (see `enqueueJob` in `src/scheduler.ts`), Relax makes teardown an atomic-ish structural operation: first it makes the DOM and runtime graph consistent, *then* it runs user code.

For background on the reasoning, React's commit phase + effect cleanup is a helpful conceptual parallel: Effects and cleanup.

9.5 Inside Component.unmount() (`src/component.ts`)

Components in Relax are created with `defineComponent(...)`. The generated class maintains several private fields that matter for unmount:

- `#vdom`: the last rendered subtree returned by `render()`
- `#dispatcher`: a per-component `Dispatcher` used to implement `emit()` and event handler wiring
- `#subscriptions`: unsubscribe closures returned from `dispatcher.subscribe()`
- `#isMounted` and `#hostEl`: mount state and mount container

9.5.1 Unmount sequence

Look at `unmount()` in `src/component.ts`:

1. It throws if the component isn't mounted.
2. It destroys the rendered subtree: `destroyDOM(this.#vdom)`.
3. It unsubscribes from internal dispatcher subscriptions: `this.#subscriptions.forEach(unsubscribe)`.
4. It clears state: `#vdom = null`, `#isMounted = false`, `#hostEl = null`, `#subscriptions = []`.

This means after unmount, both DOM and book-keeping references are gone.

9.5.2 Unmount in one annotated snippet (as JavaScript)

Here is the sequence from `src/component.ts`, rewritten in JavaScript style so you can read it as control flow:

```
unmount() {
  if (!this.#isMounted) {
    throw new Error('Component␣is␣not␣mounted')
  }

  // 1) structural teardown of rendered subtree
  destroyDOM(this.#vdom)

  // 2) detach component-owned subscriptions (not VNode-owned)
  this.#subscriptions.forEach((unsubscribe) => unsubscribe())

  // 3) clear references so GC can reclaim memory and stale use fails fast
  this.#vdom = null
  this.#isMounted = false
  this.#hostEl = null
  this.#subscriptions = []
}
```

The separation between (1) and (2) is the same architectural split as earlier chapters: extttdestroyDOM() is VNode-centric, while `unmount()` is component-centric.

Why unsubscribe in `unmount()` and not in `destroyDOM()`? Because subscriptions are a property of the component instance, not a property of the rendered VDOM nodes. extttdestroyDOM() is intentionally "VNode-centric"; `unmount()` is "component-centric".

9.6 HRBR nodes: dispose, destroy, and remove host

Relax also supports an experimental HRBR interop node type (`DOM_TYPES.HRBR`). These nodes represent "something else" mounting into a host container (a block, micro-framework component, etc.).

In `removeHrbrNode(vdom)` (`src/destroy-dom.ts`):

- If the instance provides `dispose()` and/or `destroy()`, Relax calls them (in that order).

- Then it removes the HRBR host element from the DOM.
- Finally, it deletes `vdom.instance` and `vdom.host`.

9.6.1 From scratch: HRBR is about *ownership boundaries*

HRBR nodes exist for one reason: sometimes the thing you're mounting isn't a Relax VNode subtree. It might be a compiled template block, a micro-framework widget, or a third-party imperative library.

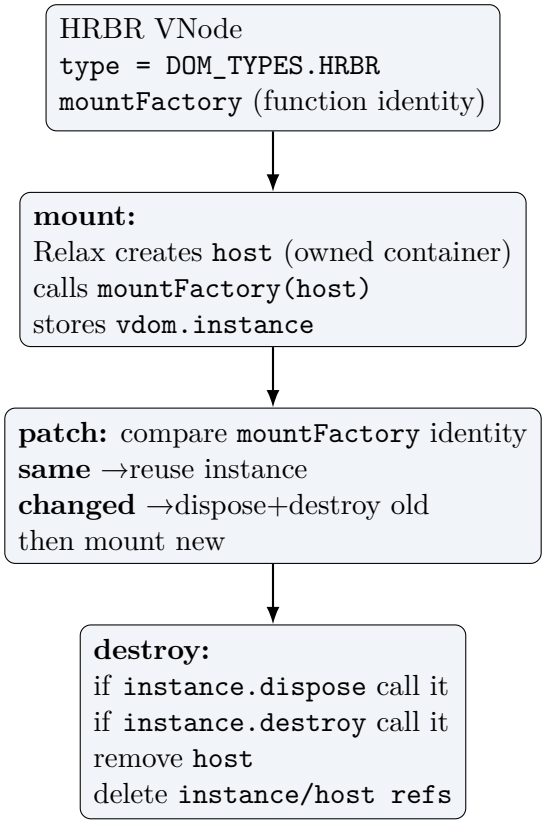
To make teardown predictable, you need a rule for **who owns what**:

- **Relax owns the host container** (a DOM element that acts as a mounting surface).
- **The interop system owns the instance** (whatever object the mount function returns).
- **Relax promises a small teardown protocol:** call optional `dispose()`, then optional `destroy()`, then remove the host.

This is very similar to a resource-management pattern in lower-level systems: *if you allocate it, you must free it.* The difference is that in UI you often split responsibility: Relax allocates the DOM "socket" (host), while the interop layer allocates the instance.

Diagram: HRBR ownership + lifecycle (mount, patch, destroy)

The easiest way to think about HRBR is: Relax plugs an *opaque instance* into a *host container*, and then manages the host's lifetime. The instance itself is treated like a resource with optional cleanup hooks.



The test suite drives this design: HRBR patching is intentionally based on *factory identity* (stable identity enables reuse; changing identity forces remount).

9.6.2 The destroy sequence (as JavaScript)

Here is `removeHrbrNode()` from `src/destroy-dom.ts`, rewritten into JavaScript style so you can focus on the protocol:

```
function removeHrbrNode(vdom) {
  const inst = vdom.instance
  if (inst?.dispose) inst.dispose()
  if (inst?.destroy) inst.destroy()

  const host = vdom.host
  if (host) host.remove()

  delete vdom.instance
  delete vdom.host
}
```

This ordering is deliberate:

- **Call hooks before removing the host** so the instance can still query/update the DOM it owns.
- **Remove the host last** so Relax reliably removes the container regardless of how well-behaved the instance is.

For background on DOM node removal semantics, see MDN: `Element.remove()`.

This mirrors the mounting/patching behavior from earlier chapters: HRBR nodes are treated like a mounted instance with an owned host container.

Interop rule of thumb Relax doesn't assume what "destroy" means. It only promises it will call well-known lifecycle hooks (`dispose`/`destroy`) if present, and remove the host container it created.

9.6.3 How HRBR teardown relates to patching (what the tests prove)

The HRBR integration tests (`src/__tests__/hrbr-integration.test.ts`) implicitly define a second contract: how HRBR behaves under *patch*, not just under destroy.

- **Stable mount function identity means reuse.** If a component re-renders and returns an HRBR node created with the *same* mount function, Relax preserves the existing instance.
- **Changed mount function identity means remount.** If the mount function identity changes, Relax must treat it like a different interop node: it tears down the old instance (`dispose`/`destroy`) and mounts a new one.

Why identity? Because it's the simplest safe signal that the meaning of "this interop node" changed. If Relax tried to "diff" two opaque interop instances, it would be forced to guess. So the system picks a clean architectural line: *either you're the same factory (reuse), or you're a new factory (remount).*

This is the same philosophy you saw in keyed VDOM reconciliation earlier: stable identity enables preservation.

For a conceptual parallel in mainstream UI libraries, see React's note on identity and preserving state across renders (keys and element identity): Keeping list items in order with `key`.

9.7 A practical mental model

It helps to treat teardown as the inverse of mounting:

- mounting creates DOM nodes and stores refs in `vdom.el`
- patching preserves DOM nodes and updates refs on the new VDOM
- destroying removes DOM nodes and deletes refs from the old VDOM

The explicit `delete vdom.el` makes it hard to accidentally keep using stale VNodes after tear-down.

9.7.1 A leak-prevention checklist you can reuse

When teardown bugs happen in real apps, they usually violate one of these checks. You can use this list as a debugging worksheet:

1. **DOM nodes removed:** is the subtree gone from the host element?
2. **Listener identities removed:** are `removeEventListener` calls using the exact wrapper that was added?
3. **VNode references cleared:** are old `vdom.el` pointers deleted so stale objects can't keep nodes alive?
4. **Component subscriptions cleared:** do per-component subscriptions get unsubscribed on unmount?
5. **Interop disposed:** do embedded instances get `dispose`/`destroy` and does the host container get removed?

The browser-side references are the subtle part. Even after DOM removal, a reference chain like: extttappState -> handler -> closure -> element can keep things alive. This is why tests focus on listener removal and reference deletion.

9.8 Takeaways

- Relax teardown is explicit and recursive.
- Element teardown removes event listeners using the stored wrapper identities.
- Component teardown is two-phase: synchronous unmount + scheduled `onUnmounted` hook.
- Deleting `.el` and clearing instance fields is a core anti-leak strategy.

9.9 Walking the tests: what they prove

This section treats the tests as a teardown requirements document.

9.9.1 `destroy a text element`

From `src/__tests__/destroy-dom.test.ts`:

- Mount a text VNode via `mountDOM` and assert `vdom.el` is a `Text` node.
- Call `destroyDOM(vdom)`.
- Assert the document is empty *and* `allElementsHaveBeenDestroyed(vdom)` returns true.

The helper `allElementsHaveBeenDestroyed()` recursively checks for the absence of `.el`. This is the strongest teardown invariant in this test suite.

9.9.2 `destroy an html element and its children`

This test ensures an `ELEMENT` node:

- removes the parent element from DOM
- still destroys (cleans up) its children VNodes

It's the "DOM removed" + "runtime graph cleaned" pairing again.

9.9.3 `remove an html element event listeners`

This is the leak-prevention test.

1. Mount a `button` with `on: { click: handler }`.
2. Trigger a click and assert the handler is called.
3. Destroy the VNode.
4. Trigger another click on the previously captured `buttonEl` and assert the handler isn't called again.

This works because during mount, Relax installs a wrapper listener and stores it in `vdom.listeners`. During destroy, `removeEventListeners(vdom.listeners, el)` uses those wrapper identities to detach correctly.

9.9.4 Recursive destruction and fragments

texttt destroy an html element and its children recursively, `destroy a fragment`, and `destroy a fragment recursively` collectively prove that:

- recursion always visits nested VNodes
- fragment nodes are teardown-neutral: they simply forward destruction to children

9.9.5 `destroy a component with subcomponents`

This test ensures that the recursion crosses component boundaries. The starting VNode is an element `div` containing a component. Destroying the root must unmount the component and destroy its rendered subtree, including nested child components.

9.9.6 Lifecycle scheduling tests (`component-lifecycle.test.ts`)

The component lifecycle tests focus on `enqueueJob`/`nextTick()` behavior:

- `onMounted` is not expected to run synchronously with `mountDOM`; the test awaits `nextTick()`.
- Symmetrically, `onUnmounted` is not expected to run synchronously with `destroyDOM`; the test awaits `nextTick()`.

This enforces a stable mental model: lifecycle hooks run in the scheduler, not "inline" within the DOM mutation operations.

9.9.7 HRBR teardown behavior (`hrbr-integration.test.ts`)

The HRBR integration suite demonstrates that `destroyDOM()` participates in interop cleanup.

- In the "can mount an HRBR block" test, the block's `destroy()` clears the host; afterwards, Relax removes the host itself.
- In "patch preserves HRBR instance", destroying triggers the instance's `destroy()` and the test asserts it was called.

9.10 Online resources

- MDN: `Element.remove()`
- MDN: `removeEventListener()` and listener identity
- MDN: JavaScript memory management (GC and references)
- React docs (conceptual parallel): Effects and cleanup
- Vue docs (conceptual parallel): Lifecycle hooks

9.11 Failure modes: the bugs youĺl actually hit

When teardown is wrong, the DOM often still "looks fine" because itś gone. The failures show up later as leaks, stale references, and re-entrancy.

Use this mapping as a debugging guide.

9.11.1 Zombie event handlers (detached element still fires)

extbfSymptom: you remove a view, but an old click handler still runs (or memory grows after repeated mount/unmount cycles).

extbfWhy it happens: a DOM node can be detached from the document yet still exist as a JavaScript object. If some code keeps a reference to the node (like the test does with `buttonEl`), it can still dispatch events and hold listener closures.

extbfRoot cause: listener identity mismatch. The function you pass to `removeEventListener` must be the same function object you passed to `addEventListener`. Relax solves this by storing listener wrappers on the VNode (`vdom.listeners`) during mount/patch, then reusing them during destroy.

Reference: `removeEventListener()`.

9.11.2 Use-after-destroy (stale VNodes still referenced)

extbfSymptom: something tries to patch using an old tree, throws because `el` is missing, or silently corrupts the UI.

extbfRoot cause: application code treated VNodes as durable objects instead of ephemeral render outputs. VNodes are only valid while they are mounted; after `destroyDOM`, that VNode graph should be considered dead.

extbfRelaxś guard rail: the shared post-step `delete vdom.el` is a fail-fast mechanism.

9.11.3 Unmount hook re-entrancy

extbfSymptom: an `onUnmounted` hook triggers an update which tries to patch while destroy is mid-flight.

extbfDesign fix in Relax: hooks are scheduled via `enqueueJob`. Teardown becomes "do all structural cleanup first, then run user code".

Conceptual parallel: Reactś effect cleanup happens after commit, not in the middle of DOM mutations: Effects and cleanup.

9.11.4 HRBR interop leaks

extbfSymptom: embedded widgets keep timers/observers alive after unmount.

extbfRoot causes:

- the mounted instance didnt́ implement `dispose/destroy`
- the factory identity changes on every render, forcing remount and skipping the "reuse" mental model

extbfFix: implement at least one cleanup hook and keep the mount factory identity stable when you intend reuse.

9.12 Edge cases worth understanding

9.12.1 Fragments and multi-root components

Fragments dont́ have a single `el`. So teardown means:

- recursively destroy children
- ensure each child loses its `.el` pointer

This is also why `Component.elements` returns an array: a component might render a fragment, and its DOM footprint is multiple siblings.

9.12.2 Double destroy

Relax does not promise idempotent teardown. In practice, calling destroy twice usually means ownership got confused (two subsystems believe they own the same subtree).

If your application needs idempotence, implement it at a higher level (router/view manager) by tracking whether a screen is currently mounted.

9.12.3 Destroy order (parent removed before child teardown)

Relax removes the parent element first, then recurses. That feels reversed until you internalize the key idea:

- removing the parent is the browser-side detach
- recursion is runtime cleanup (listeners, hooks, deleting references)

So this ordering reduces intermediate observable states in the live document, while still guaranteeing runtime cleanup.

9.13 Practical patterns for leak-free components

Relax provides hooks, but you still need discipline. Most real apps leak not because the renderer forgot something, but because application code created resources without cleaning them up.

9.13.1 Common resources you must clean up

- **Timers:** `setInterval`/`setTimeout` →clear them in `onUnmounted`
- **Global listeners:** `window.addEventListener` →remove in `onUnmounted`
- **Observers:** `ResizeObserver`/`IntersectionObserver` →`disconnect()` in `onUnmounted`
- **Fetches:** cancel via `AbortController` in `onUnmounted`

Reference: `AbortController`.

9.13.2 A small `resourceBag` pattern (plain JavaScript)

When a component grows, cleanup code tends to scatter. A tiny pattern keeps it centralized: store cleanup callbacks and run them in the teardown hook.

```
function createResourceBag() {
  const cleanups = []
  return {
    add(fn) {
      cleanups.push(fn)
      return fn
    },
    run() {
      for (const fn of cleanups.splice(0)) fn()
    },
  }
}

// Example usage:
const bag = createResourceBag()
const onResize = () => {}

window.addEventListener('resize', onResize)
bag.add(() => window.removeEventListener('resize', onResize))

// later, onUnmounted:
bag.run()
```

The important rule is ownership: only add resources that *this component created*.

9.14 Exercises

1. **Detached-element click:** recreate the listener test mentally. Why is clicking a detached element a meaningful leak test?
2. **Idempotent destroy wrapper:** design `safeDestroy(vdom)`. Where should an `isDestroyed` flag live: VNode, component instance, or app router?
3. **HRBR cleanup contract:** write a mount factory that starts an interval timer. Implement `dispose` or `destroy` so teardown stops it.
4. **Scheduling thought experiment:** list two ways inline `onUnmounted` execution could corrupt teardown correctness.

9.15 Chapter summary

- Teardown is two jobs: destroying DOM nodes and destroying the runtime graph.
- `destroyDOM` is a type-dispatcher that recursively tears down VNode trees.
- Element teardown removes event listeners using stored wrapper identities.
- Component teardown is two-phase: synchronous `unmount()` + scheduled `onUnmounted`.
- HRBR teardown is a small resource protocol (optional `dispose`/`destroy` + remove host + delete refs).

Chapter 10

Scheduling, nextTick, and Deterministic Work

Scheduling is one of those topics that every UI developer feels they "kind of" understands even while still getting surprised by it weekly. So weĺl treat the scheduler as a real subsystem with a contract, not a vague "async happens later" box.

Relax has *two* schedulers:

- A small microtask-based job queue used by the classic VDOM layer (`src/scheduler.ts`).
- A more capable lane-based scheduler used by the runtime/reactive layer (`runtime/scheduler.ts`).

They serve different purposes, but they share one theme: **make UI work predictable**.

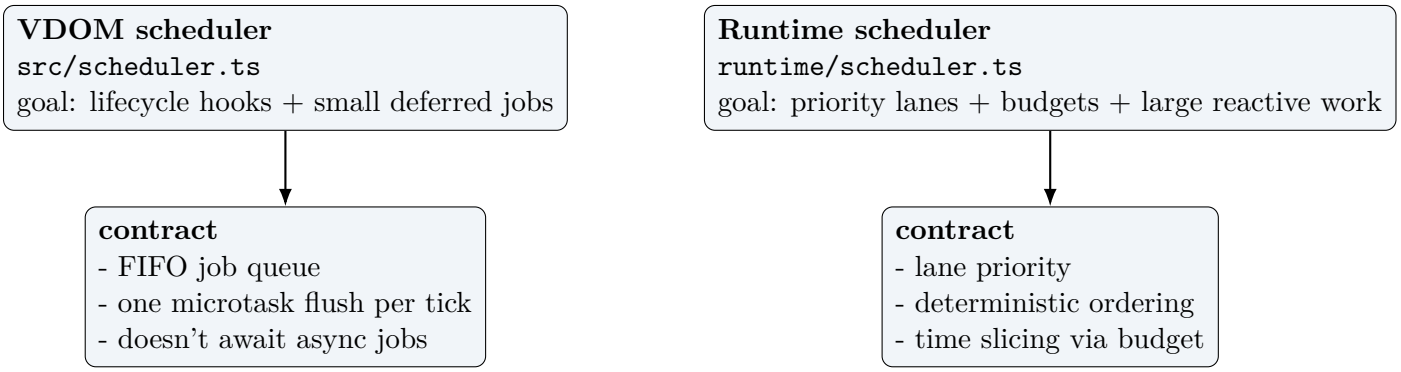
Scheduling discussions tend to get vague fast ("it's async, it'll happen later"). In Relax, scheduling is treated as part of the renderer's public contract: tests depend on it, and lifecycle semantics are defined by it.

This chapter is anchored in:

- `src/scheduler.ts` and `src/__tests__/scheduler.test.ts`
- `runtime/scheduler.ts` and `runtime/__tests__/scheduler.test.ts`

10.1 Diagram: two schedulers, two contracts

Relax has two schedulers because it has two kinds of work with different needs:



10.2 The VDOM microtask job queue (`src/scheduler.ts`)

Before we read Relaxś scheduler, we need one shared mental model: **the JavaScript event loop**.

10.2.1 From scratch: one thread, a call stack, and two queues

In a browser (and in many JS environments), your code runs on one main thread. Execution is a loop:

1. run synchronous code on the call stack
2. when the stack is empty, run microtasks (promises / `queueMicrotask`) until empty
3. then allow the next macrotask (timers, IO, UI events) to run

Two important consequences for UI frameworks:

- **Microtasks run sooner than timers.** If you want "run after this render, but before the browser handles the next event", microtasks are the sharp tool.
- **Microtasks can starve the browser if abused.** If you keep queueing microtasks from microtasks, you can delay input/paint.

References: MDN: JavaScript event loop and MDN: `queueMicrotask()`.
The VDOM layer uses a minimal scheduling primitive: `enqueueJob(job)`.

10.2.2 Why it exists

Several operations in Relax are intentionally deferred:

- component `onMounted()` is scheduled after mount completes
- component `onUnmounted()` is scheduled after unmount begins

Deferring hook execution avoids re-entrancy surprises and gives the renderer a clean boundary: "render/mount now, run hooks shortly after".

The hidden constraint: DOM consistency and re-entrancy If `onMounted()` ran inside `mountDOM()`, a component could synchronously call `updateState()`, which triggers patching while the initial mount is still underway. By scheduling hooks, Relax ensures the mount/patch/destroy primitives finish their structural work before user hook code runs.

10.2.3 Data model

`src/scheduler.ts` maintains:

- `jobs`: an array of queued callbacks
- `isScheduled`: a flag to ensure we schedule processing only once per tick

10.2.4 enqueueJob and queueMicrotask

`enqueueJob(job)` pushes the job and calls `scheduleUpdate()`.

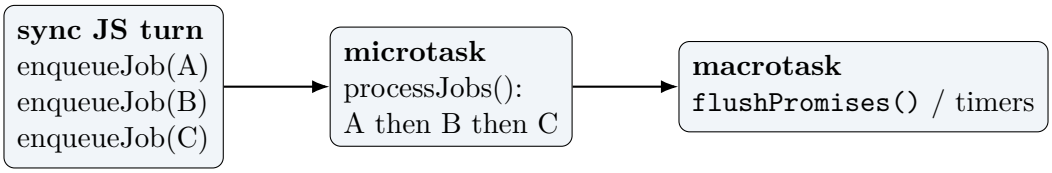
`scheduleUpdate()` is the key:

```
if (isScheduled) return
isScheduled = true
queueMicrotask(processJobs)
```

So no matter how many jobs you enqueue, you'll process them in a single microtask flush.

Diagram: microtask batching timeline

This is the behavior the tests rely on:



This is also why hook scheduling avoids re-entrancy: hooks run after the renderer finishes its synchronous DOM work.

Batching rule The `isScheduled` latch is the batching mechanism:

- enqueue one job →schedule one microtask
- enqueue many more jobs before the microtask runs →still one microtask

This makes lifecycle hooks cheap even when many components mount in the same synchronous turn.

10.2.5 processJobs: ordering and async behavior

`processJobs()` drains `jobs` in FIFO order. For each job:

- it calls `job()`
- it wraps the result with `Promise.resolve(result).then(...)` to log async errors

Important nuance (documented in the code): **async jobs do not block the queue**. The queue keeps running other jobs; the promise continuation runs later.

A code walk (as JavaScript)

Here is the scheduler rewritten into minimal JavaScript so the control flow is obvious. It mirrors `src/scheduler.ts` exactly, just without TypeScript types:

```
let isScheduled = false
const jobs = []

export function enqueueJob(job) {
  jobs.push(job)
  scheduleUpdate()
}

function scheduleUpdate() {
  if (isScheduled) return
  isScheduled = true
  queueMicrotask(processJobs)
}

function processJobs() {
  while (jobs.length > 0) {
    const job = jobs.shift()
    const result = job()

    // Important: we don't await async jobs.
    // We only attach a rejection handler so async errors are visible.
    Promise.resolve(result).then(
      () => {},
      (error) => console.error(`[scheduler]: ${error}`)
    )
  }
  isScheduled = false
}
```

From this we can state the contract precisely:

- Jobs run **after** the current synchronous call stack.
- Jobs run in **FIFO order** within a single flush.
- The queue does **not await** async jobs.
- Async errors are surfaced (logged) instead of turning into silent unhandled rejections.

Why the queue doesn't await The VDOM scheduler is designed for hook-style work ("start it soon, don't stall unrelated hooks, but surface errors"). If it awaited each job, one slow async hook could hold up every other component's hook.

This is tested in `src/__tests__/scheduler.test.ts`:

- `Asynchronous jobs enqueue more jobs` expects order `[1, 3, 2]`.

That test shows:

1. async job pushes `1`
2. it awaits a microtask, so it pauses
3. the next (sync) job runs and pushes `3`
4. then the async job resumes and pushes `2`

10.2.6 nextTick(): what it guarantees (and what it doesn't)

`nextTick()` is the testing and integration helper:

```
scheduleUpdate()
return flushPromises()
```

`flushPromises()` is implemented as `setTimeout(resolve)`. That means **nextTick resolves after the microtask flush and one macrotask turn**.

The comment in `src/scheduler.ts` is explicit:

> If the jobs are asynchronous, the promise will resolve before all the jobs have completed.

So the contract is:

- It ensures the queued jobs have been *started* and synchronous effects have run.
- It does not await completion of async work inside a job.

Practical testing guidance `nextTick()` is a boundary for "scheduler-driven work started", not "all async completed". If you write a hook that does work after an `await`, you may need an additional wait (or design your test so the awaited promise resolves immediately).

Why does `flushPromises()` use `setTimeout`?

This is subtle but important. Relaxś `nextTick()` is:

```
scheduleUpdate()
return flushPromises() // setTimeout(resolve)
```

If `nextTick()` only awaited a microtask, you could end up in situations where:

- the scheduler flush runs in a microtask
- a job itself queues more microtasks
- and your test resumes before the environment has advanced a real "turn" (macrotask)

Using `setTimeout(resolve)` forces the promise to settle on the next macrotask, which is a stronger "breathing point" for tests.

Reference: MDN: `setTimeout()`.

Batching correctness: a tiny proof sketch

The batching mechanism is the `isScheduled` latch. Hereś the key claim:

> Within a single synchronous JS turn, regardless of how many times `enqueueJob()` is called, `processJobs()` will be scheduled at most once.

Reasoning:

1. The first `enqueueJob()` sees `isScheduled === false`, flips it to true, and schedules one microtask.
2. Every later `enqueueJob()` in the same turn hits `if (isScheduled) return` and schedules nothing.
3. Only after the microtask flush completes does `processJobs()` set `isScheduled = false` again.

So: one turn →one microtask flush →all jobs drained.

Timeline: why the tests produce those arrays

The scheduler tests are easiest to understand as timelines.

Jobs run after synchronous code (expects `[4, 1, 2, 3]`)

1. Sync: enqueue jobs `1,2,3`. Scheduler schedules one microtask flush.
2. Sync: push `4` directly into the array.
3. Microtask: scheduler flush runs and executes `1,2,3`.

So the sync side-effect (`4`) happens before the microtask side-effects.

Asynchronous jobs enqueue more jobs (expects `[1, 3, 2]`) This is the "async jobs dont́ block" rule.

1. Microtask flush starts.
2. Job A runs: pushes `1`, then hits `await Promise.resolve()` and yields.
3. Scheduler continues the flush loop and runs Job B (sync): pushes `3`.
4. Later microtask: Job A continuation runs and pushes `2`.

So `3` appears between `1` and `2`.
Reference: MDN: `Promise` (microtask semantics).
Tests in `src/__tests__/scheduler.test.ts` cover:

- jobs run after `nextTick`
- jobs run in order
- jobs run after synchronous code

10.3 The runtime lane scheduler (`runtime/scheduler.ts`)

The runtime scheduler is built for reactive workloads and larger apps. Instead of a single FIFO queue, it manages multiple priority lanes:

- `sync`
- `input`
- `default`
- `transition`
- `idle`

This scheduler is deterministic by design. It uses stable tie-breakers, explicit budgets, and a promotion system (aging + deadlines) so low-priority work eventually runs.

This section is grounded directly in `runtime/scheduler.ts` and `runtime/__tests__/scheduler.test.ts`.

10.3.1 Creating a scheduler

The entry points:

- `createScheduler(options)`
- `createRequestFlush(strategy)` for how flush is triggered
- `createBrowserScheduler(...)` convenience wrapper

10.3.2 Budgets and time slicing

A scheduler flush is run with a budget (in milliseconds). The flush loop:

- picks the next task
- runs it
- checks elapsed time
- if budget is exceeded and work remains, schedules another flush

This behavior is tested in `runtime/__tests__/scheduler.test.ts`:

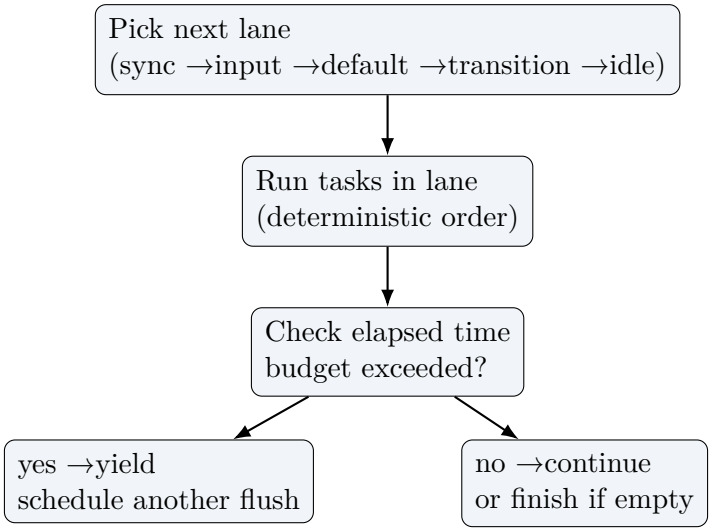
- `budget enforcement: flush splits work across multiple flush calls when budget is tight`
- `setFrameBudget caps the budget used by scheduled flushes`

10.3.3 Lane priority and deterministic ordering

Lane priority is declared in `LANE_PRIORITY` and selection order is:

```
sync > input > default > transition > idle
```

Diagram: lane selection and budgeted flushing



Within a lane, tasks are sorted with deterministic tie-breakers:

1. earlier deadline
2. earlier timestamp
3. lower task id

Test: `orders tasks within a lane by explicit deadline (then timestamp/id)`.
In code, the deterministic ordering is implemented by sorting each lane queue using:

```
(a.deadline - b.deadline) || (a.timestamp - b.timestamp) || (a.id - b.id)
```

The final tie-breaker (`id`) is the determinism anchor: it makes equal-time scheduling stable.

10.3.4 Aging and promotion (starvation prevention)

To prevent starvation, the scheduler promotes work that is overdue or old.
The implementation:

- defines `AGING_MS = 250`
- in `pickNextTask()`, checks queue heads in weaker lanes
- if a head task is overdue (deadline reached) or aged, it is promoted one lane upward

Tests:

- `ages/pushes starving tasks upwards (idle -> transition)`
- `promotes a task when its deadline is reached (even if it is not old enough)`
- `starvation prevention: repeated promotions eventually let idle work run even under continuous input load`

This is the scheduler equivalent of "don't let background work get stuck forever".

10.3.5 withBudget(): integrating user work

The scheduler also offers `withBudget(budgetMs, fn)`. It executes user code and, if that work consumes the budget while tasks remain, schedules continuation.

Test: `withBudget schedules continuation when user code exhausts budget and tasks remain`.

10.4 How these schedulers relate to the book

In Part II and Part IV we saw that the VDOM layer uses `enqueueJob` to schedule lifecycle hooks.

In later parts of the book (signals, blocks, hydration), we'll lean more on the runtime scheduler. The runtime lane model is a foundation for:

- fair scheduling of effects
- batching updates
- making performance tunable (frame budgets, flush strategy)

10.5 Takeaways

- `src/scheduler.ts` is tiny but intentional: microtask batching + safe error logging.
- `nextTick()` is a boundary for tests and integration, but does not await async job completion.
- `runtime/scheduler.ts` adds lanes, deadlines, aging, and budgets for scalable reactive workloads.

10.6 Walking the tests: what they prove

You can treat these as the scheduler's executable specification.

10.6.1 VDOM scheduler tests (`src/__tests__/scheduler.test.ts`)

Enqueued jobs run after nextTick Proves `enqueueJob()` is deferred and does not run synchronously.

Enqueued jobs run in order Proves FIFO execution order in `processJobs()`.

Jobs run after synchronous code Proves microtasks run after the current synchronous call stack.

Asynchronous jobs enqueue more jobs Proves the "async doesn't block" rule: a job that yields allows later jobs to run, producing `[1, 3, 2]`.

10.6.2 Runtime scheduler tests (`runtime/__tests__/scheduler.test.ts`)

The runtime scheduler suite uses a manual clock (`now()`) and a manual flush trigger (`requestFlush(cb)` pushes `cb` into an array). That technique eliminates timing flakiness and makes time-based scheduling deterministic.

Key tests and the guarantees they enforce:

runs tasks by lane priority Proves lane selection order and that `sync` runs immediately.

budget enforcement / setFrameBudget Proves time slicing across multiple flush calls when the budget is tight.

ages/pushes starving tasks upwards Proves aging-based promotion (`AGING_MS = 250`).

promotes a task when its deadline is reached Proves deadline-based promotion even when a task isn't old.

withBudget schedules continuation Proves user code that consumes budget can trigger a continuation flush if tasks remain.

createRequestFlush(messageChannel/raf) Proves flush strategy selection when platform APIs exist.

10.7 Online resources

- MDN: `queueMicrotask()`
- MDN: JavaScript event loop (microtasks vs. macrotasks)
- WHATWG HTML: Microtask queue (spec)
- React (conceptual parallel): Queued updates and batching
- MDN (conceptual parallel): Cooperative background work

10.8 Pitfalls and debugging (senior notes)

This section is where real-world scheduler bugs live.

10.8.1 Mistaking `nextTick()` for "wait for all async"

`nextTick()` does not promise to await async jobs. It promises the scheduler had a chance to flush and start them.

If a lifecycle hook does:

```
async onMounted() {
  await fetch(...)
  this.updateState(...)
}
```

then `await nextTick()` only guarantees the hook started. It does *not* guarantee the `fetch` completed.

10.8.2 Microtask storms (starving input/paint)

If jobs keep enqueueing more jobs in a tight cycle, you can monopolize the microtask queue. In production this looks like:

- input lag
- delayed paints
- "the browser froze" reports

This is one reason the runtime scheduler exists: it introduces budgets and yielding.

10.8.3 Re-entrancy and why hooks are scheduled

The component lifecycle tests (`src/__tests__/component-lifecycle.test.ts`) enforce the contract:

- `mountDOM`/`destroyDOM` do structural work synchronously
- `onMounted`/`onUnmounted` run later via the scheduler

This keeps the renderer from entering "mount triggers patch triggers mount" spirals.

10.9 Exercises

1. **Order reasoning:** explain why the scheduler test expects `[4, 1, 2, 3]` in `Jobs run after synchronous code`. Write the timeline in terms of call stack vs microtasks.
2. **Async ordering:** explain why `Asynchronous jobs enqueue more jobs` yields `[1, 3, 2]`. What would have to change in the scheduler for it to yield `[1, 2, 3]` instead?
3. **Design exercise:** if you wanted a variant `nextTickAll()` that awaited async jobs too, what bookkeeping would you need? What new failure mode could that introduce (hint: one slow job blocks all hooks)?
4. **Lane intuition:** when would you choose `transition` lane over `input` lane? Give a UI example (typing vs expensive filtering).

10.10 Chapter summary

- The VDOM scheduler is intentionally tiny: FIFO microtask batching for lifecycle hooks.
- The key mechanism is the `isScheduled` latch, collapsing many enqueues into one flush.
- `nextTick()` is a useful flush boundary for tests, but it is not an "await all async" primitive.
- The runtime scheduler adds lanes, deadlines, budgets, and starvation prevention for large reactive workloads.

Chapter 11

Slots and Content Projection

Components are most reusable when they can accept external structure. In Relax, *slots* are the mechanism for content projection.

What problem are slots solving? Without slots, a component can only vary by *data* (props/state). But many UI building-blocks need to vary by *structure*. Examples:

- **A Card** needs a customizable header/body/footer.
- **A TableRow** needs a caller-provided cell.
- **A Layout** needs to accept "whatever children" and place them inside a container.

In other frameworks, you might see this as: Web Components `<slot>`, React children, or Vue slots.

Relax takes a deliberately small approach: model the slot as a *temporary marker VNode*, then erase it with a deterministic tree rewrite. This gives us content projection without making mounting or patching "slot-aware".

Tiny contract (keep this in your head)

- **Input**: A component renders a VNode tree that may contain SLOT markers (created by `hSlot()`).
- **Rewrite**: If the component received external children, `fillSlots()` rewrites the first slot position(s) into fragments containing chosen content.
- **Output**: After rewriting, the tree contains only normal VNodes (elements, fragments, text, components). No slots remain.

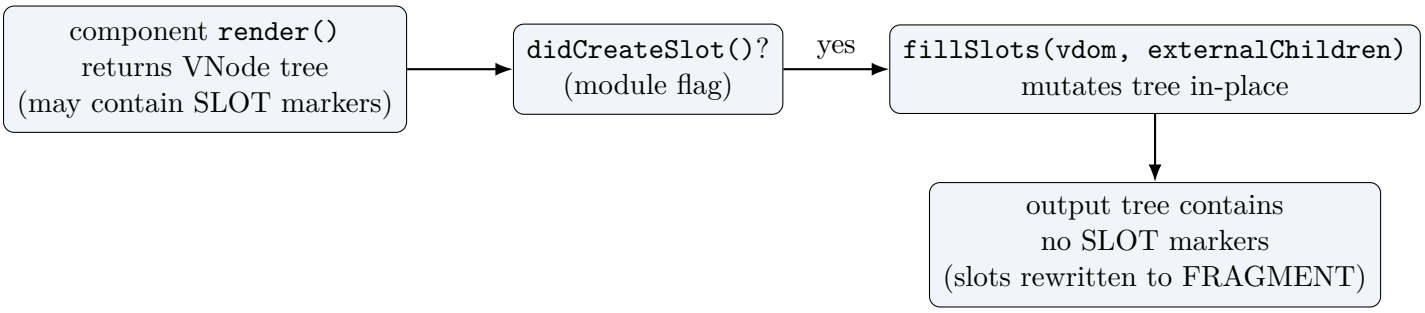
Relax's slot model is intentionally minimal:

- there is exactly one slot "kind" (no named slots)
- there is no runtime slot registry
- slots are compiled away into normal VDOM shape before mount/patch does serious work

That constraint keeps the renderer small and makes the behavior easy to specify with tests.

11.1 Diagram: component render →slot rewrite →normal VDOM

Slots are easiest to understand as a short rewrite pass that happens *between* component render and mounting/patching:



After this rewrite, slots disappear as a concept: mount/patch just see fragments and children arrays.

Slots in the VDOM layer are intentionally small and test-driven:

- VNode marker: `DOM_TYPES.SLOT` built by `hSlot()` in `src/h.ts`
- Slot filling algorithm: `fillSlots(vdom, children)` in `src/slots.ts`
- Specs: `src/__tests__/slots.test.ts` and `src/__tests__/component-slots.test.ts`

11.2 Slot VNodes: `hSlot` and the SLOT marker

A slot is represented as a VNode whose `type` is `DOM_TYPES.SLOT`.

In `src/h.ts`:

- `DOM_TYPES` includes `SLOT: 'slot'`.
- `hSlot(children = [])` returns `{ type: SLOT, children }`.
- `hSlot` also sets a module-level flag `hSlotCalled = true`.

That flag is read via `didCreateSlot()` and reset via `resetDidCreateSlot()`.

11.2.1 Why track `didCreateSlot`?

Relax uses this flag as a cheap "did the render tree contain any slots?" signal. The component renderer (`src/component.ts`) uses it:

- if a slot was created during `render()`, it calls `fillSlots(vdom, externalChildren)`
- then it resets the flag

This avoids doing a full traversal to *search* for slots when there are none.

How the optimization is wired (`src/component.ts`) The component render pipeline is:

1. call user `render()` to produce a VNode tree
2. if any `hSlot()` call happened while rendering (module flag), run `fillSlots(vdom, externalChildren)`
3. reset the module flag so the next render starts clean

This is literally the code:

```
if (didCreateSlot()) {
  fillSlots(vdom, this.#children)
  resetDidCreateSlot()
}
```

11.2.2 Authoring from scratch: what the user writes vs. what Relax sees

It helps to keep two mental views:

- **Author view**: you "render a slot" and "pass children".
- **Runtime view**: you render a placeholder VNode of type `SLOT`, then run a rewrite that replaces it.

Here's a simple author-level example:

```
// A component with a slot position
const Card = defineComponent({
  render() {
    return h('section', { class: 'card' }, [
      h('header', {}, [h('h2', {}, ['Title'])]),
      h('main', {}, [hSlot([h('em', {}, ['Default␣body'])])]),
    ])
  },
})

// Call site provides external children
const app = h(Card, {}, [h('p', {}, ['Hello␣from␣the␣outside'])])
mountDOM(app, document.body)

// After slot rewrite, patch/mount only sees normal children under <main>
// (slot marker is gone)
```

Even if you never think about slots again, this explains why they're cheap: the renderer doesn't need special mount logic.

So slot filling is a *conditional rewrite pass* executed only when a slot marker was created.

The tests explicitly check this optimization:

- `when the component has slots, the "fillSlot()" function is called`
- `when the component has no slots, the "fillSlot()" function is not called`

(See `src/__tests__/component-slots.test.ts`).

11.3 fillSlots(): projecting external children into a component

The slot filling algorithm lives in `src/slots.ts`:

`fillSlots(vdom, children)`

At a high level:

- It traverses a VDOM tree.
- It finds SLOT nodes.
- It replaces a SLOT node with a FRAGMENT node containing either:
  - external content (children passed into the component), or
  - default slot content (children attached to the SLOT marker).

11.3.1 Contract

- Input: a rendered VDOM subtree `vdom`, and an array of `children` coming from the component call-site.
- Output: it mutates the tree in-place.
- Post-condition: every SLOT marker encountered is rewritten into a FRAGMENT node.

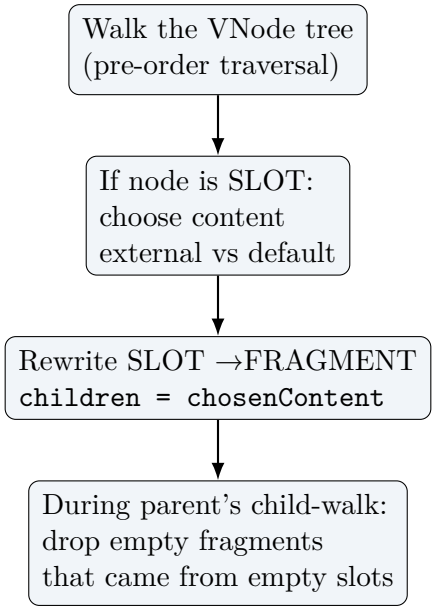
11.3.2 A more precise contract: invariants you can rely on

If you're debugging a slots issue, hand-wavy descriptions aren't enough. The tests assume these invariants:

- **No slot markers survive**: after the rewrite, `DOM_TYPES.SLOT` must not appear in the tree that gets mounted/patched.
- **One component = one filling context**: a component only fills slots within the VDOM tree returned by its own `render()`.
- **External children are late-bound per render**: on every component render, the caller-provided children array is considered "the current external content".
- **Mount/patch remain slot-agnostic**: any observable behavior must reduce to normal fragment/child reconciliation.

This design is why slots are "easy" in Relax: they are implemented as a deterministic pre-processing pass.

Diagram: the traversal + rewrite policy



Why FRAGMENT? The renderer needs to splice *zero, one, or many* children into a single position in the parent. In Relax's VDOM model, fragments are the standard encoding for "a list of children with no wrapping element". By rewriting slots to fragments, everything downstream (mounting, patching, diffing) can treat the result as normal VDOM.

11.3.3 Choosing between default and external content

When `fillSlots` sees `vdom.type === SLOT`, it computes:

- `defaultContent` from `vdom.children`
- `externalContent` from the passed `children`

Then it uses a simple rule:

> If external content is provided, it wins. Otherwise the default content is used.

In code, this is:

```
const content = externalContent.length > 0 ? externalContent : defaultContent
```

This is exactly what `src/__tests__/slots.test.ts` asserts:

- `set default content if no external content`
- `set the external content if provided`

11.3.4 Slots with no content are removed

If a slot has neither default nor external content, Relax removes it.

Implementation detail:

- A SLOT node is converted into a FRAGMENT.
- If the chosen content is empty, `vdom.children = []`.
- During the parent traversal pass, empty fragments that were slots are dropped.

Test: `remove the slot if no content or default content`.

A subtle implementation detail extttfillSlots() itself doesn't have access to a parent pointer. So it can't "remove the slot node" directly. Instead it uses a two-step approach:

1. rewrite SLOT →empty FRAGMENT (`children = []`)
2. during the parent's child-walk pass, drop empty fragments that were produced by empty slots

That design keeps the algorithm simple and makes behavior local to the traversal.

11.3.5 A subtle but crucial point: `children` is not "DOM children"

In Relax, component call-sites pass *VNode children*, not DOM nodes. That means "slot content" participates in the same rules we've already learned:

- strings/numbers passed as children are normalized into TEXT VNodes (by `h()` / child mapping)
- arrays are structurally meaningful (fragments flatten in some places; slot-default content has a special shape rule)
- children are diffed like any other VNode list after slot filling

If you come from Web Components: HTML `<slot>` projects "real DOM". Relax is closer to "project a subtree description". Conceptual reference: MDN `<slot>`.

11.3.6 A code-walk of `fillSlots` (in plain JavaScript)

The real implementation is in `src/slots.ts`, but this version captures the exact intent and shape. The key is: **slot rewrite is mutation**, not allocation of a second tree.

```
function fillSlots(vdom, externalChildren) {
  if (!vdom) return

  // 1) Slot node found: rewrite it into a fragment
  if (vdom.type === DOM_TYPES.SLOT) {
    const defaultContent = Array.isArray(vdom.children) ? vdom.children : []
    const passed = Array.isArray(externalChildren) ? externalChildren : []
    const content = passed.length > 0 ? passed : defaultContent

    vdom.type = DOM_TYPES.FRAGMENT
```

```
    // Empty slot => become empty fragment; parent traversal later drops it
    if (content.length === 0) {
      vdom.children = []
      return
    }

    // Shape rule: hSlot([defaultContent]) means ‘content‘ is [[...]]
    const first = content[0]
    if (Array.isArray(first) && content.length === 1) {
      vdom.children = content
    } else {
      vdom.children = [content]
    }

    return
  }

  // 2) Traverse children, but treat components as opaque boundaries
  if (Array.isArray(vdom.children) && vdom.type !== DOM_TYPES.COMPONENT) {
    const next = []
    for (const child of vdom.children) {
      fillSlots(child, externalChildren)

      // Optional cleanup: drop empty fragments produced by empty slots
      if (child && child.type === DOM_TYPES.FRAGMENT && (!child.children || child.children
          .length === 0)) {
        continue
      }
      next.push(child)
    }
    vdom.children = next
  }
}
```

If you've implemented compilers, this should feel familiar: it's a single-pass rewrite with a few invariants.

Important: the real implementation differs slightly from the teaching version In the actual `src/slots.ts`, external content is set as:

> fragment children = `content`

not `[content]`. This matters for patching. If you accidentally wrap external content in another array, you'd introduce an extra nesting level, which would make the fragment children shape wrong.

Interpretation: extbfdefault content has a special container shape; external content is already a list.

11.3.7 Shape detail: why `hSlot([defaultContent])` is special

There's a slightly odd but intentional shape rule documented in `src/slots.ts`:

> `hSlot([defaultContent])` passes an array whose first element is itself an array.

So default slot content is often authored like:

```
hSlot([ h('span', {}, ['Hello']) ])
```

In this case the algorithm sees: "content is an array with a single element, and that element is also an array". It preserves that container shape so that:
extbfa slot expands to exactly one fragment child in the parent, matching how the tests assert the resulting tree:

```
expect(vdom.children).toEqual([hFragment(defaultContent)])
```

11.4 Why fillSlots skips component children during traversal

The traversal logic in `fillSlots` explicitly ignores children whose type is `COMPONENT`.

This is important. Consider:

- You are filling a slot for component `A`.
- Inside `A`'s subtree there might be a nested component `B` that also uses slots.

Relax wants **each component to fill its own slots** using its own external content. If the parent traversal reached into component children, it could fill the wrong slots with the wrong content.

Another way to say this: `fillSlots` treats `COMPONENT` vnodes as *opaque*. That matches the rest of the renderer: components are patched/mounted as black boxes, and only the component instance is responsible for producing/maintaining its internal subtree.

This is a major boundary rule:

> Slot rewriting is done per-component and must not cross component vnode boundaries.

11.4.1 Why this boundary matters in real apps

If the parent traversal rewrote nested component internals, you'd get extremely confusing coupling:

- A parent could accidentally fill a child component's slot with the wrong content.
- A refactor that wraps content in an extra component would silently change slot behavior.
- Slot filling would become order-dependent across renders (hard to test, hard to reason about).

By treating components as tuples of `(props, children)` and letting each component own its own slot rewrite, Relax preserves a clean mental model: **slots are part of the component API surface, not the global tree.**

It also aligns with how the renderer treats components elsewhere: components are patched/-mounted as black boxes whose internal VDOM is managed by the component instance.

11.4.2 From-scratch example: why crossing the boundary would be a bug

Imagine this:

```
const Inner = defineComponent({
  render() {
    return h('div', {}, [hSlot([h('em', {}, ['Inner default'])])])
  },
})

const Outer = defineComponent({
  render() {
    return h('section', {}, [
      // Outer has a slot too
      hSlot([h('span', {}, ['Outer default'])]),

      // And it renders Inner
      h(Inner, {}, []),
    ])
  },
})

// Call site passes ONE set of children to Outer
mountDOM(h(Outer, {}, [h('b', {}, ['Projected into Outer'])]), document.body)
```

If `fillSlots` descended into the `Inner` component vnode, it could (incorrectly) fill Inner's slot with Outer's external children. That would mean Inner's content depends on who rendered it (bad encapsulation).

Relax avoids this by making `COMPONENT` a hard boundary during traversal.

This behavior is tested in `src/__tests__/slots.test.ts`:

- `ignores slots whose parent is a component (they are handled by the component)`

11.5 Slots in real components (`component-slots.test.ts`)

The integration tests show what authoring looks like.

11.5.1 Basic projection

`can have a slot where external content can be inserted`:

- Component renders `<div>{slot}</div>`.
- Call site provides `<span>Hello</span>`.
- The mounted DOM contains the span.

11.5.2 Default content

`when no external content is provided, the default content is used`.

This is the standard "slot fallback" semantics.

11.5.3 Conditional slots

`conditionally rendered slots` verifies that slots can appear/disappear depending on component state, and the patcher will reconcile accordingly.

This is where slots meet the patching story from Chapter 8: slot output is just fragments and normal children after filling.

11.5.4 Updating slot content between renders

`slot content updated between renders` has a container component that alternates between:

- a single `<span>Hello</span>`
- or two children `<p>World</p><span>!</span>`

The important point: slot external content is treated like regular children arrays and diffed normally after filling.

Practically:

- the container component re-renders and passes a different external children array
- the slotted component re-renders, sees a slot marker, and re-runs `fillSlots`
- after that rewrite, the patcher sees ordinary FRAGMENT/ELEMENT/TEXT children and reconciles them

11.5.5 Nested slots inside lists

The last integration scenario in `component-slots.test.ts` is important because it looks like "normal app code": build a list, render repeated components, and project a different child into each one.

Mentally, it's just two loops:

- The parent maps `items` to `TableRow` component vnodes.
- Each `TableRow` render tree contains a slot marker in a `<td>`.

In author-level code (simplified):

```
const TableRow = defineComponent({
  render() {
    return h('tr', {}, [
      h('td', {}, [hSlot([h('em', {}, ['(empty)'])])]),
    ])
  },
})

const Table = defineComponent({
  render() {
    const titles = ['One', 'Two']
    return h('table', {}, [
      h('tbody', {}, titles.map((t) =>
        // Each row receives different external children.
```

```
      h(TableRow, {}, [t])
    )),
  ])
 },
})
```

The slot rewrite story stays exactly the same: each `TableRow` instance stores its own external children (here: `['One']` or `['Two']`) and fills the slot while rendering.

Senior rule of thumb:

> If you can explain repeated slots-in-lists, you understand slots. Everything else is a smaller case.

11.6 Walking the integration tests in slow motion (event-loop + render pipeline)

Slots can feel magical until you align them with the component pipeline from earlier chapters. At a high level, mounting a component with slot children looks like this:

1. **Mount begins** at the parent vnode.
2. When the renderer hits a `COMPONENT` vnode:
   (a) create the component instance
   (b) store external children array on the instance (conceptually: `this.#children`)
   (c) call `render()` to get an internal subtree
   (d) if `didCreateSlot()` was set by `hSlot()`, run `fillSlots(subtree, this.#children)`
   (e) mount the rewritten subtree to DOM

In other words, slot filling is part of "turn render output into mountable VDOM". The patcher never sees a slot marker.

This is why the spy tests in `component-slots.test.ts` are important: they confirm the optimization boundary is hooked up correctly.

`slot in a nested component` uses a table row component with a slot in a `<td>`. The parent maps titles `['One','Two']` into multiple `TableRow` instances.

This shows that slot content can be:

- a raw string child (which becomes a TEXT VNode via `h()` normalization)
- repeated many times across a list of nested components

11.7 Takeaways

- Slots are recorded as a VNode marker produced by `hSlot()`.
- Slot filling is a tree rewrite: SLOT → FRAGMENT with chosen content.
- External content wins over default content.
- Empty slots are removed by dropping empty fragments.
- Each component fills its own slots; parent traversal does not cross component boundaries.

11.8 Walking the tests: what they prove

Slots are specified in two layers: a pure tree-rewrite unit suite and a DOM-level integration suite.

11.8.1 Unit semantics (`src/__tests__/slots.test.ts`)

remove the slot if no content or default content Proves the empty-slot removal behavior: a slot with no default and no external content is eliminated (via empty-fragment dropping).

set default content if no external content Proves fallback behavior and the default-content shape rule. The assertion expects `[hFragment(defaultContent)]`.

set the external content if provided Proves the precedence rule: external content overrides default content.

ignores slots whose parent is a component Proves the traversal boundary rule: `fillSlots` must not rewrite nested component internals.

11.8.2 Integration semantics (`src/__tests__/component-slots.test.ts`)

can have a slot where external content can be inserted Proves end-to-end projection into mounted DOM.

default content is used Proves fallback behavior at DOM level.

slot is empty Proves empty slots become empty output (no wrapper nodes appear).

conditionally rendered slots Proves slots can appear/disappear and patch correctly when state changes.

slot content updated between renders Proves that changing the external children array results in normal DOM reconciliation after slot rewrite.

slot in a nested component Proves nested component usage with lists and primitive children (strings become text nodes via `h()` normalization).

fillSlots spy tests Proves the `didCreateSlot()` optimization works: slot rewrite only runs when `hSlot()` was actually used.

11.9 Online resources

- Web Components: MDN `<slot>` (conceptual comparison)
- Vue: Slots and content projection
- React: Passing JSX as children
- SolidJS (conceptual): Children utilities

11.10 Common pitfalls (senior notes)

- **Mistaking slots for "render props"**: Relax slots are structural children, not callbacks. If you need code execution with parameters, pass functions as props.
- **Forgetting the rewrite step exists**: by the time `mountDOM/patchDOM` runs, there is no slot node. Debug by inspecting the post-rewrite VDOM.
- **Component-boundary confusion**: if your slot isn't filling, check where you placed `hSlot()` – it must be inside the component that receives the children.
- **Default content shape surprises**: remember that `hSlot([defaultContent])` uses a container shape that becomes a single fragment child.

If you internalize one idea: **Relax slots are a simple, deterministic rewrite into fragments**. Everything else follows.

11.11 Performance notes (why this is fast in practice)

Two design choices keep this lightweight:

- **Conditional traversal**: the `didCreateSlot()` flag lets the component skip traversal when there are no slots.
- **In-place rewrite**: `fillSlots` mutates existing vnode objects. No cloning of the full tree.

That matters because slot filling runs on every render where slots are present. This is the same kind of micro-optimization you see in production UI frameworks: avoid whole-tree work unless a feature is actually used.

Reference (conceptual): React render/commit mental model.

11.12 Exercises

1. Modify the `fillSlots` teaching implementation so it matches `src/slots.ts` exactly—especially the external-content shape.
2. Write a small component pair `Outer/Inner` (like the boundary example) and predict what would go wrong if traversal crossed into components.
3. Explain (in 3-5 sentences) why "slots compiled away into fragments" simplifies the patcher.

11.13 Chapter summary

- Relax implements slots as a VNode marker (`DOM_TYPES.SLOT`) created by `hSlot()`.
- Slot behavior is a deterministic rewrite pass: SLOT is rewritten to FRAGMENT containing either external or default content.
- Component vnodes are traversal boundaries: each component owns slot filling for its own render output.

- After filling, mount and patch operate on normal VDOM only—no special slot logic downstream.

Chapter 12

HRBR Signals: a Small Deterministic Reactive Graph

Part III begins the HRBR runtime. Unlike the VDOM runtime (`src/*`), HRBR is a "compiled fast path" whose job is to:

- track fine-grained dependencies (signals)
- schedule updates deterministically (scheduler lanes)
- update DOM directly in blocks (later chapters)

This chapter focuses on the reactive graph implementation in `runtime/signals.ts`.

12.1 Why signals exist (and why HRBR needs them)

Up to this point, Relax updates UI via a Virtual DOM pipeline: you re-run `render()`, diff children, then patch the DOM. That model is flexible and easy to reason about, but it can do more work than necessary when updates are tiny and frequent.

Signals exist to make a different trade-off:

> Track *exactly* which computations depend on a value, and when that value changes, re-run only those computations.

This is the core HRBR idea: the compiler can translate template expressions into tiny computations (effects/memos) that update very specific DOM nodes. So instead of reconciling a large subtree, a signal write can result in "update this text node".

Compare the philosophies:

- **VDOM:** change state →rerender →reconcile structure.
- **Signals:** change signal →schedule dependent computations →update exactly where read.

For external context (not Relax-specific), see: SolidJS signals (fine-grained model) and React render/commit (VDOM model).

12.2 Public API: the contract we have to satisfy

The file `runtime/index.ts` exports these symbols from `runtime/signals.ts`:

- `createSignal(initial) -> [read, write]`
- `createEffect(fn, options?)  -> { dispose() }`
- `createMemo(fn, opts?)  -> readMemo`
- `batch(fn) -> result`
- `untrack(fn) -> result`

The executable specification lives in:

- `runtime/__tests__/signals.test.ts`
- `runtime/__tests__/signals.determinism.test.ts`
- `runtime/__tests__/signals.fuzz.test.ts`

12.3 Mental model

You can read `runtime/signals.ts` as a deliberately small version of a Solid-like graph:

- A **signal** is a node with a current value and a list of observers.
- An **effect** is a computation that re-runs when its dependencies (signals/memos) change.
- A **memo** is a computation that behaves like a derived signal.

Two design constraints keep showing up in the implementation and tests:

1. **Determinism**: observer notification runs in stable creation order.
2. **Leak-safety**: disposing a computation must detach it from all sources.

There's a third, more pragmatic constraint you'll see in the code:

- **Low ceremony**: everything is implemented with plain objects and a few intrusive linked lists.

This is not a "fully-featured" reactive system; it's a compact kernel meant to be embedded into compiled blocks.

12.4 The tiny contract (keep this in your head)

Signals are small, so their contract can be crisp:

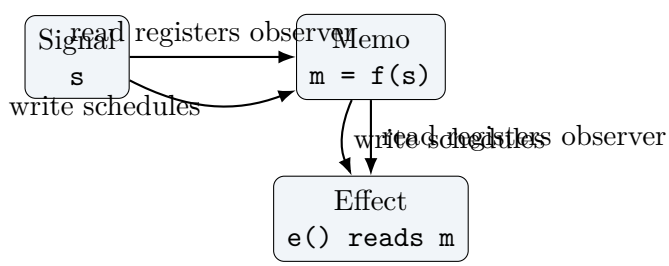
- **Read** returns a value and optionally registers a dependency edge.
- **Write** updates the value and schedules observers *in deterministic order*.
- **Effect** runs once initially, then re-runs when its dependencies change.
- **Memo** caches a derived value and notifies dependents when it changes.
- **batch(fn)** coalesces multiple writes into a single downstream flush.
- **untrack(fn)** temporarily disables dependency capture.

And the failure modes we must avoid (the tests are built around these):

- **Nondeterminism**: the observer order changes across runs.
- **Leaks**: disposed computations still run after future writes.
- **Re-entrancy recursion**: an effect writes to what it reads and causes stack overflow.
- **Stale dependencies**: conditional reads don't update subscriptions.

12.5 Diagram: signals graph (sources →computations →derived values)

The runtime is easiest to visualize as a graph where reads register edges and writes schedule computations:



The rest of this chapter explains how Relax keeps this graph deterministic and leak-safe with intrusive linked lists.

12.6 Core data structures: intrusive lists for stable order

In `runtime/signals.ts` you will see a set of internal types:

- `SignalNode<T>` holds `value` and an intrusive doubly-linked list of observers: `observersHead/observersTail`.
- `Computation` represents an effect or memo computation.

- `ObserverLink` is the node stored in the signal's observer list.
- `SourceLink` is a reverse edge stored on the computation so it can detach from sources on re-run/dispose.

The key technique: when a computation reads a signal, `addObserver(node, currentComputation)` inserts the computation into the signal's observer list *sorted by computation id*.

Computation ids come from a global counter:

```
let nextId = 0

comp._id = nextId++
```

Because ids are monotonically increasing, the observer list is stable and deterministic by construction.

12.6.1 Why two linked lists (observer list + source list)?

Each dependency edge is represented twice:

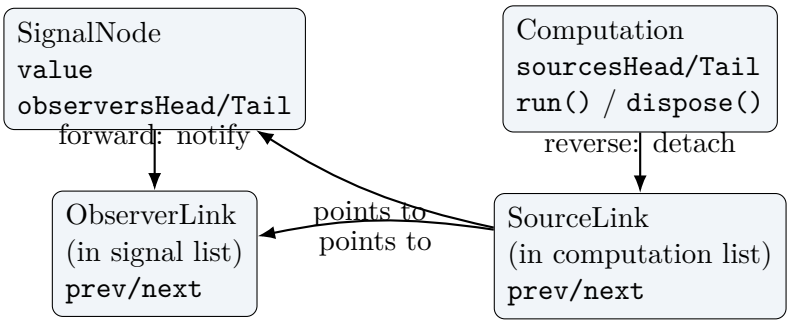
- a forward link in the signal's `observersHead/observersTail` list (who should be scheduled when the signal changes)
- a reverse link in the computation's `sourcesHead/sourcesTail` list (what should be detached before re-run/dispose)

This dual representation is what makes "conditional dependencies" safe: each run starts by detaching previous sources and then re-registering based on the reads that actually occur.

12.6.2 Diagram: forward observers vs. reverse sources

This is the part that makes signals feel like a real runtime (not a toy): every edge is represented twice.

- The **forward edge** lives on the signal: "who should I notify when I change?"
- The **reverse edge** lives on the computation: "what did I subscribe to last run, so I can detach?"



In `runtime/signals.ts`, this is the practical outcome:

- `addObserver(node, comp)` inserts an `ObserverLink` into `node.observers...`
- and it appends a `SourceLink` into `comp.sources...` that points back to that observer link
- `unlinkAllSources(comp)` walks `comp.sources...` and removes each referenced observer link from its signal

Senior takeaway:

> If you don't keep the reverse list, you can't implement conditional dependencies without leaks.

12.7 Dependency capture: `currentComputation`

Reactive systems need a way to know "who is currently running".

This file uses a single global pointer:

```
let currentComputation: Computation | null
```

12.7.1 Signal reads

In `createSignal()`, the reader is:

- return `node.value`
- if `currentComputation != null`, also call `addObserver(node, currentComputation)`

That means dependencies are collected by running the effect/memo function and letting reads "register".

From scratch: the global pointer trick

The `currentComputation` pointer is the entire core of dependency tracking. When a computation is running, it sets this global. Signal reads check it and attach the dependency edge.

Here's a very small teaching implementation in plain JavaScript (not the real Relax code):

```
let currentComputation = null

function createSignal(initial) {
  const node = { value: initial, observers: [] }

  function read() {
    if (currentComputation) node.observers.push(currentComputation)
    return node.value
  }

  function write(next) {
    const nextValue = typeof next === 'function' ? next(node.value) : next
    if (Object.is(nextValue, node.value)) return
    node.value = nextValue

    for (const comp of node.observers) {
      comp.run()
```

```
    }
  }

  return [read, write]
}

function createEffect(fn) {
  const comp = {
    run() {
      const prev = currentComputation
      currentComputation = comp
      try {
        fn()
      } finally {
        currentComputation = prev
      }
    },
  }
  comp.run()
  return { dispose() {} }
}
```

Relax's real implementation adds the hard parts:

- leak-safety via a reverse list (so it can detach old edges)
- deterministic order via monotonically increasing ids + sorted insertion
- correct batching + re-entrancy via a pending set and deterministic flush

No dedupe (by design) You may notice `addObserver()` explicitly does not deduplicate observers. This is safe because every run begins with `unlinkAllSources(comp)`, which removes all prior edges. So duplicates can only occur within a single run, and a well-behaved effect/memo typically reads each signal at most once.

12.7.2 `untrack(fn)`

`untrack(fn)` temporarily sets `currentComputation = null` while executing `fn`.

Spec (`runtime/__tests__/signals.test.ts`):

- `untrack prevents dependency registration`

In other words: you can read a signal without subscribing.

12.8 Writes: schedule observers, preserve order

In `createSignal()`, the writer:

1. computes `nextValue` (supports "functional set" style)
2. bails out if `Object.is(nextValue, node.value)`
3. updates `node.value`

4. walks the observer list and schedules each observer

There's a subtle but important line in the loop:

> Capture `next` first, because running an observer can unsubscribe/resubscribe and mutate the list.

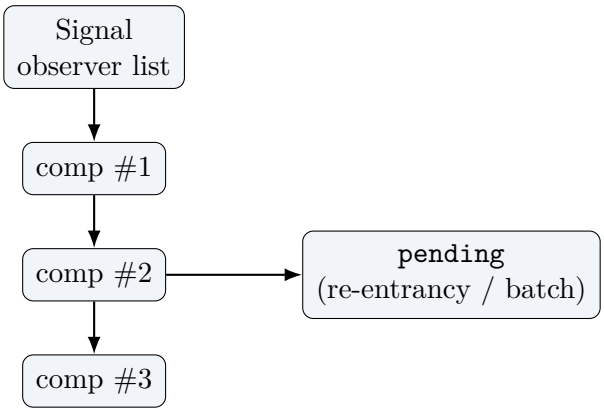
In code (comments preserved):

```
const next = cur.next
scheduleComputation(cur.comp)
cur = next
```

This ensures we traverse the original linked list in deterministic order.

Diagram: observer notification preserves creation order

Observer lists are kept in stable creation order (by computation id). A write walks that list and schedules computations without losing determinism:



The key implementation trick is the one shown in the code snippet above: capture `next` before scheduling so list mutation during computation runs doesn't break traversal.

What gets scheduled? When a signal is written, Relax schedules the dependent computations. Scheduling does not necessarily mean "run immediately" (see batching and lanes), but it always preserves the *creation order* contract.

12.8.1 A precise code-walk: `scheduleComputation` and the `pending` set

All the tricky correctness properties (re-entrancy, batching, lane scheduling) flow through one choke point: extttscheduleComputation(computation).

In the real implementation (`runtime/signals.ts`), there are four gates. Reading them as a ruleset helps:

1. **Disposed computations are inert.**
2. **Running computations don't re-enter.** If a computation schedules itself while running, it is queued in `pending` and flushed after the current stack unwinds.

3. **Lane-aware computations don't run inline.** If a computation has `_schedule`, scheduling is delegated to the runtime scheduler.
4. **Batching defers work.** If `isBatching > 0`, add to `pending` instead of running.

Here's the same logic in a teaching version (simplified JavaScript), preserving the ordering of checks:

```
function scheduleComputation(comp) {
  if (comp._disposed) return

  // Re-entrancy: don't recurse.
  if (comp._running) {
    pending.add(comp)
    return
  }

  // Lane scheduling: hand off to the scheduler.
  if (comp._schedule) {
    comp._schedule(comp)
    return
  }

  // Batching: defer until end of outer batch.
  if (isBatching > 0) {
    pending.add(comp)
    return
  }

  // Default: run inline.
  comp.run()
}
```

Why re-entrancy and batching share the same queue

At first glance, using one `pending` set for two distinct situations looks odd. It's actually a good simplification because both situations mean the same thing:

> We are not allowed to run this computation right now, but we must not lose the fact that it became dirty.

The only additional requirement is determinism. That's why `flushPendingIfNeeded()` sorts the set by `_id` before scheduling.

Timeline: the re-entrancy convergence test

Spec: `re-entrancy: effect that schedules itself via writes does not recurse infinitely`.

The effect is:

```
createEffect(() => {
  const v = s()
  seen.push(v)
  if (v < 3) setS(v + 1)
})
```

The important thing is not just that it ends at `3`, but that it does not recurse. The run looks like this:

1. Start run: comp becomes `_running = true`
2. Reads `s()` (dependency edge registered)
3. Calls `setS(v + 1)` which tries to schedule the same computation
4. Because it is `_running`, scheduling goes to `pending` (no re-entry)
5. End run: `_running = false`, then we flush `pending`

This repeats until the signal hits 3. That is exactly why the expected log is deterministic: `[0, 1, 2, 3]`.

Reference (conceptual background): Solid's reactivity avoids re-entrancy recursion via queuing and batching as well: Solid: execution order.

12.9 Effects: cleanup, re-run, disposal

12.9.1 Cleanup semantics

Effects may return an optional cleanup function:

```
createEffect(() => {
  ...
  return () => cleanup()
})
```

The runtime guarantees cleanup runs:

- before an effect re-runs
- when the effect is disposed

Spec: `effect cleanup runs before re-run and on dispose`.

In code, the cleanup is stored on the computation object as `comp.cleanup` and executed in a try/finally block both in `run()` and `dispose()`.

12.9.2 Detaching and resubscribing: `unlinkAllSources(comp)`

Before each run, the runtime calls `unlinkAllSources(comp)`. That function walks the computation's `sourcesHead` list and unlinks each corresponding `ObserverLink` from the signal's observer list.

This gives the common reactive invariant:

> An effect's dependencies are exactly the signals it read during its last run.

It also enables conditional dependencies (read signal A in one run, read signal B in another) without leaking observers.

12.9.3 Conditional dependencies in slow motion (the reason `unlinkAllSources` exists)

Conditional dependencies are the most common place reactive systems silently leak. The problem looks like this:

> An effect reads `a()` in one run, but reads `b()` in the next run. If you never detach old edges, the effect ends up subscribed to both forever.

Relax avoids this by making `unlinkAllSources(comp)` step 0 of every run. Then the effect/memo re-subscribes only to the signals it *actually reads* this time.

Here's a minimal example to build intuition:

```
const [useA, setUseA] = createSignal(true)
const [a, setA] = createSignal(0)
const [b, setB] = createSignal(0)

createEffect(() => {
  if (useA()) {
    console.log('A␣path:', a())
  } else {
    console.log('B␣path:', b())
  }
})
```

Now imagine this sequence:

1. Initial run: `useA() == true` so the effect reads `useA()` and `a()`.
2. Flip to `false`: the effect re-runs, but now it reads `useA()` and `b()`.

If the runtime does not detach, a later `setA(...)` would still schedule the effect even though it no longer reads `a()`. That is wasted work and (in large graphs) becomes a performance cliff.

What the implementation guarantees

This is the practical contract enforced by the forward+reverse lists:

- Before each run, unlink all previously tracked sources.
- During the run, every signal/memo read adds new source links.
- After the run, the computation is subscribed to exactly the set of reads that occurred.

That is why the dual list structure matters: the reverse list is what makes detaching cheap and precise.

Conceptual reference: Solid's explanation of fine-grained dependency tracking is very similar: Solid: how it works.

Leak-safety note extttunlinkAllSources() also deliberately nulls out fields on the link objects (with `@ts-expect-error` comments). That's not required for correctness, but it makes accidental retention less likely in this prototype implementation.

12.10 The tests as specs: what behavior is non-negotiable

If you want to understand signals fast, read the tests first. They encode the contract in a way that prevents "almost works" implementations.

12.10.1 Core semantics: `runtime/__tests__/signals.test.ts`

The suite asserts:

- dependency tracking (reads register; writes re-run)
- `Object.is` bail-out prevents no-op writes from causing work
- `untrack` disables edge registration
- `batch` is a flush boundary (including nested batching)
- cleanup is run before re-run and on dispose
- dispose fully detaches
- re-entrancy converges without recursive re-entry
- non-sync lanes schedule through the runtime scheduler (tests intercept `setTimeout`)
- memos behave like derived signals and notify dependents

A senior reading of the suite: what it forces in the implementation

The most important thing these tests enforce is that signals are not just a pub/sub primitive. They are a mini runtime with strict invariants:

- **Idempotent writes**: `Object.is` bail-out means no "useless work" on repeated values.
- **Precise tracking**: `untrack` demonstrates that dependency capture is opt-in per read, not an ambient global subscription.
- **Deterministic event ordering**: effect order must be stable even under batching.
- **Cleanup correctness**: cleanup must run before the next run, not after.

If you've ever debugged a real UI bug caused by nondeterministic ordering, you'll appreciate why determinism is treated as a contract, not an "implementation detail".

12.10.2 Determinism + leak-ish checks: `signals.determinism.test.ts`

The determinism suite checks a stronger property: *same seed + same operations produce identical logs across two fresh graphs.*

That forces the runtime to be stable in ways a single unit test won't catch. It also includes a practical disposal check: once an effect is disposed, later writes must not produce any new log entries for it.

12.10.3 Fuzz/property checks: `signals.fuzz.test.ts`

The fuzz test generates nested batches, sets, and disposals. The primary assertion is determinism (two runs produce identical logs). There is also a "smoke" property related to disposal: if effects were disposed, later mass-writes shouldn't explode log volume.

12.10.4 Dispose

Disposing an effect sets `_disposed` and detaches from all sources.

Specs:

- `disposed effects fully detach: further writes do not schedule work`
- leak-ish checks in `runtime/__tests__/signals.determinism.test.ts`

12.11 Re-entrancy: scheduling while running

In `scheduleComputation(computation)` there is an explicit re-entrancy rule:

- if the computation is `_running`, do not recursively call `run()`
- instead, add it to a `pending Set`

Then `flushPendingIfNeeded()` runs pending computations after the current run unwinds.

12.11.1 Two reasons computations are delayed

The same `pending` set is used for two distinct delay cases:

- **re-entrancy**: a computation schedules itself while it's already running
- **batching**: writes should not trigger intermediate runs until the batch boundary

Both cases end up in the same deterministic flush code path.

Spec: `re-entrancy: effect that schedules itself via writes does not recurse infinitely`.

The test constructs an effect that reads a signal and writes it again until a fixed point is reached. The expected (deterministic) log is:

```
[0, 1, 2, 3]
```

12.12 Batching: a flush boundary

`batch(fn)` increments `isBatching` and makes writes collect work into `pending`. When the outermost batch finishes, it flushes the pending set sorted by computation id.

Specs (`runtime/__tests__/signals.test.ts`):

- `batch groups multiple writes into a single effect re-run`
- `batch() is a flush boundary: nested batches still only produce one re-run after outer finishes`

- `batch() returns the callback result and preserves deterministic effect order after flush`

This is the first place you see HRBR's theme: *deterministic scheduling is part of the user-facing contract.*

Implementation detail: deterministic flush At batch end, pending computations are flushed by sorting on `_id`:

```
Array.from(pending).sort((a,b) => a._id - b._id)
```

This is why the tests can assert stable order like `['a:2', 'b:2']`.

12.13 Memos: derived signals with equality

`createMemo(fn, opts?)` creates a computation whose output is stored in a `SignalNode<T>`.

Important details:

- the memo lazily computes on first read (`if (!hasValue) comp.run()`)
- it supports an equality function `opts.equals` (defaults to `Object.is`)
- it notifies observers only when the memo value actually changes

Spec: `createMemo computes from signals and updates dependents`.

Memo invalidation path When the memo value changes, it walks `node.observersHead` and schedules each observer computation. So effects that read a memo are notified exactly like effects that read a base signal.

12.14 Lanes: effects can be scheduled via the runtime scheduler

`createEffect` accepts options:
`{ lane?: Lane, budgetMs?: number }`.

If `lane === 'sync'` (the default), the initial run executes immediately.

If `lane !== 'sync'`, `createEffect` creates a scheduler via `createScheduler()` (from `runtime/scheduler.ts`) and schedules runs into that lane.

12.14.1 How non-sync lanes are tested

In `runtime/__tests__/signals.test.ts`, the tests override `globalThis.setTimeout` to capture scheduled flush callbacks into an array. This makes scheduler-driven effects deterministic in unit tests:

- createEffect(lane: 'default') does *not* run immediately
- the test manually drains the flush queue to trigger the run

Specs:

- `createEffect( lane ) schedules initial run (non-sync)`
- `createEffect forwards budgetMs to scheduler.schedule`

The tests intercept `setTimeout` to make scheduler flushing deterministic in unit tests.

12.15 Determinism tests and fuzz tests

Determinism is explicitly tested at three levels:

- Unit test: `effects run deterministically (creation order)`
- Determinism suite: build a random-but-seeded graph and compare logs across fresh instances (`signals.determinism.test.ts`)
- Property/fuzz: randomly generate operations (set/batch/dispose) and ensure identical logs for the same seed (`signals.fuzz.test.ts`)

These tests are the reason the implementation invests in sorted observer lists and stable flush ordering.

12.16 Walking the tests: what they prove

This chapter is easiest to internalize by reading the tests as executable requirements.

12.16.1 Core semantics (`runtime/__tests__/signals.test.ts`)

tracks dependencies and re-runs Proves that reads register dependencies and writes schedule effects. Also proves the `Object.is` bail-out on repeated values.

untrack prevents dependency registration Proves that `untrack` temporarily disables dependency capture.

batch groups writes Proves that multiple writes inside a batch yield a single re-run.

nested batch is a flush boundary Proves `isBatching` is a counter and only the outermost batch flushes.

deterministic effect order Proves flush order is creation order (based on `_id`).

cleanup runs before re-run and on dispose Proves cleanup semantics for both re-run and disposal.

disposed effects fully detach Proves `unlinkAllSources` works: disposed computations aren't scheduled by later writes.

re-entrancy convergence Proves the pending-set mechanism prevents infinite recursion and produces a deterministic sequence.

lane scheduling + budgetMs forwarding Proves non-sync effects are scheduled through the runtime scheduler and that `budgetMs` is passed through.

memos Proves memo recomputation + dependent effect notification.

12.16.2 Determinism suite (`signals.determinism.test.ts`)

This suite tests the strongest property: *same seed + same operation sequence yields the same log across fresh runtime instances.*

It also includes a practical leak-ish check: after disposing effect 0, it mutates signals repeatedly and asserts there are no new log entries starting with `e0:`.

12.16.3 Fuzz/property tests (`signals.fuzz.test.ts`)

The fuzz suite generates nested batches, writes, and disposals. The primary assertion is determinism (two independent runs produce identical logs).

It also has a smoke property about disposal effectiveness: if effects were disposed, the final mass-write should not trigger an absurd number of logs (catching obvious detach bugs).

12.17 Online resources

These systems are common in UI runtimes; the references below help connect the dots without assuming the exact same API surface.

- SolidJS (conceptual baseline): Fine-grained reactivity
- Preact Signals (conceptual): Signals and effects
- React (contrast): Render/commit model vs. fine-grained graphs
- A good "reactivity under the hood" talk: SolidJS reactivity deep dive (video)
- Linked list background (for intrusive lists): Linked list

12.18 Takeaways

- Signals store intrusive observer lists, sorted by computation creation id.
- Effects detach and re-subscribe each run, enabling conditional dependencies safely.
- Batching and re-entrancy are implemented with a pending set plus deterministic flush ordering.
- Lanes integrate with the scheduler so work can be delayed and budgeted.

12.19 Exercises

1. In `runtime/signals.ts`, find the exact line in the signal write loop that captures `next` before scheduling. Explain why this matters when observers unsubscribe/resubscribe while running.
2. Build a conditional dependency like the `useA/a/b` example and write down exactly which writes should schedule the effect in each phase.
3. Using the tests as a guide, explain the difference between `isBatching` and `pending`: what does each represent, and why are both needed?
4. Explain why the `pending` set is a `Set` and not an array. What bug would an array allow in batch-heavy code?

12.20 Chapter summary

- Signals implement fine-grained dependency tracking via `currentComputation`: reads register edges.
- Determinism is a user-facing guarantee: observers are ordered by creation id, and pending flushes are sorted.
- Leak-safety comes from the dual representation of edges (forward observers + reverse sources) and aggressive detaching via `unlinkAllSources`.
- Batching and re-entrancy share the same mechanism: a `pending` set flushed after the current stack unwinds or after the outer batch ends.
- Effects can be lane-scheduled via the HRBR scheduler so the compiler can choose when work runs.

Chapter 13

HRBR Deterministic Scheduler: Lanes, Budgets, and Aging

Signals answer *what* should re-run. A scheduler answers *when* and in what order.

This chapter is the bridge from "reactivity is correct" to "reactivity feels fast". Correctness is table stakes. The hard part in UI systems is correctness *under time pressure*: you get 16ms for a 60Hz frame, and you don't control when the browser will deliver input.

If you're coming from React, the intuition you already have is that work can be prioritized and time-sliced. React's documentation explains the conceptual split between render and commit (and why keeping commits responsive matters): https://react.dev/learn/render-and-commit.

If you're coming from Solid, you already know the "signals + effects" world; this scheduler is the HRBR answer to: "when should effects flush, and how do we avoid long synchronous waves?" Background reading: https://www.solidjs.com/docs/latest#scheduling.

In HRBR, scheduling is part of the public contract. We want:

- deterministic ordering (same inputs, same execution order)
- configurable latency via lanes (input vs transition vs idle)
- budget enforcement (avoid long flushes that block the main thread)
- starvation prevention (idle work should still eventually run)

This chapter is grounded in:

- Implementation: `runtime/scheduler.ts`
- Specs: `runtime/__tests__/scheduler.test.ts`
- Devtools counters/events: `runtime/devtools.ts` (used by scheduler and signals)

13.1 The contract: schedules tasks, flushes deterministically

13.1.1 Why a scheduler exists (not just "because React has one")

Chapter 12 showed how signals can precisely compute *what* is dirty. However, once you have a list of dirty computations, you still need policy:

- **Latency policy**: should an update run now (input), soon (default), or later (idle)?
- **Budget policy**: how long can the runtime keep running tasks before yielding?
- **Determinism policy**: when multiple tasks are eligible, what is the stable tie-breaker?

The HRBR scheduler makes these policies explicit and testable.
The scheduler exports three main concepts (see `runtime/scheduler.ts`):

- `Lane: 'sync' | 'input' | 'default' | 'transition' | 'idle'`
- `ScheduledTask: { id, lane, priority, timestamp, deadline, budgetMs, run }`
- a factory: `createScheduler(options?) -> scheduler`

The returned scheduler exposes:

- `schedule(lane, run, budgetOrOptions?)`
- `flush({ budgetMs })`
- `hasPending()`
- `withBudget(budgetMs, fn)`
- `setFrameBudget(ms)` and `setDefaultBudget(ms)`

The tests treat these as the source of truth.

13.1.2 A concrete mental model: 5 queues + 1 deterministic pick loop

At runtime you can picture five arrays (one per lane), and a function that repeatedly:

1. optionally promotes one starving head task
2. picks the next lane in fixed order
3. shifts one task and runs it
4. stops when its time budget is exhausted

Even the "browser integration" is just a `requestFlush(cb)` option (`setTimeout`, `MessageChannel`, or `requestAnimationFrame`).

```
import { createBrowserScheduler } from './runtime/scheduler'

// In the browser you can pick a flushing strategy.
const scheduler = createBrowserScheduler({
  strategy: 'messageChannel',
  defaultBudgetMs: 5,
})

scheduler.schedule('input', () => {
  // keep interactions snappy
})

scheduler.schedule('idle', () => {
  // background bookkeeping
})
```

A tiny scheduler contract We can summarize what the runtime scheduler promises in four bullets:

- **Pick order is deterministic**: same task creation order + same time evolution →same execution order.
- **Latency is explicit**: lane names are the only priority vocabulary.
- **Work is time-sliced**: every flush loop has a budget, and work may continue later.
- **Starvation is mitigated**: low-priority tasks can be promoted based on age/deadline.

13.2 Lanes: latency categories and fixed priority order

Lane priority is encoded as a small integer map:

```
const LANE_PRIORITY = {
  sync: 0,
  input: 1,
  default: 2,
  transition: 3,
  idle: 4,
}
```

But the scheduler's actual selection order is hard-coded as an array:

```
['sync', 'input', 'default', 'transition', 'idle']
```

Spec: `runs tasks by lane priority (sync > input > default > transition > idle)`.

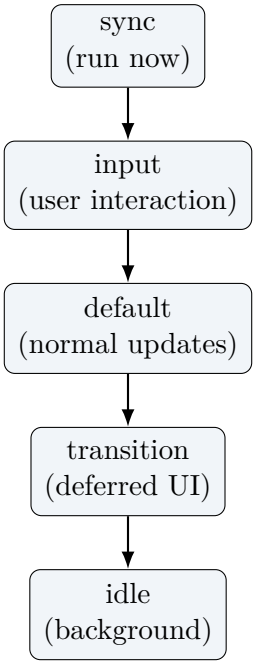
13.2.1 What lanes are (and what they are not)

It's easy to overthink lanes. In HRBR they're not threads, not preemption, and not "multiprocessing". A lane is just a *label* on a task that determines its relative ordering.

This matters because it keeps the model honest:

- tasks still run to completion
- the only question is: which task runs *next* and when do we continue later?

Diagram: lane priority at a glance



This ordering applies unless aging/deadline promotion temporarily pushes work upward.

13.2.2 What is a lane in practice?

Lanes are not preemption. They're an ordering and continuation policy:

- tasks run to completion one-by-one
- lane selection chooses which queue head runs next
- aging/deadlines can temporarily override lane order via promotion

13.2.3 The sync lane

A `sync` task is special: `schedule('sync', ...)` triggers an immediate flush.

In `schedule()`:

- the task is still enqueued
- then the scheduler calls `flush({ budgetMs: Infinity })`

This gives a "run now" lane while still reusing the same task machinery.

Why still enqueue sync tasks? Because determinism still matters. Enqueueing before flushing means sync work still participates in the same stable ordering rules if multiple sync tasks are scheduled.

13.3 Flush strategies: how work gets a time slice

The scheduler accepts `requestFlush(cb)` in its options. If you don't provide one, it uses `createRequestFlush('tim`

13.3.1 createRequestFlush(strategy)

`createRequestFlush()` supports:

- `'timeout'` (universal fallback): `setTimeout(cb, 0)`
- `'messageChannel'`: uses `MessageChannel` if available
- `'raf'`: uses `requestAnimationFrame` if available

Specs in `runtime/__tests__/scheduler.test.ts` cover:

- `createRequestFlush(messageChannel) uses MessageChannel when available`
- `createRequestFlush(raf) uses requestAnimationFrame when available`

13.3.2 createBrowserScheduler() convenience

`createBrowserScheduler( strategy )` is a wrapper that wires the chosen strategy into `requestFlush`.

Spec: `createBrowserScheduler wires a strategy into requestFlush (smoke)`.

Why these strategies exist The scheduler is designed to be embeddable:

- in Node-like environments (`setTimeout`)
- in browsers where `MessageChannel` is often lower-latency than timeouts
- in UIs where `requestAnimationFrame` aligns work with the next frame

13.4 Deterministic ordering inside a lane

Within a lane, tasks are sorted with explicit tie-breakers:

1. earlier `deadline`
2. earlier `timestamp`
3. lower `id`

This ordering is implemented by `sortQueueInLane()`:

```
q.sort((a, b) => (a.deadline - b.deadline)
  || (a.timestamp - b.timestamp)
  || (a.id - b.id))
```

Spec: `orders tasks within a lane by explicit deadline (then timestamp/id)`.

13.4.1 Where deadlines come from

If you don't provide an explicit `deadline`, `schedule()` computes:

```
deadline = timestamp + budgetMs
```

So tasks with smaller budgets naturally get earlier deadlines and bubble forward within their lane. This is a useful "do small things first" heuristic that is still deterministic.

13.4.2 Three tie-breakers, one guarantee (and why we need all three)

It's tempting to think only deadlines matter, but the scheduler needs a *total* ordering to avoid accidental nondeterminism. Inside a single lane, two tasks can easily share a deadline (same budget) and timestamp (scheduled in the same tick). The created `id` is the final "stable sort" key.

The guarantee we want is:

> If you schedule the same tasks in the same order, you will flush them in the same order.

This is why the implementation stores `id` and sorts by `(deadline, timestamp, id)`.

13.5 Budgets: splitting work over multiple flushes

There are two different "budget" knobs:

- `defaultBudgetMs`: the per-task budget used to compute default `deadline` and default `budgetMs`
- `frameBudgetMs`: a global cap for *scheduled flush loops*

13.5.1 What flush(budgetMs) does

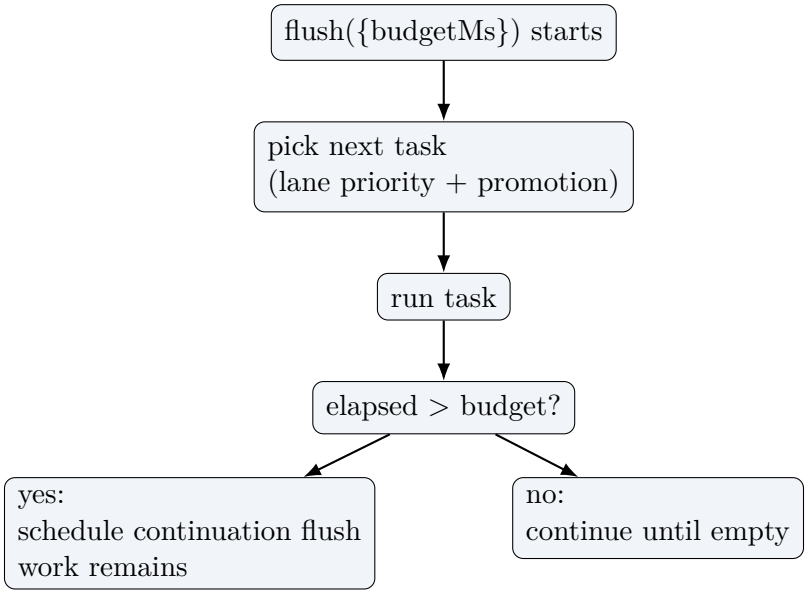
`flush({ budgetMs })`:

- emits devtools events `flushStart` and `flushEnd`
- runs tasks in priority order until:
  - queues are empty, or
  - elapsed time exceeds `budgetMs`
- if budget is exceeded *and* work remains, it schedules a continuation flush

Specs:

- `budget enforcement: flush splits work across multiple flush calls when budget is tight`
- `setFrameBudget caps the budget used by scheduled flushes`

Diagram: the flush loop (budgeted work)



13.5.2 setFrameBudget(ms)

Even if tasks provide their own budgets, scheduled flushes are capped by `frameBudgetMs`.

The spec demonstrates this by setting `frameBudgetMs = 1` and scheduling tasks that each consume `2ms`; only one task runs per flush, and the second flush runs the remainder.

13.5.3 Frame budget versus task budget (important distinction)

In `runtime/scheduler.ts`, `budgetMs` is stored on each task, but it isn't directly used by `flush()`. Instead:

- **task budget** influences default deadlines (and therefore ordering)
- **flush budget** (`flush({budgetMs})` or `frameBudgetMs`) limits how long the flush loop runs before continuation is scheduled

The test suite focuses on flush budgets because those directly affect responsiveness.

13.5.4 Walkthrough: tight budget slicing (mapping a test to code)

Read the spec named `budget enforcement: flush splits work across multiple flush calls when budget is tight`. It schedules three tasks; each task increments a manual clock by 3ms. Then it runs two flush calls with `budgetMs = 5`.

The implementation in `runtime/scheduler.ts` does exactly this:

1. stores `start = now()`
2. runs one task at a time
3. recomputes `elapsed = now() - start`
4. stops once `elapsed >= budgetMs`

So with 3ms per task:

- task 1 leaves us at 3ms (still under 5ms)
- task 2 leaves us at 6ms (over budget) →stop and continue later
- task 3 remains queued for the next flush

This is "cooperative scheduling": tasks cooperate by being small and letting the runtime yield. If you want more intuition about why we care, the web performance RAIL model is a good framing for latency budgets: https://web.dev/rail/.

13.5.5 Timeline example: lane priority, aging, and budget all at once

The trickiest behavior is when *multiple policies apply*: lane order, promotion, and a flush budget. Let's walk a concrete timeline using the same concepts as `makeManualScheduler()` in the tests.

Assume:

- `AGING_MS = 250`
- `frameBudgetMs = 5`
- each task costs `3ms` (it advances the manual clock)

We schedule:

```
// T = 0
scheduler.schedule('idle', () => tick(3))

// time passes; the idle head is now "old"
tick(300)

// scheduling default work ensures a flush is scheduled
scheduler.schedule('default', () => tick(3))
```

Now what happens during `flush()`?

1. **Promotion step**: before lane selection, the scheduler checks the idle head. It has aged, so it is promoted to `transition`.
2. **Lane selection**: the scheduler picks the highest-priority non-empty lane. At this point we have `default` (new) and `transition` (promoted). The lane order says `default` wins.
3. **But why do tests show idle can run first?** Because the implementation promotes by *moving the task to a higher-priority lane* before selection. Across repeated picks (and depending on the exact mix in queues), promotion can change which lane is "non-empty" at the top.

The important lesson isn't the specific interleaving—it's that the promotion rule is deterministic and testable. When you change the policy, the tests will immediately surface the new ordering.

13.5.6 Edge cases to keep in mind

When you build APIs on top of this scheduler (signals, blocks), three edge cases matter:

- **Re-entrancy**: a running task may schedule more tasks. Determinism requires those tasks get consistent timestamps/ids.
- **Budget 0**: `setFrameBudget(0)` makes scheduled flushes yield aggressively. That's allowed, but it can create many continuation flushes.
- **Clock source**: using `performance.now()` in production versus a manual clock in tests is fine, but it means "timestamp equality" is common only in tests.

13.6 Deadlines and aging: starvation prevention via promotion

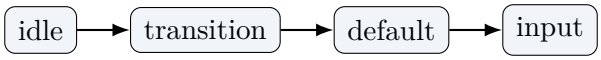
HRBR's scheduler includes an aging/promotion mechanism.

In the implementation:

- `AGING_MS = 250`
- lanes eligible for promotion checks: `idle`, `transition`, `default`
- promotion moves one step toward higher priority: `idle -> transition -> default -> input`

Promotion runs *before* picking the next task, and it only considers the **queue head** in each lane.

Diagram: promotion ladder



The scheduler promotes a task when it's overdue (deadline reached) or old (aging threshold). This is the mechanism behind the starvation-prevention specs.

A task is promoted if:

- it is overdue: `now() >= head.deadline`, or
- it is old: `now() - head.timestamp > AGING_MS`

Specs:

- `ages/pushes starving tasks upwards (idle -> transition)`
- `promotes a task when its deadline is reached`
- `starvation prevention: repeated promotions eventually let idle work run even under continuous input load`

One subtlety the tests call out:

> Promotion happens before lane selection, so an aged/overdue idle task can execute before later-scheduled default work.

This is a deliberate fairness trade-off.

13.6.1 Fairness without nondeterminism

The promotion rule checks only **queue heads** and promotes at most one task per candidate lane per pick. That has two benefits:

- it's cheap (no scanning the entire queue)
- it stays deterministic (the head is a deterministic choice)

13.6.2 Why promotion checks only queue heads

The promotion policy is intentionally restricted to heads:

- It's **predictable**: given the same inputs and the same clock, you'll promote the same tasks.
- It's **O(1)** per lane: no scanning and no "who is most starving" heuristics.
- It preserves within-lane ordering as much as possible.

This is a classic engineering trade-off: accept a slightly simplified fairness model to buy determinism and low overhead.

13.7 withBudget(): bridging user work and scheduler work

Real apps do work outside of the scheduler too (rendering, event handlers, parsing). `withBudget(budgetMs, fn)` is a helper that:

- runs `fn`
- if `fn` exceeds the budget and tasks remain, it schedules a continuation flush

Spec: `withBudget schedules continuation when user code exhausts budget and tasks remain`.

This is the "be nice to the main thread" glue between imperative work and queued reactive work.

13.7.1 A practical use case: big event handler + pending effects

Imagine an event handler that does some synchronous work (parsing, filtering, sorting) and also triggers signal writes. Those writes schedule computations, but you still want control: if your handler took too long, queue a flush continuation.

```
import { withBudget } from './runtime'

withBudget(4, () => {
  // expensive user work
  // ... parse, compute, validate ...
  // signals may schedule effects while we do this
})
// if the 4ms budget was exceeded and work is pending,
// the scheduler will ensure another flush is scheduled.
```

13.7.2 Global helpers in `runtime/index.ts`

The runtime also exports shared budgeting helpers that use a module-level global scheduler:

- `withBudget(budgetMs, fn)`
- `setFrameBudget(ms)`

This is covered by the smoke test `runtime withBudget() helper returns the callback result and is callable`.

13.8 Devtools and instrumentation hooks

The scheduler calls `emitDevtoolsEvent()` (`runtime/devtools.ts`) when flushing:

- `flushStart`
- `flushEnd{ durationMs }`

These feed counters like `flushes` and `flushTimeMs`.
Signals also emit `scheduleComputation` events when a computation is scheduled.
(We'll use these counters later when measuring block update costs.)

13.8.1 Why "events + counters" is a good runtime pattern

Two competing goals exist:

- keep the runtime fast (no heavy logging)
- make the runtime observable (so you can measure and debug)

The implementation in `runtime/devtools.ts` uses a tiny event stream plus aggregated counters. That means you can do "always-on" cheap counting (flush count/time) and keep deeper instrumentation behind a toggle.

13.9 Walking the tests: what they prove

The runtime scheduler suite is unusually readable because it uses a manual clock and a manual flush queue. That makes time and continuation behavior deterministic.

13.9.1 Core semantics (`runtime/__tests__/scheduler.test.ts`)

runs tasks by lane priority Proves the lane order and that `sync` executes immediately.

orders tasks within a lane by explicit deadline Proves the sort tie-breakers inside a lane.

respects budget and continues in a later flush Proves a scheduled flush loop may stop early and reschedule continuation.

budget enforcement: flush splits work Proves deterministic time slicing under a small budget.

setFrameBudget caps scheduled flush budget Proves continuation flushes use `frameBudgetMs` as a cap.

ages/pushes starving tasks upwards Proves age-based promotion and the fairness trade-off (promoted work can run before later default work).

promotes a task when its deadline is reached Proves deadline-based promotion independent of age.

starvation prevention under input load Proves repeated promotions eventually allow idle work to run.

createRequestFlush(messageChannel/raf) Proves strategy selection uses platform APIs when present.

createBrowserScheduler smoke Proves wiring strategy into `requestFlush`.

13.9.2 How to read these tests like a spec

The tests are valuable because they remove nondeterminism from time. The helper `makeManualScheduler()` provides:

- a manual clock: `tick(ms)`
- a list of pending flush callbacks: `runNextFlush()` / `runAllFlushes()`

So when you see an assertion like `expect(order).toEqual([...])`, you're basically reading the scheduler's language definition.

13.10 Online resources

- MDN: `MessageChannel`
- MDN: `requestAnimationFrame`
- MDN: Event loop (tasks, microtasks)
- React (conceptual parallel): Render/commit (scheduling intuition)
- Cooperative background work (conceptual): `requestIdleCallback`

13.11 Takeaways

- Lanes provide a small, explicit latency model with a deterministic priority order.
- Each lane is internally sorted by deadline, then timestamp, then id.
- Budgeting is enforced at flush time to time-slice work.
- Aging/promotion makes starvation unlikely without making scheduling non-deterministic.
- `withBudget` lets user code cooperate with the scheduler.

Chapter 14

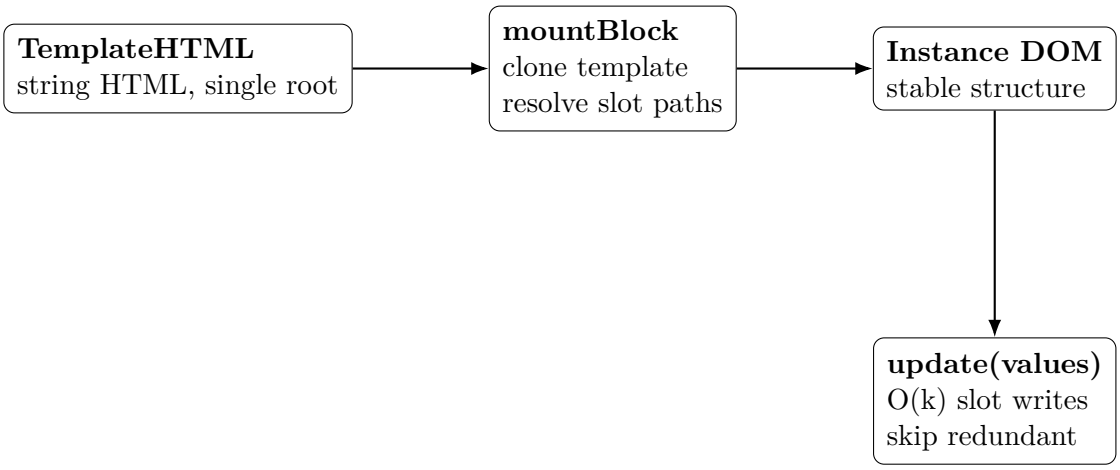
HRBR Blocks: Template + Slots (the Compiled Fast Path)

Chapters 12 and 13 gave us two halves of a reactive UI runtime:

- **Signals** decide *what* values changed.
- **Scheduler lanes** decide *when* to apply work.

Blocks are the third half: they decide *how* to touch the DOM.

A **block** is the "fast DOM" path: it describes a stable DOM template and a set of precise write locations (slots). Updates become *O(k)* slot writes instead of building a VDOM tree and diffing children.



`mountCompiledBlock(..., options)` accepts `lane` and/or a custom `scheduler`.

- **input-latency sensitive** (use `input` or `sync`)
- **deferable** (use `transition` or `idle`)

If you remember only one thing about blocks, make it this:

extbfBlocks replace diffing with addressing. The template fixes structure. Slots give exact addresses. Updates become a handful of targeted DOM writes.

This chapter is grounded in:

- Implementation: `runtime/block.ts`
- Specs: `runtime/__tests__/block.test.ts`
- Slot matrix: `runtime/__tests__/block-slot-matrix.test.ts`
- Input controls: `runtime/__tests__/block-input-controls.test.ts`
- Disposal correctness: `runtime/__tests__/block-disposal.test.ts`

14.1 Why blocks exist (what VDOM cannot do efficiently)

VDOM diffing is a *general-purpose* way to update UI. That generality costs work:

- allocate a new tree of VNodes
- compare old/new trees
- figure out where the DOM should change

Compiled UI wants the opposite trade-off: specialize up-front so runtime work is minimal. Blocks do that by encoding two things that VDOM typically learns at runtime:

- **Structure is stable**: the template defines the DOM shape.
- **Dynamic writes are explicit**: slots pre-compute where updates go.

The result is a small set of direct DOM operations per update. This is the same general idea behind other compiler-first frameworks. Svelte has a well-known write-up for the intuition (even if you disagree with the framing, the mental model is useful): https://svelte.dev/blog/virtual-dom-is-pure-overhead.

14.2 A minimal block from scratch (end-to-end)

Let's build the smallest meaningful block: a greeting with a dynamic name and a click handler. This is not pseudo-code; it matches the contracts in `runtime/block.ts`.

```
import { defineBlock, mountBlock } from './runtime/block'

const HelloBlock = defineBlock({
  templateHTML: `<div>Hello <span>world</span> <button>ok</button></div>`,
  slots: {
    // root.childNodes = [Text('Hello '), span, Text(' '), button]
    // span.childNodes = [Text('world')]
    name: { kind: 'text', path: [1, 0] },

    // button is childNodes[3]
    onClick: { kind: 'event', path: [3], name: 'click' },
  },
})
```

```
const host = document.getElementById('app')
const mounted = mountBlock(HelloBlock, host, {
  name: 'Ravi',
  onClick: () => console.log('clicked'),
})

// later: this is O(1) targeted DOM writes
mounted.update({ name: 'Relax' })

// cleanup
mounted.destroy()
```

Two details matter more than they look:

- The `text` slot must point to an *existing* Text node in the template. That's why the template uses `<span>world</span>` instead of `<span><!-placeholder-></span>`.
- The `path` walks `childNodes`, not `children`. Text nodes (including whitespace) count.

14.3 A tiny contract (what a block guarantees)

Treat this as the "promise" the runtime makes to compiled code:

1. **Single stable root:** `templateHTML` must produce exactly one root element, which is cloned and appended to the host.
2. **Exact addressing:** each slot points to a node using a `path: number[]` walk over `childNodes` indices.
3. **Specialized patching:** slot kinds are patched by small, explicit routines (text/attr/prop/-class/style/event).
4. **No needless DOM writes:** redundant values are skipped via `Object.is`.
5. **Safe lifecycle:** `destroy()` is idempotent, removes listeners, and after destroy further `update()` calls are ignored.

14.4 Block vocabulary

14.4.1 TemplateHTML

A block definition includes a `templateHTML: string`. This must produce exactly one root element.

Implementation detail: `parseTemplate(templateHTML)` uses a `<template>` element and asserts:

```
templateHTML must have a single root element
```

The implementation is intentionally strict:

- it does `templateHTML.trim()` before parsing
- it reads `tpl.content.firstElementChild`
- it asserts non-null and returns the root element

This makes "multiple root nodes" a loud error, which is what you want for compiled templates.

14.4.2 Slot

A **slot** is a metadata record describing *where* and *how* to write a dynamic value into a template instance.

Slots are keyed by string (slot key) and come in these kinds:

- `text`: write to a Text node
- `attr`: set/remove attributes
- `prop`: set DOM properties
- `class`: write `class` attribute
- `style`: write inline styles
- `event`: bind events once and swap handler references

Each slot includes a `path: number[]` which is a **childNodes index walk** from the template root.

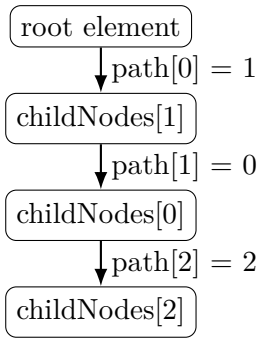
Two practical consequences:

- `childNodes` counts text nodes and comments, so whitespace in templates matters.
- Slot paths are best generated by a compiler that controls the template serialization.

14.5 Why paths (and not querySelector)?

Paths are the "compiled fast path" address:

- they're *exact* (no ambiguities)
- they're cheap (depth walk, no subtree scan)
- they enable prefix caching across multiple slots



One subtle point the tests implicitly enforce: `childNodes` includes text nodes (including whitespace). That means a compiler must be in control of template serialization and whitespace if it wants stable paths.

14.5.1 Path stability: whitespace is part of your ABI

This is worth slowing down for, because it's the most surprising part of the block model.

extbfPaths are an ABI between the compiler and runtime. If the compiler emits `path: [3, 0]`, it is assuming a specific `childNodes` layout.

That means these two templates are *not equivalent* from the runtime's perspective:

```
// Template A
`<div><span>hi</span><button>ok</button></div>`

// Template B (note the whitespace/newlines)
`<div>
  <span>hi</span>
  <button>ok</button>
</div>`
```

Template B contains additional text nodes for the whitespace/newlines. So a slot path that points to the `button` in template A may point to a whitespace Text node in template B.

Practical rule for compilers Compilers should either:

- serialize templates in a whitespace-stable canonical form, or
- generate paths from the exact serialized string they ship as `templateHTML`

Otherwise you'll get "works in dev, breaks in prod" bugs when minifiers/formatters rewrite whitespace.

14.6 The public API (surface area)

The core exports in `runtime/block.ts` are:

- `defineBlock(def)`: identity helper for typing and readability
- `mountBlock(def, host, initialValues?, options?) -> MountedBlock`
- `mountCompiledBlock(def, host, compiledSlots, options?) -> MountedBlock & { dispose() }`
- `mountNestedBlock(parent, slotKey, def, initialValues?)`
- `mountNestedFallback(parent, slotKey, mount)`
- `resolvePath(root, path)` and `resolvePathCached(root, path, slotKey, dev?)`

Reading guide:

- `mountBlock` is the non-reactive primitive (mount + `update`).
- `mountCompiledBlock` wires signals into `update` with effects and the HRBR scheduler.
- `mountNestedBlock`/`mountNestedFallback` are the structural escape hatches.

14.7 defineBlock(): the type boundary

textttdefineBlock(def) doesn't transform input. It exists so "compiled" code can be explicit and type-safe.

In `runtime/block.ts`, it is literally an identity function:

```
texttexport function defineBlock(def: BlockDef): BlockDef { return def }
```

That sounds trivial, but it creates a clean "compiler boundary" and a stable type surface.

Tests use it in every case:

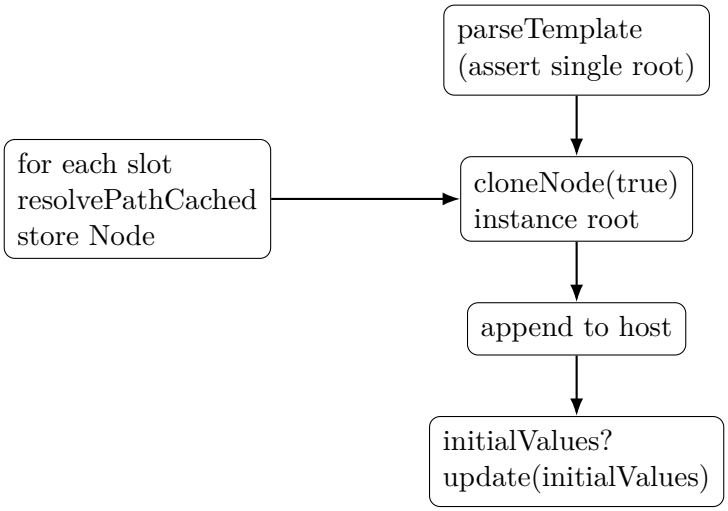
```
const block = defineBlock({ templateHTML: ..., slots: ... })
```

14.8 mountBlock(): instantiate template + resolve slot nodes

`mountBlock()` does the non-reactive mount:

1. parse template HTML into a root element
2. clone it to create an instance
3. append instance root to the host
4. resolve each slot `path` into a concrete DOM `Node`
5. apply `initialValues` via `block.update(initialValues)`

The key optimization is "resolve once, write many": paths are resolved at mount time and stored in `slotNodes`. Updates never traverse the DOM tree; they dereference and write.



This diagram is also a good mental model for the difference between `mountBlock` and `mountCompiledBlock`:

- `mountBlock` stops after returning `update()`.
- `mountCompiledBlock` adds the reactive wiring on top (effects + scheduler lanes) and then calls `update()` as signals change.

The returned `MountedBlock` includes:

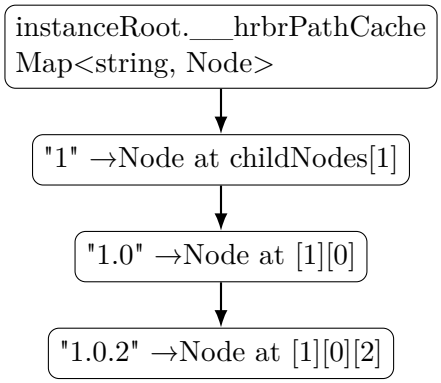
- `root`: the mounted root element
- `slotNodes`: mapping from slot key to resolved node reference
- `update(values)`: apply slot writes
- `destroy()` / `dispose()` lifecycle

14.8.1 Path resolution and caching

`resolvePath(root, path)` is the simple implementation. But `mountBlock` uses `resolvePathCached(instanceRoo path, slotKey, dev)`.

The cache is a `Map<string, Node>` stored on the instance root as `__hrbrPathCache`. Prefixes are stored as dot-joined strings like `"1.0"`.

The caching algorithm is incremental: as it walks the path, it caches each prefix result (`"1"`, then `"1.0"`, etc.). Multiple slots that share a prefix benefit immediately.



This cache is not about speeding up a single slot; it pays off when there are many nearby slots (a common outcome of compilation).

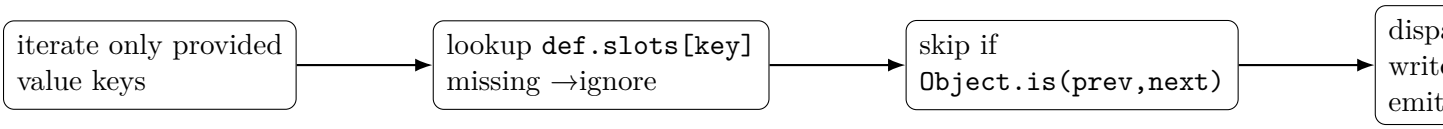
Spec: `mountBlock caches repeated path prefixes across slots`.

14.8.2 Dev diagnostics

When `options.dev === true`, the assertions include the slot key in the error.

Spec: `dev diagnostics: mountBlock includes slot key in invalid path errors (dev mode)`.

14.9 update(values): slot patching rules



The update loop iterates over provided keys only. If a key doesn't exist in `def.slots`, it is ignored.

Two important performance rules:

- The block stores `prevValues[key]` and skips redundant writes using `Object.is`.
- It emits a devtools event `slotWrite` for each actual write.

The use of `Object.is` is deliberate: it treats `NaN` as equal to itself and provides a cheap "same reference" check for objects (like style objects or class arrays).

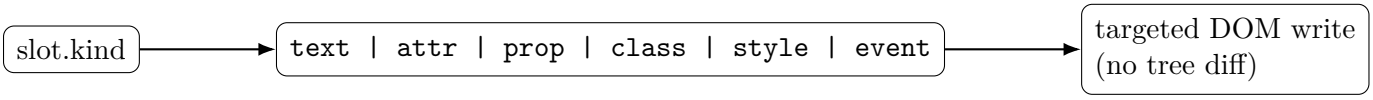
It also affects correctness in subtle-but-useful ways:

- **Stable function identity:** if a compiled block passes the same handler function reference repeatedly, the event slot can skip work.
- **Object slots:** if style/class are represented as objects/arrays, reusing the same value prevents unnecessary DOM churn.

Spec: `skips redundant writes for unchanged values (text + attr)`.

14.9.1 Slot kind dispatch at a glance

The update routine is small because each slot kind is a focused patcher. It's useful to keep the matrix in your head:



14.9.2 text slots

Text slots must resolve to a real Text node. The runtime asserts:

```
slot 'x' expected a Text node
```

Writes use `Text.nodeValue` and normalize nullish values to the empty string.

Spec: `mounts a template and updates text slots by path`.

DOM ops (what actually happens) This is conceptually equivalent to:

```
textNode.nodeValue = next == null ? '' : String(next)
```

No nodes are created and no subtree scans happen.

Gotcha: Text slots are strict about node type The runtime asserts `nodeType === Node.TEXT_NODE`. So if a compiler points a text slot at an Element (for example, thinking in terms of `textContent`), updates will throw. This strictness is a feature: it turns silent misaddressing into a loud, debuggable error.

14.9.3 attr slots (including boolean attributes and SVG xlink)

Attribute writes route through `setAttr(el, name, next)`:

- `null | false` removes the attribute
- boolean-ish attributes are present/absent (set to empty string)
- SVG `xlink:*` attributes use `setAttributeNS`

Implementation notes:

- SVG detection uses `namespaceURI` (not `instanceof SVGElement`) for broader runtime compatibility.
- Boolean attribute names are normalized (`readOnly` maps to `readonly`).

Specs:

- `updates attribute slots by path`
- `treats boolean attributes as present/absent (attr slots)`
- `sets xlink:* attributes with the xlink namespace on SVG elements`

DOM ops summary

- `null | false` →`removeAttribute` (or `removeAttributeNS` for SVG xlink)
- boolean-ish →`setAttribute(name, '')` (presence indicates truth)
- other values →`setAttribute(name, String(value))`

14.9.4 prop slots (input correctness)

Property slots route through `setProp(el, name, next)`.

Special-case correctness for inputs:

- `value`: written as a string property, nullish clears to `""`
- `checked`: written as boolean

This matches standard controlled-input behavior: writing to properties updates the live control state, while attributes can drift.

Specs:

- `updates property slots by path`
- `updates input value/checked via property semantics for prop slots`
- input control suite in `block-input-controls.test.ts`

Why prop slots exist at all Because for form controls the live state is on properties. Setting an attribute might update the HTML, but not the user-visible control state in the same way. The tests in `block-input-controls.test.ts` are basically saying: "treat blocks like controlled inputs".

14.9.5 class slots

Class slots write the `class` attribute:

- `null | false` removes the attribute
- arrays are joined with spaces after filtering falsy values
- otherwise coerced to string

14.9.6 style slots

Style slots accept:

- `null | false`: remove whole style attribute
- string: set `style` attribute directly
- object: per-key `style.setProperty` / `removeProperty`

This small matrix is validated by `block-slot-matrix.test.ts`.

14.9.7 event slots: bind once, swap handler

Event slots are designed to avoid listener leaks and avoid churn:

- the first time a slot is set, the runtime binds a stable listener function
- the listener reads `rec.current` to find the latest handler
- updates replace `rec.current` without re-binding
- destroying the block removes the listener

Specs:

- `supports event slots:  binds once and updates handler without leaking listeners`
- `destroy removes event listeners (no further handler calls)`
- `skips redundant event listener churn when handler identity is unchanged`

Matching Relax VDOM semantics: hostComponent binding

`mountBlock(..., { hostComponent })` optionally calls handlers with `this === hostComponent`, matching Chapter 7's event binding behavior in the VDOM runtime.

14.10 Lifecycle: destroy vs dispose, and idempotency

`mountBlock` is not reactive by itself, so `dispose()` is just an alias for `destroy()`.

`destroy()` is idempotent:

- it guards with a `destroyed` flag
- it destroys composed children
- it removes event listeners
- it removes the block root from the DOM

After `destroy()`, `update()` becomes a no-op (guarded by the same `destroyed` flag). That prevents stale scheduled work from mutating detached DOM.

14.10.1 Why the destroyed guard matters in a scheduled world

Once you introduce lanes + budgets, "later" becomes a first-class concept. So the runtime must assume:

- a compiled block can be disposed while a flush task is still pending
- events may still fire on detached nodes in test environments

The destroyed guard plus listener removal turns those situations into safe no-ops. The disposal tests exist to make this behavior non-negotiable.

Specs:

- `destroy is idempotent even with event slots and children`
- `lifecycle:  destroy() is idempotent and prevents further updates`

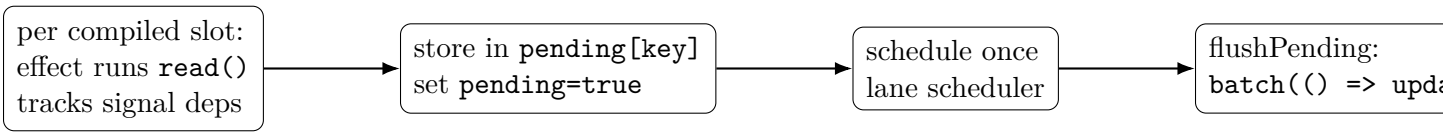
14.11 mountCompiledBlock(): wiring signals to slot writes

14.11.1 The contract the compiler relies on

The compiler's job is to turn template expressions into a list of `CompiledSlot` entries:

- `key` must match a slot key in `def.slots`
- `read()` must be a pure read of the current values (it may read signals)

The runtime's job is to evaluate these reads inside effects, coalesce them, and commit them through `block.update()`.



`mountCompiledBlock` is the bridge between the reactive graph and the block DOM writer.
Input: `CompiledSlot[]` where each entry is:

```
{ key: string, read: () => unknown }
```

Important: `read()` can access signals; that's how dependency tracking happens.
Mechanism:

- compute initial values by running each `read()`
- mount a non-reactive block with those initial values
- create one `createEffect()` per compiled slot
- coalesce multiple invalidations into a single scheduled `flushPending`
- execute `block.update(values)` inside `batch()` to keep effects consistent

The coalescing logic (`pending` + `scheduled` flag) is the performance heart: many effects can invalidate in one turn, but only one scheduled task drains `pending` and commits to the DOM.
Scheduling goes through the deterministic lane scheduler from Chapter 13:

texttttscheduler.schedule(lane, flushPending)

Spec: `changing a signal updates only the intended DOM node (mountCompiledBlock)`.

14.11.2 Why coalescing matters (and how it stays deterministic)

Without coalescing, a burst of signal writes could schedule and run many DOM commits. With coalescing:

- each effect recomputes its slot value and writes it into a `pending{}` object
- a single scheduled task flushes `pending` to the DOM

This mirrors the scheduler philosophy from Chapter 13: run tasks in a deterministic order, but avoid doing redundant work.
In code form, the shape looks like:

```
// Conceptual shape (names simplified)
const pending = {}
let scheduled = false

function flushPending() {
  scheduled = false
  batch(() => block.update(pending))
  // reset pending
}

createEffect(() => {
  pending[key] = read()
  if (!scheduled) {
    scheduled = true
    scheduler.schedule(lane, flushPending)
  }
})
```

The actual implementation is in `runtime/block.ts` and uses the same primitives you've already learned: `createEffect`, `batch`, and the deterministic scheduler lanes.

14.11.3 Timeline: what happens when two signals change in the same tick

The spec `changing a signal updates only the intended DOM node (mountCompiledBlock)` is tiny, but it implies a timing model. Here's the practical timeline:

1. Two signals are written (often in the same event handler).
2. Their dependent slot effects re-run and compute new values.
3. Each effect writes its value into `pending[key]`.
4. The first effect that notices `scheduled === false` schedules `flushPending` exactly once.
5. The scheduler later runs `flushPending` (based on lane + flush strategy).
6. `flushPending` calls `block.update(pending)` inside `batch()`.

This is the block-runtime equivalent of a "commit phase": compute reactively, then commit a coalesced set of DOM writes.

14.11.4 Annotated walkthrough: `mountCompiledBlock()` line-by-line

This section is a literate read of the real implementation in `runtime/block.ts`. The goal is to understand the exact control flow that turns *many reactive invalidations* into *one scheduled DOM commit*.

Listing 14.1: The full `mountCompiledBlock()` implementation (from `runtime/block.ts`)

```
export function mountCompiledBlock(def, host, slots, options = {}) {
  const lane = options.lane ?? 'default'
  const scheduler = options.scheduler ?? createScheduler()

  // initial values from slots
  const initialValues = Object.create(null)
  for (const s of slots) initialValues[s.key] = s.read()

  const block = mountBlock(def, host, initialValues)

  // Coalesce multiple slot invalidations into a single scheduled update.
  let scheduled = false
  const pending = Object.create(null)

  function flushPending() {
    scheduled = false
    const values = Object.assign(Object.create(null), pending)
    for (const k of Object.keys(pending)) delete pending[k]
    batch(() => block.update(values))
  }

  const effects = slots.map((s) =>
    createEffect(() => {
```

```
      const next = s.read()
      pending[s.key] = next
      if (scheduled) return
      scheduled = true
      scheduler.schedule(lane, flushPending)
    })
  )

  let disposed = false

  return Object.assign(block, {
    dispose() {
      if (disposed) return
      disposed = true
      for (const e of effects) e.dispose()
      block.destroy()
    },
  })
}
```

Now let's go line-by-line and name the invariants.

`lane` + `scheduler`: where the commit runs

- `const lane = options.lane ?? 'default'`: the compiler (or framework integration) chooses the lane that this block uses for DOM commits.
- `const scheduler = options.scheduler ?? createScheduler()`: by default each mounted compiled block gets a scheduler instance; passing one in allows sharing a scheduler across many blocks so their commits interleave deterministically (Chapter 13).

Initial read: mount with a consistent snapshot

- `for (const s of slots) initialValues[s.key] = s.read()`: we read every compiled slot once up front.
- This matters because `mountBlock(...)` immediately calls `block.update(initialValues)` internally, so the DOM is created already reflecting the current reactive state.
- The read happens *before* we install effects, so we don't accidentally schedule any work while mounting.

The coalescing state: `pending` + `scheduled`

- `pending`: a plain dictionary `{ [slotKey]: nextValue }` of the latest computed values.
- `scheduled`: a one-bit latch that says whether a flush task has already been enqueued.

This pair implements a standard UI-runtime trick: *many producers, single consumer*. Every slot effect can write into `pending`, but only the first invalidation schedules a consumer run.

`flushPending()`: the commit phase

- `scheduled = false`: we open the gate so that new invalidations can schedule the next flush.
- `const values = Object.assign(Object.create(null), pending)`: we take a snapshot of the bag.
- `for (const k of Object.keys(pending)) delete pending[k]`: clear `pending` in-place.
- `batch(() => block.update(values))`: commit the DOM writes, but keep the signals runtime in a batched mode so downstream effects don't thrash while we patch nodes.

Two subtle details here are worth calling out:

- **Snapshot then clear:** if a slot changes *while* we're in `block.update`, it writes into the cleared `pending` bag and can schedule a *new* flush (because `scheduled` was reset first).
- **The update payload is sparse:** `values` only contains keys that changed since the last flush, so `block.update` iterates fewer entries.

Per-slot effects: producers that only schedule once

- `createEffect(() => { const next = s.read(); ... })`: each compiled slot becomes an effect.
- The effect writes `pending[s.key] = next` on every re-run.
- `if (scheduled) return`: if a flush is already enqueued, we stop here.
- Otherwise we set `scheduled = true` and enqueue `flushPending` into the scheduler lane.

This is the key guarantee: *no matter how many times effects re-run before the scheduler flushes, we only enqueue one task.* The tasks queue stays small, and the DOM work happens at a predictable point.

Disposal: stop reactive producers, then remove DOM

- `let disposed = false` + `if (disposed) return`: disposal is idempotent (safe to call twice).
- `for (const e of effects) e.dispose()`: the critical step. If we destroyed DOM first but kept effects alive, a later signal write would try to update detached (or missing) nodes.
- `block.destroy()`: removes DOM and event listeners (from `mountBlock`), and also destroys any nested blocks registered via composition helpers.

For additional background on why frameworks separate "compute" from "commit" phases (even when the implementation is small like this), see React's render/commit model https://react.dev/learn/render-and-commit.

14.11.5 Lane choice is part of the compiled contract

`mountCompiledBlock(..., options)` accepts `lane` and/or a custom `scheduler`. That lets compiled code decide whether DOM writes are:

- **input-latency sensitive** (use `input` or `sync`), or
- **deferable** (use `transition` or `idle`).

So blocks aren't just a faster patching mechanism; they're also a practical location to attach scheduling intent.

That test also checks identity stability:

- the static DOM elements are not recreated
- text node identity remains stable; only `nodeValue` changes

Spec: `mounting does not recreate static DOM during updates`.

14.11.6 Disposal stops reactive updates

`mountCompiledBlock` returns a `dispose()` method that:

- disposes all effects
- destroys the block DOM
- is idempotent

Specs:

- `mountCompiledBlock.dispose stops reactive updates and removes DOM`
- `lifecycle: mountCompiledBlock.dispose() stops reactive updates and is idempotent`

14.12 Composition: nested blocks and fallback regions

Blocks are stable and fast, but real UIs still need dynamic structure. `runtime/block.ts` provides two small composition helpers:

- `mountNestedBlock(parent, slotKey, def)`: mount a child block into an Element slot
- `mountNestedFallback(parent, slotKey, mount)`: mount a structural fallback region into an Element slot

Both helpers:

- clear the host element's existing children
- mount into it
- register a child destructor so destroying the parent destroys the child

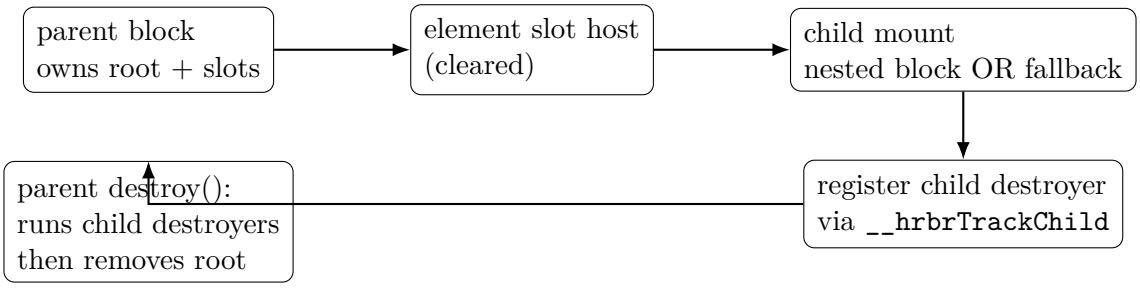
Specs in `runtime/__tests__/block.test.ts`:

- `block composition: can mount a nested block ... and destroys it with parent`
- `block composition: can mount a nested fallback region ... and disposes it with parent`

This is HRBR's interop story:

> Compiled stable subtrees use blocks; dynamic subtrees route to fallback reconciliation.

Implementation detail: composition is tracked with a small internal child registry. textttmountBlock exposes an internal `__hrbrTrackChild` function, and the composition helpers use it to register child destroyers so teardown order is correct.



14.13 Walking the tests (the executable spec)

14.13.1 `runtime/__tests__/block.test.ts`

This test file locks down the baseline behavior of each slot type and of the compiled/reactive mount.

- **text slots:** the slot must resolve to a real Text node; updates rewrite `nodeValue`; destroy removes DOM.
- **attr slots:** `null` removes the attribute.
- **prop slots:** input `value` is updated via property semantics.
- **class/style:** `null` removes attrs, arrays join for class, style objects set properties.
- **event slots:** click once, swap handler, click again; after `destroy()` clicks do nothing.
- **boolean attributes:** present/absent behavior is asserted with `disabled`.
- **SVG xlink:** uses `getAttributeNS` to validate namespace behavior.
- **path caching:** asserts `__hrbrPathCache` contains prefix keys (e.g. `"0"`, `"0.0"`).
- **dev diagnostics:** invalid path errors include the slot key in dev mode.
- **mountCompiledBlock:** changing one signal updates only its text node, and node identity is stable across updates.

14.13.2 `runtime/__tests__/block-slot-matrix.test.ts`

This is a matrix-style smoke test that mounts one template with **all slot kinds** and asserts both initial values and updated values, including event swapping.

14.13.3 `runtime/__tests__/block-input-controls.test.ts`

This suite documents the intended controlled-input semantics for `value` and `checked` when patched as properties. It also includes a note about radio group behavior in jsdom.

14.13.4 `runtime/__tests__/block-disposal.test.ts`

This suite locks down lifecycle safety:

- `destroy()` removes event listeners (clicking a detached element must not call handlers).
- `destroy()` is idempotent.
- `mountCompiledBlock.dispose()` stops reactive updates and removes DOM permanently.

14.14 Online resources

- MDN: `<template>` (template parsing and cloning)
- MDN: `Node.childNodes` (why paths include text/comment nodes)
- MDN: `addEventListener` (listener identity + removal)
- MDN: `HTMLInputElement` (property semantics for `value` and `checked`)
- MDN: `xlink:href` (SVG namespaced attribute background)
- Svelte: "Virtual DOM is pure overhead" (compiler-driven DOM updates intuition)

14.15 Takeaways

- Blocks are "template + slots": a stable DOM instance plus precise write locations.
- Slot patching is specialized per kind, with correctness for input controls and SVG namespaces.
- Updates skip redundant writes via `Object.is`.
- Event slots bind once and swap handler references; destroy removes listeners.
- `mountCompiledBlock` wires signals to slots and coalesces updates into scheduled flushes.
- Composition helpers let blocks nest blocks and fallback regions safely.

Chapter 15

Hydration: Attaching HRBR to Server HTML

What you'll build

In this chapter I'll build a mental model for v1 hydration in Relax, and you'll learn how to reason about it by reading the real runtime code. Concretely, you'll be able to:

- take an HTML string that was produced from a BlockDef, insert it into the DOM, and *attach* a live HRBR block to it without rewriting nodes
- explain why Relax's v1 hydration is strict and why that strictness is a feature (not a limitation)

What you'll learn

- The hydration contract: what must match between server HTML and `templateHTML`
- The two safety rails: `root tag check` and `slot-path validation`
- The cost model: why hydration is cheap when it succeeds, and cheap to bail out when it must
- How v1 hydration handles DOM updates (same slot kinds as Chapter 14) and where it differs (events)

Prereqs

- Chapter 14: Blocks, templates, and slots (slot kinds + path addressing)
- Chapter 12: Effects and dependency tracking (why `mountCompiledBlock` works)

15.1 Why hydration exists (from scratch)

Hydration is the bridge between two worlds:

- The **server** produced HTML as a string. That's great for first paint and SEO.
- The **client** needs live behavior: event handlers, reactive updates, and cleanup.

If you don't hydrate, you have only two choices on the client:

- **Remount by recreating DOM** from scratch, which throws away the server's nodes (and can introduce a visible flash), or
- **Try to patch your way into the server DOM** with a general reconciler, which is powerful but hard to make airtight.

Relax HRBR v1 chooses a third option:

> extbfAttach by validating structure and resolving slot node references. If validation fails, remount.

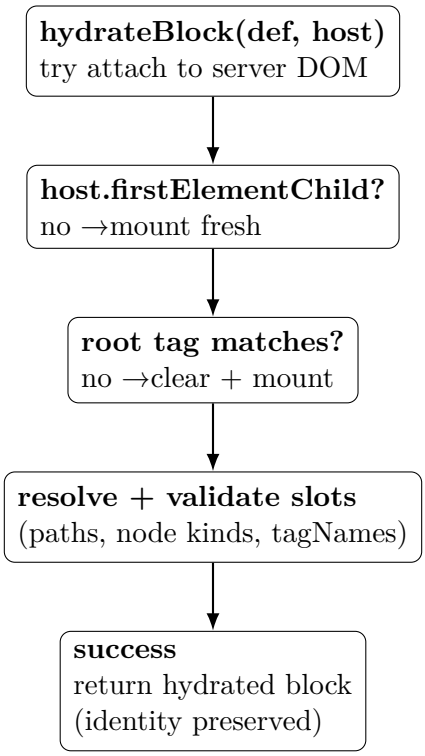
This conservative posture is intentional. It makes hydration predictable: the moment we say "hydrated", we're also saying the block's template ABI matches the DOM we're about to mutate.

15.2 The big picture: where hydration sits in HRBR

A block runtime has a natural relationship with server rendering:

- The compiler produces a stable template shape (`templateHTML`).
- The server can render the template to HTML.
- The client can *attach* to the existing DOM instead of rebuilding it.

In Relax HRBR v1, hydration is deliberately conservative. If anything looks wrong, it bails out and remounts client-side.



This chapter is grounded in:

- `runtime/hydration.ts` (hydrate implementation)
- `runtime/ssr.ts` (string rendering helper used in tests)
- `runtime/__tests__/hydration.test.ts`
- `runtime/__tests__/ssr.test.ts`

15.3 A tiny contract (what hydrateBlock promises)

Hydration is the moment where two worlds meet:

- The **server** produced HTML as a string.
- The **client** wants a live block instance with slot node references.

In v1, Relax uses a strict approach that is easy to reason about and hard to get subtly wrong. You can treat `hydrateBlock(def, host, initialValues)` as enforcing this contract:

1. If there is no DOM to attach to (`host.firstElementChild == null`), hydration simply mounts a fresh block.
2. If the root element tag does not match the template's root tag, hydration clears the host and mounts fresh.
3. If slot paths resolve to the wrong node types or to the wrong element tag names (compared to the parsed template), hydration clears the host and mounts fresh.
4. If everything matches, hydration returns a block-like object whose `root` and `slotNodes` refer to the server DOM nodes.

The headline design choice is: **when in doubt, remount**. That makes correctness obvious at the cost of being less clever than incremental/partial hydration systems.

15.4 Build it: a minimal end-to-end hydration loop

I like to teach hydration as a 3-step loop: **render** → **attach** → **update**.

In Relax, the pieces are:

- `renderBlockToString(def, { initialValues })` (SSR helper)
- `host.innerHTML = html`
- `hydrateBlock(def, host, initialValues)` (attach)
- optionally: `mountCompiledBlock(def, host, compiledSlots)` to wire signals/effects (Chapter 14)

Here's the smallest "looks runnable" snippet that matches what the tests demonstrate.

Listing 15.1: SSR + hydration + update: the smallest end-to-end loop

```
import { defineBlock, mountCompiledBlock } from ’./runtime/block’
import { hydrateBlock } from ’./runtime/hydration’
import { createSignal } from ’./runtime/signals’
import { renderBlockToString } from ’./runtime/ssr’

const Counter = defineBlock({
  templateHTML: ‘<div class="c"><button id="inc">+</button><span id="t"> </span></div>‘,
  slots: { value: { kind: ’text’, path: [1, 0] } },
})

// "Server" side
const html = renderBlockToString(Counter, { initialValues: { value: ’A’ } })

// "Client" side
const host = document.getElementById(’host’)
host.innerHTML = html

// Attach to existing nodes (no rewrite if markup matches)
const hydrated = hydrateBlock(Counter, host)

// Wire reactive updates on top (optional, but this is the HRBR story)
const [value, setValue] = createSignal(’A’)
const mounted = mountCompiledBlock(Counter, host, [{ key: ’value’, read: () => value()
    }])

setValue(’B’)
mounted.update({ value: ’B’ })

mounted.dispose()
hydrated.destroy()
```

15.4.1 Why these steps matter

- **SSR produces a shape:** If SSR and the template disagree, hydration must not "half attach".
- **Hydration creates references:** the essential output is `slotNodes[key] = Node`.
- **Updates are the same contract:** a call to `update({ key: value })` still means “write into slot nodes”.

15.5 Hydration contract (v1)

`hydrateBlock(def, host, initialValues?)` returns a `HydratedBlock` (type alias of `MountedBlock`). Its docstring states three key assumptions:

1. `host` contains exactly one root element matching the template.
2. Hydration does not mutate DOM; it only resolves slot node references.
3. On mismatch, hydration bails out and remounts client-side.

This is a strict, "safety first" strategy: it prefers correctness over partial hydration tricks.
It's also worth noticing what the contract does *not* promise:

- It does not try to recover from mismatches by patching individual nodes.
- It does not attempt to reconcile text/comment whitespace differences.
- It does not do event delegation or partial listener attachment.

15.6 The fast path: attach without rewriting

Given server markup like:

```
<div><span>A</span></div>
```

and a matching block definition:

```
templateHTML: `<div><span>A</span></div>`
slots: { a: { kind: 'text', path: [0, 0] } }
```

The hydration algorithm is:

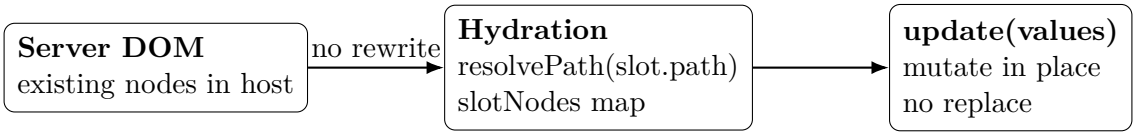
1. Read `serverRoot = host.firstElementChild`.
2. Validate root tagName against the template (cheap check).
3. Resolve every slot path against `serverRoot`.
4. Validate slot node types, and element tagNames against the template DOM.
5. Return a `HydratedBlock` whose `update()` writes into those resolved nodes.

Two details in `runtime/hydration.ts` make this efficient:

- Root tag validation is a fast regex over `templateHTML` (`getRootTagName`). This allows an early bail-out without parsing a template DOM.
- Slot resolution is just a `childNodes` index walk (`resolvePath`). It's similar to `runtime/block.ts`, but hydration keeps its own copy with different error strings.

Spec: `hydrates without rewriting DOM when markup matches`.
What the spec is really asserting is **node identity preservation**. You can think of hydration as creating a *mapping* from (slot key → DOM node reference) without changing the structure:

- before hydration: DOM nodes already exist (from server HTML)
- after hydration: we have slot node references pointing at those exact nodes
- on update: we mutate those nodes in place



That test captures node identities before hydration and asserts identity stability after calling `hydrateBlock`:

- same root element
- same inner `<span>`
- same text node

Then it updates slot values and checks the content changed without replacing nodes.

15.7 Mismatch detection: when to bail out

Hydration needs a mismatch policy. In v1 it is intentionally conservative.

15.7.1 Root tag mismatch

The first check is easy: compare the root tagName.

Implementation: `getRootTagName(templateHTML)` uses a small regex for the first tag. If root tags don't match, hydration does:

```
host.innerHTML = ''
return mountBlock(def, host, initialValues)
```

This is the first key safety property: mismatched DOM never becomes a "half-hydrated" zombie tree. We either attach cleanly or we tear down and replace.

This strict policy gives you an important debugging invariant:

> If hydration succeeds, later slot writes will never throw because of a missing node or wrong element type (for the validated paths).

It's also why HRBR v1 hydration doesn't try to be clever about whitespace or comment-node drift: the moment your markup generation stops matching the compiler's structural expectations, hydration deliberately stops and remounts.

Spec: `mismatch triggers remount for subtree`.

15.7.2 Slot path mismatch: element tagName mismatch

Root tag matching is not enough. You can have the same root element but different internal structure.

The conservative check in `runtime/hydration.ts` is:

- Parse the expected template root into a detached DOM node.
- For each non-text slot:
  - resolve the same path against the expected template DOM
  - assert both are Elements
  - assert `expected.tagName === actual.tagName`

Why compare tag names at slot paths (instead of deep-walking the entire tree)?

- Slots are where we will write later, so validating those boundaries catches many mismatches that would otherwise crash later.
- It avoids paying for a full tree diff during hydration.
- It keeps the rule extremely easy to explain and test.

If any assertion fails, hydration catches and bails out with client remount.
Spec: `slot path element mismatch triggers bailout remount (conservative)`.

15.8 Why this design? (tradeoffs, alternatives, and invariants)

Hydration is an area where it's easy to ship something that works on your demo and breaks in production. So I made a deliberate v1 choice: **strict validation + bailout**.

Here are the tradeoffs I'm optimizing for:

- **Invariant: "hydrated means safe to update."** If hydration returns successfully, the slots you'll write to later have been validated against the template.
- **Low implementation risk.** Partial hydration systems can be correct, but they require more state and more surface area for mismatches.
- **Predictable failure mode.** When something is off (wrong tag, wrong path, wrong slot kind), we don't keep walking a tightrope. We clear and remount.

Alternatives (all valid, just not v1 Relax):

- **Deep tree diff during hydrate:** robust but expensive and complex.
- **Marker-based hydration:** embed explicit markers in HTML (comment anchors / data attributes). This is a common approach, and it's likely the direction for a more advanced Relax hydration.
- **Island / partial hydration:** hydrate only interactive regions. Great UX story, more tooling.

For a framework-agnostic overview of hydration strategies and tradeoffs, see https://web.dev/rendering-on-the-web/.

The test intentionally renders:
`<div><em>X</em></div>`
but the template expects:
`<div><span></span></div>`
So hydration remounts and you end up with a `<span>`.

15.9 Hydrated update semantics: same slot kinds as mountBlock

Hydration returns an object that looks like a mounted block:

- `root` is the server root element
- `slotNodes` maps slot keys to resolved DOM nodes
- `update(values)` writes into those nodes
- `destroy()` removes hydrated listeners and removes `root`

The update logic supports the full set of slot kinds:

- `text`: write `Text.nodeValue`
- `attr`: boolean attribute semantics and SVG xlink namespace
- `prop`: input correctness for `value` / `checked`
- `class`: string or array join
- `style`: string or object diff
- `event`: bind event handler(s)

Spec: `hydrates with full slot semantics (event/attr/prop/SVG) without rewriting existing DOM`.

Most of the slot patching logic is a copy of the mount-time block patching rules (Chapter 14), but hydration differs in one important place: event slots.

That test verifies:

- DOM identity is preserved for root/button/input/link
- clicking calls the hydrated handler
- boolean attribute `hidden` is present
- `checked` property toggles
- SVG `xlink:href` is set in the xlink namespace

15.9.1 Redundant write skipping

Hydration tracks `prevValues` and skips updates when `Object.is(prev, next)`. This matches the block runtime's "skip redundant writes" optimization.

This is a subtle performance win because it also prevents unnecessary listener churn (given hydration's v1 event strategy below).

15.10 Event slots: v1 difference vs mountBlock

There's an intentional simplification in hydration:

- `mountBlock` binds once and swaps `rec.current` (no listener churn).
- `hydrateBlock` removes the previous listener and re-adds on update.

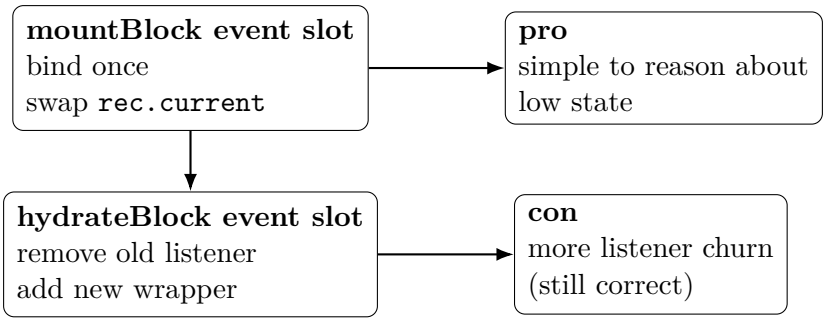
You can see this directly in `runtime/hydration.ts`:

- On `event` updates, it looks up `listenersByKey[k]`.
- If present, it calls `removeEventListener(prev.type, prev.handler)`.
- If the new value is a function, it creates a fresh wrapper handler and calls `addEventListener(slot.name, handler)`.

This is not as optimal as the bind-once approach in `mountBlock`, but it is still correct and easy to reason about.

Two important correctness notes (easy to miss):

- **Redundant writes matter more with this strategy.** If you didn't skip redundant writes, a repeated `update()` could churn listeners (remove+add) even when the handler didn't change.
- **Destroy must remove hydrated listeners.** Otherwise the host subtree could be detached but still reachable through listener closures.



The key safety property is lifecycle cleanup: `destroy()` removes any listeners hydration attached, so even in the "remove + re-add" strategy you don't leak handlers across updates.

It still avoids leaks because `destroy()` removes any listeners it attached.

This choice keeps hydration simpler while still correct. If hydration becomes a performance hot spot, it can be refactored to reuse the bind-once strategy.

15.11 SSR helper: renderBlockToString()

`runtime/ssr.ts` provides a minimal string renderer used mainly for tests.

Contract:

- create a detached host element
- `mountBlock(def, host, initialValues)`

- read `host.innerHTML`
- destroy the mounted block

This is explicitly a v1 teaching tool:

- It uses the browser's DOM implementation to do serialization.
- It's not the kind of streaming SSR engine you'd use in production.

But it has an important property: it guarantees that the HTML string shape matches `mountBlock` semantics, because it's literally produced by the same code.

Spec: `renders a block to HTML string, hydrates it, and updates via mountCompiledBlock`.

This gives a full lifecycle demo:

1. render HTML string from a block
2. insert it into the document
3. hydrate it without rewriting
4. mount a compiled block on top of the hydrated DOM and update via signals

15.12 Takeaways

- Hydration v1 is conservative: it validates structure and bails out on mismatch.
- When markup matches, hydration preserves DOM identity (no rewrites).
- Hydrated updates support the full slot matrix (text/attr/prop/class/style/event + SVG xlink).
- SSR v1 uses the DOM to produce strings for tests; it's simple and correct, not optimized.

15.13 Walking the tests (the executable spec)

Hydration and SSR are notoriously easy to get "mostly right" and still ship broken edge cases. Relax locks down the v1 behavior with two test files.

15.13.1 `runtime/__tests__/hydration.test.ts`

This suite defines the hydration safety model.

- **hydrates without rewriting DOM when markup matches** captures `root/span/text` node identities before hydration, hydrates with an initial value, and asserts:
  - the same elements/text nodes are still present (identity preserved)
  - only the content changes
- **hydrates with full slot semantics (event/attr/prop/SVG) without rewriting existing DOM** verifies the "slot matrix" works even when DOM is pre-existing:
  - event handler is attached and called

  - boolean-ish attr (`hidden`) is present
  - `checked` is set as a property
  - SVG `xlink:href` is set in the xlink namespace

- **mismatch triggers remount for subtree** uses the wrong root tag (`<section>`) and asserts hydration replaces the entire subtree.
- **slot path element mismatch triggers bailout remount (conservative)** uses the same root tag but the wrong inner tag at a slot path (`<em>` vs expected `<span>`). The test asserts that hydration bails out and remounts so the template structure is restored.

15.13.2 `runtime/__tests__/ssr.test.ts`

This test is a full lifecycle demo "in miniature":

1. **render** a block to an HTML string via `renderBlockToString`
2. insert the HTML into a host
3. **hydrate** without changing root/text node identities
4. **mountCompiledBlock** to wire signals and update the hydrated DOM

It's intentionally pragmatic: rather than relying on timing, it calls `mounted.update( value: 'B' )` to assert the DOM is updated. The key idea being demonstrated is: *hydration produces a block-shaped surface that the rest of HRBR can build on.*

15.14 Online resources

- MDN: `<template>` (hydration compares template structure by parsing)
- MDN: `firstElementChild` (the attach point)
- MDN: `Node.childNodes` (path-based DOM addressing)
- MDN: events + listener removal
- MDN: `xlink:href` (namespaced SVG attribute semantics)
- A broader hydration overview (framework-agnostic): web.dev: Rendering on the Web.

15.15 Common pitfalls

These are the mistakes I see most often when people attempt hydration systems (or when I debug my own early versions):

- **Treating whitespace as "not real DOM".** Paths are over `childNodes`, so whitespace text nodes count.
- **Hydrating when the template changed.** If `templateHTML` is an ABI, you must deploy server+client together.

- **Forgetting idempotence.** Hydration is often retried; cleanup must be safe even if called twice.
- **Listener leaks.** If you attach listeners during hydration, `destroy()` must remove them.

15.16 Mini-exercise

Open `runtime/hydration.ts` and make the mismatch error messages more actionable.

- Add the slot key and the path (like `[2,0]`) in the "invalid path" assertion.
- Ensure the tests in `runtime/__tests__/hydration.test.ts` would still pass conceptually.

The goal is to practice the key engineering habit for hydration: when things go wrong, logs must tell you *exactly* what structural assumption failed.

Chapter 16

SSR Authoring (v1): Rendering Blocks to HTML Strings

Server-side rendering (SSR) is the process of producing HTML *before* your client JavaScript boots. In practical terms, SSR answers one very specific question:

> What HTML should the server send so the client can *attach* to it without rewriting?

In HRBR v1, SSR is intentionally minimal. It primarily exists to make hydration testable and to provide a predictable authoring workflow. The key property is not raw server performance; it's **compatibility** with hydration.

16.1 What you'll build

By the end of this chapter you'll be able to:

- render a `BlockDef` to a string via `renderBlockToString`
- understand why SSR v1 is implemented as *mount into a detached host* → *serialize*
- reason about what breaks hydration, and how the SSR/hydration tests detect those breaks

16.2 What you'll learn

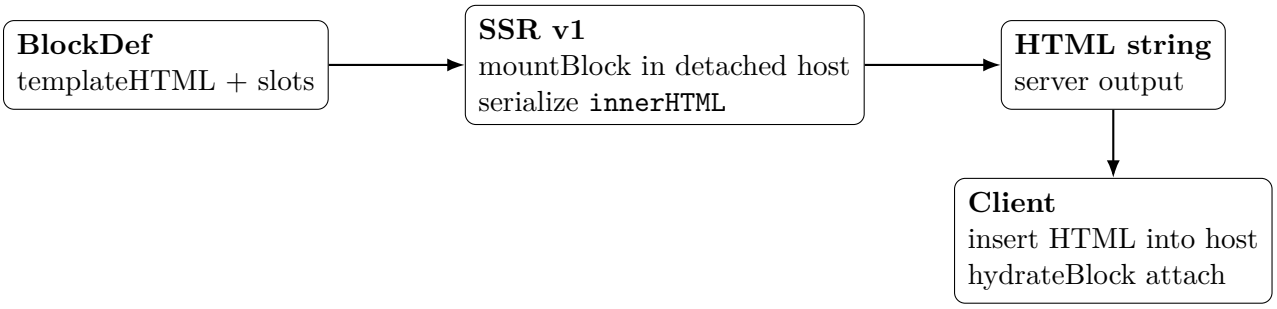
- the "ABI" idea: `templateHTML` is a stable structure shared by server render and client hydrate
- why HRBR treats SSR as a special case of the same slot-write machinery used on the client
- what SSR v1 explicitly does *not* attempt (DOM-free HTML emission, streaming, partial hydration)

16.3 Prerequisites

This chapter builds on:

- Chapter 14 (blocks: `templateHTML`, slots, and stable paths)

- Chapter 15 (hydration: validate + attach, else bailout/remount)



This chapter is grounded in:

- `runtime/ssr.ts`
- `runtime/__tests__/ssr.test.ts`
- and the follow-on behavior in `runtime/hydration.ts` (Chapter 15).

16.4 Why SSR exists in HRBR

If hydration is "attach to existing DOM", SSR is "produce DOM that is safe to attach to". So SSR in this runtime is less about doing rendering on the server in the abstract, and more about enforcing a strong invariant:

> The server must output markup whose structure matches what `mountBlock()` would create.

This is why SSR v1 reuses the block runtime: it reduces the number of places where HTML/slot semantics can diverge.

16.5 A tiny contract (what SSR v1 guarantees)

SSR is not "render an app" in the abstract. In HRBR v1 it is narrowly scoped:

1. You provide a `BlockDef` (template + slots).
2. You optionally provide `initialValues` (a slot-key → value map).
3. The runtime returns an HTML string that matches the same template structure the client will mount/hydrate.

Put differently:

> SSR v1 is *mountBlock + serialize*. Hydration v1 is *validate + attach, else remount*.

That quote is a useful mental checksum: if you ever change block slot semantics (Chapter 14), you should expect SSR and hydration tests to fail until all three layers agree again.

This tight coupling is deliberate. It makes SSR and hydration "round-trip compatible" by construction.

16.6 The contract: `renderBlockToString(def, options?) -> string`

The only exported function is:

```
renderBlockToString(def: BlockDef, options?: { initialValues?: Record<string, unknown> }): stri
```

The options only support one thing today: `initialValues`.
This mirrors the core block update contract:

> A block update is just a map `{ [slotKey]: value }`.

SSR is simply the "apply values, then serialize" version of that idea.

16.6.1 A minimal end-to-end usage (SSR → hydrate → update)

The fastest way to internalize SSR v1 is to look at how it composes with hydration. The test suite models this exact authoring loop:

```
import { defineBlock, mountCompiledBlock } from 'relaxjs/runtime/block'
import { renderBlockToString } from 'relaxjs/runtime/ssr'
import { hydrateBlock } from 'relaxjs/runtime/hydration'

const Counter = defineBlock({
  templateHTML: `<div class="c"><button id="inc">+</button><span id="t"> </span></div>`,
  slots: {
    value: { kind: 'text', path: [1, 0] },
  },
})

// "Server" side: first paint
const html = renderBlockToString(Counter, { initialValues: { value: 'A' } })

// "Client" side: insert + attach
const host = document.getElementById('host')
host.innerHTML = html
const hydrated = hydrateBlock(Counter, host)

// Later: incremental updates can happen via hydration...
hydrated.update({ value: 'B' })

// ...or via compiled slots (Chapter 14/15 composition story)
const mounted = mountCompiledBlock(Counter, host, [{ key: 'value', read: () => 'C' }])
mounted.update({ value: 'C' })

mounted.dispose()
hydrated.destroy()
```

Two important observations:

- SSR returns *just a string*. The correctness part is whether the client can attach without rewriting.
- Both hydration and compiled mounts boil down to the same contract: `update({ [slotKey]: value })`.

One way to think about it:

- **Client mount:** template → DOM instance + slot node references.
- **Client update:** slot values → DOM writes.
- **Server render:** slot values → DOM writes → serialize.

16.7 Implementation: DOM-based SSR for correctness

`runtime/ssr.ts` implements SSR using the platform DOM:

1. create a detached host element: `document.createElement('div')`
2. mount the block into that host via `mountBlock(def, host, initialValues)`
3. read `host.innerHTML`
4. destroy the mounted block to avoid leaks
5. return the HTML string

This is documented directly in the function comment:

- it uses a detached DOM
- it serializes via `innerHTML`
- it's primarily intended for tests/examples
- a future version could emit HTML without a DOM

The crucial correctness choice is that SSR applies `initialValues` by calling `mountBlock(def, host, initialValues)`. That means:

- all the slot semantics from Chapter 14 apply (boolean attributes, SVG namespaces, input properties, etc.)
- the generated markup shape matches what the client expects to attach to

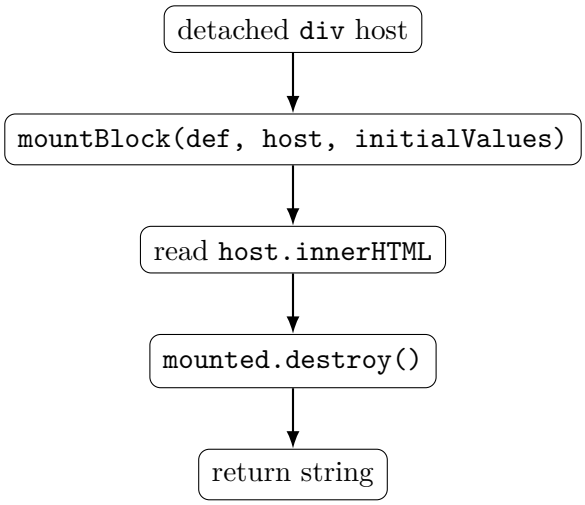
16.7.1 Why DOM-based SSR is acceptable in v1

For a production SSR renderer, involving a full DOM implementation server-side can be expensive.

But for v1 HRBR, this has two big advantages:

- it is hard to get HTML escaping and attribute semantics wrong
- it produces markup exactly consistent with the same block runtime used in the client

So correctness comes for free.



There's also a subtle lifecycle advantage: because SSR uses `mountBlock`, tearing down is just `mounted.destroy()`. That ensures listeners are removed and the temporary root is cleaned up, even in a test environment.

16.7.2 The implementation is short on purpose

SSR v1 is deliberately a thin adapter. The actual code in `runtime/ssr.ts` is essentially:

```
export function renderBlockToString(def, options = {}) {
  const host = document.createElement('div')
  const mounted = mountBlock(def, host, options.initialValues ?? {})
  const html = host.innerHTML
  mounted.destroy()
  return html
}
```

This design matters because it centralizes correctness. Instead of re-implementing HTML escaping, boolean attributes, SVG namespaces, and slot kinds in a second renderer, SSR simply uses the same mount/update semantics as the client.

16.8 SSR must round-trip with hydration

SSR isn't useful unless the client can hydrate it.

Hydration v1 makes strong assumptions (Chapter 15):

- the root element tagName matches `templateHTML`
- slot paths resolve to the expected node types
- element slots match expected tagNames along the path

Therefore, SSR output must preserve the template's structure. Because SSR uses `mountBlock()`, it naturally meets the same invariant:

> SSR uses the same template parser and slot patching semantics as client mount.

This is the important "authoring rule" for SSR v1:

> If you change `templateHTML` shape, you must change both server render and client hydrate together.

You should treat `templateHTML` as a stable ABI between server and client.

16.8.1 Authoring rule: structure changes are breaking changes

Hydration v1 validates aggressively and bails out if anything looks unfamiliar. That implies a simple rule of thumb:

- Changing slot *values* is safe.
- Changing `templateHTML` *structure* is a breaking change for hydration.

One subtle (and common) structural foot-gun: paths are `childNodes`-based. Whitespace can introduce text nodes and shift indices. So "just add formatting" in the HTML string can become a breaking change if it changes the node tree.

16.9 The end-to-end spec: render → hydrate → update

The key test is `runtime/__tests__/ssr.test.ts`:

1. Define a block with `templateHTML` and a text slot.
2. Render HTML with `renderBlockToString(block, { initialValues })`.
3. Insert the returned string into the DOM.
4. Call `hydrateBlock(block, host)`.
5. Verify DOM identity is preserved (same root, same text node).
6. Mount a compiled block via `mountCompiledBlock(...)` and update.

This accomplishes two things:

- it proves that SSR output is "hydrate compatible"
- it shows how hydration and compiled blocks can be composed (a realistic client story)

One implementation-level nuance in the test is worth calling out. After updating the signal with `setCount('B')`, it also calls:

> textttmounted.update({ value: 'B' })

That makes the assertion deterministic and keeps the test independent of scheduler timing. It's modeling the fact that `mountCompiledBlock` can schedule work, and test environments often differ in how timers/microtasks flush.

16.9.1 Why the test checks identity, not just text

It's easy to write a system that "hydrates" by discarding server HTML and remounting. But HRBR's hydration story is explicitly about preserving DOM identity when markup matches. So the test stores references like:

- `rootBefore = host.firstElementChild`
- `textBefore = host.querySelector('#t')?.firstChild`

and asserts they're the same after `hydrateBlock()`. That's the heart of the v1 contract: *attach without rewriting, or remount fully.*

16.10 Limitations and the intended direction

The HRBR whitepaper explicitly lists a non-goal for v1:

> A DOM-free string renderer (v1 SSR uses a DOM-based approach for tests/examples)

A future SSR implementation could:

- parse `templateHTML` once into a token stream
- apply slot writes into a string builder
- avoid constructing DOM nodes entirely

Even if SSR becomes DOM-free later, the invariants from hydration still apply:

- the root tag must match
- the structure along slot paths must match
- text nodes must exist where text slots expect them

But even then, the contract remains:

> SSR output must match the template structure so hydration can do a no-rewrite attach.

16.11 Takeaways

- SSR v1 is intentionally minimal: mount a block in a detached DOM and serialize `innerHTML`.
- `initialValues` uses the same slot update contract as client updates.
- The primary goal is hydration compatibility, not server performance.
- The end-to-end test shows SSR + hydration + compiled updates working together.

16.12 Walking the tests (the executable spec)

SSR v1 has one test file, but it covers the entire story.

16.12.1 `runtime/__tests__/ssr.test.ts`

The test `renders a block to HTML string, hydrates it, and updates via mountCompiledBlock` enforces these guarantees:

- **SSR applies initial values:** `renderBlockToString(..., { initialValues })` must produce HTML containing `'A'`.
- **Hydration preserves identity:** after `hydrateBlock`, the root element and the text node under `#t` are the same objects as before hydration.
- **Compiled updates work on hydrated DOM:** wiring a signal through `mountCompiledBlock` and then updating should change text to `'B'`.
- **Cleanup is explicit:** the test disposes the compiled mount and destroys the hydrated block.

Because this file touches SSR, hydration, signals, and compiled blocks, it acts as a integration-level "safety rail" for the whole pipeline.

16.13 Common pitfalls

- **Thinking SSR is string concatenation:** in HRBR, correctness is driven by slot semantics (attr vs prop vs SVG namespaces).
- **Forgetting paths are `childNodes`-based:** whitespace and formatting can change indices.
- **Assuming partial hydration:** v1 is conservative; mismatch triggers remount.
- **Forgetting cleanup in examples/tests:** SSR cleans up its own temporary mount; hydration/compiled mounts still require explicit `destroy()/dispose()`.

16.14 Mini-exercise

Extend the SSR test scenario conceptually by adding an attribute slot and updating it after hydration.

1. Add a `hidden attr` slot on the `<button id="inc">`.
2. SSR render with `hidden: false` (should not emit the attribute).
3. Hydrate without rewriting.
4. Update with `hidden: true` and verify the attribute exists.

This exercise forces you to connect all three layers: slot semantics (Chapter 14), hydration attach rules (Chapter 15), and SSR reuse of mount semantics (this chapter).

16.15 Online resources

- HTML serialization via DOM: MDN: `Element.innerHTML`.
- Server rendering and hydration overview: web.dev: Rendering on the Web.
- Template element (used by mounting/hydration): MDN: `<template>`.
- A compiler-first argument for skipping VDOM in the steady state: Svelte: "Virtual DOM is pure overhead".

Chapter 17

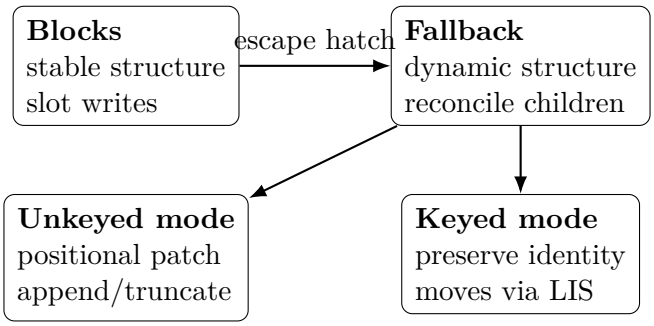
Fallback Reconciler: Dynamic Structure When Blocks Aren't Enough

Blocks are fast because structure is stable: the template doesn't change, only slot values do.

But JSX in the real world has loops, conditionals, fragments, and dynamic lists. HRBR needs a safe escape hatch for those regions.

That escape hatch is the **fallback reconciler**:

- a runtime mount: `runtime/fallback.ts`
- a child-list reconciler: `runtime/reconciler.ts`



This chapter is grounded in:

- `runtime/fallback.ts`
- `runtime/reconciler.ts`
- `runtime/__tests__/fallback.test.ts`
- `runtime/__tests__/reconciler.test.ts`
- `runtime/__tests__/reconciler.fuzz.test.ts`

17.1 What you'll build

You're going to build a correct mental model for HRBR's "dynamic structure" pipeline. By the end, you should be able to:

- represent a dynamic list as an array of `ReconcileItem` specs
- predict exactly when HRBR preserves DOM identity (keyed) vs when it can't (unkeyed)
- understand why `ReconcileRange` exists (one logical item → multiple DOM nodes)
- use `mountFallback(host, render)` to wire a signal-driven list to DOM updates

17.2 What you'll learn

- why fallback exists in a compiler-first renderer: not a failure, a deliberate escape hatch
- the direct-children-only contract (the reconciler patches a list; each item owns its subtree)
- keyed reconciliation mechanics: range metadata, old-index sequence, LIS to minimize moves
- the relationship between fallback and the scheduler (updates are scheduled, not always immediate)

17.3 Prerequisites

This chapter assumes you've read:

- Chapter 14 (compiled blocks: stable template + slot writes)
- Chapter 15 (hydration: preserving node identity is the whole point)
- Chapter 16 (SSR: markup must round-trip with client attachment)

If you're new to keyed list rendering concepts, React's "why keys matter" background is useful context (even though HRBR's implementation is different): https://react.dev/learn/rendering-lists.

17.4 Why fallback exists (the honest reason)

The HRBR block model is intentionally strict: structure must be stable. That gives you very cheap updates (slot writes) and very strong guarantees (paths must resolve, node types must match).

But real UI code contains *structural* changes:

- conditionals toggle subtrees
- loops add/remove/reorder items
- fragments and multi-root components produce multiple siblings

Trying to force all of that into a single stable `templateHTML` would either:

- explode the template into a huge "unrolled" superset, or
- require a second structural patcher anyway.

So fallback is not "blocks didn't work". Fallback is the explicit boundary where HRBR chooses correctness and generality over the block fast-path.

17.5 A tiny contract (what fallback + reconciler promise)

The fallback reconciler is the HRBR answer to "dynamic structure":

- Blocks are optimal when structure is stable.
- Fallback reconciliation is the safe, general mechanism when structure changes.

The contract is intentionally narrow:

1. `reconcileChildren(host, next)` only patches **direct children** of `host`.
2. Each child spec is responsible for building/patching its own subtree (if any).
3. In keyed mode, keys must be unique among siblings.
4. In keyed mode, DOM identity is preserved by key when possible; the algorithm minimizes moves, but *correctness* is the first goal.

`mountFallback(host, render)` is the reactive wrapper:

- `render()` can read signals.
- When those signals change, fallback schedules a reconcile pass in a scheduler lane.
- Disposal stops further reactive updates.

17.6 Contracts

17.6.1 The reconciler contract: direct children only

`reconcileChildren(host, next, options?)` patches `host.childNodes` so they match `next`.

Important: it reconciles **direct children only**. Each child spec is responsible for patching its own subtree.

That design choice keeps the reconciler small and composable:

- A "child spec" can represent a component, a fragment, or even an embedded compiled block.
- Nested reconciliation happens inside that child spec's own `patch()` function.

Types (`runtime/reconciler.ts`):

- `ReconcileNode`: a logical node that maps to exactly one DOM node
- `ReconcileRange`: a logical item that maps to multiple DOM nodes
- `ReconcileItem = ReconcileNode | ReconcileRange`

Each item supports:

- `create()` to create DOM
- `patch(existing)` to patch in place
- optional `destroy(existing)` to cleanup when removed

17.6.2 From scratch: what is a `ReconcileItem`?

When you use `reconcileChildren(host, next)`, you aren't passing DOM nodes. You're passing *instructions* for how to create or patch them:

- **Create:** how to build the DOM the first time.
- **Patch:** how to update an existing DOM node (or nodes) when re-rendering.
- **Destroy (optional):** cleanup hooks for removed items.

The smallest useful example is a keyed text node spec:

```
function keyedText(key, text) {
  return {
    key,
    create() {
      const el = document.createElement('span')
      el.textContent = text
      return el
    },
    patch(node) {
      node.textContent = text
    },
  }
}
```

Now strengthening this idea: the reconciler doesn't know (or care) if this `span` represents a component, a list item, or a compiled block. It only cares about mapping "logical items" to DOM children.

17.6.3 Keyed vs unkeyed mode

`reconcileChildren` chooses mode:

- if `options.keyed` is provided, use it
- else infer keyed mode if any next item has a key

It also emits dev-only warnings for suspicious usage patterns when `opts.dev === true`:

- mixing keyed and unkeyed siblings (unkeyed nodes become "unstable")
- replacing an entire keyed list with a disjoint set of keys (often unstable keys like array index)

The two algorithms are:

- `reconcileChildrenUnkeyed(host, next)`
- `reconcileChildrenKeyed(host, next, dev)`

17.7 Why ranges exist (one logical item => multiple DOM nodes)

In UI rendering, one logical item can expand to multiple nodes:

- a fragment
- a component that returns multiple roots
- a compiled block or template that expands into siblings

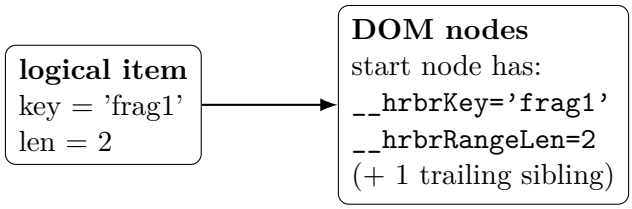
HRBR models this with `ReconcileRange`:

```
{ kind: 'range', key?, create(): Node[], patch(nodes: Node[]) }
```

The reconciler stores the range length on the first DOM node:

- `__hrbrKey`: key of the logical item (on the start node)
- `__hrbrRangeLen`: number of DOM nodes in the range (on the start node)

This metadata is the core trick that lets the reconciler treat DOM as a list of *logical items* rather than raw nodes.



The helper `buildCurrentRanges(host)` reads `childNodes` and groups them into ranges by repeatedly:

- read `len = __hrbrRangeLen ??  1` from the next start node
- slice `len` nodes and treat them as one logical item
- advance by `len`

This is also why correctness requires normalization when items change shape (node ↔ range). Spec: `keyed:  supports range items (logical item => multiple DOM nodes)`.

17.7.1 Range authoring example (fragment-like output)

If one logical thing needs to output two siblings, model it directly:

```
function keyedRange(key, aText, bText) {
  return {
    kind: 'range',
    key,
    create() {
      const a = document.createElement('span')
      a.textContent = aText
      const b = document.createElement('span')
```

```
      b.textContent = bText
      return [a, b]
    },
    patch(nodes) {
      nodes[0].textContent = aText
      nodes[1].textContent = bText
    },
  }
}
```

Internally, HRBR tags the first DOM node so it can re-discover logical items later: `__hrbrKey` stores the logical key and `__hrbrRangeLen` stores the range length. That's how a flat DOM child list becomes a list of logical items again.

17.8 Unkeyed reconciliation: simple and predictable

`reconcileChildrenUnkeyed` relies on positional patching:

1. patch common prefix (min(oldLen, newLen))
2. remove extra old nodes
3. append new nodes

Spec: `unkeyed: patches in order and truncates/appends`.
Unkeyed mode is fast and simple, but it can't preserve identity across reorders.
Unkeyed mode is best when:

- lists are small
- items rarely reorder
- you don't care about preserving DOM identity across reorder (e.g. no input state)

17.9 Keyed reconciliation: preserve identity and minimize moves

Keyed mode tries to:

- reuse DOM nodes by key
- patch in place instead of recreating
- move existing ranges/nodes to new positions
- minimize DOM moves using a Longest Increasing Subsequence (LIS)

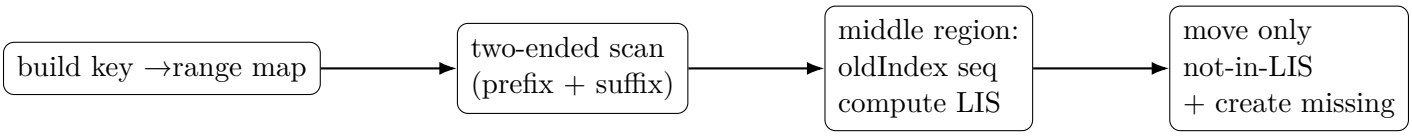
Specs in `runtime/__tests__/reconciler.test.ts` cover the basic expectations:

- `keyed: moves DOM nodes instead of recreating them`
- `keyed: removes nodes that are no longer present`
- `keyed: inserts new nodes at the right position`

17.9.1 Two-ended scan + LIS

The reconciler uses a standard sequence:

1. Build a map from key to current range (start..end, nodes[]).
2. Fast two-ended scan for common prefix/suffix keys.
3. For the middle region:
   - build an "old index" sequence for new keys
   - compute LIS indices
   - move only the ranges not in the LIS



Why LIS?

- If a subsequence of items already appears in increasing old-index order, those items can stay where they are.
- Everything else must move (or be created), but LIS keeps that set minimal.
- The implementation uses `lisIndices(seq)` over old indices, ignoring `-1` (new items).

17.9.2 Why keyed mode still needs dev diagnostics

Keyed algorithms only work when the keys are meaningful. That's why `reconcileChildren(..., { dev: true })` warns in two important cases (see `runtime/__tests__/reconciler.test.ts`):

- **Mixed keyed and unkeyed siblings:** unkeyed nodes will be treated as unstable.
- **No overlap between previous and next keys:** often indicates unstable keys (like array index).

This is a practical "foot-gun detector". The runtime can't fix unstable keys for you, but it can tell you when your list semantics are likely wrong.

17.10 Fallback mount: wiring reconciliation to reactivity

The reconciler itself is just a pure patcher: you call it when you want.

`mountFallback()` is the bridge from signals (Chapter 12) to structural patches:

- your `render()` reads signals
- an effect tracks those reads
- when signals change, the fallback schedules a reconcile pass via the scheduler

The key detail in `runtime/fallback.ts`: updates are *scheduled*. So tests use a tick (`setTimeout(0)`) to wait for the scheduled reconcile.

17.10.1 Fallback example (mirrors the integration test)

This is essentially `runtime/__tests__/fallback.test.ts`, expressed as a teaching example:

```
const host = document.createElement('div')
const [items, setItems] = createSignal(['a', 'b', 'c'])

const mounted = mountFallback(host, () => {
  const children = items().map((k) => keyedText(k, k.toUpperCase()))
  return { children }
})

// Initial paint is immediate
host.textContent // => "ABC"

// Reorder + insert
setItems(['c', 'a', 'b', 'd'])
// ... later (scheduled)
host.textContent // => "CABD"

mounted.dispose()
```

Notice what this proves:

- structural change is handled (reorder, insert, remove)
- DOM identity is preserved by key (the test asserts the same nodes moved)

Implementation cues:

- `buildCurrentRanges(host)` groups DOM nodes into logical ranges by `__hrbrRangeLen`.
- `lisIndices(seq)` computes LIS over old indices (ignoring `-1`).
- `moveRangeBefore(host, range, anchor)` moves nodes end-to-start to preserve order.

17.10.2 Dev diagnostics for suspicious usage

Keyed mode includes two dev-only warnings:

- Mixed keyed and unkeyed children in the same list.
- Next keys have no overlap with previous keys (often indicates unstable keys).

Specs:

- `dev diagnostics: warns on mixed keyed/unkeyed children in keyed mode`
- `dev diagnostics: warns when next keys have no overlap with previous keys`

It also asserts that keys are unique among siblings.
That uniqueness assertion is a hard error, not a warning:

textttassert(!seen.has(k), '[hrbr/reconciler] duplicate key ...')

This is intentional: duplicate keys make identity preservation ill-defined.

17.10.3 Shape-change safety: node ↔ range

A tricky edge case: a key might render as a singleton node in one render and as a range in another.

The reconciler handles this explicitly:

- if key matches but logical shape changes (node vs range), it removes the old nodes and recreates
- it aggressively removes any *unkeyed trailing nodes* that might remain after a range collapses into a singleton

This is where the metadata invariants matter. The code contains normalization passes that:

- remove unkeyed nodes owned by keyed mode
- delete stale `__hrbrRangeLen` when the logical item is now a singleton

The fuzz test exists largely to guard these invariants.

One specific implementation detail to notice is `cleanupTrailingRangeNode(start, key)`. When a keyed item collapses from a range into a singleton node, the DOM may still contain trailing unkeyed nodes that used to be part of that range. The reconciler aggressively removes any contiguous unkeyed siblings immediately after the start node.

This is later reinforced by a final normalization pass that removes any unkeyed nodes interleaved between keyed logical items.

17.11 Property/fuzz testing as an executable oracle

`runtime/__tests__/reconciler.fuzz.test.ts` runs 250 seeded steps.

It compares `reconcileChildren(host, ...)` to a reference renderer that rebuilds the list each step but reuses nodes by key.

The key correctness check is not "minimal moves". It is:

> For the same list of logical items, the resulting DOM represents the same logical structure and text content.

The fuzz harness reads the logical structure with:

- start key: `__hrbrKey`
- range length: `__hrbrRangeLen` (default 1)

Then it compares the logical tuples (key, len, texts[]) against the reference.

The fuzz test is doing something important for maintenance:

- It doesn't claim the reconciler is optimal.
- It claims that for any seeded sequence of updates, the reconciler produces the *same logical output* as a simpler reference "rebuild" strategy.

That's exactly the right kind of oracle for reconciliation code: optimality is a performance concern, but correctness is a binary property.

17.12 Fallback mount: reconciling signal-driven dynamic children

`runtime/fallback.ts` provides a reactive mount function:

`mountFallback(host, render, { lane?, scheduler?, keyed? })`

Contract:

- `render()` may read signals.
- When signals change, schedule a reconcile pass in the requested lane.
- The render result is `{ children: ReconcileItem[], keyed? }`.

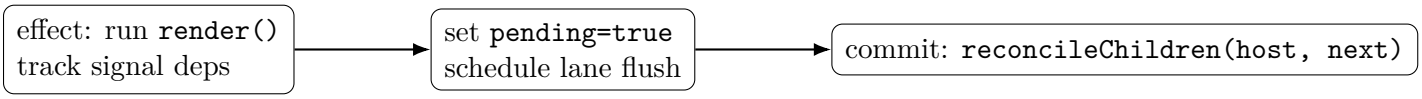
Implementation uses two passes:

- An effect that runs `render()` just to track dependencies, then schedules.
- A `runOnce()` pass that actually calls `reconcileChildren()`.

This split is subtle but intentional:

- The tracking effect should be very cheap; it exists to subscribe to signals.
- The DOM work should be scheduled and batched (via `batch(() => runOnce())`).
- A `pending` flag ensures multiple invalidations coalesce into one scheduled reconcile.

In other words, `mountFallback` mirrors the same "coalesce and commit" philosophy used by `mountCompiledBlock` (Chapter 14): collect changes reactively, then perform one DOM commit.



Spec: `reconciles structural changes when signal-driven list changes` (`fallback.test.ts`). That integration test verifies that:

- reorders preserve node identity by key
- inserts create new nodes
- removals shrink the DOM
- disposing stops reactive updates

17.13 Takeaways

- The fallback reconciler gives HRBR a safe strategy for dynamic structure.
- Keyed reconciliation preserves identity and minimizes moves via two-ended scan + LIS.
- Ranges let one logical item expand to multiple DOM nodes, enabling fragments and multi-root outputs.
- Metadata invariants (`__hrbrKey`, `__hrbrRangeLen`) are crucial; fuzz tests enforce them.
- `mountFallback` bridges signals/scheduler to `reconcileChildren` for real apps.

17.14 Walking the tests (the executable spec)

17.14.1 `runtime/__tests__/reconciler.test.ts`

This suite locks down the core expectations:

- **Dev warnings are emitted:** by temporarily replacing `console.warn`, the tests prove warnings fire for mixed keyed/unkeyed children and for disjoint key sets.
- **Keyed reorder preserves identity:** the "moves DOM nodes" test captures original nodes `a0/b0/c0` then reorders and asserts node identity is preserved.
- **Keyed removals:** nodes that disappear from `next` are removed, while survivors keep identity.
- **Keyed inserts:** missing keys are created and inserted at the correct position.
- **Unkeyed mode is positional:** patches first, then truncates/appends, without identity guarantees across reorder.
- **Ranges work:** a range key produces two DOM nodes and behaves as a single logical item during moves/inserts/removals.

17.14.2 `runtime/__tests__/reconciler.fuzz.test.ts`

This is the long-term safety net. It runs 250 seeded steps and compares the reconciler output against a reference renderer.

Key idea: the test reads *logical structure* from the DOM via `__hrbrKey` and `__hrbrRangeLen`, then compares (key, len, per-node text) tuples. If there's a mismatch, it throws an error that includes seed, step, next specs, and DOM dumps to make replay/debugging practical.

17.14.3 `runtime/__tests__/fallback.test.ts`

This test shows the reconciler in its intended reactive setting:

- a signal produces a keyed list
- changing the signal reorders, inserts, and removes
- identity is preserved for reused keys
- `dispose()` stops further reactive updates

The test uses `await new Promise((r) => setTimeout(r, 0))` to wait for the scheduled reconcile to run.

17.15 Online resources

- A classic keyed diff discussion (and why identity matters): React docs: Rendering Lists (keys).
- Longest Increasing Subsequence (the core move-minimization idea): Wikipedia: Longest increasing subsequence.

- A VDOM implementation reference with LIS-based keyed diff (for historical comparison): Vue docs: Maintaining State with `key`.
- DOM child manipulation primitives used by the reconciler: MDN: `Node.insertBefore` and MDN: `Node.removeChild`.

Chapter 18

VDOM ↔ HRBR interop (mixed trees)

This chapter covers the experimental interop layer that allows compiled HRBR output to live *inside* a Relax VDOM tree. The goal is practical: you can adopt HRBR incrementally (or only for hot paths) without rewriting your whole app.

What youĺl build

By the end of the chapter youĺl be able to:

- return an HRBR block from a normal Relax VDOM component (mixed trees)
- understand why HRBR needs a wrapper host element and why that host is the thing the VDOM "owns"
- predict when patching will reuse an existing HRBR instance vs dispose/destroy/remount it
- reason about event handler binding semantics when HRBR uses VDOM-style component instances

Prerequisites

You should already be comfortable with:

- the Relax vnode model (Chapter **??**)
- the mount/patch/destroy split in the DOM runtime (Chapters on `mountDOM` and `patchDOM`)
- the HRBR mental model: "mount into a host, then the runtime owns whatś inside"

Why this matters (the real-world motivation)

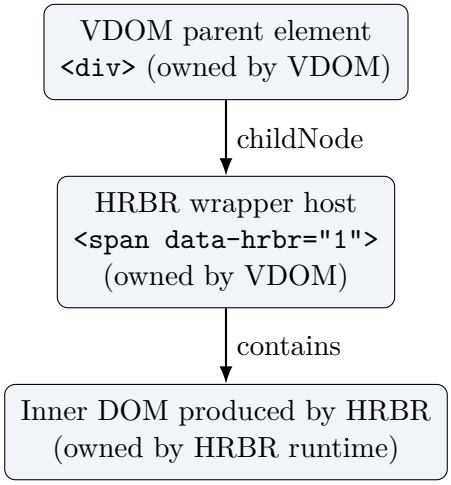
In production UIs, you rarely get to rewrite everything. Interop gives you a migration path:

- keep most of the app in VDOM where the ergonomics are great
- drop in HRBR for hotspots (large lists, heavy patching, micro-interactions) where you want tighter control

- avoid building "two apps" and stitching them manually

This design is also a useful general lesson: if you ever embed one rendering runtime into another, you need a *boundary object* that both runtimes can agree on. In Relax, that boundary object is a wrapper host element.

Diagram: where HRBR sits inside a VDOM tree



This "wrapper host" trick is what makes generic VDOM replace/move logic still work: VDOM only needs a single stable child node to own.

18.1 The tiny contract: an HRBR vnode is just a mount factory

In the VDOM layer, HRBR is represented by a new vnode type, `DOM_TYPES.HRBR`, implemented in `src/h.ts`.

18.1.1 VNode shape

The HRBR vnode type is defined as `HrbrVNode`:

- `type: DOM_TYPES.HRBR`
- `mount(host: Element) -> instance`
- internal bookkeeping fields: `host`, `instance`, `el`

The important part is that `mount` is referentially meaningful. Two HRBR vnodes are treated as the "same" subtree iff:
textttold.mount === new.mount.

In other words, *mount factory identity is the key.* This identity flows through three separate places that must agree:

- `src/nodes-equal.ts`: HRBR equality is `old.mount === new.mount`.
- `src/patch-dom.ts`: HRBR patching reuses the mounted instance on that same condition.

- `src/destroy-dom.ts`: HRBR teardown calls the instance hooks and removes the host wrapper.

This isn't an accident: it lets the VDOM runtime treat HRBR the same way it treats components. "Same function identity" means "keep state/instance".

18.1.2 A tiny contract you can hold in your head

If you forget everything else, remember this:

> The VDOM runtime treats an HRBR vnode like a component instance whose "type" is the `mount` function identity.

That yields a very concrete set of guarantees (and constraints):

- If you keep returning the same `mount` function (same reference), you keep the same mounted HRBR instance.
- If you return a new `mount` function reference, you are asking the VDOM runtime to throw away and rebuild that subtree.
- The VDOM runtime does *not* diff the DOM inside the HRBR host. Thatś HRBRś job.

This kind of contract shows up in many UI runtimes:

- React treats function component identity as the key to preserving state ("same function, same hooks state"). See https://react.dev/learn/preserving-and-resetting-state.
- Solid treats compiled templates as stable "owners" that update reactive expressions without remounting DOM. See https://www.solidjs.com/docs/latest#the-compiler.

18.1.3 HRBR instances: a minimal lifecycle interface

The VDOM runtime does *not* understand HRBR internals. It only requires that `mount(host)` returns an object with the following hooks:

- `destroy(): void` — required (DOM and internal cleanup).
- `dispose?: () => void` — optional (reactivity teardown, unsubscribe).
- `update?: (values: any) => void` — optional (to push new values).

This interface matches the objects returned from the HRBR runtimes (compiled blocks and fallback mounts), but it's also easy to fake in tests.

18.1.4 Creating HRBR vnodes: `hBlock()`

The helper `hBlock()` in `src/h.ts` wraps a mount factory into an HRBR vnode:

```
hBlock((host) => mountCompiledBlock(def, host, compiledSlots))
```

This is the shape the compiler produces (or that you can use manually for experiments).

Code: the actual `hBlock()` helper

The source implementation is intentionally tiny: itś just a tagged vnode:

```
function hBlock(mount) {
       return { type: DOM_TYPES.HRBR, mount }
}
```

That simplicity is a feature:

- the VDOM runtime doesnt́ need to know about blocks, templates, or signals
- HRBR is "just another vnode type" as far as mount/patch/destroy are concerned

18.2 Mounting: why HRBR gets its own host container

Mounting happens in `src/mount-dom.ts`. The main `mountDOM()` switch includes a new case:

```
case DOM_TYPES.HRBR: createHrbrNode(vdom, parentEl, index)
```

18.2.1 `createHrbrNode()`: mount into a dedicated `<span>`

`createHrbrNode()` creates a dedicated host element (currently a `span`) and inserts it into the parent:

- It sets `host.dataset.hrbr = '1'` to make the host easily recognizable in the DOM.
- It stores the host on the vnode: `vdom.host = host`.
- It calls `vdom.mount(host)` and stores the returned instance.
- It sets `vdom.el = host`.

The decision *to set `vdom.el` to the host container* is subtle but critical.

Why `vdom.el = host` matters

Relax's patcher has a generic "replace" path:

> if nodes aren't equal ⇒ find index of `oldVdom.el` in `parentEl.childNodes` ⇒ destroy old ⇒ mount new at the same index.

That index scan only works if `oldVdom.el` is actually a direct child of `parentEl`. For HRBR, the direct child is the wrapper host, not the inner DOM nodes produced by the HRBR runtime.

So `createHrbrNode()` intentionally makes the wrapper the unit of ownership:

- moves are moves of the wrapper
- removals remove the wrapper
- inner churn stays encapsulated in HRBR

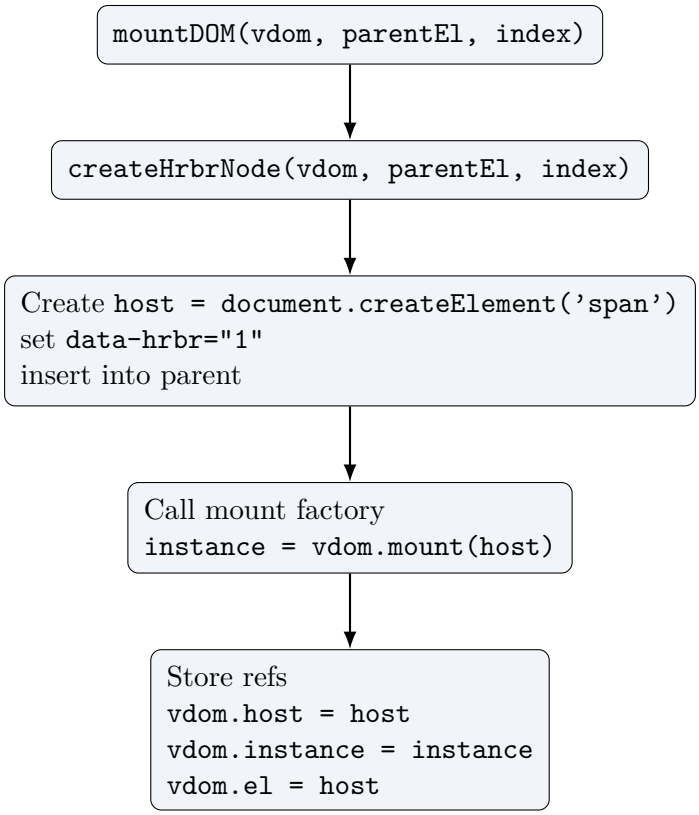
Even though the vnode type includes an `el?: Element` field described as "first element produced by the block" in `src/h.ts`, the mount code currently treats `el` as the wrapper host. This aligns with "VDOM owns the wrapper, HRBR owns the inside".

18.2.2 Edge cases this design avoids

Owning the wrapper host solves a bunch of subtle problems youĺl hit otherwise:

- **Replace correctness:** VDOM replace logic can always locate the subtree by scanning for `oldVdom.el` among direct children.
- **Move correctness:** keyed reorders can move one DOM node (the wrapper) instead of trying to reason about a variable-length range of nodes.
- **Encapsulation:** HRBR can freely insert/remove/patch children inside the host without violating VDOM invariants.
- **Teardown safety:** removing the host element guarantees the whole subtree is gone, even if an inner runtime makes mistakes.

Diagram: mount path (VDOM creates host, HRBR fills it)



A small but useful marker: `data-hrbr="1"`

The wrapper uses `host.dataset.hrbr = '1'`. This isn't required for correctness, but it's very handy when inspecting mixed trees in DevTools: you can quickly spot the boundaries between VDOM-managed and HRBR-managed DOM.

18.3 Patching: stability by mount factory identity

Patching happens in `src/patch-dom.ts`. The HRBR case delegates to `patchHrbr(old, next)`.

18.3.1 Fast path: stable mount function means reuse

`patchHrbr()` first checks referential identity:

```
if (old.mount === next.mount) { copy host/instance/el; return; }
```

This mirrors component identity rules: if the "type" of the subtree hasn't changed, we preserve the mounted instance.

In the actual implementation, the fast path copies internal bookkeeping from the old vnode to the new vnode:

- `host`
- `instance`
- `el`

That copy step is important because the patcher returns the *new* vnode object, and subsequent patches/destroys must be able to reach the original HRBR host and instance.

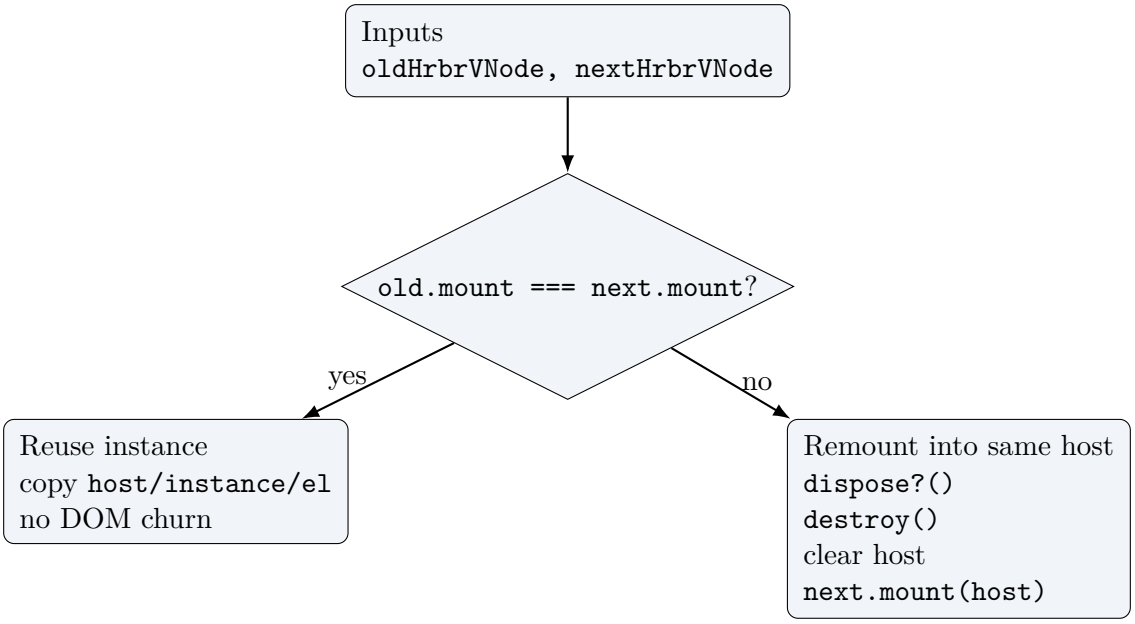
Code: fast-path reuse in `patchHrbr()`

The happy path is just a bookkeeping transfer:

```
function patchHrbr(oldVdom, newVdom) {
        if (oldVdom.mount === newVdom.mount) {
                // Keep the existing already-mounted instance.
                newVdom.host = oldVdom.host
                newVdom.instance = oldVdom.instance
                newVdom.el = oldVdom.el
                return
        }

        // ...remount path...
}
```

When you get this right, an HRBR subtree behaves like a normal component subtree with preserved state.

Diagram: patch decision (reuse vs remount)



18.3.2 Slow path: mount changed means dispose+destroy+remount

When `mount` changes, `patchHrbr()` treats it as a subtree replacement *inside the same outer HRBR vnode type*:

- reuse the same `host` container
- if the instance supports `dispose()`, call it
- call `destroy()` (required)
- clear the host's children
- call `next.mount(host)` and store the new instance

Note the interplay with `areNodesEqual()`:

- If `areNodesEqual(old, next)` returns *false*, `patchDOM()` goes through the full replace path (destroy old vnode + mount new vnode at the same index).
- If equality says they are the "same type" (both HRBR), then `patchHrbr()` is responsible for remounting when the mount factory changes.

This makes HRBR a first-class citizen in the patcher without teaching the patcher HRBR internals.

Code: remount ordering (dispose, destroy, clear, mount)

The slow path is still small, but order matters:

```
function patchHrbr(oldVdom, newVdom) {
        if (oldVdom.mount === newVdom.mount) return

        const host = oldVdom.host
        const inst = oldVdom.instance

        // 1) detach reactivity / cancel scheduled work
        if (inst && inst.dispose) inst.dispose()

        // 2) cleanup DOM and internal state
        if (inst && inst.destroy) inst.destroy()

        // 3) ensure the host is empty even if destroy() was partial
        while (host.firstChild) host.removeChild(host.firstChild)

        // 4) mount the new subtree into the same wrapper host
        const nextInst = newVdom.mount(host)
        newVdom.host = host
        newVdom.instance = nextInst
}
```

Thereś a deliberate "belt and suspenders" step here: even if `destroy()` forgets to remove some nodes, the host is forcibly emptied. That makes remounts deterministic.

18.3.3 A note on where remounts happen

There are two remount scenarios and they get handled in different layers:

- **VNode type changes** (or HRBR `mount` identity changes per `areNodesEqual()`): handled by the generic replace-path in `patchDOM()` (destroy + mount at the same index).
- **Same vnode type** but HRBR `mount` changes: handled by `patchHrbr()` (dispose/destroy/-clear/remount into the same wrapper).

The integration tests (Section 18) mostly focus on the second path, because the first isn't HRBR-specific.

18.4 Destroying: one call site, two optional hooks

Destroying happens in `src/destroy-dom.ts` via `removeHrbrNode(vdom)`.

The HRBR instance contract is designed to work for both compiled blocks and fallback runtime objects:

- `destroy():` `void` is required.
- `dispose():` `void` is optional (signals cleanup / reactive graph unsubscribe).

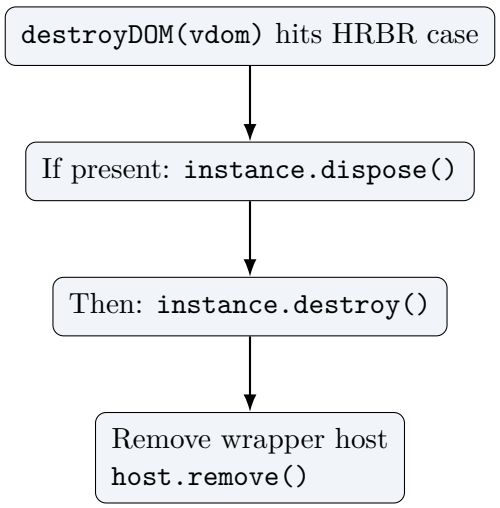
The destroy path:

1. call `instance.dispose()` if present
2. call `instance.destroy()` if present

3. remove the host element itself from the DOM

Because the host wrapper is removed, the HRBR subtree can't leak DOM nodes even if the inner runtime forgets to clean up.

Diagram: destroy ordering (dispose is optional, destroy is required)



18.4.1 Why both `dispose()` and `destroy()`?

This pattern matches the rest of the HRBR runtime in this book:

- `dispose()` is for disconnecting from reactive graphs (signals/effects) and releasing scheduled work.
- `destroy()` is for deterministic DOM cleanup, including nested mounts.

18.5 Tests as specs: what the integration tests guarantee

The behavior in this chapter is backed by a small integration test suite in:

extttsrc/___tests___/hrbr-integration.test.ts

Itś worth reading these tests like a contract. Here are the key guarantees they encode.

18.5.1 Mounting an HRBR block returned from a VDOM component

The first test verifies a basic shape: a VDOM component can return `hBlock(mount)`, and mounting the VDOM tree calls `mount(host)` exactly once.

```
const mount = (host) => {
        const el = document.createElement('div')
        el.textContent = 'HRBR'
        host.appendChild(el)
        return { destroy() { host.innerHTML = '' } }
}
```

```
const Comp = defineComponent({
	render() {
		return hBlock(mount)
	},
})
```

The point of this test isn't the DOM details. It's the ownership boundary: the VDOM runtime gives you a host element, and you're free to fill it.

18.5.2 Patching: stable mount function preserves instance

Another test asserts the most important property: if the mount factory is stable, patching doesn't remount.

Implementation detail: this works because `src/nodes-equal.ts` defines HRBR vnode equality as `old.mount === new.mount`.

18.5.3 Patching: changing mount function forces remount

Finally, a test changes the function identity and expects a dispose/destroy/remount into the same host. This is the contract you lean on when you want to intentionally reset an HRBR subtree.

18.6 Common pitfalls (and how to avoid them)

18.6.1 Pitfall: creating a new mount function every render

If you write:

```
render() {
	return hBlock((host) => mountBlock(def, host, slots))
}
```

then you're creating a new function every render, so patching will treat it as a different subtree and remount.

Fix: hoist the mount function to a stable reference (module scope, constructor/init, memo), or structure your API so the mount function is stable and you pass changing values via an `update()` hook.

18.6.2 Pitfall: forgetting that VDOM owns the wrapper, not the inner nodes

When writing debugging tools or DOM traversal helpers, don't assume the HRBR subtree's "root element" is a direct child of the VDOM parent. The direct child is always the wrapper host (currently `<span data-hrbr="1">`).

18.6.3 Pitfall: leaking event handlers without a teardown hook

If an HRBR runtime attaches global listeners (`window`, `document`) or schedules work, it should put the cleanup in `dispose()` and/or `destroy()`. The VDOM layer will faithfully call them.

18.7 Mini-exercise: design an HRBR block API that updates without remounting

Goal: avoid remounting on every render.

1. Define a block mount factory that returns `{ update(values), destroy() }`.
2. Keep the `mount` function reference stable.
3. On VDOM component updates, call `instance.update(nextValues)` instead of returning a brand new mount function.

This is the same idea as "props update" in a component system, just expressed as a tiny imperative contract at the boundary.

The VDOM runtime calls both (when present) so HRBR implementations can keep those concerns separate.

18.8 Tests-as-spec: the integration suite

The file `src/__tests__/hrbr-integration.test.ts` is the executable contract for everything above. It contains four key behaviors.

18.8.1 Test harness: what this suite is actually exercising

This isn't a unit test of the HRBR runtime itself; it's testing the integration points:

- `hBlock()` produces a vnode with `type: DOM_TYPES.HRBR`.
- `mountDOM()` correctly creates the wrapper host and calls the mount factory.
- `patchDOM()` decides reuse vs remount using mount identity.
- `destroyDOM()` tears down and removes the wrapper.

18.8.2 1) Mount an HRBR block returned from a VDOM component

The first test constructs a component whose render returns `hBlock(mount)`. The mount function appends a child to the provided host and returns an object with `destroy()`.

Assertions:

- mounting the VDOM mounts the HRBR region (DOM contains `HRBR`)
- the mount function is called exactly once
- destroying the VDOM destroys the HRBR region (DOM becomes empty)

18.8.3 2) Event semantics: bind to the host component

The second test verifies the *host component binding* convention. An HRBR block is mounted from inside a VDOM component, and the block defines an event slot:
`{ kind: 'event', path: [], name: 'click' }`.

The key detail is the `mountBlock(..., { hostComponent: this })` option. It ensures the handler's `this` matches the VDOM component instance, mirroring Relax's VDOM event behavior.

Assertions:

- clicking the button calls the handler
- the handler sees `this === vdom.component`
- the argument is an `Event`

This test is a key "mixed tree" guarantee: HRBR doesn't invent a new event binding model when embedded inside Relax VDOM; it can reuse Relax's convention of binding handlers to the component instance.

18.8.4 3) Patch: stable mount identity preserves instance

The third test mounts a component that always returns `hBlock(mount)` where `mount` is stable. It then patches the VDOM with another render of the same component.

Assertions:

- `mount` was called only once
- the instance array length remains 1
- DOM remains `hello`

This is the "no remount" fast path.

Notice what is (and isn't) asserted:

- It doesn't require HRBR to implement any diffing—the integration layer simply promises not to tear it down.
- It asserts the VDOM runtime carries the old instance forward by copying `host/instance` into the new vnode.

18.8.5 4) Patch: changing mount identity remounts

The fourth test swaps `currentMount` from `MountA` to `MountB` and triggers a component re-render via `updateState()`.

Assertions:

- `MountA` called once
- `MountB` called once
- DOM changes from `A` to `B`

The mechanism used here is also important: the test triggers a component re-render via `updateState()`, which causes the component's `render()` to run again and return an HRBR vnode with a different `mount` reference. This matches real application code where mount factories can accidentally change due to closures.

18.9 Practical guidance (and foot-guns)

- **Keep mount factories stable** if you want patch-time reuse. In practice, that means hoisting the mount function out of render (or memoizing it).
- **Always implement `destroy()`** in your HRBR instance. Without it, detaching the host wrapper still removes DOM, but reactive resources and event hooks can leak.
- **Pass `hostComponent`** when you mount blocks from components, if you want event handlers to behave like VDOM events.
- **Don't assume DOM ownership beyond the host**. The VDOM layer treats the host wrapper as the unit of reconciliation.

18.9.1 Common failure modes in mixed trees

- **Accidental remounts:** if you create the mount function inline (a new closure every render), it will remount every patch. Prefer a stable function identity.
- **Forgetting to clear:** `patchHrbr()` explicitly clears `host` before remount. If your custom HRBR instance assumes an empty host but doesn't clean on `destroy()`, you'll get surprising leftover nodes in custom setups.
- **Destroying the wrong thing:** the integration treats the wrapper host as the owned node. Removing inner nodes manually from outside HRBR can break assumptions inside the HRBR runtime.

In Part IV's later chapters, we'll connect this interop layer to the compiler output: the compiler emits `hBlock(() => mountCompiledBlock(...))` for static templates and `hBlock(() => mountFallback(...))` when it detects dynamic structure.

18.10 Online resources

- DOM node insertion and wrapper ownership patterns:
  MDN: Node.appendChild,
  MDN: Node.insertBefore,
  MDN: ChildNode.remove.
- "Identity is state" in UI runtimes (keys and function identity are the rough analogs used here):
  React docs: list keys,
  Vue docs: maintaining state.
- Event handler binding and the meaning of `this` in JavaScript:
  MDN: `this`.
- Data attributes used as lightweight, inspectable markers:
  MDN: Using data attributes.

Chapter 19

Compiler architecture overview (prototype + Babel transform)

This chapter explains how Relax's HRBR compiler takes TSX/JSX input and produces output that mounts *compiled blocks* (the fast path) or routes to the *fallback runtime* when the template's structure is dynamic.

What youĺl build (mentally)

This chapter is about building a reliable mental model, not shipping a production compiler. By the end, you should be able to:

- explain the compiler/runtime boundary in one sentence
- read transformed output and understand where `templateHTML`, `slots`, and `read()` closures come from
- predict when compilation must route to `mountFallback()` because the DOM structure isnt́ stable
- debug slot path bugs by reasoning about `childNodes` indexing (elements *and* text nodes)

Prerequisites

Youĺl get the most value if you already understand:

- HRBR blocks: `defineBlock()` + `mountCompiledBlock()` (Chapter **??**)
- fallback reconciliation: `mountFallback()` + structural patching (Chapter **??**)
- VDOM ↔ HRBR interop boundary: `hBlock(mount)` (Chapter 18)

Why this matters

Compilers are optional in UI libraries until they arent́. The moment you want both:

- a stable template that never re-creates DOM, and
- ergonomic authoring in JSX/TSX,

you end up building a compiler or transform.

This chapter is deliberately practical: weĺl treat tests as specs, and weĺl keep the compiler contract small enough that you can maintain it.

The compiler currently exists in two forms:

- A TypeScript-AST prototype compiler in `compiler/index.ts` (used as an executable spec for template extraction).
- A Babel plugin scaffold in `compiler/jsx/transform.ts` (used by build tooling to transform real TSX/JSX files).

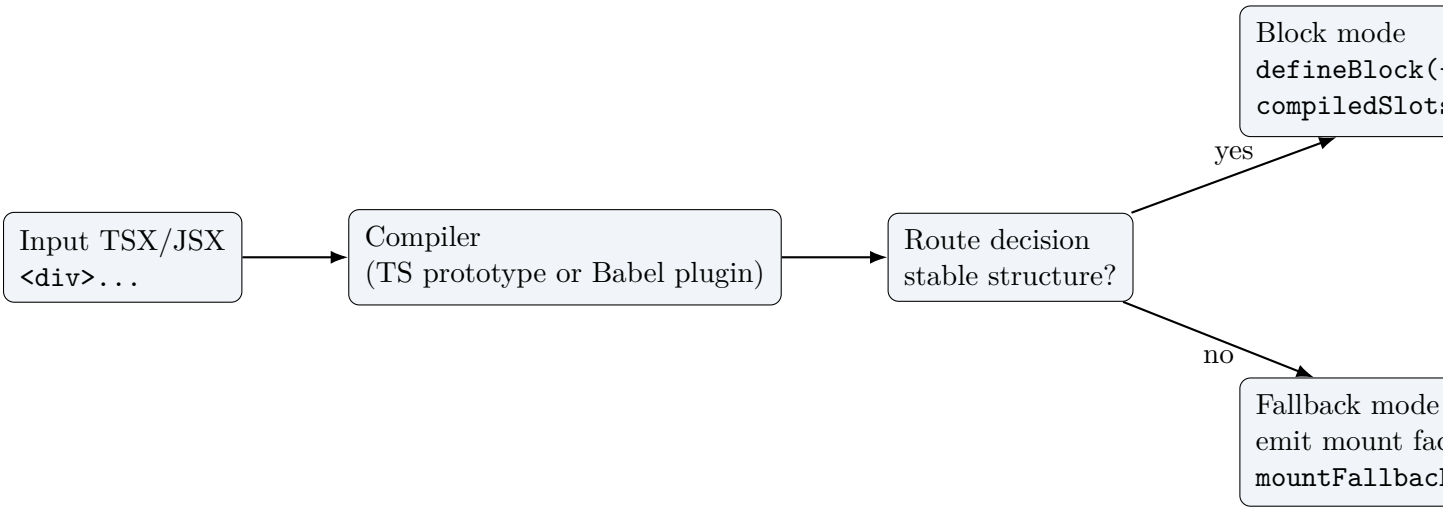
The purpose of having both is pedagogical and practical:

- The TS compiler is a small, deterministic "reference" that focuses on *template + slot-path extraction*.
- The Babel plugin is the real integration point for user codebases.

19.1 The output contract (what the runtime needs)

Before we talk about parsing and visitors, it's worth making the compiler/runtime boundary explicit. The compiler's job is to emit *data and closures* that satisfy a small runtime API. Everything in the implementation is organized around making that boundary stable.

Diagram: the compiler-to-runtime boundary (block vs fallback)



This is the architectural backbone: the compiler emits either a *BlockDef + read closures* (fast) or a *fallback mount factory* (safe).

19.1.1 Tiny contract (2 bullets)

Given a JSX expression, the compiler must be able to produce one of:

- **Block mode (fast path):** `defineBlock( templateHTML, slots )` plus a mount factory that calls `mountCompiledBlock(def, host, compiledSlots)`.
- **Fallback mode (safe path):** a mount factory that calls `mountFallback(...)` and delegates structural reconciliation to the runtime.

The runtime intentionally does not know how compilation works; it only requires these shapes. The HRBR runtime expects two main pieces of data.

19.1.2 1) A `BlockDef`: stable template + slot metadata

Defined in `runtime/block.ts`, a `BlockDef` contains:

- `templateHTML: string`, the stable HTML string that will be parsed once.
- `slots: Record<string, BlockSlot>` , the mapping from a slot key to its write kind and a DOM path.

A `BlockSlot` describes *what* to write (text/attr/class/style/prop/event, etc.) and *where* to write it (a childNodes index path).

19.1.3 2) A `CompiledSlot[]` list: how to read live values

At runtime, `mountCompiledBlock(def, host, compiledSlots)` needs an array of objects shaped like:

```
{ key: string, read: () => unknown }
```

The compiler can't always inline the read closures in a pure data file. In particular, the TypeScript prototype compiler deliberately returns no reads; it records expression source code so a caller with lexical scope can build closures.

In contrast, the Babel transform *can* emit `read: () => expr` arrow functions directly into the output program, precisely because it is already transforming a file with the correct lexical scope.

19.1.4 A 3-part boundary you should memorize

At the boundary, the compiler emits three things:

1. a `BlockDef` (pure data: stable HTML + slot metadata)
2. a `compiledSlots[]` list (closures: how to read live values)
3. a mount factory (a closure boundary: where lexical scope lives)

That last bullet is easy to miss. Itś the reason the Babel transform can emit `read: () => expr` directly, while the TypeScript prototype compiler intentionally stops at "expression source code".

For a background overview of why JSX is typically compiled, see: https://react.dev/learn/writing-markup-with-jsx.

19.2 Routing decision: block vs fallback

The compiler's core architectural decision is: *can this TSX be represented as a stable HTML template?* If not, we must use the fallback reconciler.

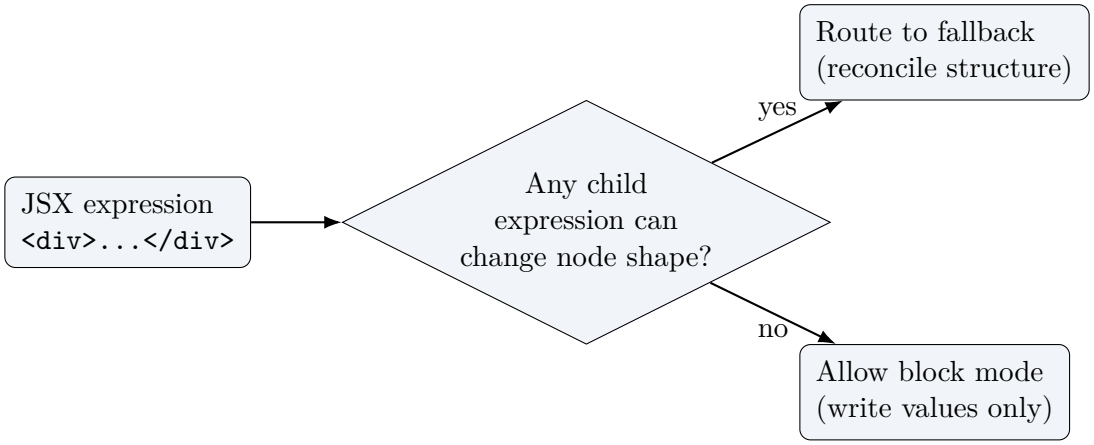
19.2.1 Dynamic structure

The term **dynamic structure** means "the set/shape/order of DOM nodes can change as state changes." Examples:

- conditionals: `cond ? <div/> : null`
- loops/lists: `items.map(...)`
- fragments that change child count
- dynamic tag names or component tags
- spreading attributes (often implies unknown keys)
- event handlers (in the current compiler phase) if they can't be represented as static bindings

Compiled blocks are designed for *stable structure* with dynamic *values*. When structure is dynamic, we let the fallback runtime reconcile children.

Diagram: what counts as "dynamic structure" (conservative)



Relax's current compilers deliberately err on the safe side: anything that could introduce or remove DOM nodes is treated as "dynamic structure".

19.2.2 A useful rule of thumb

If an expression could evaluate to:

- `null`
- `false`
- an array of nodes

- a `<div>` sometimes and `<span>` other times

then it's structural and must go to fallback.

If it always evaluates to a *value* that can be written into an existing node (text/attr/prop/-class/style), then it can stay in block mode.

This is conservative, but it keeps the compiler honest.

19.2.3 How "block vs fallback" relates to Chapter 18

The compiler output doesn't mount directly into the VDOM. Instead, it produces a *mount factory* that matches the HRBR vnode contract described in Chapter 18. That is why the VDOM type `DOM_TYPES.HRBR` stores a `mount(host)` function: it is the common denominator between compiled blocks and fallback regions.

19.3 TypeScript prototype compiler (Phase 5 reference)

The TypeScript implementation lives in `compiler/index.ts`. It provides two public APIs:

- `compileTSXToBlock(source, opts?) -> CompileResult`
- `compileTSXOrFallback(source, opts?) -> CompileOrFallbackResult`

19.3.1 `compileTSXToBlock()`: parse -> extract -> render template

Inputs

- `source: string`: a single-root TSX expression (or a statement containing one).
- `opts.fileName?`: helps TypeScript provide useful source locations.
- `opts.strictTemplate?` (default true): reject patterns that can't be expressed as a static HTML template.

Main pipeline

1. Parse the input with TypeScript: `ts.createSourceFile(..., ScriptKind.TSX)`.
2. Find the first expression via `findFirstExpression()`.
3. Ensure the root is `JsxElement` or `JsxSelfClosingElement`.
4. Build a `TemplateBuild` using `buildTemplate()`.
5. Return `{ block: {templateHTML, slots}, slots: [], meta }`.

19.3.2 Determinism: why the TS compiler is a good executable spec

The implementation in `compiler/index.ts` is deliberately small and deterministic:

- It parses with TypeScript's parser (so syntax + TSX rules match what users write).
- It walks the syntax tree into a simple string renderer that produces `templateHTML`.
- It computes slot paths using an explicit `childIndex` counter.

This makes it ideal for teaching and for pinning down invariants (especially path semantics) before committing to a more complex Babel visitor implementation.

19.3.3 Paths: the compiler and runtime must agree

The most important shared invariant between compiler and runtime is how a "path" is interpreted.

In this prototype compiler, each element maintains a `childIndex` counter, and each child node increments the counter as it is rendered. When the compiler emits a slot, it records the current path (a list of child indices) that identifies a target node under the template root.

Why this works The block runtime later parses `templateHTML` into a DOM tree and walks `childNodes[index]` repeatedly to locate the slot target.

19.3.4 The one gotcha that causes most bugs: `childNodes` includes text nodes

If you take nothing else from this chapter, take this:

> extbfSlot paths are based on DOM `childNodes`, not `children`.

That means whitespace and literal text in your template count as nodes. So when you look at markup like:

> exttt Hello {name()}

the compiled `templateHTML` might contain literal spaces or newlines, and those become `Text` nodes when parsed.

This is also why the tests talk about paths like `[3, 1]` (not `[0, 0]`). In the Babel snapshot test (`compiler/__tests__/babel-transform.test.ts`), the template contains indentation and newlines. When parsed, they become text nodes that affect indexing.

19.3.5 Walkthrough: compute a slot path by hand

Letś do a small thought experiment that matches the compilerś rules.

Input TSX:

```
const App = () => (
       <section id="root" className={cls()}>
              <h1>Title</h1>
              <p>Hello {name()}</p>
       </section>
)
```

The Babel transform snapshot illustrates this exact structure as a static template plus two slots:

- `s0`: a `class` slot on the root element, with `path: []`
- `s1`: a `text` slot inside the `<p>`, with `path: [3, 1]`

Why `[3, 1]`?

- `[]` means ṡtart at the template root element (the `<section>`).
- `3` means g̈o to the 4th `childNodes` entry under `<section>`. In formatted templates, the `childNodes` often look like: newline text, `<h1>`, newline text, `<p>`, newline text.

- `1` then means g̈o to the 2nd `childNodes` entry under `<p>`.: Under `<p>`, you often have: `Text('Hello ')` then `Text('__hrbr__')` (or an empty placeholder) depending on how the compiler encodes it.

This is ugly but reliable. And reliability is the whole point: the compiler and runtime just need to agree.

For background on DOM node lists, see MDN: https://developer.mozilla.org/en-US/docs/Web/API/Node/childN

19.3.6 What the TS prototype actually returns (and why the reads are empty)

In `compiler/index.ts`, `compileTSXToBlock()` returns:

- `block: { templateHTML, slots }`
- `slots: []` (yes, empty)
- `meta` about dynamic structure, lanes, etc.

Thatś not a bug. Itś an explicit constraint: a string-only compiler doesnt́ have access to the lexical scope where `count()`, `cls()`, or `name()` live. So it records the expression source code internally (`reads: {key, code}[]`) but does not pretend it can create closures.

The end-to-end test in `compiler/__tests__/compiler.e2e.test.ts` shows the intended integration: the caller wires in the `read` closures manually.

```
const { block } = compileTSXToBlock(`(<div className={count()}>Count</div>)`)

const mounted = mountCompiledBlock(block, host, [
       { key: 's0', read: () => count() },
])
```

The key behavioral contract in that test is worth repeating:

> extbfUpdating signals must never recreate the root element.

In other words, compilation is only useful if it reduces work to *writes*, not *mount/unmount*.

19.4 Babel transform: how the output program is shaped

The Babel plugin in `compiler/jsx/transform.ts` exists to solve the closure problem. It runs inside your real source file, so it can emit `read: () => expr` in-place.

19.4.1 The shape (as asserted by tests)

The unit tests in `compiler/__tests__/babel-transform.test.ts` basically define the public contract. The transform must emit:

- an import from `relax/hrbr` that includes `defineBlock` and `mountCompiledBlock`
- a module-scope `const __hrbr_block_N = defineBlock(...)`
- component output that returns a **mount factory**: `host => __hrbrBlock(host, def, slots)`

- a wrapper function `__hrbrBlock(host, def, slots)` that calls `mountCompiledBlock(def, host, slots)`

Hereś the key part of the snapshot (reformatted as teaching code):

```
import { defineBlock, mountCompiledBlock } from "relax/hrbr";

const __hrbr_block_0 = defineBlock({
       templateHTML: "<section␣id=\"root\"␣class=\"\">\n...",
       slots: {
              "s0": { kind: "class", path: [] },
              "s1": { kind: "text", path: [3, 1] },
       }
});

const App = () => host => __hrbrBlock(host, __hrbr_block_0, [
       { key: "s0", read: () => cls() },
       { key: "s1", read: () => name() },
]);

function __hrbrBlock(host, def, slots) {
       return mountCompiledBlock(def, host, slots);
}
```

There are two deeply important design choices here:

1. **The `BlockDef` is module-scope data.** It should be created once, not per render.
2. **The mount factory is per render, and it closes over reads.** Thatś where `cls()` and `name()` are actually in scope.

This pattern is very similar in spirit to Solidś compilation strategy (static DOM + fine-grained updates), though the runtime and APIs differ. For a conceptual reference, see: https://www.solidjs.com/docs/latest#co

19.4.2 Lane pragmas: compiler-to-scheduler hints without changing semantics

Both compilers support a tiny l̈ane pragmas̈yntax:

```
<div
       className={/*@lane transition*/ cls()}
       data-x={/*@lane input*/ x()}
/>
```

The Babel test asserts that these land in slot metadata as `lane: "transition"` and `lane: "input"`.

Why it matters: when you start integrating compilation into an app/runtime scheduler, this gives you a way to nudge *when* updates flush without changing *what* updates mean.

19.5 Debugging: when a slot path is wrong

The most common failure mode when you build a compiler like this is: extbfthe slot writes go to the wrong node.

When that happens, you should immediately suspect one of these:

- you counted `children` instead of `childNodes`
- your template string gained or lost whitespace (newlines/indentation)
- you treated a void tag as having children (or forgot it closes itself)

Mini-exercise (tests-as-spec)

Open `compiler/__tests__/babel-transform.test.ts` and find the inline snapshot where `path: [3, 1]`. Then:

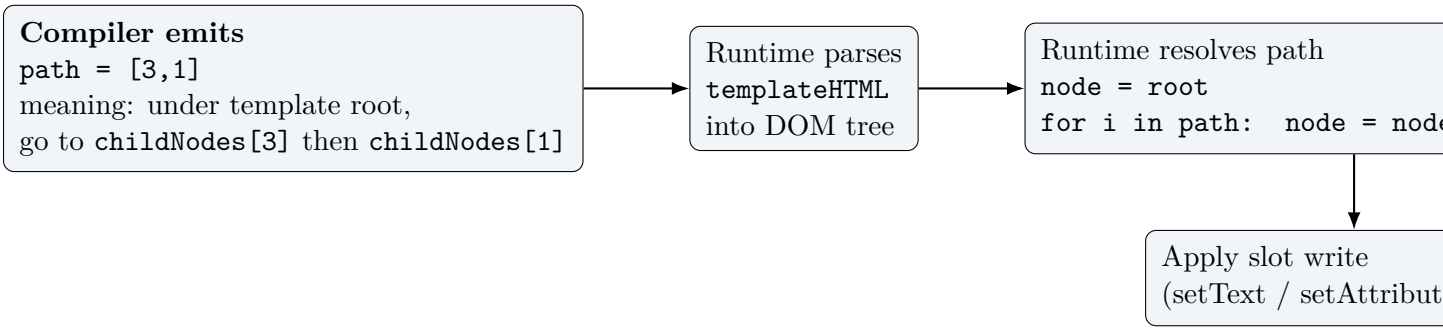
1. remove the whitespace/newlines in the template emitter (so the HTML becomes a single line),
2. update the snapshot, and
3. explain (in one sentence) why the path changed.

This is the simplest way to internalize the `childNodes` rule.

The critical detail is that the compiler counts *every child node* (elements *and* text nodes), because `childNodes` includes text nodes. In `compiler/index.ts` this is represented by `PathState{ childIndex }`.

This is also why the compiler uses explicit sentinels for some dynamic expressions: it needs to ensure a real, countable node exists at the path.

Diagram: how slot paths are interpreted via `childNodes`



This is why both compilers count *text nodes* as children: the runtime uses `childNodes`, not `children`.

19.5.1 Text expression handling: sentinel + slot path

In `renderJSX()`, when a child is `JsxExpression`:

- the compiler marks `meta.hasDynamicStructure = true`
- it allocates a slot key `sN`
- it emits a unique sentinel string `__hrbr_sN__` into the template
- it records a `text` slot whose path points to that text node

- it records the expression's source text as a "read" entry

This is a deliberately conservative design: any child expression could be a conditional or a list, so the prototype treats it as structure-changing. The Babel compiler (later in this book) uses heuristics to allow some expressions as "text-like" without falling back.

19.5.2 Void elements and template correctness

The TS compiler treats HTML void tags specially via the `VOID_TAGS` set. For these tags it emits `<input ...>` rather than `<input ...></input>`. This matters because the runtime parses `templateHTML` through `<template>` and relies on the browser's HTML parser to build a stable node tree. Producing non-HTML-canonical markup can shift the parsed tree and break slot paths.

19.5.3 Attributes: literals vs expression slots

The prototype supports:

- boolean attributes: `<div hidden />`
- string literal attributes: `<div id="x" />`
- expression attributes: `<div className={count()} />`

Expression attributes allocate slot keys and record `kind` and `name`:

- `className` normalizes to `class`
- `style` maps to a `style` slot
- anything else becomes an `attr` slot

The compiler also records lane hints (per slot) and "signal-like" hints:

- `meta.lanesByKey`: a compiler-to-runtime hint used by schedulers in later phases
- `meta.signalLikeByKey`: a heuristic that recognizes `foo()` style getters

Even when later phases choose to ignore (or reinterpret) these hints, keeping the meta separate from the structural template makes it possible to evolve scheduling policy without changing the fundamental block ABI.

19.5.4 Fallback decision wrapper: `compileTSXOrFallback()`

This wrapper is the "policy layer" on top of template extraction. It answers one question: *is it safe to run in block mode?*

At the top level, the TS prototype exposes a convenience API: extttcompileTSXOrFallback(source).

Its semantics are intentionally boring:

```
const res = compileTSXToBlock(source)

if (res.meta.hasDynamicStructure) {
       return { kind: 'fallback', reason: 'dynamic-structure', meta: res.meta }
}

return { kind: 'block', ...res }
```

This is important because it makes "fallback" a first-class output, not an exception. The compiler is allowed to say "I don't know".

19.6 Dynamic structure detection (what triggers fallback)

Now we'll make the fuzzy idea from earlier (*structure vs values*) precise. The easiest way is to turn the implementation into a checklist.

19.6.1 Decision table: TS prototype vs Babel transform

Relax currently has a *conservative* detector in the TS prototype and a *slightly more permissive* detector in the Babel transform. They differ because Babel has access to more information (and can safely reject with fallback by returning `null` from template building).

extbfInput pattern	**TS prototype (`compiler/index.ts`)**	**Babel transform (`compiler/jsx/transform.ts`)**
Child expression: `<p>Hello {expr}</p>`	Marks `meta.hasDynamicStructure = true` (always)	Allows only *text-like* expressions; otherwise returns `null` to route to fallback
Conditional/logical children: `cond ? <A/> : <B/>`, `cond && <A/>`	Treated as dynamic structure (child expression)	Treated as "not text-like" (`ConditionalExpression`/`LogicalExpressi` routes to fallback
Non-intrinsic tags: `<Component/>`, `<Foo.Bar/>`	Marks dynamic structure when tag is not lowercase	Template builder returns `null` when tag is not a lowercase intrinsic
Event handlers: `onClick={fn}`	Marks dynamic structure (events not representable in static HTML)	Immediately returns `null` (block mode not supported for events yet)
Spread attributes: `<div {...props} />`	Marks dynamic structure (and throws in strict template mode)	Returns `null` (not supported in block template)
Void tags: `<input/>`, `<img/>`	Emitted with correct HTML void syntax via `VOID_TAGS`	Emitted with a regex-based void-tag check

This table is the practical meaning of *"dynamic structure"* in Relax today.

19.6.2 Babel's `isTextLikeExpr`: the tiny heuristic that buys you a lot

The Babel transform has a helper called `isTextLikeExpr(expr)`. The idea is simple: if a child expression could evaluate to an element/fragment/array, then the template structure isn't stable. But if it's always a value that can be written into `textContent`, we can keep block mode.

From the scaffold (summarized):

```
function isTextLikeExpr(expr) {
        if (expr is JSXElement or JSXFragment) return false
        if (expr is ConditionalExpression) return false
        if (expr is LogicalExpression) return false
        return true
}
```

Notice what this buys us:

- **Text updates stay cheap.** Most UIs do a lot of "render some string" work.
- **You can still escape hatch.** The moment an expression might be structural, it falls back.

This is a deliberately small heuristic. In later chapters, you'll see how it grows, and how tests keep it honest.

19.7 Babel compilation from scratch: how one JSX element becomes a block

So far we've looked at the *shape* of the output. Now we'll build a mental model for the *mechanics* of how the Babel transform creates that output.

The transform in `compiler/jsx/transform.ts` does three jobs:

1. build a static HTML template string
2. allocate slots + paths for the dynamic parts
3. emit a mount factory that closes over live reads

If you understand those three, the whole transform stops feeling magical.

19.7.1 Step 1: pick a stable root (or give up)

The transform only wants to compile stable, intrinsic HTML tags like `div`, `section`, and `button`. If the tag is not a lowercase intrinsic (for example `<Component/>` or `<Foo.Bar/>`), there is no obvious static HTML representation. So the template builder returns `null`, and the plugin routes to fallback.

Why is this the right default?

- In JSX, a capitalized tag means "call some function and let it return anything".
- That means the compiler can't promise stable structure.

Block mode only works when the compiler can promise that *the same DOM nodes exist forever*.

19.7.2 Step 2: render a string that the browser will parse deterministically

Think of compilation as emitting a `<template>` string. Your compiler isn't building DOM; the browser will. So your job is just to produce a string that will be parsed into a stable tree.

That implies two constraints:

- **Void tags matter.** `<input>` can't accidentally become `<input></input>`.
- **Whitespace matters.** Newlines/indentation become text nodes and shift `childNodes` indices.

This is why the snapshot tests in `compiler/__tests__/babel-transform.test.ts` are so valuable. They lock down not just the slot paths, but the *exact* template string (including newlines).

19.7.3 Step 3: allocate slots: "static HTML with holes"

When the transform sees something dynamic, it does *not* generate DOM operations directly. It just records metadata that the runtime knows how to apply later.

You can think of slot allocation as making a table that answers:

> When the runtime mounts this template, *where* is the thing to update, and *how* should it be updated?

Examples you should expect as you read the code:

- **Attribute slot:** dynamic attribute value (`kind: 'attr'`).
- **Class slot:** `className={...}` normalized to `class` (`kind: 'class'`).
- **Text slot:** dynamic text content inside an element (`kind: 'text'`).

Every slot includes a `path`, which is just a frozen version of the `childNodes` walk we discussed earlier.

19.7.4 Step 4: emit reads as closures (the key Babel advantage)

The TS prototype can't produce `read: () => expr` because it doesn't know which variables exist. But Babel is transforming your actual file, so it can emit closures that reference `cls()`, `name()`, or any local binding.

The key idea is:

> extbfAll dynamic values are read through closures passed to `mountCompiledBlock()`.

This means compilation doesn't change your state model. Signals are still signals. The compiler just gives the runtime a faster way to apply their values.

19.7.5 Step 5: fallback routing is just "emit a different mount factory"

In the Babel plugin, "fallback" isn't a special runtime mode. It's just a different function call in the generated output:

- block mode: emit `mountCompiledBlock(def, host, compiledSlots)`
- fallback mode: emit `mountFallback(host, () => <JSX...>)` (shape varies as the scaffold evolves)

The important part is the architecture: both modes still return a mount function. So the caller doesn't care which one it got.

For conceptual background on why many UI frameworks separate "render" from "commit" (build a description then apply it), see: https://react.dev/learn/render-and-commit.

19.8 Tests as specs: how to validate compiler changes safely

Compilers are notoriously easy to break with tiny edits. Relax deals with this by treating tests as the spec. If you're modifying the compiler, keep these two mental buckets:

- **Behavioral contracts:** integration tests that assert DOM stability.
- **Shape contracts:** snapshot tests that assert exact emitted output.

19.8.1 Spec-to-test mapping (start here when debugging)

- **"Don't recreate root DOM" contract:** `compiler/__tests__/compiler.e2e.test.ts`
- **"Exact codegen shape" contract (imports, defineBlock, wrapper):** `compiler/__tests__/babel-tran`
- **"Path semantics" contract (whitespace affects `childNodes`):** also `babel-transform.test.ts` snapshots

When you're unsure whether a change is safe, prefer adding a tiny unit test over adding another heuristic. Heuristics get clever; tests keep you honest.

Now weĺl make the fuzzy idea from earlier (*structure vs values*) precise. The easiest way is to turn the implementation into a checklist.

19.8.2 Decision table: TS prototype vs Babel transform

Relax currently has a *conservative* detector in the TS prototype and a *slightly more permissive* detector in the Babel transform. They differ because Babel has access to more information (and can safely reject with fallback by returning `null` from template building).

extbfInput pattern	**TS prototype (`compiler/index.ts`)**	**Babel transform (`compiler/jsx/transform.ts`)**
Child expression: `<p>Hello {expr}</p>`	Marks `meta.hasDynamicStructure = true` (always)	Allows only *text-like* expressions; otherwise returns `null` to route to fallback
Conditional/logical children: `cond ? <A/> : <B/>`, `cond && <A/>` "not text-like" (`ConditionalExpression`/`LogicalExpression`); routes to fallback	Treated as dynamic structure (child expression)	Treated as
Non-intrinsic tags: `<Component/>`, `<Foo.Bar/>`	Marks dynamic structure when tag is not lowercase	Template builder returns `null` when tag is not a lowercase intrinsic
Event handlers: `onClick={fn}`	Marks dynamic structure (events not representable in static HTML)	Immediately returns `null` (block mode not supported for events yet)
Spread attributes: `<div {...props} />`	Marks dynamic structure (and throws in strict template mode)	Returns `null` (not supported in block template)
Void tags: `<input/>`, `<img/>`	Emitted with correct HTML void syntax via `VOID_TAGS`	Emitted with a regex-based void-tag check

This table is the practical meaning of *d̈ynamic structure*¨ in Relax today.

19.8.3 Babelś `isTextLikeExpr`: the tiny heuristic that buys you a lot

The Babel transform has a helper called `isTextLikeExpr(expr)`. The idea is simple: if a child expression could evaluate to an element/fragment/array, then the template structure isnt́ stable. But if itś always a value that can be written into `textContent`, we can keep block mode.

From the scaffold (summarized):

```
function isTextLikeExpr(expr) {
        if (expr is JSXElement or JSXFragment) return false
        if (expr is ConditionalExpression) return false
        if (expr is LogicalExpression) return false
        return true
}
```

Notice what this buys us:

- `<span>count()</span>` can be compiled as a text slot.
- `<span>count() > 0 ? <b/> : null</span>` must go to fallback.

Even though this is conservative, it gets you most of the performance wins for the common case (text/attr/class/style bindings).

19.9 How fallback routing looks in the Babel output (scaffold)

When `buildTemplateFromJsxElement()` returns `null`, the plugin constructs a minimal `mountFallback()` call. This is still early-stage code, but itś worth understanding the design:

- block mode emits a `BlockDef` and a tight `CompiledSlot[]` list
- fallback emits a *render function* that returns reconciliation specs (create/patch)

That means fallback is not V̈DOMïn the React sense. Itś a small structural reconciler that still aims to avoid recreating nodes whenever possible.

If you want a conceptual baseline for why compilers do this s̈tatic template + runtime writess̈plit, Reactś docs on render/commit are a good mental model: https://react.dev/learn/render-and-commit.

Spec-to-test mapping (what to read next)

When youŕe debugging compiler behavior, treat these tests as your spec:

- **No DOM recreation on updates**: `compiler/__tests__/compiler.e2e.test.ts`
- **Output program shape**: `compiler/__tests__/babel-transform.test.ts`

If a behavior isn't asserted there yet, it's not stable.

`compileTSXOrFallback()` runs `compileTSXToBlock()` and then returns:

- `{ kind: 'fallback', reason: 'dynamic-structure' }` if `meta.hasDynamicStructure` is true
- otherwise `{ kind: 'block', ...result }`

This is the high-level policy the real Babel transform is evolving toward.

Note the important nuance: `compileTSXToBlock()` still returns a `BlockDef` even when `meta.hasDynamicStruct` is true. That mirrors how the Babel plugin can still hoist block defs for other expressions in the same file even if one subtree needs fallback.

19.10 Babel transform scaffold (real-world integration)

The Babel plugin entrypoint is exported as `hrbrJsxTransform` from `compiler/jsx/index.ts` and re-exported from `compiler/index.ts`. The implementation lives in `compiler/jsx/transform.ts`.

19.10.1 Roles of the Babel plugin

Compared to the TS prototype, the Babel plugin has additional responsibilities:

- Operate on full program files, not just an isolated expression string.
- Inject runtime imports: `defineBlock`, `mountCompiledBlock`, and optionally `mountFallback`.
- Generate wrapper helpers (e.g. `__hrbrBlock`) that connect emitted data literals to runtime calls.
- Derive stable slot keys in dev mode (optionally) so diffs are readable.

19.10.2 "Block mode" is best-effort: return `null` to fall back

The internal function `buildTemplateFromJsxElement()` tries to convert a JSX element into:

- `templateHTML`
- `slots` map
- `compiledSlots` array literal (with `read: () => expr`)

If it sees unsupported patterns (events, dynamic elements, structural expressions), it returns `null` and marks that fallback imports are needed.

19.10.3 Babel transform: what "best-effort block mode" means in code

In `compiler/jsx/transform.ts`, block mode is essentially:

- Try to turn a `JSXElement` into `{ templateHTML, slots, compiledSlots }` using `buildTemplateFromJsxEl`
- If any unsupported structure appears, return `null`.
- The caller then chooses a fallback mount factory instead of a compiled-block mount factory.

The current transform is strict on purpose (e.g. it rejects events and spreads in block mode), because the project is trying to keep the block subset crisp while the fallback runtime is still evolving.

19.10.4 Slot key strategies: sequential vs stable (dev mode)

The Babel compiler contains two different slot key allocation styles:

- **Default (prod-like):** sequential keys `s0, s1, ...`.
- **Dev mode:** location-based keys like `s_text_12:18` derived from source positions.

Location-based keys are helpful because they reduce "slot churn" in diffs and make debugging invalid-path errors more readable.

19.10.5 Lane pragmas (compiler-to-scheduler hint)

The Babel plugin recognizes comment pragmas in expression containers:

```
className={/*@lane transition*/ cls()}
```

It extracts the lane and stores it on the slot metadata. Later chapters will connect this to the deterministic scheduler.

Concretely, `extractLanePragmaFromExpressionContainer()` scans leading comments for a pattern like:
texttt@lane transition. When found, the compiler attaches `lane: "transition"` to the slot metadata object.

19.11 Tests-as-spec

The compiler has three layers of tests:

- Unit compilation tests (shape of emitted template/slots): `compiler/__tests__/compiler.test.ts`
- Snapshot tests for stability: `compiler/__tests__/compiler.snapshot.test.ts`
- End-to-end tests that mount the result in jsdom:
  - `compiler/__tests__/compiler.e2e.test.ts` (TS prototype -> mountCompiledBlock)
  - `compiler/__tests__/babel-transform.e2e-jsdom.test.ts` (Babel output -> evaluate -> mountCompiledBlock)

In the rest of this section, we'll treat the tests as a specification of the compiler/runtime boundary. We won't run them here, but reading them tells you what changes are allowed without breaking the public contract.

19.11.1 Unit tests (TS): compile + mount sanity

extbfFile: `compiler/__tests__/compiler.test.ts`

This test anchors the smallest happy path:

- `compileTSXToBlock()` accepts a simple intrinsic element with a dynamic attribute expression.
- The caller supplies `CompiledSlot{ key: 's0', read: () => name() }`.
- `mountCompiledBlock()` produces DOM matching the template, and the dynamic slot lands in the right place.

One subtle detail: the test disposes the mounted block via `mounted.dispose()`. This keeps the contract clear: compiled mounts are still reactive, and disposal is part of the lifecycle.

19.11.2 TS end-to-end: compile -> mount -> update without recreating root

In `compiler/__tests__/compiler.e2e.test.ts`:

1. Create a signal: `const [count, setCount] = createSignal(1)`.
2. Compile TSX: `(<div className={count()}>Count</div>)`.
3. Mount via `mountCompiledBlock(block, host, { key: 's0', read: () => count() })`.
4. Update the signal, wait for the scheduler (microtask + `setTimeout(0)`).
5. Assert the root element identity is preserved and only the class changed.

This is the key end-to-end contract for compiled blocks.

This test also documents a scheduling assumption: it comments that HRBR scheduling uses `setTimeout(0)` by default in tests, so updates require waiting at least one timer tick.

19.11.3 Babel transform unit tests: shape of emitted program

extbfFile: `compiler/__tests__/babel-transform.test.ts`

These tests assert that the plugin is producing the intended *program shape*:

- it injects an import from the configured runtime path (default `relax/hrbr`)
- it references `defineBlock` and `mountCompiledBlock`
- it hoists a `const __hrbr_block_0 = defineBlock(...)`
- it emits a wrapper mount function of the form `host => __hrbrBlock(host, def, slots)`

The snapshot test in this file is especially important: it pins down the exact emitted structure (templateHTML string, slots map, and compiledSlots array with `read: () => expr`).

The suite also asserts two "developer experience" guarantees:

- lane pragmas are propagated to slot metadata
- dev mode can emit location-based keys

19.11.4 Babel transform errors: the intentional block-mode subset

extbfFile: `compiler/__tests__/babel-transform.errors.test.ts`

This suite documents what is currently *rejected* in block mode:

- event handlers (e.g. `onClick`)
- spread attributes (`<div {...props} />`)
- fragments (`<>...</>`)
- non-intrinsic tags (components like `<Foo />`)

Even if future versions route these to fallback instead of throwing, the existence of these tests is useful: it makes the current compilation subset explicit and prevents accidental expansion that would silently generate incorrect templates.

19.11.5 Babel transform E2E (jsdom): evaluate emitted literals + mount

extbfFile: `compiler/__tests__/babel-transform.e2e-jsdom.test.ts`

This test checks something subtle but crucial: the compiler output is not just syntactically valid—its emitted *data* can actually be evaluated and mounted.

Notable aspects:

- It extracts the object literal passed to `defineBlock(...)` from the emitted text.
- It extracts the compiledSlots array literal from the wrapper call.
- It evaluates those two pieces with `new Function(...)` while providing `cls()` and `name()` from the test scope.
- It mounts using `mountCompiledBlock` and asserts identity preservation across updates.

This approximates how "real code" will behave: the compile-time output must work with runtime scope (signals) and produce stable DOM.

19.12 Worked example: derive `templateHTML` and slot paths from the snapshot

Now weĺl do the most valuable "learn it from scratch" exercise: take the golden snapshot in extttcompiler/___tests___/babel-transform.test.ts and derive the template and slot paths by hand. If you can do this once, debugging a compiler becomes much less mysterious.

19.12.1 The input JSX (what the author writes)

The snapshot test compiles this input:

```
const App = () => (
        <section id="root" className={cls()}>
                <h1>Title</h1>
                <p>Hello {name()}</p>
        </section>
)
```

At a high level, the compiler must split this into:

- **Structure:** a static HTML template string.
- **Values:** a list of dynamic slots with deterministic DOM paths.

19.12.2 Step 1: the static template string (structure + placeholders)

In the normalized snapshot, the transform emits a `defineBlock( templateHTML, slots )` where `templateHTML` is a single string. It contains the `<section>` with `id="root"`, plus an empty `class=""` placeholder.

In the snapshot, the template also includes indentation and newlines. Thatś not just cosmetic: when the runtime parses HTML, whitespace becomes text nodes.

Why the placeholders matter:

- `className={cls()}` becomes `class=""` in the template so the runtime can later write the real attribute value.
- `Hello {name()}` becomes `Hello ` in the template (leaving a stable "spot" for the text slot).

If you ever see a compiler try to "just concatenate strings" at runtime, this is the key difference: HRBR wants the template to be parsed once, and then updated via direct DOM writes.

19.12.3 Step 2: the slot declarations (what to write + where to write it)

The same snapshot asserts two slot declarations:

- `s0`: `kind: "class"`, `path: []`
- `s1`: `kind: "text"`, `path: [3, 1]`

Think of `path` as a precomputed set of array indexes into `childNodes`. No search. No diff. Just indexing.

19.12.4 Step 3: compute `path: [3, 1]` by hand (the whitespace trap)

We'll compute `[3, 1]` mechanically.

First hop: `3` means we're selecting `section.childNodes[3]`. Because the template contains whitespace between elements, the `childNodes` list includes *text nodes*:

1. `childNodes[0]`: text (newline/indent before `<h1>`)
2. `childNodes[1]`: `<h1>`
3. `childNodes[2]`: text (newline/indent between `</h1>` and `<p>`)
4. `childNodes[3]`: `<p>`
5. `childNodes[4]`: text (newline/indent before `</section>`)

So hop 1 lands on the `<p>`.

Second hop: `1` means `p.childNodes[1]`. The `<p>` contains static text (`Hello `) plus a dynamic text slot. The compiler's job is to ensure there's a stable text node representing the dynamic value. So we get:

- `p.childNodes[0]`: text node for `"Hello "`
- `p.childNodes[1]`: text node that will receive `name()` updates

That's what `path: [3, 1]` means: "the second text node inside the paragraph."

This is the most important compiler invariant in HRBR: if you change the template emitter and accidentally add/remove whitespace, you can shift every downstream path.

19.12.5 Step 4: compiled slot readers (how values cross the boundary)

The final piece the snapshot asserts is the compiled slot array literal:

```
const App = () => host => __hrbrBlock(host, __hrbr_block_0, [
       { key: "s0", read: () => cls() },
       { key: "s1", read: () => name() },
])
```

This is where Babel shines: `read` is a closure in the output program. Thereś no serialization format for expressions. The emitted code just calls the original functions.

19.12.6 Debugging checklist (tests-first)

When a compiled block updates the wrong node, treat it like a deterministic math error. Use the tests to narrow it down:

1. Check the snapshot: `compiler/__tests__/babel-transform.test.ts`. If your change modified whitespace in `templateHTML`, you probably shifted paths.
2. Recompute the `childNodes` list by hand and see if the asserted `path` still lands on the intended node.
3. Run the E2E that actually mounts the evaluated literals: extttcompiler/___tests___/babel-transform.e2e-jsdom.test.ts.

If you get into the habit of adding a snapshot test first, youĺl catch most path regressions before they reach runtime.

19.12.7 Babel fallback E2E (jsdom): structural expression routes to `mountFallback`

extbfFile: `compiler/__tests__/babel-transform.fallback.e2e-jsdom.test.ts`

This suite pins down the fallback routing policy:

- A conditional expression in children is treated as **dynamic structure** (unsafe for block mode).
- The transform output must include `mountFallback`.
- The emitted module is evaluated, and calling `App()` returns a mount function.
- Updates are reactive: signal changes trigger DOM updates without throwing.

The test includes a comment that the MVP fallback spec doesn't yet clear conditional text in all cases; the contract being asserted is deliberately smaller: the pipeline compiles, mounts, and updates safely.

19.13 Where we go next

Chapter 20 will zoom in on the Babel visitor model and show how the plugin emits the full final shape that user code will run. We will also connect the "fail block mode" paths to the fallback runtime and reconciler described earlier.

19.14 Online resources

- TypeScript compiler API: parsing source and working with AST nodes:
  TypeScript Wiki: Using the Compiler API.
- Babel plugin authoring and visitor model:
  Babel docs: plugins,
  Babel docs: plugin API.
- JSX specification details (useful when deciding what constitutes "dynamic structure"):
  JSX spec (historical reference).
- Why HTML parsing matters for template correctness (void elements, tree building, etc.):
  MDN: HTML elements reference.
- Source maps basics (relevant to dev mode and debugging transforms):
  Source Map Revision 3 Proposal.

Chapter 20

Babel plugin in detail (JSX transform)

This chapter explains the current HRBR Babel transform in `compiler/jsx/transform.ts`. It is intentionally a *scaffold*: it covers a useful subset, has crisp failure modes, and is wired end-to-end in jsdom tests.

The transform's job is to recognize TSX/JSX expressions that can be compiled into a stable `BlockDef` + `CompiledSlot[]` and to rewrite them into a `mount(host) => instance` factory that the VDOM interop layer can consume (see Chapter 18).

When it can't safely compile a JSX expression into a stable template, it routes to `mountFallback()`.

20.1 Public entrypoint and options

The plugin is exported as `hrbrJsxTransform`:

- `compiler/jsx/index.ts`: re-exports the default plugin
- `compiler/index.ts`: re-exports it as part of the public compiler API

20.1.1 Tiny contract: what this plugin must guarantee

The transform is successful if it guarantees two things:

- The emitted program is **self-contained**: all runtime references are imported (or intentionally omitted when not needed).
- Any transformed JSX expression becomes a **mount factory** of the form `(host: Element) => MountedInstance`, matching the HRBR vnode contract from Chapter 18.

Everything else (slot heuristics, fallback subset size, key readability) is an implementation choice that can evolve as long as the tests keep passing.

The options type `HrbrJsxTransformOptions` (in `compiler/jsx/transform.ts`) controls a few key behaviors:

- `runtimeImport`: module path for runtime helpers (default `"relax/hrbr"`)
- `mountWrapperName`: helper function name used to call `mountCompiledBlock` (default `__hrbrBlock`)
- `dev`: enables more readable output and debug-oriented metadata
- `stableSlotKeys`: when true, derive slot keys from source locations (defaults to `dev`)

20.2 High-level rewrite shape

Given a JSX expression like:

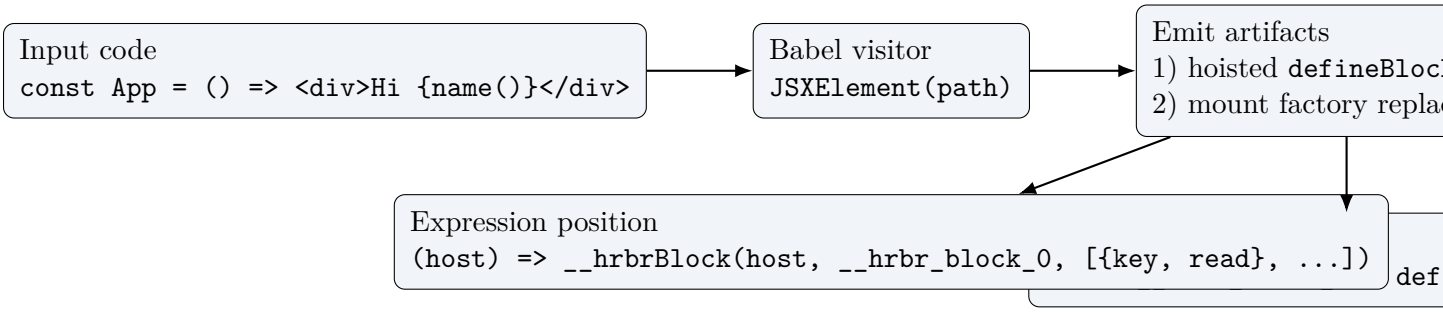
```
const App = () => <div className={cls()}>Hi {name()}</div>
```

The plugin emits three conceptual things:

1. An import of runtime functions:
   `import { defineBlock, mountCompiledBlock } from "relax/hrbr"`
2. A module-scope `const __hrbr_block_N = defineBlock (...)` containing:
   - `templateHTML: "<div ...>..."`
   - `slots: { key: { kind, path, name?, lane? }, ... }`
3. A replacement of the JSX expression with a mount factory:
   `host => __hrbrBlock(host, __hrbr_block_N, [{key, read}, ...])`

This shape matches the interop contract in `src/h.ts`: an HRBR vnode is just a mount function that returns an instance.

Diagram: one JSX expression becomes a hoisted block + mount factory



The key optimization is hoisting: the template is parsed once per module, while the mount factory runs per render/mount.

20.3 Program-level state and synthesis (imports + wrappers)

The plugin keeps per-file state variables:

- `needsRuntimeImport`: whether to inject `defineBlock/mountCompiledBlock`
- `needsFallbackImport`: whether to also import `mountFallback`
- `needsWrapper`: whether to emit the wrapper function `__hrbrBlock`
- `nextBlockId`: a counter used to generate `__hrbr_block_0`, `__hrbr_block_1`, ...
- `currentFileNamecurrentFileCode`: used for context-rich error messages

These are reset in the `Program.enter` visitor and materialized in `Program.exit`:

- `ensureRuntimeImport(programPath)` uses `unshiftContainer('body', ImportDeclaration)`.
- `ensureWrapper(programPath)` appends `function __hrbrBlock(host, def, slots) { return mountCompiledBlock(def, host, slots) }`.

Implementation note: this scaffold avoids depending on Babel "types" helpers by emitting raw AST node objects.

Diagram: per-file state at `Program.enter/exit`

This structure prevents unused imports/helpers: nothing is emitted unless some subtree required it.

20.3.1 Why inject at `Program.exit`?

The plugin defers emissions until `Program.exit` so it can answer questions like:

- Did we ever successfully compile a block? If so, we need `defineBlock` and `mountCompiledBlock`.
- Did any subtree route to fallback? If so, we need `mountFallback` too.
- Did we generate a wrapper call? If so, we need to emit the wrapper function body.

This design keeps the output minimal: files that contain no transformable JSX don't get imports they don't use.

20.4 The core visitor: `JSXElement(path)`

The plugin's heart is the `JSXElement` visitor. For each JSX element in an expression position, it calls:

```
const built = buildTemplateFromJsxElement(path)
```

20.4.1 Success path: compile into a block

When `built` is non-null:

1. Mark `needsRuntimeImport = true` and `needsWrapper = true`.
2. Insert a module-scope `const __hrbr_block_N = defineBlock({ templateHTML, slots })`.
3. Replace the JSX expression with:
   `host => __hrbrBlock(host, __hrbr_block_N, built.compiledSlots)`

The slot objects are turned into AST literals through `literalizeSlot(slot)`.

Hoisting: why `defineBlock(...)` lives at module scope

The plugin uses `unshiftContainer('body', ...)` on the program node to emit:

texttttconst ___hrbr_block_N = defineBlock(templateHTML, slots)

Hoisting matters for two reasons:

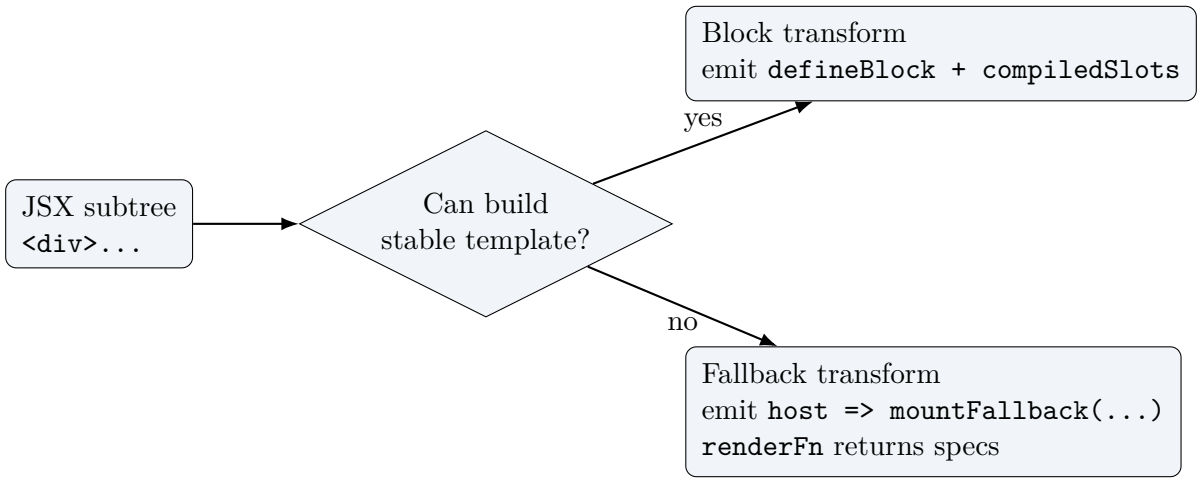
- **Performance:** we want template parsing and slot-path validation to happen once per module, not once per render.
- **Identity:** the mount factory will refer to a stable `__hrbr_block_N` constant.

20.4.2 Failure path: route to fallback

When `built` is null, the plugin attempts a *minimal fallback transform*.

Why fallback exists Block mode requires a stable template. Expressions that can produce different node shapes at runtime (conditionals, JSX inside expressions, etc.) must be executed by the fallback runtime, which uses the reconciler.

Diagram: fallback routing when template extraction fails



In other words: block mode is an optimization; fallback mode is the semantic firewall.

Current fallback subset The fallback routing in `transform.ts` is intentionally small:

- It only supports intrinsic lowercase root tags.
- It only supports children that are `JSXText` or `JSXExpressionContainer`.
- It allows only static string attributes (and ignores dynamic attrs).
- It rejects spread attributes and event handlers.

In this subset, the plugin synthesizes a `render()` function (a closure-free data + expression composition) and emits:

```
host => mountFallback(host, () => ({ children:  [spec] }))
```

Each child spec contains:

- `create(): Element` -> creates the element and sets attributes/text
- `patch(node)` -> updates textContent based on the latest expressions
- optional `key` if a `key=` attribute is present

This is enough to prove the end-to-end pipeline and to connect the compiler to the fallback runtime from earlier chapters.

20.4.3 Fallback synthesis: what gets captured (and what doesn't)

The fallback compiler intentionally avoids capturing complex structures. It builds a `renderFn` with the shape:
texttt() => (children: [spec])

Where `spec.create()` and `spec.patch(node)` both close over the user expressions.

Important implications:

- **Reactivity comes from runtime:** `mountFallback` re-runs `renderFn` under tracking and schedules reconciliation.
- **This is not a general JSX renderer:** it is a deliberate minimal bridge used to validate the "block or fallback" architecture.

20.5 Template extraction: `buildTemplateFromJsxElement()`

The function `buildTemplateFromJsxElement()` is a best-effort compiler for a supported subset. It returns either a full block build or `null`.

20.5.1 Supported subset (today)

In block mode, the transformer supports:

- intrinsic lowercase tags only (e.g. `div`, `section`)
- nested elements
- static string attributes
- expression attributes -> slot kinds `class/style/attr`
- dynamic text children -> only when the expression is "text-like"

So far this chapter has described the transform like a black box. Now weĺl build the mental model from scratch: how a Babel plugin works, which AST nodes it sees, and how this particular scaffold chooses to represent output.

20.6 Babel plugins from scratch: the minimum you need to know

If youv́e never written a Babel plugin, the key idea is simple:

- Babel parses your file into an AST (a tree of node objects).
- A plugin gives Babel a *visitor*: a set of functions like `Program()`, `JSXElement()`, `Identifier()`, etc.
- As Babel walks the tree, it calls your visitor functions with a *path* object. A path is like "a cursor pointing at this node", with helpers to replace, insert, and inspect context.

In Relaxś scaffold we rely on three path operations:

- `path.replaceWith(newNode)`: turn `<div/>` into `host => ...`.
- `programPath.unshiftContainer('body', stmt)`: insert imports and hoisted constants.
- `programPath.pushContainer('body', stmt)`: append helper functions like `__hrbrBlock`.

What makes compiler work feel hard is not Babel itself. Itś the discipline of preserving semantics:

- when can we treat an expression as *just a string*? (text slot)
- when does it become *structure*? (must use fallback)
- how do we keep paths stable so runtime updates dont́ hit the wrong node?

20.7 A tiny contract for this file (inputs/outputs/error modes)

Treat `compiler/jsx/transform.ts` as implementing this contract:

- **Input:** a JS/TS program containing JSX elements.
- **Output (block success):** the JSX expression is replaced by a mount factory exttt(host) => ___hrbrBlock(host, def, compiledSlots).
- **Output (fallback):** the JSX expression is replaced by a mount factory exttt(host) => mountFallback(host, renderFn).
- **Errors:** unsupported constructs in the scaffold (fragments, spreads, components, events) throw with a friendly code-frame.

Notice that both success and fallback produce the *same outer type*: a mount function. Thatś what makes the rest of the system clean. The VDOM layer doesnt́ need to know which mode was chosen.

20.8 Why this scaffold emits "raw AST objects" instead of Babel helpers

If youŕe used to Babel plugins, you might expect to see heavy usage of `@babel/types` helpers like `t.identifier('x')`. This scaffold goes the other direction: it constructs node objects like:

```
{
  type: 'ImportDeclaration',
  specifiers: [
    { type: 'ImportSpecifier', imported: { type: 'Identifier', name: 'defineBlock' },
        local: { type: 'Identifier', name: 'defineBlock' } }
  ],
  source: { type: 'StringLiteral', value: runtimeImport }
}
```

Why it matters:

- It keeps the scaffold dependency surface small.
- It makes it obvious what Babel is doing: everything is an AST node object.
- Itś easier to read snapshots because the output is less abstracted.

As the compiler grows, you may still migrate toward Babel helpers for correctness and less boilerplate. But the raw-node style is an excellent teaching tool.

20.9 Program-level synthesis: `ensureRuntimeImport` and `ensureWrapper`

The transform maintains file-scoped flags:
extttneedsRuntimeImport, `needsFallbackImport`, `needsWrapper`.

These flags solve a real "compiler ergonomics" problem: we dont́ want to inject imports or helpers unless the file actually used JSX in a supported way.

20.9.1 How imports are synthesized

In block mode, we need `defineBlock` and `mountCompiledBlock`. In fallback mode, we also need `mountFallback`.

So `ensureRuntimeImport(programPath)` emits exactly:

```
import { defineBlock, mountCompiledBlock } from "relax/hrbr"
// (plus mountFallback if needed)
```

This is asserted by `compiler/__tests__/babel-transform.test.ts`.

20.9.2 Why we have a wrapper helper at all

The wrapper is intentionally boring:

```
function __hrbrBlock(host, def, slots) {
  return mountCompiledBlock(def, host, slots)
}
```

So why keep it? Because it gives you one stable call site to instrument later (devtools, tracing, invariants, feature flags). A compiler that emits a dozen ad-hoc call sites is harder to evolve.

20.10 Template extraction as a recursive renderer

The heart of block mode is `buildTemplateFromJsxElement(elPath)`. Itś easiest to understand it as a recursive function that tries to *render JSX into HTML* while simultaneously recording slots.

20.10.1 Key data structures

Inside `buildTemplateFromJsxElement()` the compiler builds three things:

- `templateHTML`: a string that can be parsed as an element.
- `slotMap`: `{ key -> { kind, path, name?, lane?  } }`.
- `compiledSlots`: an array of objects with `{ key, read:  () => expr }`.

The first two belong to `defineBlock` (module scope). The third belongs next to the mount call (runtime scope).

20.10.2 Paths: tracked while rendering

The recursive helper `renderElement(node, pathToHere, childIndexRef)` incrementally builds the DOM path. Every time we emit a child node into the HTML string, we increment a counter. When we encounter a dynamic expression thatś allowed as a text slot, we record:

extttpath: [...pathToHere, childIndex]

This is exactly the scheme you reverse-engineered in Chapter 19 via the snapshot.

20.10.3 The "text-like" gate: why `isTextLikeExpr` exists

Remember: **block mode cannot represent structural variability**. So if a child expression might yield an element/fragment/conditional structure, we must route to fallback.

In this scaffold the rule is encoded as:
extttif (!isTextLikeExpr(expr)) return null.

The important thing here is not the exact heuristic. Itś the *architectural pattern*:

- template extraction is best-effort
- any ambiguous construct returns `null`
- the caller chooses fallback

This makes the compiler safe by construction.

20.11 Fallback synthesis: the semantic firewall (minimal but real)

When block mode returns `null`, the transform builds a minimal fallback mount. You can see this end-to-end in:

- `compiler/__tests__/babel-transform.fallback.e2e-jsdom.test.ts`

That test compiles:

```
const App = () => (
  <div>{show() ? name() : ''}</div>
)
```

Even though the child expression is "text-like" at runtime, itś a `ConditionalExpression`, and the current heuristic treats that as structural. So the plugin emits `mountFallback` and the test asserts that updates are reactive and do not throw.

20.11.1 From scratch: what the fallback compiler is building

The fallback compiler doesnt́ build a vdom tree. It builds a tiny *reconciliation spec* object with two operations:

- `create(): Node` — build initial DOM.
- `patch(node): void` — mutate existing DOM in place.

Then it wraps those specs in a `renderFn`:

exttt() => ({ children: [spec] })

And finally emits:
exttthost => mountFallback(host, renderFn).

Why this matters: by giving fallback the same outer mounting shape, the rest of Relax stays composable. Block mode and fallback mode become interchangeable implementations behind one vnode contract.

20.12 Failure modes are part of the design (and are tested)

Not everything should silently route to fallback—especially in a scaffold. The project currently chooses to throw for several constructs. Thatś intentional because it makes the supported subset crisp.

The tests in `compiler/__tests__/babel-transform.errors.test.ts` document the current hard constraints:

- event handlers like `onClick`
- spread attributes like `{...props}`
- fragments `<>...</>`
- non-intrinsic tags like `<Foo />`

As the fallback runtime becomes more feature complete, you can change the policy (throw vs fallback)—but you should always encode it in tests first.

It explicitly rejects (in block mode):

- event handlers (`onClick=` etc.)
- fragments (`<>...</>`)
- non-intrinsic tags (components, capitalized)
- spread attributes (`{...props}`)

20.12.1 How paths are computed in the Babel transform

Like the TS prototype compiler (Chapter 19), this transform must produce `childNodes`-based paths.

In `renderElement(...)` the transform:

- keeps a per-parent counter `childState.i` that increments for each child node
- uses `pathToHere` plus the current index to form paths like `[3,1]`
- emits a placeholder text node (a single space) when it needs to guarantee the existence of a text node for a dynamic text slot

This is why the snapshot test shows a template that contains *extra whitespace* in places where text slots are inserted. That whitespace isn't an aesthetic choice; it's a structural anchor.

20.12.2 Slot keys: sequential vs location-based

Slot keys are allocated in `allocSlotKey()`.

- Default: sequential keys `s0`, `s1`, ...
- Dev mode (`stableSlotKeys=true`): keys include source location: `s_<kind>_<line>:<col>`.

This is validated in `compiler/__tests__/babel-transform.test.ts`.

One nuance: location-based slot keys depend on Babel preserving `loc` information and comments; this is why dev mode is framed as "debug output" rather than a semantic requirement.

20.12.3 Lane pragmas: `/*@lane transition*/`

The helper `extractLanePragmaFromExpressionContainer()` scans leading comments and looks for:

```
@lane (sync|input|default|transition|idle)
```

If present, the emitted slot metadata includes `lane: "..."`. The tests verify that the transformed output contains the lane strings.

20.12.4 Text-like safety

In `isTextLikeExpr()`, the transform tries to reject expressions that can yield JSX elements or otherwise change DOM structure.

Examples treated as *not text-like*:

- returning JSX elements/fragments
- conditionals/logicals that could yield elements (conservatively rejected)

When an expression is accepted as text-like, the transform:

- emits a sentinel space in the template (so a text node exists)
- records a `text` slot pointing to that position

- emits a compiled slot `read: () => expr`

The predicate `isTextLikeExpr(expr)` is the key safety gate. It explicitly rejects:

- `JSXElement` and `JSXFragment` (obviously structural)
- `ConditionalExpression` and `LogicalExpression` (conservatively structural for now)

This conservative choice is why the fallback e2e test uses `show() ? name() : ""` to force fallback routing: it is not considered "text-like" yet.

20.13 Failure modes are part of the API

This compiler phase aims for clarity and determinism. Unsupported syntax produces explicit errors via `fail(path, message)` which attempts to build Babel code-frame errors.

The file `compiler/__tests__/babel-transform.errors.test.ts` locks in these failure modes:

- event handlers should throw
- spread attributes should throw
- fragments should throw
- non-intrinsic tags should throw

While these errors might eventually become "route to fallback" behaviors, they're still valuable today: they prevent the compiler from silently generating templates that would be incorrect (for example, by treating an event handler like an attribute).

20.14 End-to-end tests: jsdom execution proves output shape

Two e2e suites validate that the emitted output is not just syntactically correct but *runnable*.

20.14.1 Block mode e2e: transformed output mounts and updates

`compiler/__tests__/babel-transform.e2e-jsdom.test.ts` compiles a small component, extracts the generated `defineBlock(...)` literal and the `compiledSlots` array literal, evaluates them with `new Function(...)` and mounts the result with `mountCompiledBlock`.

The key invariant: updating a signal updates the DOM without recreating the root element.

This suite also proves a deeper property: the emitted output can be *partially evaluated*. The test extracts only the `defineBlock(...)` object literal and the compiled slots array literal, then evaluates them with `new Function`. So the plugin isn't just emitting runnable code; it's emitting code whose pieces behave like data.

20.14.2 Fallback routing e2e: structural text expression goes to `mountFallback`

`compiler/__tests__/babel-transform.fallback.e2e-jsdom.test.ts` compiles an expression with a structural conditional:

```
<div>{show() ?  name() :  "}</div>
```

This should include `mountFallback` in output and remain reactive under signal updates.

The spec here is intentionally small:

- it asserts that `mountCompiledBlock` should *not* be called
- it asserts that mounting succeeds and subsequent signal changes cause updates
- it does *not* assert perfect minimal DOM operations yet

That matches the current purpose of fallback in this compiler phase: correctness-first safety for dynamic structure.

20.15 What to improve next (guided by tests)

The current transform is intentionally conservative. The next incremental steps typically look like:

- add safe event slot support (mapping `onClick` to HRBR event slots)
- support fragments by lowering them to concatenated children in block mode
- expand fallback routing to support nested children specs (not just a single root spec)
- improve text-like analysis so common conditionals can stay in block mode when both arms are text

20.16 Full trace: one `JSXElement` visitor call, end to end

You now have all the pieces. Letś glue them together by tracing a single JSX expression through the plugin.

Weĺl use the smallest example that still exercises the three interesting mechanics:

- a dynamic attribute (becomes a slot)
- a nested element (forces recursion)
- a dynamic text child (becomes a text slot)

20.16.1 Author input (JSX in user code)

```
const App = () => (
  <section id="root" className={cls()}>
    <h1>Title</h1>
    <p>Hello {name()}</p>
  </section>
)
```

This is the same structure used by the inline snapshot in extttcompiler/___tests___/babel-transform.test.ts. Thatś intentional: we want a trace that is already pinned by tests.

20.16.2 Step A: `Program.enter` resets per-file state

Before any JSX is transformed, the plugin resets:

- `needsRuntimeImport = false`
- `needsFallbackImport = false`
- `needsWrapper = false`
- `nextBlockId = 0`

This ensures a plugin run is deterministic per file. If you ever see "block ids jump" across compiles, this is the first place to look.

20.16.3 Step B: `JSXElement(path)` attempts block compilation

When Babel visits the `<section ...>` node, the visitor calls:

extttconst built = buildTemplateFromJsxElement(path)

Think of `built` as three outputs:

- `built.templateHTML`: the static HTML string
- `built.slots`: the slot metadata map (kind + path + name/lane)
- `built.compiledSlots`: the runtime `[ key, read ]` array

If `built` is null, the visitor routes to fallback. But in this example it succeeds.

20.16.4 Step C: `buildTemplateFromJsxElement` renders HTML and records slots

Inside the template builder, `renderElement` walks:

1. Root: `<section id="root" className={cls()}> ...`
2. Child element: `<h1>Title</h1>`
3. Child element: `<p>Hello {name()}</p>`

Along the way it produces two slots:

- a `class` slot on the root element (path `[]`)
- a `text` slot inside the `<p>` node (a deeper path)

The important implementation trick is how text slots are anchored: when a text expression is accepted, the builder emits a literal space in `templateHTML`. That guarantees a real text node exists at mount time.

20.16.5 Step D: the visitor hoists `defineBlock(...)` at module scope

Once we have `built`, the visitor sets:

- `needsRuntimeImport = true`
- `needsWrapper = true`

And it emits a module-scope constant:

```
const __hrbr_block_0 = defineBlock({
  templateHTML: "...",
  slots: {
    "s0": { kind: "class", path: [] },
    "s1": { kind: "text", path: [3, 1] }
  }
})
```

Why this is hoisted: it makes template parsing a one-time cost per module. At runtime, multiple mounts just reference the same `__hrbr_block_0`.

20.16.6 Step E: the JSX expression becomes a mount factory

Finally, the plugin replaces the original JSX expression with an arrow function:

```
const App = () => host => __hrbrBlock(host, __hrbr_block_0, [
  { key: "s0", read: () => cls() },
  { key: "s1", read: () => name() }
])
```

This is the key compiler/runtime boundary:

- `defineBlock(...)` is static data (safe to hoist)
- `read: () => expr` is dynamic code (must live in runtime scope)

The snapshot in `babel-transform.test.ts` asserts that this shape stays stable. If you change output shape, you should expect snapshot churn.

20.16.7 Step F: `Program.exit` injects imports and the wrapper helper

At the end of the file walk, `Program.exit` materializes any pending artifacts:

- an import from `runtimeImport`
- the `__hrbrBlock` wrapper function

This is the "emit only what you used" policy: no JSX compiled, no injected runtime surface.

20.17 Exercises (tests-first, like a real compiler engineer)

If you want to really internalize this transform, do these small, test-backed exercises.

20.17.1 Exercise 1: explain a slot path without running code

Open the snapshot in `compiler/__tests__/babel-transform.test.ts` and pick the `text` slot `path` (for example `[3, 1]`). Write out the `childNodes` array layout that makes the path correct.

20.17.2 Exercise 2: add a new slot kind (carefully)

The builder already classifies `class`, `style`, and generic `attr`. Add a test that forces a `prop` slot (for example `value={val()}` on `<input>`). Then implement the minimal compiler support in `buildTemplateFromJsxElement`.

20.17.3 Exercise 3: widen `isTextLikeExpr` with a test

Right now the transform routes `show() ? name() : "` to fallback. Add a unit test that asserts this can remain in block mode *only when* both arms are "text-like". Then update `isTextLikeExpr` to accept that case.

If you can complete Exercise 3 without breaking the E2E suites, youv́e learned the right lesson: compiler changes are always negotiated via tests.

The rule is always: add a targeted test first (unit + e2e), then widen syntax support.

When you widen the subset, remember the three invariants all previous chapters rely on:

- slot paths must match `childNodes`
- stable mount function identity enables reuse in VDOM interop
- fallback must remain safe when structure can change

20.18 Online resources

- Babel plugin authoring and visitor lifecycle:
  Babel docs: plugins,
  Babel docs: plugin API.
- JSX AST node shapes (useful when reading `JSXElement`, `JSXText`, and `JSXExpressionContainer` visitor logic):
  Babel parser AST spec.
- Source maps and why transforms try to preserve locations:
  Source Map Revision 3 Proposal.
- “Static template + dynamic slots" as a general pattern (useful for intuition):
  Svelte docs: compiler-first model.

Chapter 21

Devtools and instrumentation (optional)

Relax's HRBR runtime is designed to be deterministic and debuggable. This chapter documents the small *optional* devtools surface in `runtime/devtools.ts` and how the rest of the runtime reports events into it.

The design goals are:

- **Low overhead by default**: counters for heavyweight instrumentation (DOM ops, allocations) are disabled unless explicitly enabled.
- **Always hookable**: events can be observed via a hook regardless of whether counting is enabled.
- **Stable semantics**: the runtime emits a small number of event types at well-defined points.

21.1 The API surface: five functions and two helpers

All of devtools lives in `runtime/devtools.ts`. The public API is:

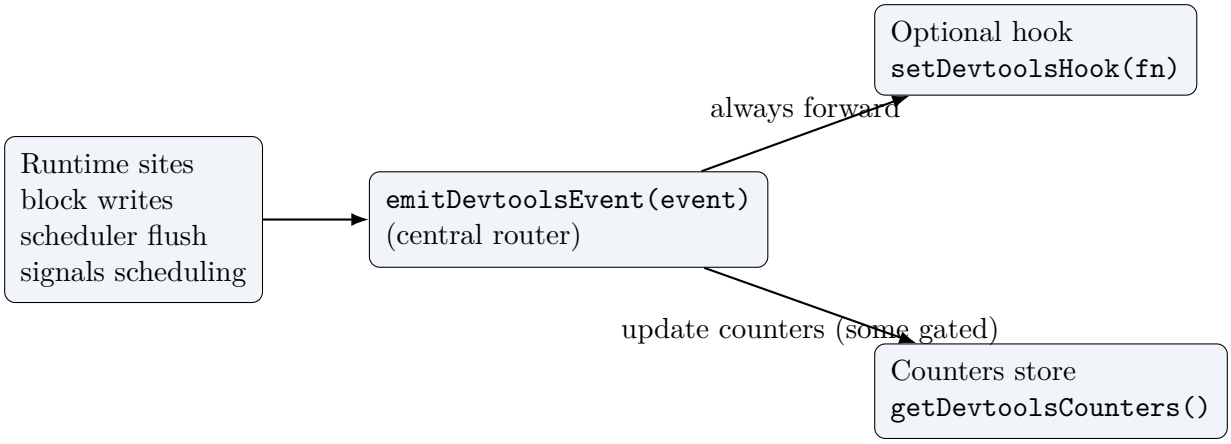
- `setDevtoolsHook(hook | null)` -> register/unregister a global event sink
- `getDevtoolsCounters()` -> get a defensive copy of counters
- `resetDevtoolsCounters()` -> zero all counters
- `setInstrumentationEnabled(boolean)` -> enable counting for expensive instrumentation
- `isInstrumentationEnabled()` -> query the flag

And the helper emission functions used across the runtime:

- `emitDevtoolsEvent(event)` -> core increment + forward logic
- `emitDomOp(op)` and `emitAlloc(kind, count?)` -> convenience wrappers

The reactivity and scheduling layers re-export these through `runtime/index.ts` so "public HRBR" users can import them.

Diagram: devtools module as a hub (hooks + counters)



This captures the intent: event forwarding is always possible, while some counter updates are optional for performance.

21.1.1 Tiny contract (what you can rely on)

The devtools module is deliberately small, but it still has a stable contract:

- If you install a hook with `setDevtoolsHook()`, it will receive *every* emitted event.
- Counters are best-effort metrics intended for debugging/perf investigation, not correctness.
- Expensive counters (DOM ops, allocations) are disabled by default and must be explicitly enabled.
- `getDevtoolsCounters()` returns a copy (so the internal counter object can't be mutated externally).

21.2 Event model

Devtools events are a tagged union `DevtoolsEvent`:

- `slotWrite`: emitted when a block slot write is attempted
- `scheduleComputation`: emitted when a reactive computation is scheduled
- `flushStart` / `flushEnd(durationMs)`: emitted by the deterministic scheduler
- `domOp(op)`: emitted by the block runtime around DOM mutations (counted only when instrumentation is enabled)
- `alloc(kind, count?)`: emitted by allocation sites (counted only when instrumentation is enabled)

The idea is that you can build either:

- simple counters (the built-in ones), or
- richer tooling (log streams, flamegraphs, dashboards) via the hook.

21.3 Counters: what is tracked and when

The internal counters live in a module-local object `_counters`:

- `slotWrites`
- `scheduledComputations`
- `flushes`
- `flushTimeMs`
- `domOps` (instrumentation gated)
- `allocs` (instrumentation gated)

21.3.1 Defensive reads: `getDevtoolsCounters()`

`getDevtoolsCounters()` returns `{ ..._counters }`. This is important: tests and users can't mutate internal state by holding onto the object.

The corresponding reset function, `resetDevtoolsCounters()`, mutates the internal object in-place. That choice keeps object identity stable for any internal references while still providing deterministic test setup.

21.3.2 Why DOM ops and allocs are gated

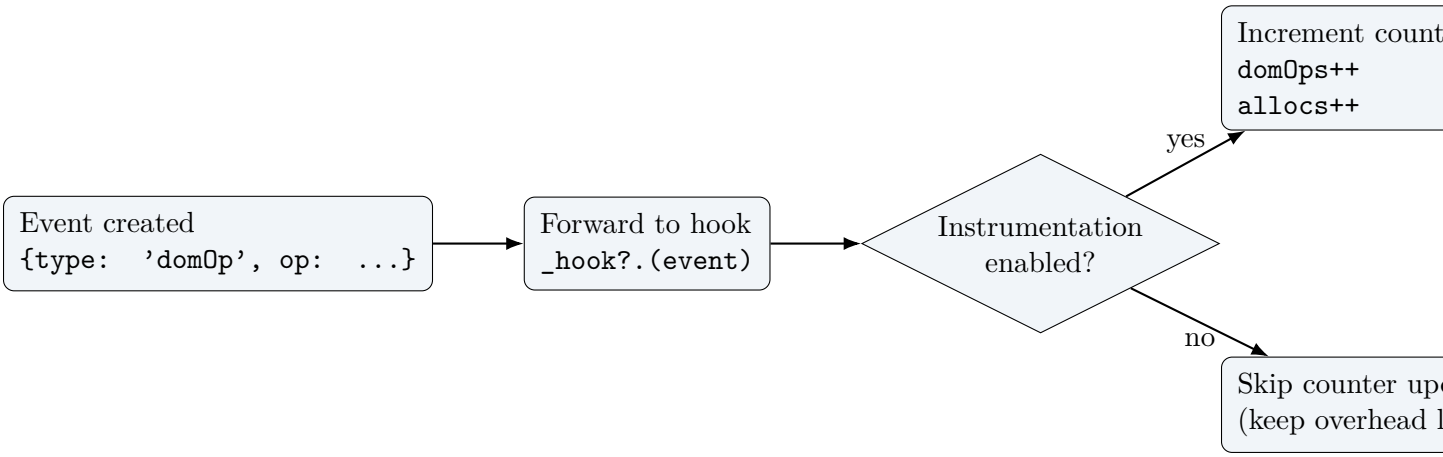
Counting DOM operations and allocations often requires adding emission calls in hot loops. The devtools system separates *event forwarding* from *counter increments*:

- Events are always forwarded to the hook.
- Counter increments for `domOp` and `alloc` only happen when `_instrumentationEnabled` is true.

This split is implemented directly in `runtime/devtools.ts`:

- `emitDevtoolsEvent()` always forwards to `_hook?.(event)`.
- Only the `domOp` and `alloc` branches check `_instrumentationEnabled` before touching counters.

Diagram: gating policy (forward always, count sometimes)



This is the perf contract: you can always observe events, but you only pay for counting when you asked for it.

21.4 Emission: increment + forward

`emitDevtoolsEvent(event)` does two things:

1. updates counters based on the event type
2. forwards the event to `_hook` if present

In pseudo-code, the key policy is:

```
always forward to hook; only count domOp/alloc when enabled
```

The convenience wrappers `emitDomOp()` and `emitAlloc()` add one more optimization:

- if instrumentation is disabled *and no hook is installed*, they return immediately

This prevents needless event objects from being allocated on the hot path.

21.4.1 A subtle performance point: "no hook" short-circuit

Notice the early return:

texttt if (!enabled && !hook) return

This means there are three distinct operating modes:

- **Default mode (recommended for production):** no hook; instrumentation disabled. Most hot-path emitters become no-ops.
- **Streaming mode:** hook installed; instrumentation disabled. Events still flow, but counters don't track domOps/allocs.
- **Instrumentation mode:** instrumentation enabled (hook optional). Counters track domOps/allocs.

21.5 Where events come from in the runtime

The core runtime emits events from a few key paths:

21.5.1 Blocks: slot writes and DOM operations

In `runtime/block.ts`:

- Every slot write emits `{ type: 'slotWrite', slotKind, slotKey }`.
- Many DOM mutations are annotated with `emitDomOp('setText')`, `emitDomOp('setAttribute')`, etc.

This provides a cheap "what changed" signal even without full DOM diffing.

Slot writes: one event per attempted update

In `runtime/block.ts`, every slot application includes:

texttttemitDevtoolsEvent({ type: 'slotWrite', slotKind: slot.kind, slotKey: key })

This is intentionally emitted even when the runtime later decides to skip a DOM write (for example when values are `Object.is` equal). That makes slotWrites a measure of "reactive updates attempted" rather than "DOM updates performed".

DOM operations: a vocabulary of `domOp` strings

The block runtime calls `emitDomOp(...)` around operations like:

- `setText`
- `setAttribute` / `removeAttribute`
- `setProperty`
- `styleSetProperty` / `styleRemoveProperty`
- `addEventListener` / `removeEventListener`

The string payload keeps the event cheap to allocate and flexible to extend.

21.5.2 Scheduler: flush timing

In `runtime/scheduler.ts`, the flush loop emits:

- `flushStart` at the beginning
- `flushEnd(durationMs)` at the end

This is the basis for simple performance dashboards.
In `runtime/scheduler.ts`, these events are emitted inside `flush(...)`:

- `flushStart` at the beginning

- `flushEnd(durationMs)` at the end (duration computed using the scheduler's `now()` source)

Because the scheduler is designed to be deterministic (queues sorted by lane/priority and stable promotion rules), these timing events are also useful when you want to prove "same inputs produce the same flushing pattern".

21.5.3 Signals: scheduled computations

In `runtime/signals.ts`, scheduling a non-sync computation emits `scheduleComputation`. The devtools counter is intentionally 'how many computations were scheduled', not 'how many ran'.

The payload includes optional `name` and `lane` metadata taken from the computation object. That metadata isn't required for semantics, but it makes event streams much more useful in practice (you can group by lane or by effect name when debugging).

21.6 Tests-as-spec

There are two test files that define the intended behavior.

21.6.1 `runtime/__tests__/devtools.test.ts`

This suite checks two high-level properties:

- **slotWrites are counted** when `mountBlock()` applies initial values and when `update()` applies new values.
- **scheduledComputations and flush metrics are counted** when the scheduler runs and when signals schedule work.

In the scheduler portion, the test uses `createScheduler( requestFlush: (cb) => cb() )` to make flushing deterministic.

Reading this test as a contract gives you the intended semantics:

- Slot writes must be observed during both mount and update.
- Scheduler flush events must be emitted even when a custom `requestFlush` runs synchronously.
- Signal scheduling should increment a counter even if the actual effect execution happens immediately (depending on lane/batching).

21.6.2 `runtime/__tests__/instrumentation.test.ts`

This suite locks in the *gating* behavior:

- without instrumentation enabled, `domOps` stays at 0 even though the runtime emits DOM op events
- after calling `setInstrumentationEnabled(true)`, DOM-related operations increment `domOps`

This is a very important behavioral line for performance: many hot paths call `emitDomOp()` all the time, and the test documents that these calls must not inflate counters unless instrumentation is explicitly enabled.

21.7 How you might use this in practice

21.7.1 From scratch: what problem are we solving?

When people say "devtools" they often mean a glossy UI. But the hard part comes first: you need a *stable and cheap* stream of runtime facts. In this repo, the HRBR runtime is fast specifically because it avoids whole-tree diffing; that also means you don't get a built-in "diff" you can inspect.

So the question becomes:

> *If the runtime doesn't diff a tree, what do we observe to debug performance and correctness?*

The devtools layer answers that by emitting *small, well-timed events*:

- A slot write attempt tells you "reactivity wanted to change something."
- A DOM op tells you "we actually touched the DOM" (when instrumentation is enabled).
- Scheduler flush boundaries tell you "work was batched and committed now."

Why it matters: these events let you debug the two hardest classes of UI problems:

- **Too much work:** you are doing redundant updates, thrashing the DOM, or flushing too often.
- **Work at the wrong time:** you are flushing in a tight loop or starving lower-priority work.

The design is intentionally minimal so it stays truthful: a few counters and an event hook are much harder to accidentally "lie" with than a large, stateful instrumentation system.

21.7.2 A tiny devtools "panel" in 50 lines (hook + DOM)

The runtime doesn't ship a UI, but it gives you everything you need to build one. Here's a minimal pattern you can paste into a demo page: install a hook, aggregate in userland, and render a tiny dashboard.

```
import {
	setDevtoolsHook,
	getDevtoolsCounters,
	resetDevtoolsCounters,
	setInstrumentationEnabled,
} from 'relax/hrbr'

export function installHrbrDevtoolsPanel(hostEl) {
	// Optional: enable expensive counters.
	setInstrumentationEnabled(true)
	resetDevtoolsCounters()

	const state = {
		domOpsByKind: Object.create(null),
		lastFlushDurationMs: 0,
	}
```

```
	setDevtoolsHook((e) => {
		if (e.type === 'domOp') {
			state.domOpsByKind[e.op] = (state.domOpsByKind[e.op] ?? 0) + 1
		}
		if (e.type === 'flushEnd') {
			state.lastFlushDurationMs = e.durationMs
		}
	})

	function render() {
		const c = getDevtoolsCounters()
		hostEl.innerHTML = `
			<pre>
slotWrites: ${c.slotWrites}
scheduledComputations: ${c.scheduledComputations}
flushes: ${c.flushes}
flushTimeMs: ${c.flushTimeMs}
domOps: ${c.domOps}
allocs: ${c.allocs}

lastFlushDurationMs: ${state.lastFlushDurationMs}
domOpsByKind: ${JSON.stringify(state.domOpsByKind, null, 2)}
			</pre>
		`
		requestAnimationFrame(render)
	}

	render()
	return () => setDevtoolsHook(null)
}
```

Two important details in this example are grounded in `runtime/devtools.ts`:

- The hook sees *every* event, even if expensive counters are disabled.
- If you don't install a hook *and* you keep instrumentation disabled, hot-path emitters are designed to become no-ops.

This is why the API is split into both counters and a hook: you can build UIs without forcing the runtime to count every single thing.

21.7.3 How to interpret the counters (and common misreads)

Counters are not correctness. They are an intentionally blunt instrument. Here are the interpretations that match the tests:

- **`slotWrites`**: how many slot updates were *attempted* by the block runtime. This includes cases where the runtime decides a DOM write is unnecessary.
- **`domOps`**: how many DOM mutations were observed *when instrumentation is enabled*. The instrumentation tests lock in that this must stay at 0 by default.
- **`scheduledComputations`**: how often signals scheduled work (not necessarily how often it ran).

- **`flushes`** and **`flushTimeMs`**: coarse scheduler batch metrics.

Common misread: "slotWrites went up, therefore DOM changed." Not necessarily. In this runtime, slotWrites measure how often reactivity *wanted* to update; domOps measure how often the runtime actually mutated the DOM (and only when the expensive counters are enabled).

21.7.4 A tiny contract you can build tooling around

If you want to build real tooling (like a profiler timeline), define your own minimal contract on top of the hook:

- Treat `flushStart` / `flushEnd` as a bracket for "commit work".
- Count `domOp` events inside that bracket to get "DOM work per flush".
- Group `scheduleComputation` by lane to visualize priority.

This is deliberately similar to the idea of tracing spans, but implemented with a tiny custom event union. If you want the conceptual background, see OpenTelemetry's overview: https://opentelemetry.io/docs/what-is-opentelemetry/.

21.8 Exercises (tests-first)

1. **Write a test that proves the "streaming mode" behavior:** install a hook, keep instrumentation disabled, trigger a block update, and assert that you receive `domOp` events while `getDevtoolsCounters().domOps` remains 0. (Hint: this should be an addition to `runtime/__tests__/instrumentation.test.ts`.)
2. **Build a per-op counter:** implement a tiny helper `createDomOpsHistogram()` that installs a hook and keeps a histogram `{ [op]: count }`. Prove in a test that `setText` happens during a text slot update.
3. **Add a new event type (safely):** add an optional `flushBudgetMs` field to `flushStart` (or introduce a new event) and thread it from the scheduler. Update the tests to lock in the behavior. If you can't update all call sites cleanly, you're changing the contract too aggressively.

Two common patterns:

- **Counters dashboard**: poll `getDevtoolsCounters()` and display deltas per second.
- **Event stream**: set a hook via `setDevtoolsHook((e) => { ... })` and record or filter events.

A practical rule of thumb:

- enable instrumentation only in debug builds or perf investigations
- leave it off in production unless you have a carefully designed sampling strategy

21.8.1 A simple pattern: hook + manual aggregation

If you want detailed breakdowns without touching the built-in counters, install a hook and aggregate in userland:

- Count per-domain domOps (e.g. `setText` vs `setAttribute`)
- Track flush durations into histograms
- Group schedule events by lane

This works even when instrumentation is disabled, because the hook still receives events.

21.9 Online resources

- Performance measurement pitfalls and best practices (timers, main-thread work, long tasks):
  web.dev: RAIL performance model,
  MDN: `performance.now()`.
- Observability patterns for UI runtimes (profilers, tracing, counters):
  OpenTelemetry: overview.
- Source maps and error locations are why devtools often care about source positions:
  Source Map Revision 3 Proposal.

Chapter 22

Benchmarks methodology (repeatability + fairness)

Benchmarks are where performance stories can go wrong fast. It's easy to produce a chart that looks convincing while accidentally measuring something else (layout thrash, dev-mode checks, GC noise, or even the benchmark harness itself).

This chapter is the "source of truth" for the Relax.js benchmark harness:

- harness code: `benchmarks/harness.ts`
- runner UI: `benchmarks/index.html` + `benchmarks/main.ts`
- methodology notes: `docs/benchmarks.md`

The core goals are:

- **Repeatability**: seeded randomness, stable profiles, and a clear warmup/measure separation.
- **Fairness**: compare like-for-like work (same DOM outcomes, comparable commit boundaries), and control build modes (dev vs prod).
- **Interpretability**: report metrics that catch jank (p95 and frame drops), not just averages.

22.1 A tiny contract for what we are measuring

It helps to make the benchmark harness explicit as a small contract:

- **Input:** a benchmark case that can mount into a host element and perform one unit of work per tick.
- **Output:** per-tick timings and a few summary statistics (avg, p95, drops).
- **Success criterion:** when framework A is faster than framework B, they are doing comparable work and reaching a comparable "committed" state before the timer stops.
- **Error mode:** if one framework returns early (async batching) while another blocks until DOM is updated, the comparison is not fair.

Most of this chapter explains how Relax's harness tries to stay on the right side of that contract.

22.2 Benchmarks run in a real browser

22.3 Benchmarks run in a real browser

The harness is designed to run in a browser environment:

- Uses `requestAnimationFrame` to align work with frames.
- Uses `performance.now()` for timing.
- Bundles through Rollup configs that target `browser: true`.

See `benchmarks/index.html` and the entry script `benchmarks/main.ts`.

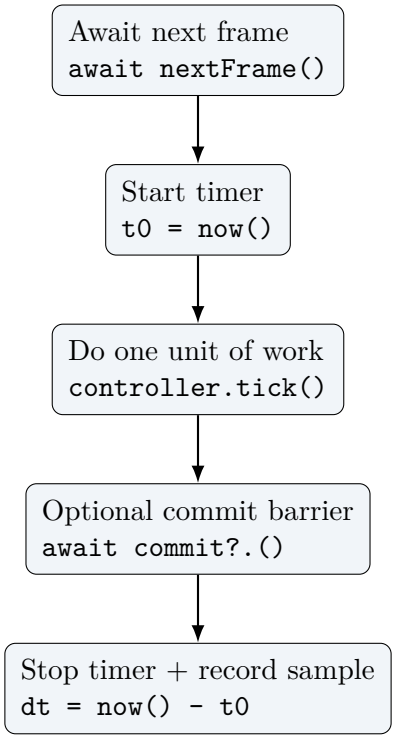
22.3.1 Why "real browser" matters

Most UI framework performance costs are dominated by:

- DOM mutation costs (properties, attributes, insertion/reordering)
- layout and style recalculation
- paint/compositing
- batching policies (microtasks, macrotasks, rAF)

These are browser behaviors. Node + jsdom is useful for correctness tests, but it doesn't model layout/paint and it changes timing behavior. That's why the canonical benchmark environment is a browser.

Diagram: how a benchmark tick is measured



This is the heart of "fairness" in the harness: every framework is asked to do one `tick()`, and the measured duration only finishes once the case reports that it has committed.

22.3.2 A small but important detail: Node/jsdom fallback exists

The harness is designed for browsers first, but `benchmarks/harness.ts` contains a compatibility fallback:

- `now()` prefers `performance.now()` but falls back to `Date.now()`.
- `nextFrame()` prefers `requestAnimationFrame` but falls back to `setTimeout(..., 16)`.

This is *not* intended to make Node-based benchmarking "accurate". It's there so that small smoke runs and tooling can still function in non-browser environments.

22.4 The harness contract

The core harness is `benchmarks/harness.ts`.

22.4.1 Benchmark case shape

A benchmark case is a `BenchmarkCase`:

- `name`: stable identifier for reporting
- `setup(host)`: returns a controller with
  `tick()` -> perform one unit of work
  `teardown()` -> clean up resources and DOM
- optional `commit()`: a barrier that resolves only once the framework has committed for the tick

The `commit()` hook is how the harness stays fair across frameworks that batch work differently (e.g. React flush/paint boundaries).

22.4.2 What counts as "one tick"? (and why that matters)

The harness intentionally measures a *steady-state update loop*. The goal is not to measure initial mount (which can be relevant, but is often a different set of costs), and not to measure bundling or download size.

Each tick should represent something plausible in UI work:

- update 1% of a large list
- shuffle keyed rows
- update styles in a grid
- toggle a boolean prop on many nodes

If a case is too synthetic (e.g. "just mutate one text node a million times"), it can still be useful — but only if we interpret it correctly as a microbenchmark.

22.4.3 Tiny contract for a fair `BenchmarkCase`

To keep comparisons honest, the benchmark cases should follow a simple contract:

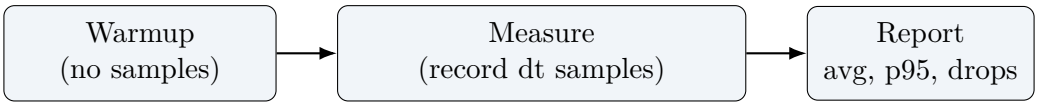
- `tick()` should perform one repeatable unit of work and avoid random extra allocations.
- `teardown()` must remove event handlers / timers / subscriptions (avoid cross-case leaks).
- If a framework batches updates asynchronously, provide `commit()` that resolves after work is actually committed.

22.4.4 Warmup vs measure windows

`runBenchmark()` splits execution into two windows:

- **Warmup frames** (default 30): let JIT, memoization, layout caching, and internal queues settle.
- **Measure frames** (default 120): record per-tick durations.

Diagram: warmup vs measure windows (and what gets recorded)



Warmup frames intentionally do the same work as measured frames; the only difference is whether the duration samples are kept.

Implementation details in `runBenchmark()`:

- For each tick: await `nextFrame()` , record `t0` , run `tick()` , await `commit()` , compute `dt`.
- A **frame drop** is counted when `dt > frameBudgetMs` (default 16.7ms).

22.4.5 Reported metrics: avg + p95 + drops

The harness reports:

- `avgMs` , average time per tick
- `p95Ms` , 95th percentile tick time (tail latency)
- `frameDrops` , number of frames over budget

22.4.6 How to read results (avg vs p95 vs drops)

In UI work, **drops are the user experience**. Averages can look great while still producing jank if there are occasional slow frames.

- **avgMs** answers: "How much work do we typically do per frame?"
- **p95Ms** answers: "How bad are the spikes?"

- **frameDrops** answers: "How often do we miss our frame budget?"

At 60Hz, the typical budget is about **16.7ms**. If a case shows `drops: 120` for `frames: 120`, it means *every* measured tick exceeded the budget.

This is why the harness reports drops alongside avg/p95: it is the easiest way to distinguish "fast enough to feel smooth" from "fast on average but still janky".

Percentiles are computed on sorted samples with a simple index selection. This is good enough for stable comparisons at the harness scale.

The harness computes p95 with:
texttidx = floor(p * (n - 1)) (clamped). This is a simple deterministic estimator, which is a feature here: the same sample list always produces the same percentile.

22.5 Profiles and seeded runs

Repeatability relies on standard profiles and seeded randomness.

22.5.1 Profiles via URL parameters

The profile parser lives in `benchmarks/profile.ts`.

It reads query params from `benchmarks/index.html`:

- `profile`: `quick | default | stress`
- `seed`: integer seed for randomized update patterns
- `warmup`, `frames`, `budget`: overrides for fine-grained control

22.5.2 Benchmark variables (every knob you can turn)

These are the key variables that affect results. When you publish results, list them, because changing any of them can change the numbers:

- **`profile`**: selects baseline `warmup` + `frames` window sizes.
- **`seed`**: makes randomized update patterns repeatable.
- **`warmup`**: number of warmup frames (no samples recorded).
- **`frames`**: number of measured frames (samples recorded).
- **`budget`**: frame budget in milliseconds (drives "drops").
- **build mode**: dev vs production bundles (especially affects React).
- **browser + OS + CPU governor**: real-world environment noise.

This harness keeps the first five variables explicit and reproducible.

22.5.3 Examples: URL parameters in practice

The runner prints the profile and seed at the top of the output. Example:

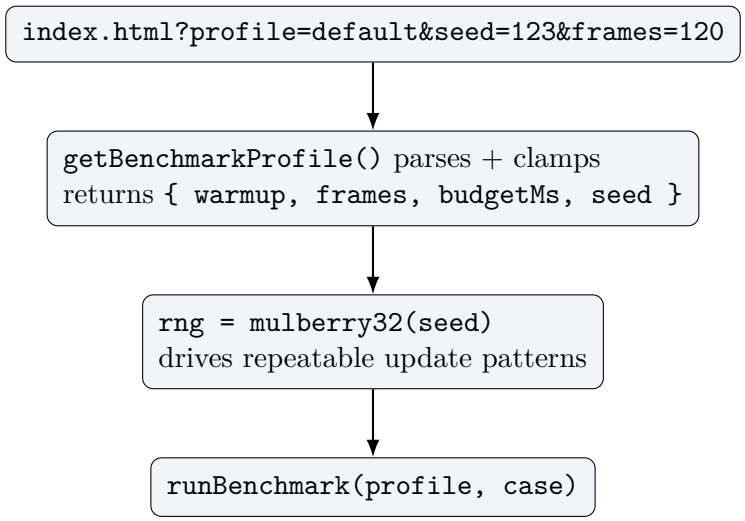
> textttRunning...
> profile: default
> seed: 12345

Typical URLs:

- Quick sanity pass:
  textttbenchmarks/index.html?profile=quick&seed=1
- Longer run that makes tail latency clearer:
  textttbenchmarks/index.html?profile=stress&seed=42&budget=16.7
- Debugging a suspicious regression by increasing sample size:
  textttbenchmarks/index.html?profile=default&frames=600&seed=12345

Defaults are intentionally "reasonable and stable" for local runs.

Diagram: profile parameters flow



Implementation note: the profile parser clamps overrides to avoid accidental runaway configurations:

- `warmup` is clamped to `[0, 10000]`
- `frames` is clamped to `[1, 100000]`

Seed values are forced into uint32 range using `>> 0`, which keeps the PRNG behavior consistent.

22.5.4 Seeded RNG

Randomized patterns (e.g. which list indices change each tick) should be deterministic. Benchmark code uses a seeded RNG in `benchmarks/rng.ts`.

A key methodology rule: the seed must be part of the report output so results can be reproduced.

The benchmark RNG implementation is `Mulberry32` (`benchmarks/rng.ts`). It is simple, fast, and deterministic—perfect for selecting indices or toggling boolean flags in a way that remains repeatable across runs.

22.6 The benchmark runner UI

The main entrypoint `benchmarks/main.ts` wires a button click to sequential benchmark runs.

Key details:

- It resets `host.innerHTML = ''` between runs so each case starts clean.
- It prints formatted results to a `<pre>`.
- It runs both HRBR and VDOM variants, and includes adapters for React/Solid.

Because different frameworks have different setup/teardown costs, the harness keeps the per-tick loop stable and encourages cases to isolate the steady-state behavior.

22.6.1 Why `host.innerHTML = ''` between runs matters

Clearing between runs gives you two strong properties:

- **Isolation:** DOM nodes and event listeners from a previous framework/case do not bleed into the next.
- **Repeatability:** each case starts from the same initial DOM shape.

This is also why safe `teardown()` behavior is part of the case contract: the harness clears DOM, but frameworks may have timers/schedulers that need explicit disposal.

22.7 Cases: what they measure

Cases live under `benchmarks/cases/`. The intent is to cover:

- **Pure text throughput**: `text-1m`
- **Large lists with sparse updates**: `list-10k-1pct`
- **Many small components/widgets**: `widgets-200`
- Extra comparisons in `cases/more.ts`

Important fairness guideline: each case should produce the same DOM output across frameworks (or as close as is reasonable).

22.7.1 Case study: `text-1m`

In `benchmarks/cases/text-1m.ts`, the HRBR variant builds the smallest possible compiled block:

- `templateHTML: ‘<div><span> </span></div>‘` includes an explicit space so there is a real text node.
- Slot path `[0,0]` means: root `<div>` child 0 is `<span>`, and its child 0 is the `Text` node.
- `tick()` updates a signal in chunks of `10,000` so work remains frame-visible.

This isn't a "real app" workload, but it is an excellent microbenchmark for measuring pure slot-write throughput.

22.7.2 What should you expect from `text-1m`?

texttttext-1m is a microbenchmark. It largely measures:

- how quickly a runtime can update a text node
- how much overhead there is in the reactive/scheduling layer
- whether the work produces long-tail spikes (p95) under repeated updates

Because it does not exercise reconciliation, keyed diffing, or lots of element updates, it should *not* be used to claim "framework X is faster for apps". It should be used to claim something narrower: "The slot update path is efficient."

22.7.3 Case study: `list-10k-1pct`

In `benchmarks/cases/list-10k-1pct.ts`, the cases delegate to example mount helpers:

- HRBR: `mountList10k1pctHRBR(host, 10000)`
- VDOM: `mountList10k1pctVDOM(host, 10000)`

The contract is the same for both: return a controller with `update1pct()` and `dispose()`. Methodologically, that matters: both implementations are being asked to do the same "one tick" of work.

22.7.4 What `list-10k-1pct` tries to model

This is closer to "real UI pain":

- 10,000 rows is a large but plausible virtualized/table workload.
- Updating 1% per tick models "small changes in a large list" (sorting indicators, status updates, counters).
- The workload stresses reconciliation overhead and DOM patch strategy.

This is where HRBR often wins: it can apply targeted DOM mutations without re-diffing a full VDOM tree.

22.7.5 Case study: `widgets-200`

texttttwidgets-200 is a component-shape workload:

- many small independent widgets
- small updates in many places

This case helps distinguish:

- runtimes that do well when work is split across many small islands
- runtimes that pay a fixed overhead per component update

Implementation: `benchmarks/cases/widgets-200.ts`.

22.7.6 Extra cases in `cases/more.ts` (and what each means)

The runner also includes additional comparisons in `benchmarks/cases/more.ts`:

- `props-toggle-5k`: toggles a boolean property (`disabled`) on 5,000 buttons.
- `style-grid-1k`: updates inline styles on 1,000 elements.
- `table-2k-shuffle`: reorders rows each tick (keyed diff/reorder stress).
- `svg-1k`: updates attributes on 1,000 SVG circles.
- `input-typing-1k`: updates 1,000 input values (a classic "controlled input" stress).

These give you a more complete story than a single macro statistic. For example, a runtime can be great at list reconciliation but weaker at style updates, or vice versa.

22.8 Build modes matter (dev vs prod)

Some frameworks change scheduling and checks based on `processėnvṄODE_ENV`. The benchmark Rollup config sets this value at bundle time.

- Rollup config: `rollup.benchmarks.config.mjs`
- It uses `@rollup/plugin-replace` to define `process.env.NODE_ENV`.

NPM scripts in `package.json` expose common modes:

- `build:bench:dev`
- `build:bench:prod`

To be fair, comparisons should run with comparable modes (usually prod vs prod).

22.8.1 What the Rollup configs actually do

The benchmark bundles are built through two rollup configs:

- `rollup.benchmarks.config.mjs` (full UI runner)
- `rollup.benchmarks.smoke.config.mjs` (perf-smoke gate)

Both use `@rollup/plugin-replace` to inject:
texttttprocess.env.NODE_ENV = 'production' by default.

This matters for fairness because some libraries (notably React) perform extra checks and different scheduling policies in development builds.

22.9 Perf smoke (CI-friendly)

For CI and "obvious regression" detection, the repo includes a reduced entry:

- `benchmarks/perf-smoke.ts`
- `benchmarks/perf-smoke.html`

It builds via `rollup.benchmarks.smoke.config.mjs` into `benchmarks/dist-smoke/`.

22.9.1 What perf-smoke is actually specifying

You can read `benchmarks/perf-smoke.ts` as an executable specification of the harness:

- it obtains a profile via `getBenchmarkProfile()`
- it forces short defaults unless overridden (default profile becomes `quick`)
- it runs a small fixed set of cases (currently `text-1m` and `list-10k-1pct`, HRBR variants)
- it emits a JSON report that includes profile, timestamp, and result rows
- it enforces very wide thresholds to catch catastrophic regressions

Even though this isn't a Vitest file, it plays the same role as tests in earlier chapters: it defines what "performance sanity" means for this repository.

22.9.2 Why the thresholds are wide

`perf-smoke.ts` prints a JSON report to `console.log` and then checks very wide thresholds:

- fail only when `avgMs > 250` or `p95Ms > 500`

This is intentional: CI machines are noisy. The smoke test is designed to catch catastrophic regressions (infinite loops, accidental $\mathcal{O}(n^2)$ behavior), not micro-optimizations. This is also why the gate uses two different lenses:

- `avgMs`: for sustained runaway behavior
- `p95Ms`: for severe tail latency spikes

Tail latency matters because user experience is dominated by occasional slow frames.

22.10 Instrumentation and benchmarks: keep them separate

Optional runtime instrumentation exists in `runtime/devtools.ts` (Chapter 21).

Methodology rule: keep instrumentation disabled during benchmarks unless you are explicitly measuring it. Otherwise, the measurement includes the measurement.

22.11 Online resources

These links are useful if you want to validate (or improve) the methodology beyond this repo:

- High Resolution Time API (`performance.now()`): https://developer.mozilla.org/en-US/docs/Web/API/Perfo
- `requestAnimationFrame` timing model: https://developer.mozilla.org/en-US/docs/Web/API/Window/reques
- User Timing API (marks/measures; helpful for deeper profiling): https://developer.mozilla.org/en-US/docs/Web/API/User_Timing_API
- "The mental model" for why p95 matters (latency vs averages): https://www.brendangregg.com/Latency.html

In practice, if you want to correlate benchmark results with DOM op counts:

- run once with instrumentation off to measure baseline
- run again with instrumentation enabled (and accept the overhead) to measure the profile of operations

Comparing these two runs helps you understand whether a regression is dominated by DOM work, scheduling, or allocation overhead.

22.12 Checklist: how to run a fair comparison

A practical checklist grounded in this repo's structure:

1. Build benchmark bundle in a known mode (dev or prod) and record it.
2. Use a named profile (quick/default/stress) so warmup and frames are consistent.
3. Fix a seed and include it in the report.
4. Run multiple times and compare distributions (p95 and drops), not just averages.
5. If a result looks surprising, isolate the case and profile it (browser performance tools).

22.13 Interpreting a real output (example walkthrough)

The runner output is designed to be copied into issue reports. It includes enough context to be reproducible (profile + seed + case names).

Here is a condensed example pattern (taken from a typical run):

texttt# list-10k-1pct:relax-hrbr
frames: 120
avg: 1.333 ms
p95: 4.000 ms
drops: 0

How to read it:

- **0 drops** means it stayed under budget on every measured tick. It should feel smooth.
- **avg 1.333ms** means the typical tick is cheap.
- **p95 4ms** means spikes exist but are still far below 16.7ms.

And compare to the same case in VDOM mode:

texttt# list-10k-1pct:relax-vdom
frames: 120
avg: 62.558 ms
p95: 94.000 ms
drops: 120

This is a very different story:

- **drops 120/120** means every tick exceeded the frame budget. It will look janky.
- The avg and p95 values suggest the runtime is doing work closer to the "tens of milliseconds" per tick.

This sort of comparison is where HRBR tends to win: it avoids large VDOM diff/patch overhead by pre-compiling stable DOM addressing.

22.14 Why this benchmark approach is better than "single number" benchmarks

This harness is intentionally simple, but it has a few properties that make it more trustworthy than many "microbenchmark charts":

- **Per-frame pacing:** `requestAnimationFrame` forces you to respect frame boundaries.
- **Tail metrics:** p95 and frame drops capture user-visible jank.
- **Warmup window:** avoids measuring purely JIT/startup effects.
- **Commit barrier:** optional `commit()` avoids lying about async batching.
- **Seeded randomness:** makes results reproducible without removing realism.
- **Isolation:** host DOM is cleared between runs, and each case has a teardown contract.

No benchmark is perfect. But if you follow the checklist above and publish the variables (profile/seed/budget/mode), this harness gives you results that are much harder to accidentally misinterpret.

22.15 Online resources

- Measuring performance in the browser and why alternatives to `Date.now()` matter:
  MDN: `performance.now()`.
- About frame budgets and why 16.7ms is the common baseline:
  web.dev: rendering performance.
- Why warmup matters (JIT, inline caches, tiered compilation):
  V8 blog (background on optimization and warmup effects).
- Percentiles and tail latency as a reporting best practice:
  Wikipedia: percentile,
  Google SRE: tail latency (The Tail at Scale).
- General benchmark hygiene and common pitfalls:
  benchmarking.sites (community notes).

Chapter 23

Release-quality discipline: tests, typecheck, lint, builds

This chapter is about the habits that make a codebase shippable. Relax is a teaching framework, but it also treats its test suite as an executable spec and its build pipeline as part of correctness.

The goal of this chapter is not to list commands—it's to explain *why* each quality gate exists, what it catches, and how to debug failures without guessing.

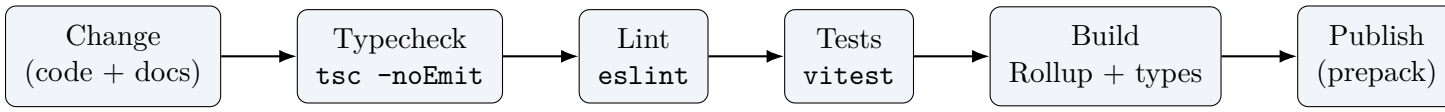
If you only remember one thing: each gate is a different lens on correctness. Typecheck defends your *data model*, lint defends your *human factors* (consistency and obvious mistakes), tests defend your *observable behavior*, and builds defend your *published artifact*.

23.1 The quality-gates contract

A "release-quality" change in this repo is expected to be green on:

- **Typecheck**: TypeScript compilation with `-noEmit`
- **Lint**: ESLint rules for consistency and obvious mistakes
- **Unit/integration tests**: Vitest (jsdom)
- **Build**: Rollup bundles for ESM + CJS, plus type bundle generation on publish

Diagram: quality gates as a pipeline



This diagram is intentionally linear. In practice you often run these in parallel locally, but conceptually each step answers a different question: "Do the types hold?", "Is the diff readable?", "Is the behavior correct?", and "Is the published artifact correct?"

These are not redundant; they fail on different classes of bugs.

23.1.1 A tiny contract (inputs, outputs, failure modes)

It helps to think about the pipeline as a function that takes a working tree and returns publishable artifacts:

- **Input**: TypeScript source under `src/`, plus experimental surfaces under `runtime/`, `compiler/`, and the TSX runtime under `relax-jsx/`.
- **Output**: compiled JS bundles in `dist/esm/` and `dist/cjs/` and a declaration bundle in `dist-types/`.
- **Success**: consumers can import from the package `exports` map and get the runtime semantics the tests describe.
- **Failure modes**: "works in tests" but fails to typecheck; "works locally" but fails to bundle; "bundles" but exports the wrong files.

23.2 Typecheck: make illegal states unrepresentable

The repo's primary TypeScript config is `tsconfig.json`. A few settings worth noticing:

- `strict: true` — the baseline.
- `noUncheckedIndexedAccess: true` — forces you to handle missing keys.
- `exactOptionalPropertyTypes: true` — optional means "may be absent", not "may be undefined".
- `jsx: react-jsx` and `jsxImportSource: relax-jsx` — enables the TSX layer.

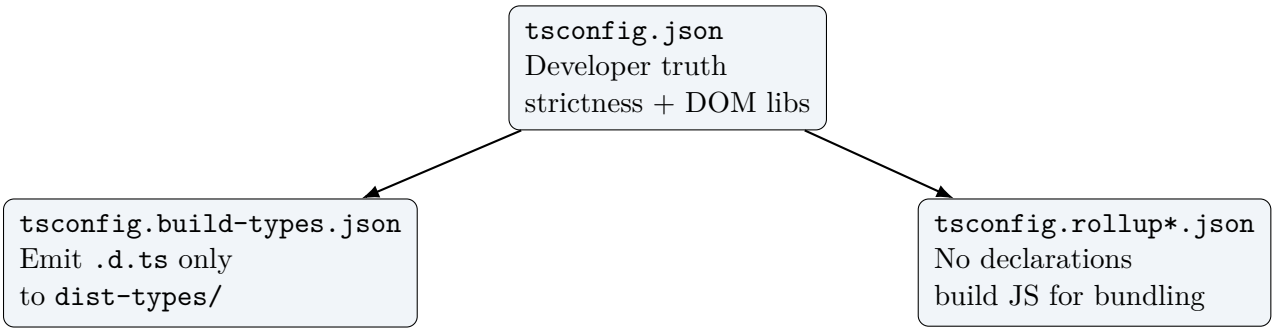
Two other settings are quietly important for how the codebase is built and consumed:

- `moduleResolution: Bundler`: TypeScript resolves imports similarly to modern bundlers. In practice, this aligns the typechecker with how Rollup (and consumer bundlers) will interpret ESM.
- `paths` for `relax-jsx`: TS can typecheck TSX code that imports the JSX runtime via the virtual specifier `relax-jsx`. This is mirrored on the test side by the Vitest alias for `relax-jsx`.

The `typecheck` script (in `package.json`) runs:

```
tsc -p tsconfig.json -noEmit
```

Diagram: why multiple tsconfig files exist



23.2.1 Common typecheck failures (and how to debug)

- **Indexing errors**: caused by `noUncheckedIndexedAccess`. Fix by narrowing or checking bounds.
- **Optional property mismatches**: caused by `exactOptionalPropertyTypes`. Fix by removing explicit `undefined` assignments or changing types.
- **DOM typing issues in tests**: Vitest uses jsdom; make sure `lib: ["DOM"]` is present (it is).

23.2.2 Why there are multiple `tsconfig` files

This repo uses a small family of configs to keep responsibilities separate:

- `tsconfig.json`: the "truth" for developer typechecking. It enables declarations and maps, targets modern JavaScript (`ES2022`), and includes the DOM library.
- `tsconfig.build-types.json`: extends the base config but turns on *declaration-only* emit. This produces the publishable `.d.ts` surface under `dist-types/`.
- `tsconfig.rollup.json` and `tsconfig.rollup.cjs.json`: extend the base config but disable declaration emission. These exist to keep Rollup builds focused on JavaScript output (ESM to `dist/esm/`, CJS to `dist/cjs/`) without creating duplicate `.d.ts` trees.

This split is release discipline in practice: build tooling stays fast and predictable, while types remain a first-class published artifact.

23.3 Lint: keep the surface area consistent

Lint config lives in `eslint.config.mjs`. It uses the ESLint 9 flat config model and `@eslint/js` recommended rules.

The lint scripts are:

- `npm run lint` — runs eslint on `src`
- `npm run lint:fix` — applies safe auto-fixes

Note: linting only `src` is a choice. HRBR runtime and compiler code are primarily guarded by tests and typecheck, but could be added to lint scope later.

23.3.1 What lint catches that typecheck doesn't

TypeScript proves things about types, but lint is often better at catching:

- accidental unused variables or imports
- inconsistent patterns that make reviews harder
- "looks correct" code that has a known foot-gun shape

Even with a mostly-recommended ESLint config, getting a clean lint run is part of keeping diffs small and reviewable.

23.4 Tests: executable specs with jsdom

All tests use Vitest (see `vitest.config.js`). The environment is `jsdom`, which means:

- DOM APIs are available (document, elements, events)
- you can test DOM manipulation deterministically without a real browser

Test inclusion patterns:

- `**/__tests__/**/*.test.ts`
- `**/__tests__/**/*.test.tsx`

23.4.1 The test suite structure

This repo uses tests in three roles:

- **Unit tests**: small functions like diffs and helpers.
- **Integration tests**: end-to-end flows (mount → patch → destroy, SSR → hydrate, compiler → mount).
- **Property/fuzz tests**: seeded tests that exercise many update sequences (e.g. reconciler and signals).

If you change invariants (like reconciliation semantics), expect to update both unit tests and fuzz oracles.

23.4.2 Tests-as-spec example: TSX wiring

The TSX layer is a good example of how "typecheck + tests" combine into a stronger guarantee. The typechecker is configured to understand TSX (`jsx: react-jsx` and `jsxImportSource: relax-jsx`), but tests still need to verify the *runtime* behavior.

In `src/__tests__/jsx-tsx.test.ts`, the test sets up a host element in jsdom, runs an example, and asserts on the resulting HTML:

- it writes `<div id="app"></div>` into `document.body`

- it locates the host element and passes it into `runHelloExample`
- it asserts that the returned HTML matches `<div class="hello">Hello Relax</div>`

The important release-quality point is the scope of the guarantee: this test doesn't just say "TSX compiles"—it says "the TSX runtime integrates with the DOM mounting layer and produces the expected structure".

23.4.3 Debugging failing tests (a workflow)

A practical workflow:

1. Re-run only the failing file (Vitest supports file targeting; use the repo scripts).
2. Read the failure output and find which invariant changed.
3. Locate the corresponding runtime function and its other tests.
4. Add a minimal regression test that isolates the bug.
5. Only then, fix the implementation and re-run the focused test.

When failures are timing-related (scheduler, signals), prefer deterministic schedulers and explicit ticking (the repo already uses this pattern).

23.4.4 What to do when a test failure seems "impossible"

The fastest way to debug tough failures is to make the implicit invariant explicit:

1. Find the smallest assertion that fails.
2. Name the invariant in one sentence (e.g. "patch preserves node identity when keys and tag match").
3. Search for other tests that mention the same concept.
4. Read the corresponding implementation with the invariant in mind.

This avoids the common trap of chasing symptoms across unrelated modules.

23.5 Builds: ESM, CJS, and types

Relax ships multiple entrypoints (see `package.json exports`):

- `"."` — the VDOM library
- `"./hrbr"` — experimental HRBR runtime
- `"./compiler"` — experimental compiler

Rollup builds:

- ESM: `rollup.config.mjs` -> `dist/esm/`

- CJS: `rollup.config.cjs.mjs` -> `dist/cjs/`

Type declaration builds:

- `npm run build:types` uses `tsconfig.build-types.json` (emit declarations only)
- `npm run prepack` runs both JS builds and types before publishing

23.5.1 Why builds are a quality gate

A green test suite doesn't guarantee:

- exports are correct
- bundling doesn't pull in unexpected dependencies
- generated output is structurally valid for consumers

For example, the compiler bundle is large; build output sizes help you notice unexpected growth.

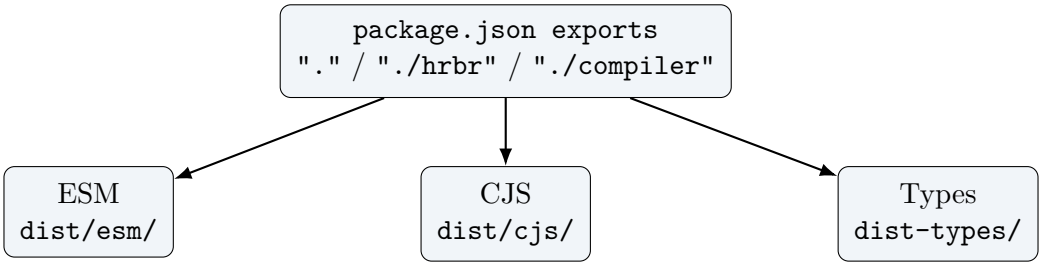
23.5.2 Build discipline: match `exports` to outputs

From `package.json`, Relax publishes multiple entry points (`"."`, `"./hrbr"`, `"./compiler"`). Release discipline means each export must be correct in three dimensions:

- **ESM path** points at something in `dist/esm/`
- **CJS path** points at something in `dist/cjs/` (with `.cjs` entry file names)
- **Types path** points at something in `dist-types/`

It's easy to accidentally break this mapping when renaming files, changing entry points, or restructuring module boundaries. That is why `prepack` builds both JS bundles and types before publish.

Diagram: package exports must match ESM, CJS, and types



23.6 Release checklist (practical)

A minimal release checklist aligned with this repo:

1. **Typecheck**: ensure TS still models your invariants.
2. **Lint**: ensure code style and basic safety checks pass.
3. **Tests**: run the suite; if you're changing HRBR internals, keep an eye on fuzz determinism.
4. **Build**: confirm ESM + CJS outputs compile.
5. **Bench smoke (optional but recommended)**: `build:bench:smoke` to catch catastrophic perf regressions.

23.7 Evidence from this repo

This chapter is grounded in real scripts and configs:

- `package.json` scripts: `typecheck`, `lint`, `test`, `build`, `prepack`
- `vitest.config.js` (jsdom environment)
- `eslint.config.mjs` (flat config)
- Rollup configs for bundling
- TypeScript configs for declaration and rollup builds

In the next section of the book (appendices), we'll turn these practices into a learner-friendly "build it from scratch" checklist and a test-reading guide.

23.8 Online resources

These references mirror the key moving parts in this chapter:

- TypeScript `tsconfig` reference (compiler options, strictness flags): https://www.typescriptlang.org/tsconfig
- `exactOptionalPropertyTypes` (why "optional" is not the same as "possibly undefined"): https://www.typescriptlang.org/tsconfig#exactOptionalPropertyTypes
- `noUncheckedIndexedAccess` (making missing keys explicit): https://www.typescriptlang.org/tsconfig#noUnc
- TypeScript JSX / `jsxImportSource`: https://www.typescriptlang.org/docs/handbook/jsx.html
- ESLint flat config (ESLint 9+): https://eslint.org/docs/latest/use/configure/configuration-files
- Vitest docs (jsdom environment, include patterns, reporters): https://vitest.dev/guide/
- jsdom project (what it simulates and what it doesn't): https://github.com/jsdom/jsdom
- Rollup configuration and multi-entry builds: https://rollupjs.org/configuration-options/
- Node.js `package.json` `exports` field (how entry points are resolved): https://nodejs.org/api/packages.html#ex

Appendix: notation

This book uses `file/path.ts` to reference source files. Symbol names are written like `h()` or `mountDOM()`.

References

Bibliography

[1] MDN Web Docs. *EventTarget: addEventListener() method*. https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener. Accessed 2026-03-01.

[2] MDN Web Docs. *Node: insertBefore() method*. https://developer.mozilla.org/en-US/docs/Web/API/Node/insertBefore. Accessed 2026-03-01.

[3] React Documentation. *Rendering Lists and Keys*. https://react.dev/learn/rendering-lists. Accessed 2026-03-01.

[4] Wikipedia contributors. *Longest increasing subsequence*. https://en.wikipedia.org/wiki/Longest_increasing_subsequence. Accessed 2026-03-01.

[5] TypeScript Documentation. *TSConfig Reference*. https://www.typescriptlang.org/tsconfig. Accessed 2026-03-01.

[6] Node.js Documentation. *Packages: package.json exports*. https://nodejs.org/api/packages.html#exports. Accessed 2026-03-01.

[7] Babel. *Babel Plugin Handbook*. https://github.com/jamiebuilds/babel-handbook/blob/master/translations/en/plugin-handbook.md. Accessed 2026-03-01.